Biomedical Signal Processing & Data Analytics

From Physiology to Machine Learning

Farhad Abtahi

2026

varkonyibarnabas@gmail.com

Biomedical Signal Processing & Data Analytics: From Physiology to Machine Learning

Farhad Abtahi

© 2026 Farhad Abtahi. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

First edition, 2026.

Pre-release beta build — Release 0 (v0.1.0-beta), built 2026-08-24. Not for distribution beyond CM2013 students.

varkonyibarnabas@gmail.com

Contents

Preface . . . viii
How to use this book . . . xi
The twelve design principles . . . xii
What you will be able to do . . . xiv
Notation and conventions . . . xiv

Chapter 1 — Biosignals & Medical Images: Origin, Physiology & Characteristics 1
1.1 Learning objectives . . . 1
1.2 What is a biosignal? . . . 2
1.3 Physiological origin: from ion to waveform . . . 2
1.4 The main biosignals and their signatures . . . 6
1.5 Medical images as signals . . . 14
1.6 The signal taxonomy . . . 15
1.7 The objectives of biosignal processing . . . 16
1.8 Before choosing a method: what kind of task is this? . . . 17
1.9 A typical pipeline: from patient to decision . . . 18
1.10 The ML/AI bridge . . . 19
1.11 Companion notebook . . . 19
1.12 Case-study tie-in . . . 21
1.13 Summary & key formulas . . . 21
1.14 Conceptual questions & exploration tasks . . . 21
1.15 Further reading . . . 22

Chapter 2 — Signal Acquisition, Sampling & Discretization 23
2.1 Learning objectives . . . 23
2.2 The measurement pipeline . . . 23
2.3 The electrode-electrolyte interface . . . 25
2.4 Electrode-skin impedance and the interface model . . . 25
2.5 How the interface creates motion artifact . . . 25
2.6 Anti-alias filtering and the front end . . . 27
2.7 Sampling I: temporal sampling and the Nyquist theorem . . . 28
2.8 Aliasing . . . 29
2.9 Quantization . . . 29
2.10 A/D conversion in practice . . . 32
2.11 Sampling II: spatial sampling and medical imaging . . . 32
2.12 Motion during acquisition . . . 33
2.13 Non-uniform sampling and resampling . . . 34
2.14 Electrical bioimpedance: an induced measurement modality . . . 36
2.15 The ML/AI bridge . . . 37
2.16 Companion notebook . . . 38
2.17 Case-study tie-in . . . 38
2.18 Summary & key formulas . . . 39

varkonyibarnabas@gmail.com

2.19 Conceptual questions & exploration tasks . . . 39
2.20 Further reading . . . 40

Chapter 3 — Fourier Analysis: Series, Transform, FFT & Time-Frequency 41
3.1 Learning objectives . . . 41
3.2 What a transformation is, and why we change representation . . . 41
3.3 Fourier series . . . 42
3.4 The Fourier transform . . . 42
3.5 The DFT and the FFT . . . 43
3.6 The FFT replaces convolution . . . 43
3.7 Frequency resolution vs. zero-padding . . . 45
3.8 Boundary handling: zero, mirror, and periodic padding . . . 47
3.9 The short-time Fourier transform and the spectrogram . . . 47
3.10 The uncertainty trade-off . . . 47
3.11 The 2-D Fourier transform for images . . . 49
3.12 Two domains, one operation: a dictionary of dual pairs . . . 50
3.13 The ML/AI bridge . . . 54
3.14 Companion notebook . . . 54
3.15 Case-study tie-in . . . 54
3.16 Summary & key formulas . . . 54
3.17 Conceptual questions & exploration tasks . . . 55
3.18 Further reading . . . 55

Chapter 4 — Windowing & Spectral Leakage 56
4.1 Learning objectives . . . 56
4.2 The finite-length problem and the origin of leakage . . . 56
4.3 Window functions . . . 58
4.4 Window properties, quantified . . . 58
4.5 The resolution-leakage trade-off . . . 60
4.6 Selection guidelines for ECG, EEG, and EMG . . . 61
4.7 The ML/AI bridge . . . 62
4.8 Companion notebook . . . 62
4.9 Case-study tie-in . . . 62
4.10 Summary & key formulas . . . 63
4.11 Conceptual questions & exploration tasks . . . 63
4.12 Further reading . . . 64

Chapter 5 — Wavelets and Other Transforms 65
5.1 Motivating question and the limits of Fourier . . . 65
5.2 Intuition first: one window size does not fit all . . . 66
5.3 The wavelet transform: CWT and DWT . . . 67
5.4 Worked example: localizing a K-complex, and an EMG envelope . . . 68
5.5 The Hilbert transform, the analytic signal, and the envelope . . . 70
5.6 Two more transforms: DCT and the z-transform . . . 70
5.7 Empirical mode decomposition and the Hilbert–Huang transform . . . 71
5.8 Signals-and-images box: one idea, two costumes . . . 71
5.9 The ML/AI bridge . . . 73
5.10 Companion notebook . . . 75

varkonyibarnabas@gmail.com

5.11 Case-study tie-in . . . 75
5.12 Summary and key formulas . . . 75
5.13 Conceptual questions and exploration tasks . . . 76
5.14 Further reading . . . 77

Chapter 6 — Stochastic Processes, Correlation and Time-Series Modelling 78
6.1 Motivating question and the random view of biosignals . . . 78
6.2 Random signals: expectation, variance, stationarity, ergodicity . . . 78
6.3 Autocorrelation: a memory meter . . . 79
6.4 Cross-correlation and coherence between channels . . . 81
6.5 The Wiener–Khinchin theorem: correlation is spectrum . . . 82
6.6 Time-series models: AR, MA and ARMA . . . 82
6.7 Model identification: ACF, PACF and order selection . . . 83
6.8 Worked example: an AR spectrum with a real peak . . . 85
6.9 The ML/AI bridge . . . 86
6.10 Signals-and-images box . . . 86
6.11 Companion notebook . . . 86
6.12 Case-study tie-in . . . 86
6.13 Summary and key formulas . . . 86
6.14 Conceptual questions and exploration tasks . . . 87
6.15 Further reading . . . 87

Chapter 7 — Spectral Estimation 89
7.1 Why we estimate, and the two problems . . . 89
7.2 The periodogram and why it is inconsistent . . . 89
7.3 The modified periodogram: windowing and leakage . . . 91
7.4 Bartlett's method: average to cut variance . . . 91
7.5 Welch's method: the everyday workhorse . . . 91
7.6 Multitaper estimation: averaging without cutting the record . . . 92
7.7 Parametric AR estimation: high resolution from short records . . . 93
7.8 The three estimators on one synthetic epoch . . . 93
7.9 Model-order selection and a decision framework . . . 95
7.10 Biomedical applications . . . 97
7.11 HRV: a worked example . . . 97
7.12 The ML/AI bridge . . . 99
7.13 Signals-and-images box . . . 100
7.14 Companion notebook . . . 100
7.15 Case-study tie-in . . . 100
7.16 Summary and key formulas . . . 100
7.17 Conceptual questions and exploration tasks . . . 101
7.18 Further reading . . . 102

Chapter 8 — Noise and Artifacts: Properties, Sources and Identification 103
8.1 Motivating question and the value of a diagnosis . . . 103
8.2 Quality control before preprocessing . . . 103
8.3 Characterizing noise: signatures and SNR . . . 105
8.4 White noise . . . 106
8.5 Coloured noise: pink and brown . . . 106

varkonyibarnabas@gmail.com

8.6 Powerline interference . . . 107
8.7 Impulsive and outlier noise — the 1-D/2-D twin . . . 107
8.8 Baseline wander and motion artifacts . . . 110
8.9 Physiological cross-contamination . . . 110
8.10 Instrumentation and quantization noise . . . 110
8.11 A diagnostic decision framework . . . 111
8.12 The ML/AI bridge . . . 111
8.13 Signals-and-images box . . . 111
8.14 Companion notebook . . . 112
8.15 Case-study tie-in . . . 112
8.16 Summary and key formulas . . . 112
8.17 Conceptual questions and exploration tasks . . . 113
8.18 Further reading . . . 113

Chapter 9 — Digital Filter Design for 1-D Signals & 2-D Images 114
9.1 Learning objectives . . . 114
9.2 Intuition first: a system is a machine, a filter is its fingerprint . . . 114
9.3 Theory: impulse response, convolution, and the frequency response . . . 116
9.4 FIR versus IIR, and the IIR families . . . 117
9.5 Zero-phase filtering, padding, and edge transients . . . 119
9.6 Worked example: an EEG band-pass + 50 Hz notch (a law and a number) . . . 120
9.7 The outlier-removal gateway: mean versus median (taught with images first) . . . 121
9.8 Time–frequency duality and 2-D linear filtering . . . 123
9.9 Worked example: Pan–Tompkins QRS detection . . . 124
9.10 The ML/AI bridge . . . 126
9.11 Companion notebook . . . 128
9.12 Case-study tie-in . . . 128
9.13 Summary, questions, and further reading . . . 128

Chapter 10 — Noise Reduction & Signal Enhancement 130
10.1 Learning objectives . . . 130
10.2 Intuition first: the signal repeats, the noise does not . . . 130
10.3 Theory: the $\sqrt{N}$ law, and the assumptions that make it true . . . 131
10.4 Worked example: SSVEP averaging for a BCI (a law and a number) . . . 132
10.5 Adaptive filtering: LMS noise cancellation . . . 133
10.6 Wavelet denoising: exploiting sparsity . . . 135
10.7 ICA and PCA: unmixing sources . . . 135
10.8 Method-selection guidelines . . . 137
10.9 The ML/AI bridge . . . 137
10.10 Companion notebook . . . 138
10.11 Case-study tie-in . . . 138
10.12 Summary, questions, and further reading . . . 139

Chapter 11 — Feature Engineering & Selection for Biosignals & Images 141
11.1 Learning objectives . . . 141
11.2 Intuition first: from a raw epoch to a feature vector . . . 142
11.3 Theory: the feature families . . . 142
11.4 Worked example: features that separate sleep stages (a number to reproduce) . . . 144

varkonyibarnabas@gmail.com

11.5 Image features — and where hand-crafting ends . . . 145
11.6 Feature scaling — and the leakage it can hide . . . 146
11.7 Feature selection and the curse of dimensionality . . . 146
11.8 The ML/AI bridge . . . 148
11.9 Companion notebook . . . 148
11.10 Case-study tie-in . . . 149
11.11 Summary, questions, and further reading . . . 149

Chapter 12 — The Machine-Learning Pipeline: Foundations, Validation & Evaluation 151
12.1 Learning objectives . . . 151
12.2 Intuition first: learning is fitting, and fitting can go wrong . . . 152
12.3 Theory: splitting, validation, and matching the split to the claim . . . 152
12.4 The failure-case clinic: how to fool yourself, and how to stop . . . 154
12.5 Class imbalance and honest metrics . . . 155
12.6 Domain metrics: from epochs to sleep architecture . . . 158
12.7 Uncertainty and calibration . . . 159
12.8 External validation . . . 160
12.9 Data curation and quality control . . . 161
12.10 Interpolation of gaps . . . 162
12.11 Missing data — is it missing at random? . . . 163
12.12 Data augmentation . . . 164
12.13 Domain shift and dataset shift . . . 164
12.14 The ML/AI bridge . . . 165
12.15 Companion notebook . . . 165
12.16 Case-study tie-in . . . 166
12.17 Summary, questions, and further reading . . . 166

Chapter 13 — Classical Learning Algorithms & Unsupervised Analysis 169
13.1 Learning objectives . . . 169
13.2 Intuition first: a boundary through a feature cloud . . . 169
13.3 Theory: four ways to draw a fence . . . 170
13.4 Worked example: three learners on the same folds . . . 173
13.5 Unsupervised learning: structure without labels . . . 174
13.6 The ML/AI bridge . . . 175
13.7 Signals-and-images box . . . 176
13.8 Companion notebook . . . 176
13.9 Case-study tie-in . . . 177
13.10 Summary, questions, and further reading . . . 177

Chapter 14 — Deep Learning & Time-Series Foundation Models for Biosignals 179
14.1 Learning objectives . . . 179
14.2 Intuition first: from designed features to learned ones . . . 179
14.3 Neural-network basics . . . 180
14.4 CNNs: the convolution you already know, learned . . . 182
14.5 Sequence models: context across the night . . . 183
14.6 Self-supervised learning and foundation models . . . 183
14.7 Practical reality: a fair fight with the forest . . . 185
14.8 Responsible AI in health . . . 186

varkonyibarnabas@gmail.com

14.9 Signals as images: encodings and the 2-D analogue . . . 188
14.10 Companion notebook . . . 190
14.11 Case-study tie-in . . . 190
14.12 Summary, questions, and further reading . . . 191

Chapter 15 — From Notebook to System: Deployment Thinking for Biomedical Signal Models 193
15.1 Learning objectives . . . 193
15.2 The deployment spectrum and the one pivotal choice . . . 193
15.3 Edge constraints, TinyML, and making the model fit . . . 195
15.4 Server, cloud, and MLOps . . . 196
15.5 Implications and responsibilities (awareness level) . . . 196
15.6 Companion notebook . . . 198
15.7 Case-study tie-in . . . 198
15.8 Summary, questions, and further reading . . . 199

Chapter 16 — Capstone: Building, Evaluating & Delivering an Automatic Sleep-Scoring System 201
16.1 Learning objectives . . . 201
16.2 The end-to-end pipeline as reusable modules . . . 201
16.3 An iterative build: baseline first, then earn the complexity . . . 203
16.4 Design decisions and their justification . . . 205
16.5 Evaluation in practice . . . 205
16.6 Reproducibility and engineering . . . 207
16.7 Teamwork . . . 207
16.8 Communicating results . . . 208
16.9 A worked walk-through: one full iteration . . . 208
16.10 Companion notebook . . . 209
16.11 Summary, questions, and further reading . . . 209

Appendix A — Getting Started with Python & MATLAB 211
A.1 An environment in five minutes . . . 211
A.2 How a biosignal gets into memory . . . 211
A.3 The shared notebook header . . . 212
A.4 A Python ↔ MATLAB mini cheat-sheet . . . 212

Appendix B — Complex Numbers & Euler's Formula 214
B.1 A number with a direction . . . 214
B.2 Euler's formula: the bridge to rotation . . . 214
B.3 Why the Fourier transform needs them . . . 215

Appendix C — Convolution, Step by Step 216
C.1 The flip-and-slide picture in one dimension . . . 216
C.2 The same picture in two dimensions . . . 216
C.3 The link to CNN kernels . . . 218

Appendix D — Correlation & Covariance 219
D.1 Covariance: do two variables move together? . . . 219
D.2 Cross-correlation: comparing signals across a lag . . . 219

varkonyibarnabas@gmail.com

D.3 Autocorrelation: a signal compared with itself . . . 220

Appendix E — Probability & Statistics Refresher 221
E.1 Random variables and distributions . . . 221
E.2 Expectation and variance . . . 221
E.3 The $\sqrt{N}$ idea . . . 222

Appendix F — Datasets & Reproducibility 223
F.1 Where to get open biosignal data . . . 223
F.2 How to cite a dataset . . . 223
F.3 Reproducibility practices . . . 223
F.4 Data governance and licensing . . . 224

Appendix G — A Task-to-Tool Atlas 226
G.1 Need → tool → cautions . . . 226
G.2 Output type → how to evaluate it . . . 227
G.3 How to use this atlas . . . 227

Appendix H — Python Implementation Map 228
H.1 The master map . . . 228
H.2 The tested helpers (`bsp`) . . . 231
H.3 The leakage-safe machine-learning pattern . . . 232
H.4 Reproducibility and verification checklist . . . 232

Appendix I — Abbreviations & Acronyms 234

Appendix J — Glossary 237

References & Further Reading 240
Accessible and biomedical foundations . . . 240
Classical signal processing (depth references) . . . 240
Machine learning and deep learning . . . 240
Evaluation and agreement statistics . . . 240
Clinical grounding and data . . . 241
Intuition-first and visual references . . . 241
Signal-processing education . . . 241

A practical, application-centred introduction in which the reader learns the full pipeline — from a physiological measurement to a validated machine-learning decision — using automatic sleep scoring as the integrating case study.

Preface

This book tells one continuous story: how a physiological event becomes a number, how a number becomes a spectrum or a feature, and how a feature becomes a decision made by a machine. Every chapter is a single link in that chain, and the chain is never broken. When you finish the last case-study chapter, you should be able to trace an unbroken path from a beating heart or a firing cortical neuron all the way to a classifier's output — and, just as

varkonyibarnabas@gmail.com

importantly, to say honestly how much you should trust that output.

The book is written for a mixed audience. It assumes you are curious and numerate, but it does *not* assume that you arrived with a signals-and-systems background, a fluent command of complex numbers, or years of programming behind you. Readers from medicine, physiology, psychology, and the social sciences are as welcome here as readers from engineering and computer science. To make that possible, the main chapters lead with intuition and worked examples, and introduce mathematics only as far as it earns its keep. The heavier prerequisites — complex numbers and Euler's formula, convolution, correlation, probability, and the mechanics of Python and MATLAB — are offloaded to short primer appendices you can read when you need them, and skip when you do not. The tone we aim for throughout is that of a friendly but rigorous introductory textbook: honest about the mathematics, but never using it to intimidate.

This is not a traditional digital signal-processing textbook with biomedical examples attached to it. It is a practical introduction to reasoning with measurements from living systems. Mathematics is introduced when it explains a method's behaviour, supports a design choice, or reveals a failure mode; fuller theoretical treatments are provided through optional primers and specialist references. The central goal is not to reproduce every derivation, but to understand what a method does, decide when it is appropriate, configure it responsibly, check whether it has worked, and develop the foundations needed to learn unfamiliar methods independently.

The order of the chapters is deliberate, and it is the spine of the book. We begin with physiology — where the wiggles in an ECG or an EEG actually come from, and what makes one recording easy and another a nightmare. We move to acquisition: how a continuous biological voltage is sampled, quantised, and turned into a stream of numbers without corrupting the very information we want. From there we build the language of transforms — Fourier analysis, windowing, wavelets, and their relatives — which let us move fluidly between the time, frequency, and time–frequency views of a signal. Because real biosignals are noisy and only partly predictable, we then treat them as stochastic processes and learn to estimate their spectra reliably even from short, imperfect records. Next comes the machinery of noise and its removal: identifying each artifact by its signature and choosing the right filter to attenuate it, quantifying the signal-to-noise ratio we gain. With a clean signal in hand, we learn to describe it numerically through feature extraction and selection, and then to learn from it with the full machine-learning pipeline — trained, and above all *honestly validated*. Two clearly-labelled extensions carry the story into deep learning and foundation models and into the realities of deployment. Finally, an integrated case study stitches every link back together into a working system.

That case study is automatic sleep staging — building a system that reads a night of EEG, EOG, and EMG and labels each thirty-second epoch with a sleep stage. We use it purely as a concrete, general biomedical application: it is rich enough to exercise every stage of the pipeline (physiology, acquisition, transforms, spectral estimation, denoising, features, classification, honest validation, and integration), and familiar enough that the intuition transfers to any other biosignal you might meet. Whenever a chapter introduces a method, we ask how it appears in the sleep-staging task — not as a homework assignment, but as a running thread that shows each idea doing a real job in a real, end-to-end system.

varkonyibarnabas@gmail.com

How this book differs from existing textbooks

It is only fair to be honest about what is, and what is not, new here. This is a comprehensive applied introduction rather than a narrow primer. It does not attempt to replace specialist texts on instrumentation, stochastic processes, medical imaging, or machine learning; instead it teaches the full biomedical signal-processing pipeline at the depth needed to build and evaluate a working system. Its contribution is curricular rather than methodological. We do not introduce a new transform, a new filter, or a new learning algorithm; the signal-processing and machine-learning tools we teach are the established ones, drawn from the shared body of knowledge the field already agrees on. What is distinctive is how those tools are *organised and taught*: as a single, continuous pipeline followed end to end on one reproducible case study.

There is real precedent for parts of this design, and we build on it gladly. Semmlow and Griffel's *Biosignal and Medical Image Processing*, for instance, deliberately treats one-dimensional signals and two-dimensional images side by side, and Rangayyan's *Biomedical Signal Analysis* is a thorough, physiology-grounded account of biosignal methods. Our debt to that tradition runs throughout the chapters. What we add is the single-pipeline narrative: rather than presenting acquisition, transforms, denoising, features, and classification as separable topics that a reader must later assemble, we link them into one unbroken chain — physiological origin → acquisition → representation → denoising → feature construction → validation → deployment — and walk that chain, in order, on a single worked problem. Automatic sleep staging is the thread that ties every link together, so that each method is met not in isolation but at the exact point where the running system needs it.

Two commitments give that narrative its character. The first is leakage-aware machine-learning discipline, taken seriously from the start rather than bolted on at the end: because a biosignal pipeline is where data leakage most easily creeps in — through improper cross-validation, subject overlap, or fitting preprocessing on test data — we treat honest, subject-independent validation as a first-class part of the pipeline, not an afterthought. The second is the companion notebooks as a figure factory: every figure in the book is generated by the notebooks, so the code and the pictures are kept in sync, and the notebooks carry additional practice depth — extra experiments, parameter sweeps, and exercises — beyond what the printed pages show. Taken together, the differentiator is not a new method but a *way of teaching the whole*: one reproducible case study, one continuous pipeline, and the discipline to know how far to trust the result.

> A note on the examples. Unless a worked example explicitly names a real, published dataset, its signals are simulated or synthetic — generated by the companion notebooks from seeded, reproducible models of the underlying physiology and noise. This has a deliberate consequence for how you should read the numbers in this book. Any κ, SNR gain, RMSE reduction, accuracy, or feature-separation figure we report is illustrative and reproducible from the companion notebooks — not an empirical benchmark drawn from clinical data. The point of a worked number here is that you can regenerate it yourself and watch it move when you change a parameter, not that it certifies how a method performs in the clinic. Where a chapter does use a named dataset, we say so explicitly.

varkonyibarnabas@gmail.com

A note on figures. The data-driven figures in this book are generated from simulated signals by the companion notebooks, so every plotted curve can be reproduced and perturbed by re-running a cell. The one photographic test image that recurs in the image-processing demonstrations is a public-domain U.S. Navy photograph of Rear Admiral Grace Hopper, bundled with the matplotlib library and used here purely as a natural scene with faces, smooth regions, fine texture, and sharp edges. We deliberately avoid the historic "Lena" image, which several venues now discourage for licensing and ethical reasons.

How to use this book

Every chapter is built on the same internal skeleton, so once you have read one you know how to read them all. Each opens with a motivating question and a short list of learning objectives. It then teaches intuition first, with a plain-language mental model and a figure, before turning to the theory — definitions, the key equations, and only the derivations that genuinely illuminate. Every method is then shown in a worked biosignal example on a named signal doing a real job, wherever possible with a *law and a number* you could reproduce yourself. A short ML/AI bridge explains how that chapter's output becomes a feature or a model input downstream. Where a method has a two-dimensional analogue, a Signals-and-images box teaches the image version alongside the one-dimensional version. Each chapter names a companion notebook, ties back to the sleep-staging case study, and closes with a summary of key formulas, conceptual questions and exploration tasks, and pointers for further reading.

The learning objectives and the assessment notes are tagged with a cognitive level, L1–L6, following the revised Bloom taxonomy — a compact way to say *what kind* of mastery an objective asks for, from recalling a fact to designing a system. The scale runs:

Level	Cognitive demand	In this book, typically
L1	*Remember* — recall facts, terms, definitions	name a signal's bandwidth; state the sampling theorem
L2	*Understand* — explain, interpret, summarise	explain why EEG is non-stationary; read a spectrogram
L3	*Apply* — use a method on a new problem	design and apply a band-pass + notch filter
L4	*Analyse* — compare, decompose, diagnose	choose and justify a feature set; diagnose a leakage failure
L5	*Evaluate* — judge, validate, defend choices	validate a classifier honestly and defend the design
L6	*Create* — integrate parts into a working whole	build and deliver the end-to-end system as a team

Higher levels build on the lower ones; the core chapters climb from L1–L3 toward L4, and the capstone reaches L5–L6.

The chapters come in two tiers. Core chapters form the assessed backbone — the full pipeline from physiology to a validated classifier, plus the integrative case study — and should be read in order. Extension chapters, covering deep learning and foundation models and the practicalities of deployment, are deliberately shorter and conceptual; they enrich the story but are optional for mastery of the core pipeline. If you are short of time, read the core end to end

varkonyibarnabas@gmail.com

and treat the extensions as a preview of where the field is going.

Readers arrive from very different backgrounds, and the book is written so each can take the shortest honest route through it. The table below suggests where to spend attention and which primers to lean on; it is a matter of emphasis, not four different books.

Reader background	Main emphasis	Optional support
Medicine, physiology, psychology	physiological meaning, task formulation, method choice, diagnostics, interpretation	Chapter 2 (sampling, in the main sequence) plus the complex-number, convolution, correlation, and probability primers (Appendices B–E)
Biomedical engineering	the full main narrative and every notebook	deeper equations and specialist references where needed
Electrical engineering / DSP	biomedical assumptions, acquisition realities, artifacts, validation, deployment	move quickly through the familiar mathematical foundations
Data science / computer science	the measurement chain, signal properties, preprocessing, leakage, clinical interpretation	the physiology and instrumentation sections

The optional routes are not separate tracks through different books. All readers share the same biomedical questions and worked examples; they differ only in how much supporting mathematics they need at each point.

Every chapter ships a companion notebook, and these notebooks are more than demonstrations: they are the book's figure factory. The figures you see in print are generated by the notebooks from a shared plotting style, so the code and the pictures are kept in sync, and any figure can be regenerated or corrected by re-running a cell. Read a chapter, then open its notebook and change a parameter — a sampling rate, a window length, a filter cutoff — and watch the figure move. That loop, between the page and the running code, is where the intuition actually forms.

The worked code is Python-primary and MATLAB-secondary. Python (with NumPy, SciPy, scikit-learn, and their scientific companions) carries the main examples; Appendix H maps every method in the book to its MATLAB equivalent in one table, and Appendix A gets you set up in both.

The twelve design principles

Twelve principles govern what is in this book and how it is taught, and it is worth stating them plainly at the outset so you can hold the authors to them.

One arc, from sampling to AI. The sequence of chapters is a single deliberate progression — physiology, acquisition, transforms, stochastic modelling and spectral estimation, noise and its removal, features, the machine-learning pipeline, deep learning, and an integrated system. Nothing appears before you need it, and everything you learn is reused downstream.

Signals *and* images, together. Where a method has a two-dimensional analogue — convolution, filtering, the Fourier transform, wavelets, learned feature maps — the image version is taught

varkonyibarnabas@gmail.com

alongside the one-dimensional version rather than quarantined in a separate chapter. Medical images enter in Chapter 1 and recur through filtering, features, and deep learning.

Teach each concept where it is clearest, then cross-map. Some ideas *see* better as pictures and some *hear* better as waveforms. The book deliberately teaches a concept in whichever domain makes it most intuitive and then transfers it. Salt-and-pepper noise on an image is the most visceral way to see why a median filter beats an averaging filter on impulsive noise — an insight then carried straight back to removing motion spikes and electrode pops from an ECG.

Classical methods and modern AI on the same page. Fourier analysis, autoregressive models, and hand-crafted features are not treated as obsolete relics but as the interpretable baseline against which deep and foundation models are measured. Each classical representation is explicitly connected to its learned counterpart — a spectrogram to a convolutional network's input, autoregressive coefficients to a learned embedding.

Python-primary, MATLAB-secondary. Worked code is Python, with every method's MATLAB equivalent collected in one table (Appendix H), so readers on either track are never stranded.

Every chapter ships a companion notebook. Each notebook reproduces the chapter's figures on biosignals, runs interactively, and doubles as the starting point for hands-on work. Because the figures are generated *from* the notebooks, the book and its code stay consistent by construction.

Format-adaptable by construction. Each chapter is written so its summary lifts cleanly into slides, its narrative into a script, and its big ideas into a dialogue. The shared per-chapter skeleton makes re-cutting the material mechanical rather than a rewrite.

No concept without a concrete example — and, where possible, a number. Every method is introduced on a named biosignal doing a real job, and its behaviour is quantified so you can predict and check it. Ensemble averaging is never left as "averaging reduces noise": it is shown pulling a clean cardiac template out of noisy beats, together with the governing law — signal-to-noise ratio improves as $\sqrt{N}$ — so you can compute how many repetitions a given target requires.

Visual-first, for understanding and retention. Figures, comparison tables, and concept diagrams are treated as primary teaching instruments, not decoration. Each chapter carries a planned set of figures generated reproducibly from its notebook, at least one side-by-side comparison table, and a summary or decision diagram, all built on a consistent visual grammar so that structure is remembered as well as content.

Question before algorithm. Every method begins with the biomedical question, the required output, and the information that must survive. You should be able to justify why a method fits the task before you run it — the task, not a favourite algorithm, is the starting point.

No result without a diagnostic. A plausible plot or a good-looking score is not evidence that a method worked. Every practical method is paired with a diagnostic, a sensitivity check, or a failure case, so that a conclusion is always accompanied by the evidence that it should be trusted.

No single best method — design for a defensible optimum. Real problems rarely have one correct technique; they admit several good designs, and the deliverable is an *optimum chosen*

varkonyibarnabas@gmail.com

against the spec — the trade-off named, the diagnostic that would settle it identified, and the runner-up acknowledged. The book trains the loop that goes with this: read the requirements, design to them, evaluate honestly, and *motivate* the choice while knowing which alternative would win under a different spec. The goal is not to memorise a single "right" filter, window, or model, but to defend a good one — and to say when you would have picked another. This is not relativism: "several good answers" never means "anything goes." For a given spec some choices are simply wrong — they discard the information the task needs, let the test set leak into training, or break a hard constraint — and no amount of tuning or validation rescues them. Designing well means choosing among the *defensible* options while recognising and ruling out the wrong ones.

What you will be able to do

The point of the book is not to memorise a catalogue of methods but to leave with a set of transferable capabilities. Stated as things you can *do*, the promise is this: given a new, unfamiliar biosignal, you will be able to —

- Characterise it — name its physiological origin, its bandwidth and dynamic range, and whether it is deterministic or stochastic, stationary or non-stationary.
- Acquire and digitise it — choose a sampling rate and anti-aliasing filter, reason about the sensor and electrode interface, and understand what quantisation costs you.
- Represent it — move fluently between the time, frequency, and time–frequency domains, and pick the transform that best exposes its structure.
- Estimate its spectrum reliably, even from short and noisy records, and model it stochastically when that is the right description.
- Clean it — identify each noise source and artifact by its signature, select the right removal method, and quantify the signal-to-noise ratio you gain.
- Describe it numerically — extract and select discriminative features across domains, for one-dimensional signals and for images alike.
- Learn from it honestly — build and *rigorously* validate a classifier, using subject-independent cross-validation, leakage-aware pipelines, and metrics that reflect what actually matters, whether the model is classical or deep.
- Ship it — reason about deploying a model under real edge and cloud constraints, with attention to privacy, regulation, and drift.
- Deliver it — integrate the pipeline end to end and communicate the result clearly to others.

These are general tools, taught on specific signals. Sleep staging is the classroom; the skills are meant for any biosignal you will ever meet.

Notation and conventions

Throughout the book we use a consistent notation. Continuous-time signals are written with round brackets, $x(t)$; discrete-time signals, obtained by sampling, use square brackets, $x[n]$, where n is an integer sample index. Frequency-domain quantities use capital letters — $X(f)$ for a spectrum — and the discrete-time system view uses the z-transform $H(z)$. Boldface

varkonyibarnabas@gmail.com

denotes vectors and matrices. The main symbols recur so often that it is worth fixing them here.

Symbol	Meaning	Typical units
f_s	Sampling frequency (samples per second)	Hz (samples/s)
$T_s = 1/f_s$	Sampling interval (time between samples)	s
N	Number of samples in a record or window	— (count)
$\Delta f = f_s/N$	DFT bin spacing of an N-point spectrum (not the practical resolution — see §3.7)	Hz
n	Discrete sample index	— (integer)
$x[n]$	Discrete-time signal (a sampled waveform)	signal-dependent (µV, mV, g, ...)
$X(f)$	Fourier spectrum of a signal	amplitude·s (e.g. V·s)
$S_X(f)$	Power spectral density of a signal (Chapters 6–7)	power/Hz (e.g. V^2/Hz)
$h[n]$	Impulse response of a filter or system	—
$H(z)$, $H(f)$	Transfer / frequency response of a system	— (gain), dimensionless
$H(e^{j\omega})$	Discrete-time frequency response (Chapter 9)	— (gain), dimensionless
$H(s)$	Continuous-time transfer function (Chapter 5)	— (gain), dimensionless
SNR	Signal-to-noise ratio	dB (or a ratio)
κ	Cohen's kappa — chance-corrected agreement	— (dimensionless, -1 to 1)

N is reused locally for a few other counts where the context makes the meaning unambiguous: the ADC bit depth (§2.9), a digital filter's feedback order (§9.3), the number of averaged trials or epochs (Chapter 10), and the number of frequency bins (§11.3) — the meaning is restated wherever it changes.

A few unit conventions are worth stating once. Electrical biosignals span a wide range of amplitudes: the EEG is measured in microvolts (µV), the ECG in millivolts (mV), and accelerometry in units of g (multiples of gravitational acceleration). Frequencies are in hertz (Hz); time in seconds (s) unless noted. Ratios such as signal-to-noise are usually reported in decibels (dB), where a factor of ten in power is 10 dB and a factor of ten in amplitude is 20 dB — a relationship worth committing to memory, because it recurs every time we quantify how much a filter or an averaging step has helped. Where a synthetic signal is drawn in arbitrary units, we say so.

varkonyibarnabas@gmail.com

Chapter 1 — Biosignals & Medical Images: Origin, Physiology & Characteristics

Where do the wiggles in an ECG or an EEG actually come from — and what is it about a biosignal that makes it "hard"? Before we can filter, transform, or classify anything, we have to know what we are looking at: what a biosignal is, what physiological machinery produced it, and what makes one recording easy and another a nightmare. This first chapter builds that vocabulary. It is the map the rest of the book fills in.

1.1 Learning objectives

By the end of this chapter you should be able to:

- Define a biosignal and describe the biomedical measurement chain that turns a living process into a stream of numbers (L1).
- Explain, at the level of resting and action potentials, where the *passive* (spontaneous) electrical biosignals come from — and distinguish them from *induced* signals such as bioimpedance, which exist only when we drive a small current through the tissue ourselves (L1, L2).
- Recognise the characteristic amplitude, bandwidth, and rhythm of the main biosignals — ECG, EEG, EOG, EMG, respiration, SpO_2/PPG, CTG, IMU/accelerometry — and of the electrodermal, gastric, heart-sound, and blood-pressure signals (EDA, EGG, PCG, ABP) (L1, L2).
- Distinguish the three levels — the raw biosignal, a derived feature or measure, and a clinical interpretation or label — and place quantities such as heart rate, HRV, SpO_2, and a diagnosis at the correct level (L1).
- Group biosignals by their transducer modality (bioelectric, bioimpedance, optical, acoustic, mechanical/inertial, pressure, gas/thermal) (L1).
- Explain what it means to treat a medical image as a two-dimensional signal, and name the major imaging modalities (L1).
- Classify a signal along the axes of the signal taxonomy: deterministic vs. stochastic, stationary vs. non-stationary, continuous vs. discrete, periodic vs. transient (L1).
- State the objectives of biosignal processing — measurement, monitoring, diagnosis, and control — and give an example of each (L1).
- Sketch the end-to-end biomedical signal-processing pipeline — acquisition, processing, analysis — as a loop from patient to decision, and map each stage to the chapter that develops it (L1, L2).

varkonyibarnabas@gmail.com

1.2 What is a biosignal?

A biosignal is any measurable quantity that carries information about a living system and varies in time (or in space, or both). The heart's contraction produces a tiny time-varying voltage at the skin; the brain's cortical activity produces another; the mechanical rise and fall of the chest is a biosignal too, as is the oxygen saturation of arterial blood and the three-axis acceleration of a wrist during a walk. What unites them is that each is a *window onto a physiological process we cannot observe directly*.

It helps from the outset to separate the signal from the system. The physiological source — the cardiac conduction pathway, a population of cortical neurons, a motor unit — is a *system*: it takes inputs (a pacemaker impulse, a synaptic drive, a motor command) and produces outputs. The signal is the observable output we manage to record. Signal processing is, in large part, the art of reasoning backwards from the signal to the system that made it.

Every real biosignal reaches us through a measurement chain, and every link in that chain leaves its fingerprint on the data:

$$\text{physiology} \;\rightarrow\; \text{transducer/electrode} \;\rightarrow\; \text{amplifier}$$
$$\rightarrow\; \text{anti-alias filter} \;\rightarrow\; \text{ADC} \;\rightarrow\; \text{digital signal } x[n].$$

A transducer or electrode converts the physiological quantity into a voltage; an amplifier makes that voltage large enough to digitise; a filter removes frequencies that would corrupt the sampling; and an analog-to-digital converter (ADC) turns the continuous voltage into the discrete numbers $x[n]$ we actually process. Chapter 2 dissects this chain link by link. For now the key idea is that the numbers on your screen are never the pure physiology — they are the physiology *seen through* an instrument, and part of a signal-processor's job is knowing which features are real and which are artifacts of the chain. (The words *sensor* and *transducer* are used interchangeably for that first stage; and in a wearable or remote monitor an optional transmitter adds a wireless telemetry link after digitisation — both are made precise in §2.2, and the "where does the processing happen" question returns in Chapter 15.)

1.3 Physiological origin: from ion to waveform

The electrical biosignals — ECG, EEG, EMG, EOG — all originate in the same microscopic machinery: the movement of ions across cell membranes. What differs, and it matters a great deal, is *which* ionic events dominate the macroscopic recording. It is tempting to say that every electrical biosignal traces back to the action potential, but that framing is only partly right: the ECG and EMG do reflect propagating depolarisation/repolarisation waves and motor-unit action potentials fairly directly, whereas the scalp EEG is not a direct observation of individual action potentials at all, as we will see below.

At rest, a nerve or muscle cell maintains a resting potential of roughly -70 mV inside relative to outside, held by ion pumps and selective channels that keep sodium out and potassium in. When the cell is sufficiently excited, voltage-gated channels open, sodium rushes in, and the membrane potential swings sharply positive and back — an action potential. Its duration depends strongly on cell type: a nerve or skeletal-muscle spike lasts about a millisecond, whereas a *cardiac* action potential is far longer — up to roughly 200–300 ms — with a sus-

varkonyibarnabas@gmail.com

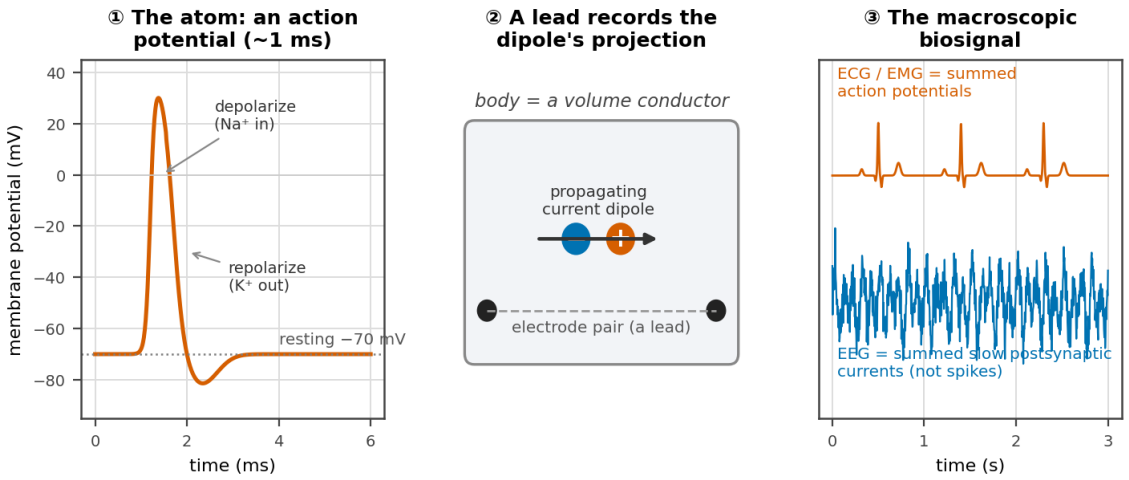
tained plateau that holds the cell depolarised while it contracts (which is why the ECG's QT interval is so long). This all-or-nothing spike is the atom of fast electrical signalling. Neurons communicate by firing action potentials that release neurotransmitters across synapses; each synapse in turn drives a slower, graded postsynaptic potential in the receiving cell, nudging it toward or away from its own firing threshold. Both kinds of event — the fast spike and the slower synaptic current — leave a mark on the fields we record, but not in equal measure across signals.

The macroscopic biosignals are the *spatial and temporal sum* of countless such events, and each one weights the two kinds of event differently. The ECG is the synchronised depolarisation and repolarisation of cardiac muscle, orchestrated by the heart's conduction system and projected onto the skin — its textbook P-QRS-T shape is the signature of atrial depolarisation (P), ventricular depolarisation (the large QRS), and ventricular repolarisation (T). The EMG arises from motor units — a motor neuron and the muscle fibres it commands — and the surface waveform is essentially the summed motor-unit action potentials of the fibres recruited; recruiting more units and firing them faster produces stronger contraction and a larger, broadband EMG. The EEG is different in kind. Scalp EEG is dominated not by action potentials but by the summed, slow postsynaptic currents of large populations of aligned cortical pyramidal neurons — cells whose parallel geometry lets their small synaptic currents add coherently into a field large enough to reach the scalp. Action potentials are too brief and too spatially scattered to sum this way, so the EEG reflects the *coordinated synaptic drive* of many cells rather than their individual spikes, which is why it is small (tens of microvolts) and rhythmic. Understanding these origins — and which event actually dominates each recording — is what lets us read a waveform diagnostically rather than merely mathematically.

One last step turns these microscopic events into something an electrode can read, and it is worth making explicit because the very idea of a lead depends on it. An action potential does not sit still: it propagates, sweeping along a muscle fibre or through the heart's conduction system as a travelling front of depolarisation trailed by repolarisation. At any instant that front behaves like a small current dipole — a separation of positive and negative charge — immersed in the salty, conductive body. Because the body is a volume conductor, that dipole sets up a potential field throughout the tissue, and a pair of surface electrodes (a lead) measures the *projection* of the dipole onto the line joining them. This is why the same heartbeat looks different in different ECG leads, and why moving an electrode changes the waveform without changing the physiology: each lead sees its own view of one underlying source. Predicting the surface signal from a known source is the forward problem; working backwards from the surface recordings to separate the mixed sources — as in EEG blind source separation (§10.7) or fetal-ECG extraction (§10.5) — is the much harder inverse problem, a distinction we return to whenever a method tries to undo the mixing that volume conduction imposes. Figure 1.1 traces the whole chain, from the single-cell spike, through summation and projection, to the macroscopic ECG, EMG, and EEG we actually record.

The ECG rewards a closer look at *how* that cardiac depolarisation is organised, because each landmark of its waveform is written by a specific leg of the heart's conduction system (Figure 1.2). The impulse begins at the sinoatrial (SA) node, the heart's natural pacemaker in the right atrium, whose spontaneous firing sets the rate. Depolarisation spreads across both atria — the P wave — and funnels into the atrioventricular (AV) node, the sole electrical bridge to the ventricles, which deliberately *delays* the signal (about 0.1 s) so the atria can finish emptying into the ventricles; that pause is the flat PR segment. From the AV node the impulse races down

varkonyibarnabas@gmail.com

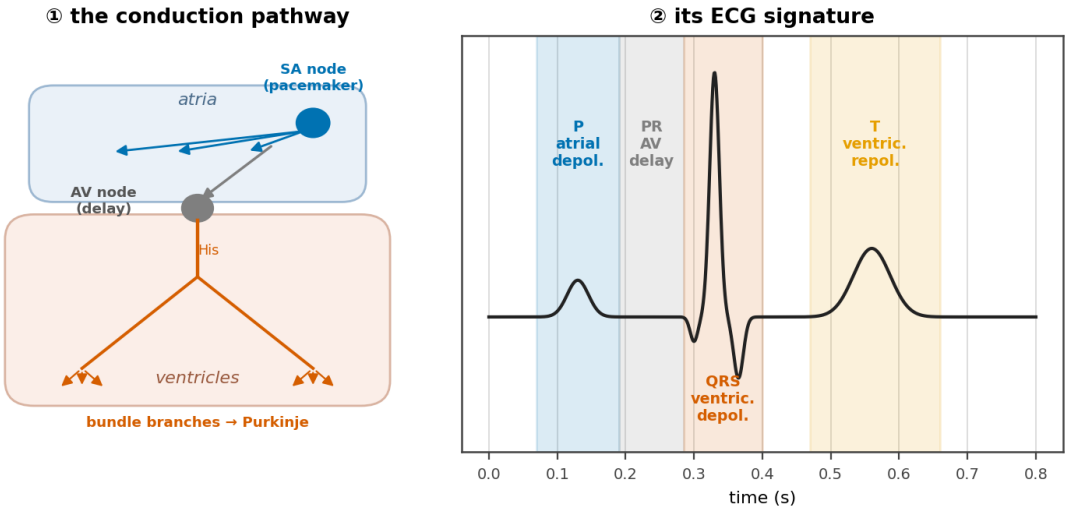


One machinery, three scales: ion currents make a millisecond spike; many cells sum and project through the body onto a lead; the recorded ECG, EMG, or EEG is that sum — spikes for ECG/EMG, synaptic currents for EEG.

Figure 1.1. From the ion to the recorded waveform. ① A schematic *neuronal or skeletal-muscle* action potential — a roughly 1 ms, all-or-nothing swing from the −70 mV resting potential, driven by Na^+ influx (depolarisation) and K^+ efflux (repolarisation), with a brief afterhyperpolarisation before it settles back to rest. (A *cardiac* action potential is much longer, ~200–300 ms with a plateau — see §1.3.) ② Many cells fire in synchrony; the propagating depolarisation front acts as a current dipole in the body, a volume conductor, and an electrode pair records the dipole's projection onto the lead. ③ The macroscopic biosignal is that sum — ECG and EMG are dominated by summed action potentials, whereas the EEG reflects the summed slow postsynaptic currents of aligned cortical pyramidal neurons, not individual spikes.

varkonyibarnabas@gmail.com

the bundle of His, splits into the left and right bundle branches, and fans out through the fast Purkinje fibres to depolarise both ventricles almost simultaneously — the tall, sharp QRS complex. Finally the ventricles repolarise, producing the T wave. So the textbook P-QRS-T is not an arbitrary squiggle but a *timed readout of a conduction pathway*: P for atrial activation, PR for the AV delay, QRS for ventricular activation, T for ventricular recovery. Reading those intervals is the basis of the fiducial ECG features of §11.3 and the QRS detector of §9.9.



The P-QRS-T is a timed readout of one conduction pathway: SA node fires (P), the AV node delays (PR), His–Purkinje drive the ventricles (QRS), which then repolarise (T).

Figure 1.2. The heart's conduction system and its ECG signature. Left: the impulse starts at the SA-node pacemaker, spreads across the atria (P), is held up at the AV node (the PR segment), then races down the bundle of His, the bundle branches, and the Purkinje fibres to depolarise the ventricles (QRS) before they repolarise (T). Right: the P-QRS-T beat, colour-coded to the conduction event that produces each deflection — the waveform is a timed readout of the pathway on the left.

All of the signals above are passive: the body is the generator, and the electrode is a passive witness that simply records the potential the tissue produces on its own. But the very same ionic machinery gives tissue a second, complementary electrical identity — its passive electrical properties, meaning how it *opposes an electrical current we apply from outside*. Reading this identity out is an induced (active) measurement: we inject a small, safe alternating current and measure the resulting voltage, and the ratio is the tissue's electrical bioimpedance. Nothing is recorded unless we drive the current ourselves, which is exactly what separates an induced signal from a spontaneous one.

The physical origin is that biological tissue is a dielectric electrolyte: the intracellular and extracellular fluids are conductive salt solutions (resistive, carrying ionic current), while the lipid cell membranes act as tiny capacitors that store charge. This gives tissue a frequency-dependent impedance. At low frequency the membrane capacitors block current, so it must detour through the extracellular fluid alone — a long path, hence *high* impedance; at high frequency the membranes conduct, current flows through both compartments, and the impedance *falls*. The smooth transition between the two regimes, governed by cell size and membrane capacitance, is the β-dispersion, and its arc is captured compactly by the empirical Cole model of tissue impedance. Because that impedance depends on tissue composition, structure, and water content, it becomes a *signal* in its own right — the

varkonyibarnabas@gmail.com

basis of the bioimpedance family (respiration, cardiac output, body composition, and tissue characterization). Its origin belongs here beside the action potential; its measurement chain and derived signals are developed in §2.14.

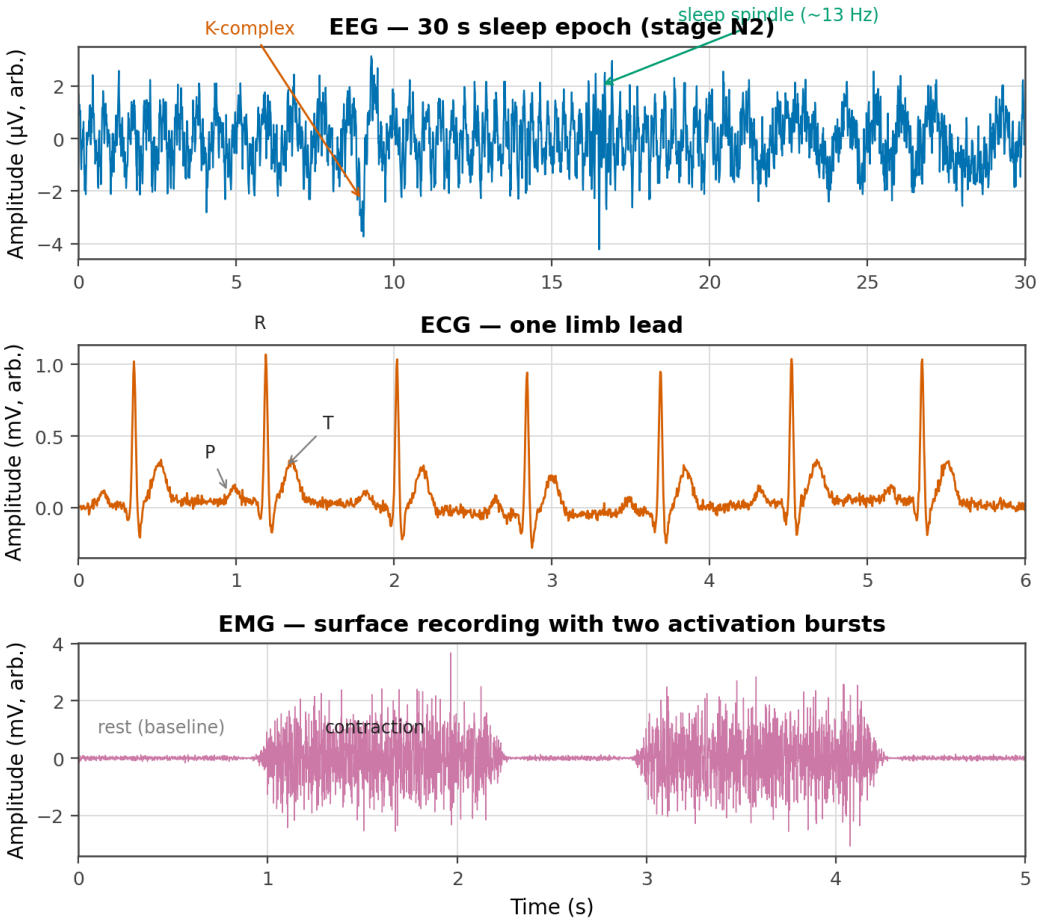
> Passive vs. induced (active) biosignals. It is worth carrying this distinction forward, drawn by a single test: *does the instrument add energy to the tissue?* Passive (spontaneous) biosignals — the bioelectric ECG, EEG, EMG, EOG, and EGG, the acoustic PCG, and the mechanical/pressure signals below — are generated by the body and merely recorded; the instrument adds no energy. Induced (active) biosignals require the instrument to *interrogate* the tissue and read its response, and they come in two energy forms. *Electrical* interrogation injects current or voltage: bioimpedance injects a current and measures a voltage; exosomatic electrodermal activity (EDA) applies a small voltage to read skin conductance; even an ECG monitor's lead-off detection drives a tiny current to sense whether an electrode is still attached (§2.14). *Optical* interrogation injects light: PPG, SpO_2, and fNIRS shine an LED into the tissue and read the absorption and scattering that come back — so, although they report an optical rather than electrical quantity, they are active measurements, not passive ones. The distinction matters practically: electrical interrogation adds a safety constraint (the applied current must stay within IEC 60601 limits) and a design knob (stimulation waveform and frequency), while optical interrogation brings its own — source wavelength, intensity, and ambient-light rejection.

1.4 The main biosignals and their signatures

Different physiological sources produce signals with very different amplitudes, bandwidths, and rhythms. Recognising these signatures on sight is a core skill; Figure 1.3 shows three of them side by side.

The ECG (electrocardiogram) is the friendliest biosignal: it is quasi-periodic, its landmark (the R peak) is usually tall and easy to spot, and its useful content spans roughly 0.05–150 Hz with amplitudes of about 0.5–4 mV at the skin (a *monitoring* bandwidth is often narrower, ~0.5–40 Hz, whereas the *diagnostic* 12-lead standard preserves up to about 150 Hz in adults and higher still in infants). This is why R-peak and heart-rate detection is a *mature* problem — reliable on clean ECG — and why the ECG is the workhorse of patient monitoring. It is not, however, fully *solved*: detection remains challenging under motion, ectopy, low SNR, and in neonatal and wearable conditions, where the tall R peak is no longer such a safe landmark. Practically, every ECG monitor also runs lead-off detection in the background — driving a tiny current through the electrodes and watching the return to sense when a lead falls off, an induced impedance measurement that shares its principle with the bioimpedance methods of §2.14. *Origin*: synchronised cardiac depolarisation/repolarisation (§1.3). *Modality*: Ag/AgCl electrodes, standardised leads/derivations (§2.6). *Artifacts*: baseline wander, powerline, motion, EMG cross-talk. *Preprocessing*: baseline-wander high-pass + 50/60 Hz notch (§9.6). *Features*: R-peaks and heart rate (§9.9), and HRV time/frequency indices (§7.11).

varkonyibarnabas@gmail.com



Each biosignal has a distinctive amplitude, rhythm and bandwidth — the fingerprint a processing pipeline exploits.

Figure 1.3. Three biosignals with distinct fingerprints: a 30-s N2 EEG epoch (small, rhythmic, carrying a transient K-complex and a sleep spindle), an ECG limb lead (a clean repeating P-QRS-T), and a surface EMG with two activation bursts rising out of a quiet baseline. Each biosignal's amplitude, rhythm, and bandwidth is the fingerprint a processing pipeline exploits.

varkonyibarnabas@gmail.com

Famous ECG abnormalities: rhythm and morphology. Normal sinus rhythm is a regular P-QRS-T at about 60–100 beats per minute (bpm). Clinically important departures each leave a recognisable fingerprint that an automatic detector learns to flag — some are abnormalities of *rate*, some of *rhythm*, some of *morphology*.

- Bradycardia — a slow rate (below ~60 bpm). The P-QRS-T keeps its normal shape but the RR intervals are long. It is often benign (in athletes, or during sleep), but when symptomatic it causes fatigue, light-headedness, or fainting (syncope) from insufficient cardiac output.
- Tachycardia — a fast rate (above ~100 bpm). *Supraventricular* tachycardias arise above the ventricles and keep a narrow QRS; ventricular tachycardia (VT) is a run of wide, abnormal QRS complexes and is a medical emergency. Symptoms range from palpitations and breathlessness to dizziness and, for VT, collapse.
- Atrial flutter — a fast, organised re-entrant circuit in the atria (~250–350 bpm) writes a regular sawtooth ("F-wave") baseline, with the ventricles usually responding at a fixed fraction (e.g. 2:1). Palpitations and breathlessness are typical.
- Atrial fibrillation (AFib) — chaotic, disorganised atrial activity replaces the SA-node rhythm. The discrete P wave disappears (a fine, wavering fibrillatory baseline takes its place) and the ventricular response is irregularly irregular — an RR-interval series with no periodicity at all. It causes palpitations and breathlessness and, importantly, raises stroke risk because blood pooling in the fibrillating atria can clot. Its irregular-RR signature makes it a natural target for the HRV and rhythm methods of Chapters 6–7.
- Long QT syndrome — a *morphology* abnormality rather than a rate or rhythm one. Ventricular repolarisation is prolonged, so the QT interval is abnormally long (measured as the rate-corrected QTc of §11.3). It may be congenital or drug-induced, is often silent, but predisposes the heart to a dangerous polymorphic VT (*torsades de pointes*) that causes syncope or sudden death — which is why QT prolongation is watched so closely, including as a side-effect of many drugs.

These patterns are the clinical stakes behind the R-peak detection of §9.9, the HRV analysis of §7.11, and the fiducial-interval features (PR, QRS, QT/QTc, ST) of §11.3.

The EEG (electroencephalogram) is far subtler: only 10–100 μV, buried under noise, and organised into classical rhythm bands — delta (0.5–4 Hz), theta (4–8 Hz), alpha (8–13 Hz), beta (13–30 Hz), and gamma (>30 Hz). Its content is *non-stationary* by nature: the mix of bands changes with cognitive and physiological state. In Figure 1.3 the EEG carries a K-complex (a large, isolated transient) and a sleep spindle (a brief ~11–16 Hz burst), two graphoelements that a sleep scorer looks for. *Origin:* summed postsynaptic currents of aligned cortical pyramidal neurons (§1.3). *Modality:* scalp Ag/AgCl electrodes in a standard montage referenced to a mastoid (§2.6). *Artifacts:* EOG (blinks/movements), EMG, powerline, electrode pop. *Preprocessing:* band-pass ~0.5–40 Hz + notch, with ICA/regression to remove ocular components (§10.7). *Features:* band powers (δ/θ/α/σ/β), spectral edge, and spindle/K-complex detection (Chapter 7, §11.3).

The exact band edges are not universal, and this is a frequent source of confusion when comparing chapters, papers, and toolboxes. The canonical partition below collects both the conventions used in this book in one place — the *classical* clinical/BCI ranges and the *sleep-analysis* ranges — so that every later chapter can be read against a single reference. This is the canonical EEG frequency-band table for the book.

varkonyibarnabas@gmail.com

Band	Classical range (Hz)	Sleep-analysis range (Hz)
Delta	0.5–4	0.5–4
Theta	4–8	4–8
Alpha	8–13	8–12
Sigma (spindle)	—	11–16
Beta	13–30	16–30 (with sigma carved out)
Gamma	>30	—

Reading the band table. Conventions vary; sleep analysis carves a sigma (spindle) band (11–16 Hz) out of low beta, so its alpha ends at 12 Hz and its beta starts at 16 Hz rather than 13 Hz. Whenever a later chapter quotes a band edge, read it against this table: the spindle/sigma band is 11–16 Hz throughout this book. Gamma (>30 Hz) is discussed in the prose but sits above the typical sleep-EEG acquisition band, so it is not always retained in a sleep pipeline. Note that the sleep-analysis alpha (8–12 Hz) and sigma (11–16 Hz) bands overlap at 11–12 Hz. That overlap is harmless for loose visual or AASM scoring, but a computational band-power feature pipeline must use non-overlapping bands, or it double-counts the energy in the 11–12 Hz bin — so when integrating band powers, define alpha strictly as 8–11 Hz (or start sigma at 12 Hz) so each frequency belongs to exactly one band. Chapter 11 (feature extraction) returns to this when it computes spectral band-power features.

The EMG (electromyogram) is broadband (roughly 10–500 Hz) and noise-like: during a contraction it looks like band-limited noise whose *envelope* — not whose fine detail — carries the information (how hard, how long). Between contractions it collapses to a quiet baseline. *Origin:* summed motor-unit action potentials of recruited fibres (§1.3). *Modality:* surface Ag/AgCl electrodes over the muscle belly (or, clinically, a needle electrode for single-unit work). *Artifacts:* ECG cross-talk, powerline, motion, and cable movement. *Preprocessing:* band-pass ~20–450 Hz + notch, then rectify and low-pass to recover the envelope (§5.5). *Features:* MAV, RMS, waveform length, zero-crossings, and mean/median frequency (§11.3).

The EOG (electrooculogram) exploits the eye's corneo-retinal standing potential: the eyeball is a fixed electrical dipole, with the cornea positive relative to the retina (a standing potential of roughly 0.4–1.0 mV maintained by metabolic ion pumping in the retinal pigment epithelium). As the eye rotates, this dipole swings toward or away from electrodes placed beside the eyes, so eye movements register as slow steps and blinks as spikes; the signal is low-frequency (roughly 0.1–10 Hz). *Origin:* the corneo-retinal dipole (§1.3, passive). *Modality:* small Ag/AgCl electrodes at the outer canthi (horizontal EOG = the difference E1–E2) and above/below one eye (vertical EOG). *Artifacts:* EEG cross-talk, electrode drift, and 50/60 Hz pickup. *Preprocessing:* band-pass ~0.1–10 Hz and baseline-drift removal. *Features:* movement amplitude and velocity, and the distinction between the rapid darting movements of REM and the slow rolling movements of drowsy onset.

Respiration is slower still (about 0.1–0.5 Hz, i.e. 6–30 breaths/min). *Origin:* the mechanical rise and fall of the chest and abdomen and the associated airflow. *Modality:* the choice of transducer matters — a piezoelectric or strain-gauge belt turns chest/abdomen circumference change into a voltage; respiratory inductance plethysmography (RIP) reads the inductance of coils sewn into chest and abdomen bands; a thermistor or thermocouple at the nostrils senses airflow through temperature; capnography measures exhaled CO_2; and impedance pneumography recovers breathing straight from the thoracic bioimpedance

varkonyibarnabas@gmail.com

between two ECG electrodes (§2.14). *Artifacts:* motion, belt slippage, and baseline drift. *Preprocessing:* low-pass filtering and detrending. *Features:* respiratory rate, effort amplitude, and inspiration/expiration timing.

PPG (photoplethysmography) and the derived SpO_2 are *optical* biosignals. *Origin:* each heartbeat drives a pulse of arterial blood whose light absorption varies. *Modality:* an LED-and-photodiode pair on a fingertip or wrist measures that pulsatile absorption. Oxygen saturation is then estimated not from a bare absorption ratio but from the ratio of the pulsatile-to-steady (AC/DC) absorption at two wavelengths — the red and infrared *ratio-of-ratios* — mapped to SpO_2 through an empirical calibration curve. That reliance on the small AC component is exactly why motion and poor perfusion degrade the reading. *Artifacts:* motion, ambient light, and poor perfusion. *Preprocessing:* band-pass around the cardiac band with motion suppression. *Features:* heart rate, pulse amplitude and morphology, and — against a second site or the ECG — pulse-transit time.

CTG (cardiotocography) pairs the *fetal* heart rate with uterine-contraction activity during labour, read together to judge fetal wellbeing. *Origin:* the fetal heartbeat and the mechanical tightening of the uterus. *Modality:* externally, fetal heart rate comes from Doppler ultrasound on the maternal abdomen while contractions are sensed by a TOCO (tocodynamometer) — a piezoelectric or strain-gauge pressure transducer strapped over the fundus that registers abdominal-wall tension; internally (once membranes rupture), a fetal scalp electrode and an intrauterine pressure catheter (IUPC) give direct signals. *Artifacts:* signal dropout and confusion of the maternal for the fetal heart rate. *Preprocessing:* gap interpolation and artifact rejection (§12.10). *Features:* FHR baseline, variability, and accelerations/decelerations read against the contraction trace.

Fetal ECG is not the same signal as the Doppler-derived FHR that CTG reports, and the distinction is worth making explicit. External CTG estimates heartbeat *timing* indirectly — an ultrasound autocorrelation algorithm infers successive beat intervals and smooths them — whereas a true fetal ECG captures the fetal heart's own depolarisation waveform, yielding a genuine beat-to-beat interval and, in principle, the waveform's morphology. Two very different routes reach it. *Non-invasively*, an array of electrodes on the maternal abdomen records a mixture: a large maternal ECG (millivolt-scale) together with a much smaller fetal ECG — typically only a few to a few tens of microvolts, one to two orders of magnitude weaker than the maternal signal riding alongside it. Pulling the fetal component out is a textbook source-separation problem: subtract an aligned maternal-ECG template (adaptive filtering, §10.5) or treat the multi-electrode recording as a mixture of statistically independent sources and unmix it with ICA (§10.7); more electrodes in the array supply more spatial diversity and a cleaner separation, at the cost of a harder acquisition. *Invasively*, once the membranes have ruptured and labour is established, a fetal scalp electrode (FSE) — a small spiral (corkscrew) stainless-steel wire twisted lightly into the presenting part, almost always the scalp — makes direct contact with the fetus and is referenced to a skin electrode on the maternal thigh. Because this route needs no maternal-signal cancellation, it delivers a clean, high-fidelity fetal ECG with genuine beat-to-beat precision, which is why it is the clinical reference standard for fetal heart-rate variability during labour, and is also the signal that intrapartum ST-analysis (STAN) — reading T-wave and ST-segment changes on the fetal ECG as a sign of fetal hypoxic stress, a layer of information heart rate alone does not carry — depends on.

IMU/accelerometry from wearables measures body motion in units of g. *Origin:* the linear ac-

varkonyibarnabas@gmail.com

celeration (and, with a gyroscope, rotation) of a body segment. *Modality:* a MEMS accelerometer — a micro-machined proof mass on springs — often paired with a gyroscope and magnetometer. *Artifacts:* the constant gravity offset, high-frequency motion transients, and slow drift. *Preprocessing:* low-pass filtering to separate the gravity component from body acceleration, then windowing. *Features:* signal-vector magnitude, dominant (cadence) frequency, and per-axis statistics — underpinning step counting, activity recognition, and sports analytics.

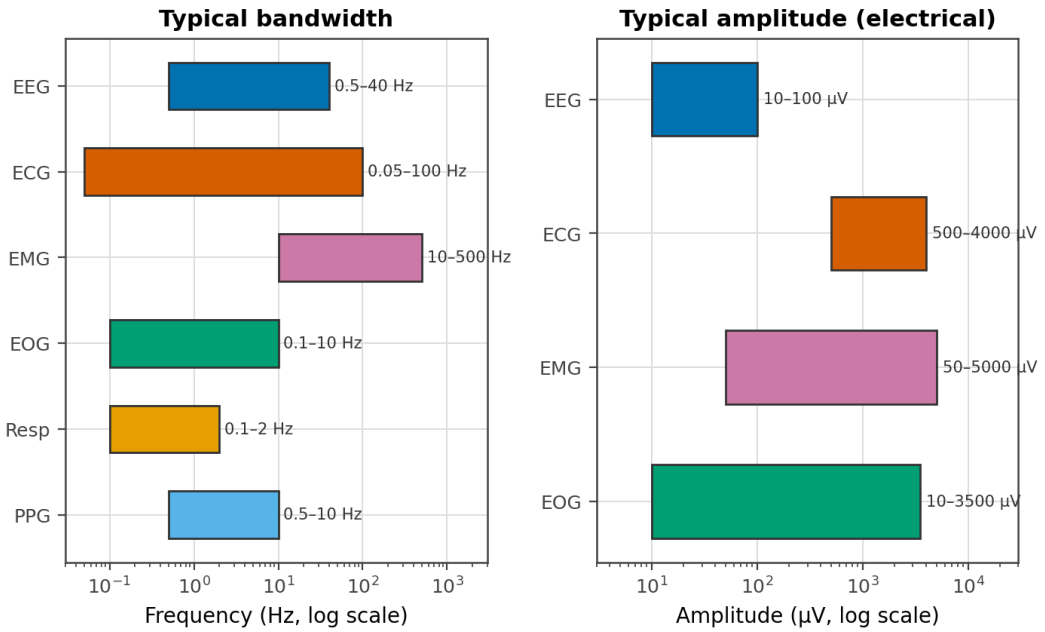
Four further biosignals round out the set a modern monitor or wearable may record. The EDA (electrodermal activity, or GSR) tracks skin conductance. *Origin:* sympathetic activation of sweat glands. *Modality:* most commonly an induced (exosomatic) measurement — a small constant voltage across two finger or palm electrodes reads the resulting current, i.e. skin conductance (§2.14); a less common endosomatic variant instead records the skin's own potential *passively*, with no applied current. Its very low bandwidth (below ~2 Hz) separates a slow tonic level (skin-conductance level) from faster phasic responses (skin-conductance responses), the classic markers of arousal and stress; motion and temperature are the main artifacts, handled by low-pass filtering and a tonic/phasic decomposition. The EGG (electrogastrogram) records the stomach's electrical slow wave. *Origin:* gastric smooth-muscle pacemaker activity at about 3 cycles per minute (~0.05 Hz). *Modality:* cutaneous electrodes over the abdomen. It is extremely low-frequency and easily swamped by respiration, motion, and ECG, so it is band-pass filtered to roughly 0.015–0.15 Hz before its dominant frequency and power are read. The PCG (phonocardiogram) is the *acoustic* biosignal of the heart. *Origin:* valve closures and blood turbulence produce heart sounds S1–S4 and murmurs. *Modality:* a chest microphone or electronic stethoscope (roughly 20–600 Hz). Ambient noise, friction, and lung sounds are the artifacts; band-pass filtering with envelope or wavelet analysis segments S1/S2 and characterises murmurs. Finally, ABP (arterial blood pressure) is a *pressure* waveform. *Origin:* the pressure pulse the heart ejects into the arteries. *Modality:* an invasive fluid-filled catheter and pressure transducer, or a non-invasive finger-cuff device. Catheter whip/damping and motion are the artifacts; after filtering and beat (foot) detection it yields systolic, diastolic, and mean pressures, pulse pressure, and dP/dt.

> The heart is two systems: an electrical trigger and a mechanical pump. It is worth separating what the ECG *does* and *does not* tell you. The ECG records only the heart's electrical command — the conduction sequence of §1.3 — not whether the muscle actually pumped blood. The mechanical consequence, the squeeze that moves blood, is watched by an entirely different family of biosignals: echocardiography (cardiac ultrasound, imaging wall motion and chamber volumes), impedance cardiography (ICG, whose dZ/dt tracks the ejection, §2.14), the arterial blood-pressure waveform, the photoplethysmogram and a finger on the pulse, and the phonocardiogram (the valve sounds S1–S2). From these come the pump's headline numbers: stroke volume (SV, the blood ejected per beat), heart rate (HR), cardiac output (CO $=$ SV $\times$ HR), and ejection fraction (EF $=$ SV/end-diastolic volume, the fraction of the filled ventricle ejected). Electrical activity always *leads* the mechanical by a short electromechanical delay — which is why the peripheral pulse arrives a fraction of a second after the ECG R-peak, and why pairing the two yields pulse-transit time and a cuffless blood-pressure estimate. The cautionary case is pulseless electrical activity: a near-normal ECG with no pump output at all — electrical *yes*, mechanical *no* — a reminder that one modality rarely tells the whole story, and part of the motivation for the multimodal feature fusion of §11.3.

varkonyibarnabas@gmail.com

> Why sleep staging needs three signals at once. No single channel tells you the sleep stage. The EEG shows the dominant rhythm and graphoelements (spindles and K-complexes flag stage N2; slow delta waves flag N3). The EOG distinguishes REM sleep by its rapid eye movements, and its slow rolling movements flag drowsy onset. The EMG (usually from the chin) shows the profound muscle atonia of REM. Only by reading EEG + EOG + EMG *together* can a scorer — human or machine — separate the stages reliably. This is the through-line case study we return to throughout the book.

Figure 1.4 collects the typical frequency and amplitude ranges into a single comparison, which is the quickest way to internalise why each signal demands a different acquisition setup.



Biosignals span three decades of frequency and amplitude — the acquisition chain must be matched to each one.

Figure 1.4. Typical bandwidth (left) and electrical amplitude (right) of the main biosignals, on log axes. Biosignals span roughly three decades of frequency and three of amplitude — so the acquisition chain (sample rate, gain, filter) must be matched to each one individually.

The table below distils the same information into the numbers you will reach for again and again when designing a pipeline.

Signal	Typical amplitude	Bandwidth (Hz)	Typical sampling rate	Main artifacts	Primary use
ECG	0.5–4 mV	0.05–150	250–500 Hz	Baseline wander, powerline, motion	Cardiac monitoring, arrhythmia
EEG	10–100 µV	0.5–40	100–256 Hz	EOG (blinks), EMG, powerline	Sleep, epilepsy, BCI
EMG	50 µV–5 mV	10–500	1–2 kHz	ECG cross-talk, powerline, motion	Muscle activity, prosthetics

varkonyibarnabas@gmail.com

Signal	Typical amplitude	Bandwidth (Hz)	Typical sampling rate	Main artifacts	Primary use
EOG	10 µV–3.5 mV	0.1–10	~100 Hz	Blinks, EEG cross-talk, drift	Eye movement, REM sleep
Respiration	(mechanical)	0.1–0.5	10–25 Hz	Motion, belt slippage, drift	Apnoea, effort
PPG / SpO_2	(optical)	0.5–10	25–125 Hz	Motion, ambient light, poor perfusion	Heart rate, oxygenation
CTG (FHR)	110–160 bpm	very low	~4 Hz	Signal dropout, maternal-heart confusion	Fetal monitoring
IMU (accel)	0–several *g*	0–20	50–100 Hz	Gravity offset, motion transients, drift	Activity, gait, sports
EDA / GSR	1–20 µS (conductance)	0–2	4–32 Hz	Motion, temperature	Arousal, stress
EGG	50–500 µV	0.015–0.15	1–4 Hz	Respiration, motion, ECG	Gastric motility
PCG (heart sound)	(acoustic)	20–600	2–8 kHz	Ambient noise, friction, lung sounds	Murmurs, valve timing
ABP	0–200 mmHg	0–20	100–250 Hz	Catheter whip, damping, motion	Haemodynamics
Bioimpedance (EBI)	mΩ–Ω on a carrier	derived 0.1–10	kHz-range carrier	Motion, electrode drift	Respiration, cardiac output, body composition (§2.14)

The EEG "0.5–40 Hz" entry is the typical sleep acquisition band, not the full physiological limit of the signal: gamma activity above 30 Hz is real and is discussed in the prose, but a sleep-EEG chain usually low-passes near 40 Hz, so it is not retained here.

Three levels: signal, feature, interpretation. It is worth fixing a hierarchy the rest of the book depends on, because conflating its levels is a common and costly mistake. A raw biosignal (level 1) is the measured waveform itself — the ECG trace, the PPG pulse, an EEG epoch, an ABP or capnogram waveform. A derived feature or measure (level 2) is a number computed *from* that waveform — heart rate, HRV, SpO_2, respiratory rate, end-tidal CO_2, RMS-EMG. A clinical interpretation or label (level 3) is a category assigned from those features — atrial fibrillation, apnoea, hypoxia, a sleep stage, fetal distress. Keeping the levels apart dissolves familiar confusions: *heart rate is not a biosignal*, it is a feature of the ECG or PPG; *SpO_2 is not a biosignal*, it is a ratio derived from two PPG wavelengths; and a *sleep stage* is a label, not a signal (the hypnogram that plots it is a time series of level-3 labels). It also marks what is not a *waveform* biosignal in the sense used here — cortisol, lactate, glucose and the like are biochemical analytes measured from a sample, not waveforms measured in time (some literature uses "biosignal" more broadly to include such biochemical measurements). This book is organised along exactly these levels: Chapters 1–2 on the signal, Chapters 3–11 on turning it into features, and Chapters 12–16 on the interpretation. The last three columns of the tables in this section — artifacts, features, primary use — already trace the same ladder from level 1 to level 3.

varkonyibarnabas@gmail.com

The signals tabulated above are the most common members of a much larger family. Grouping biosignals by their transducer modality — the physical principle that turns physiology into a number — organises the whole landscape and makes the book's central claim concrete: each is the *same* acquisition → feature → interpretation pipeline hung on a different front end. The ranges below are typical and device-dependent, not specifications.

Modality family	Transducer / principle	Example biosignals	Typical bandwidth	Unit
Bioelectric (passive)	Ag/AgCl electrodes → tissue potential	ECG, EEG, EMG, EOG, EGG, ECoG, fetal ECG	~0.01–500 Hz	µV–mV
Bioimpedance / electrodermal (induced)	electrodes + injected AC current	impedance pneumography, ICG, BIA, EIT, EDA	0–~10 Hz on a kHz carrier	Ω, µS
Optical (induced / light interrogation)	LED + photodiode (absorption/scattering)	PPG, SpO_2 (derived), fNIRS	~0.003–10 Hz	rel. intensity
Acoustic	microphone / stethoscope	PCG, lung sounds, voice, cough	~20 Hz–8 kHz	sound pressure
Mechanical / inertial	accelerometer, gyroscope, force / piezo	accelerometry, gyroscope, SCG, BCG, MMG, plantar force	~0–40 Hz	g, °/s, N
Pressure / flow	catheter + transducer, pneumotach, belt	ABP, CVP/PAP, ICP, IOP, airflow, RIP, tocogram	~0–20 Hz	mmHg, L/s
Gas & thermal	IR gas analyser, thermistor / IR camera	capnogram (CO_2), skin/body temperature	< ~10 Hz	mmHg, °C

The induced (active) measurements of §1.3 send energy into the tissue and read the response — bioimpedance and exosomatic electrodermal activity inject a *current*, and the optical PPG/SpO_2/fNIRS inject *light* — whereas the bioelectric, acoustic, mechanical/inertial, and pressure signals are recorded passively. This catalogue is a map, not a syllabus — the book develops the methods on the core signals above, but every method transfers, because the mathematics does not care which transducer produced the samples.

1.5 Medical images as signals

A signal carrying physiological information need not be one-dimensional, nor need it vary in time. It is useful here to reach for a slightly broader umbrella term: a biomedical signal is any measured quantity that carries information about a living system, whether it varies in time, in space, or in both. Under that umbrella, a grayscale medical image is a function $I(x, y)$ that assigns an intensity to every spatial location — literally a *two-dimensional signal* — and a volume or a movie adds further dimensions, $I(x, y, z)$ or $I(x, y, t)$. In this sense a medical image is a spatial biomedical signal: it is to space what a waveform is to time. Whether to *call* an image a "biosignal" is partly a matter of convention — not all authors use the word that broadly, and many reserve "biosignal" for time-varying recordings — but the mathematics does not care about the label. Almost everything we develop for 1-D signals has a 2-D analogue: sampling

varkonyibarnabas@gmail.com

becomes pixel spacing, frequency becomes *spatial* frequency, convolution and filtering carry over directly, and the Fourier transform becomes the 2-D FFT. This book teaches the two together on purpose, because seeing the same idea in both costumes cements it.

The major modalities differ in what physical quantity they map. X-ray and CT measure tissue attenuation of X-rays; CT reconstructs a cross-section from many angular projections. MRI maps the response of hydrogen nuclei in a magnetic field, acquiring data in the spatial-frequency domain (k-space). Ultrasound images the echoes of high-frequency sound. PET maps the concentration of an injected radiotracer, imaging function rather than anatomy. Microscopy images tissue at the cellular scale. Each is, at bottom, a spatial signal to be sampled, denoised, transformed, and analysed — exactly the operations we apply to waveforms.

1.6 The signal taxonomy

To reason about *methods*, we first classify *signals*. Four axes matter most, summarised in the concept map of Figure 1.5.

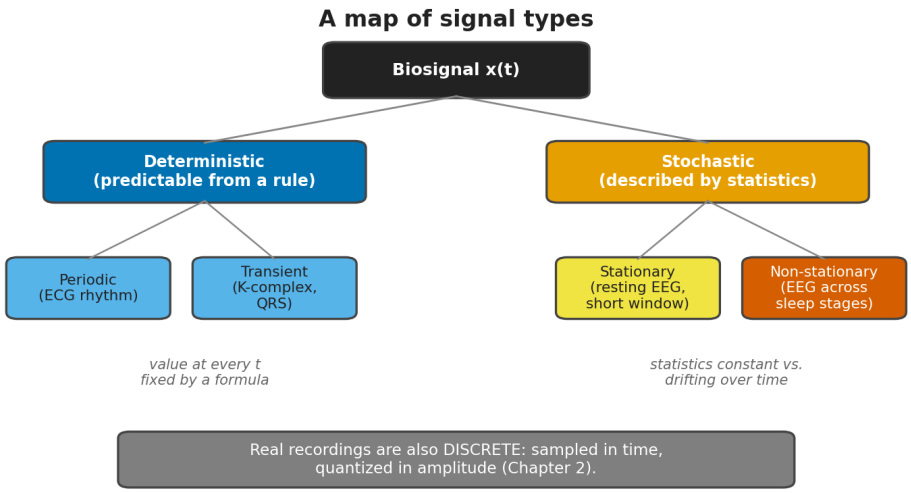


Figure 1.5. A map of signal types. A signal is deterministic (its value at every instant is fixed by a rule) or stochastic (described only by statistics); a stochastic signal is stationary if its statistics hold constant or non-stationary if they drift; and every real recording is additionally discrete in time and amplitude. Most biosignals are stochastic and non-stationary — which is exactly why they are hard.

A deterministic signal is one whose value at any time is fixed by a rule — a pure sinusoid $x(t) = A\sin(2\pi ft + \phi)$ is the archetype. A stochastic (random) signal cannot be written as a formula; it is characterised only by *statistics* — its mean, variance, and how values at different times correlate. EEG background activity is essentially stochastic. Most biosignals are best modelled as *deterministic structure plus a stochastic component*: an ECG has a deterministic beat shape riding on stochastic noise.

A stochastic signal is stationary if its statistical properties do not change over time, and non-

varkonyibarnabas@gmail.com

stationary if they do — but "stationary" is worth pinning down more carefully, because the term covers three distinct notions. Strict-sense stationarity is the strongest: the *entire* joint probability distribution of the signal is invariant to a time shift, so every statistic of every order is constant. Wide-sense (WSS) stationarity is a weaker, more practical condition that asks only that the mean be constant and the autocorrelation depend on the time lag rather than absolute time; it is the assumption most of our spectral machinery quietly relies on. Local or quasi-stationarity is weaker still, and is the one that actually applies to biosignals: the signal is not stationary overall, but is close enough to stationary *within a short window* that we may treat it as WSS over that window. Over a few seconds resting EEG may be treated as roughly stationary; but across a night of sleep the EEG spectrum shifts dramatically from stage to stage — it is profoundly non-stationary. In practice we almost always assume stationarity only within a window, and this is exactly what motivates cutting a record into short segments before analysis: epoching the EEG into 30-second windows, computing a Welch spectral estimate on short overlapping segments, or splitting a tachogram into short HRV segments. Local stationarity is *why a single global spectrum is not enough* and why we will need time-frequency methods (Chapters 3–5): you cannot summarise a signal whose content changes with one fixed set of numbers.

Signals are also continuous or discrete. The physiological process is continuous in time and amplitude; the moment we record it, sampling makes it discrete in time and quantisation makes it discrete in amplitude (Chapter 2). Finally, signals may be periodic (repeating, like the ECG rhythm) or transient (isolated events, like a K-complex or a single QRS complex). Transients are precisely the events that averaging over time destroys and that time-localised methods preserve.

> Signals and images. Every axis of the taxonomy applies to images too. A test pattern is deterministic; sensor noise is stochastic. An image of uniform texture is spatially stationary; a scene with an organ boundary is non-stationary in space. And a digital image is discrete in both position (pixels) and intensity (gray levels) — the 2-D echo of sampling and quantisation.

1.7 The objectives of biosignal processing

Why do we process biosignals at all? Four goals recur, in roughly increasing ambition. Measurement extracts a number: a heart rate, a breathing rate, a muscle-activation level. Monitoring tracks such measurements over time and raises an alarm when they leave a safe range — the core of any bedside monitor. Diagnosis classifies a recording into clinically meaningful categories: arrhythmia vs. normal rhythm, one sleep stage vs. another, seizure vs. background. Control closes the loop, using the biosignal to drive a device — a myoelectric prosthesis driven by EMG, or a brain-computer interface (BCI) driven by EEG.

BCIs deserve special mention because they reappear as a worked example later in the book. A common BCI paradigm uses evoked potentials — the brain's stereotyped response to a repeated stimulus. In SSVEP (steady-state visually evoked potential), a flickering light at, say, 12 Hz drives a small 12 Hz component in the EEG that is buried far below the ongoing background. SSVEP is a *steady-state* response to a continuous flicker and is usually analysed in the frequency domain (the FFT amplitude at the flicker frequency, or canonical correlation analysis); the signature use of *ensemble averaging*, by contrast, is recovering transient evoked

varkonyibarnabas@gmail.com

potentials — the P300, the auditory brainstem response (ABR) — where a discrete stimulus triggers a time-locked transient. Both improve with repetition: the trick that lifts a stimulus-locked response out of the noise — *ensemble averaging* over many repetitions — is a beautiful "law plus a number" story that Chapter 10 tells in full.

> Worked example — the $\sqrt{N}$ law, previewed. Averaging N repetitions of a stimulus-locked response, while the background noise is uncorrelated from trial to trial, improves the signal-to-noise ratio by a factor of $\sqrt{N}$ in amplitude. This is a mathematical consequence of the assumptions, not an empirical measurement: averaging $N = 100$ synthetic SSVEP epochs therefore buys, illustratively, a $\sqrt{100} = 10\times$ amplitude improvement — about 20 dB — enough to lift an otherwise invisible evoked response clear of the noise in the demo. The full derivation and a reproducible demonstration on simulated signals wait in Chapter 10; we flag it here so you know where the arc is heading.

1.8 Before choosing a method: what kind of task is this?

The objectives above — measurement, monitoring, diagnosis, control — describe *why* we process a signal. A second, complementary question decides *how*: what is the algorithm actually being asked to produce? The same waveform can support very different tasks, and each demands different preprocessing, tolerates different distortion, produces a different kind of output, and is judged by a different metric. It pays to name the task before reaching for a method.

Task	Central question	Typical output	Biomedical example
Representation	What structure is present?	a spectrum, time–frequency map, or embedding	reveal the rhythms in an EEG
Denoising / enhancement	What unwanted component can be attenuated without losing the target?	a cleaner waveform or image	reduce ECG baseline wander
Detection	Did an event occur, and when?	an event time or a binary decision	locate QRS complexes or spindles
Estimation	What value best describes the signal or event?	a scalar or continuous parameter	heart rate, band power, SpO_2
Segmentation	Where do states or structures begin and end?	labelled intervals or regions	a sleep-stage sequence; an organ boundary
Classification	Which predefined category fits?	a discrete label	arrhythmia type; sleep stage
Prediction	What is likely to occur later?	a risk or a future value	deterioration; a future physiological state
Control	What action should the system take?	a command or intervention	a prosthesis, BCI, or closed-loop stimulator

The same ECG may be used to *detect* QRS complexes, *estimate* heart rate, *classify* rhythm, *predict* deterioration, or *control* a device. The input is identical, but the suitable preprocessing, the acceptable distortion, the output, and the evaluation metric all change with the question. Biomedical signal processing therefore begins by defining the task, not by choosing a favourite

varkonyibarnabas@gmail.com

algorithm. We use this vocabulary consistently hereafter — QRS *detection* is not *classification*, and interval *segmentation* is not epoch *classification* — because conflating the task types is a common route to the wrong method and the wrong metric.

The pipeline in the next section shows *where* operations happen; this task taxonomy shows *why* they happen. A filter, transform, feature, or model has no meaning apart from the output the application requires.

> Design, don't pick. Naming the task is the first half of the job; the second half is accepting that it usually has *more than one good answer*. There is rarely a single best filter, window, feature set, or model — each candidate trades something away (resolution for variance, latency for accuracy, interpretability for capacity), so the real deliverable is a defensible design: the option whose trade-offs best fit *this* specification, chosen together with the diagnostic that would settle the question and the runner-up you would switch to under a different spec. This is why the recurring Design brief exercises ask you to *read the spec*, *design to it*, *evaluate it*, *motivate the choice*, *and name the alternative that would win under different requirements*. And "several good answers" is never "anything goes": a choice that blurs the very distinction the spec needs, that lets the test set leak into training, or that breaks a hard constraint is simply *wrong* for the task — spotting those is as much a part of design as weighing the good options. Whenever you finish a method and think "that is the answer," ask instead: for what specification is this the answer, what would beat it under another — and which choices would be outright wrong here, and why?

1.9 A typical pipeline: from patient to decision

The chapters that follow are not a loose collection of tricks; they are the ordered stages of one pipeline that carries a physiological event all the way to a clinical decision — and back to the patient. It helps to see the whole map before we start filling it in. Figure 1.6 lays out the canonical biomedical signal-processing application as a loop that begins and ends at the patient.

The loop has three natural groupings. Signal (data) acquisition turns physiology into numbers — the transducer, the amplifier and analog anti-alias filter, and the analog-to-digital converter (all of Chapter 2). Signal processing cleans and condenses those numbers — digital filtering and artifact removal (Chapters 8–10) and the detection of events and components such as beats, spindles, or bursts (Chapter 9). Signal analysis extracts meaning — feature extraction (Chapters 5, 7, and 11) and finally classification or diagnosis (Chapters 12–14), whose output returns to a clinician who acts on it, closing the loop back to the patient. The transforms of Chapters 3–7 are the shared language that both processing and analysis speak. Every later chapter is one block in this diagram; the table below is the reader's map.

Pipeline stage	What it does	Chapter(s)
Transducer / sensor	physiology → voltage	2
Amplifier + anti-alias filter + ADC	condition and digitise the signal	2
Transforms and representation	move between time, frequency, and time–frequency views	3–7
Digital filtering / artifact removal	attenuate noise and artifacts	8–10
Event and component detection	locate beats, spindles, bursts, waves	9

varkonyibarnabas@gmail.com

Pipeline stage	What it does	Chapter(s)
Feature extraction and selection	waveform → feature vector	5, 7, 11
Classification and diagnosis	features → decision, honestly validated	12–16

Two caveats keep this diagram honest. First, the arrows are not strictly one-way: real systems iterate, and a disappointing classification result usually sends you back to re-examine the filtering or the features rather than forward to a fancier classifier. Second, the boundary between "processing" and "analysis" is a matter of convention, not physics — and, as Chapters 13 and 14 will show, modern machine learning increasingly *absorbs* the feature-extraction block into the learning algorithm itself, a shift we take up next.

1.10 The ML/AI bridge

Everything in this chapter feeds the machine-learning pipeline that closes the book. A *classical* classifier usually consumes fixed-length feature vectors rather than raw waveforms. Modern deep models may consume raw or minimally processed waveforms, spectrograms, or learned embeddings. In every case the model consumes a *representation* of the signal — one that we engineer by hand or the model learns for itself. This book leans on the classical, hand-engineered route, and the features that work there are exactly the ones physiology tells us to compute. Because sleep spindles live at 11–16 Hz, spectral band-power in that range is a feature. Because REM is marked by chin atonia, EMG amplitude is a feature. Because the ECG is quasi-periodic, inter-beat-interval statistics are features. The taxonomy matters here too: a non-stationary signal is typically cut into short, locally quasi-stationary windows (the 30-s epoch is the conventional sleep-scoring choice) before features are computed, because a single global summary would blur the very state changes we want to detect. Knowing where a signal comes from and how it behaves is what lets us engineer — or let a network learn — features that actually separate the classes.

A deeper trade-off is lurking here, and it quietly organises the whole back half of the book. The more physiological knowledge we build into the features by hand, the *less* data a model needs in order to learn; the less we assume, the *more* data — and model capacity — it takes to learn a good representation from scratch. Classical feature engineering sits at the knowledge-rich, data-thrifty end of that spectrum; end-to-end deep learning at the data-hungry end. We lay this continuum out in full, as a ladder, in Chapter 14.

1.11 Companion notebook

The companion notebook for this chapter is a *biosignal zoo*: it loads or synthesises each signal type in turn — ECG, EEG, EMG, EOG, respiration, PPG, CTG, IMU — and plots each with annotations calling out its landmark features and its typical amplitude and bandwidth. It regenerates Figures 1.3 and 1.4 exactly, so you can change a heart rate, an EEG sleep stage, or an EMG burst timing and watch the fingerprint change. It is the hands-on counterpart to §1.4 and the natural starting point for exploring the acquisition ideas of Chapter 2.

varkonyibarnabas@gmail.com

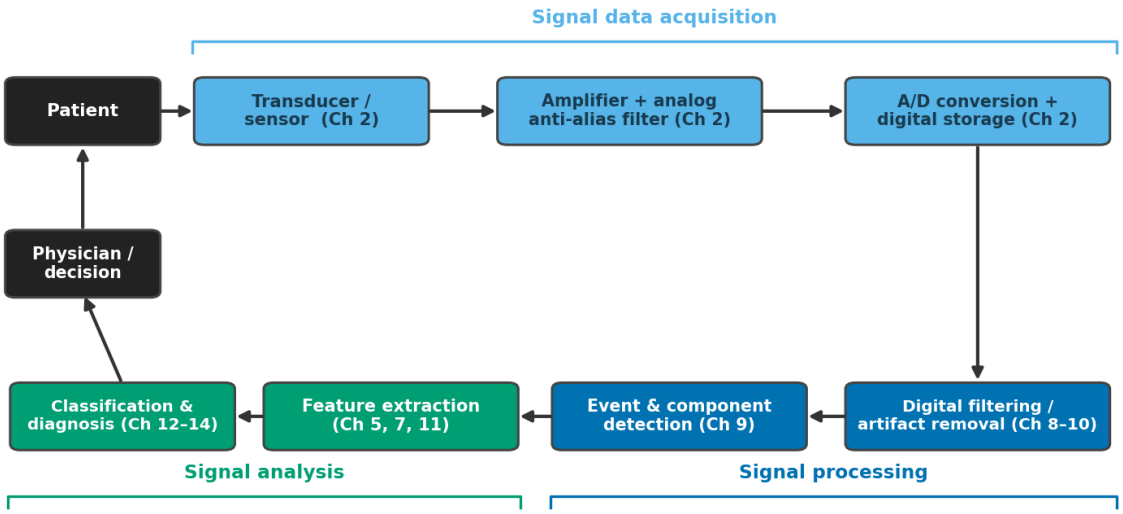


Figure 1.6. The biomedical signal-processing pipeline, and the chapter that teaches each stage. Reading clockwise from the patient: signal acquisition turns physiology into numbers (transducer → amplifier and analog anti-alias filter → A/D conversion and storage); signal processing cleans and condenses them (digital filtering and artifact removal, then event and component detection); and signal analysis extracts meaning (feature extraction, then classification and diagnosis), whose output returns to a clinician who acts on it — closing the loop back to the patient.

varkonyibarnabas@gmail.com

1.12 Case-study tie-in

The integrative case study threaded through this book is automatic sleep staging. This chapter introduces its raw material. A sleep recording provides, per 30-second epoch, several channels: EEG (the dominant rhythm and its graphoelements), EOG (rapid eye movements that flag REM), and chin EMG (the atonia that also flags REM). Chapter 1's job is to make you fluent in *why each channel is there* — EOG for REM, EMG for atonia, EEG for spindles, K-complexes, and slow waves — so that when we later extract features and train a classifier, every feature has a physiological reason for existing rather than being a number pulled from thin air.

1.13 Summary & key formulas

- A biosignal is a time- (or space-) varying measurement of a living process; we always observe it *through* a measurement chain that leaves its own fingerprint.
- The electrical biosignals originate in ion currents across membranes: resting potential ≈ -70 mV, action potentials ≈ 1 ms. But the events that dominate differ by signal: ECG and EMG reflect depolarisation/repolarisation waves and motor-unit action potentials, whereas scalp EEG is dominated by summed postsynaptic currents in aligned cortical pyramidal neurons, not individual action potentials.
- Each biosignal has a characteristic amplitude, bandwidth, and rhythm (Table in §1.4); matching the acquisition chain to these is the designer's first task.
- Biosignals sort by transducer modality (bioelectric, bioimpedance, optical, acoustic, mechanical/inertial, pressure, gas/thermal), and every quantity sits at one of three levels — raw signal, derived feature (heart rate, HRV, SpO_2), or interpretation (a diagnosis). Analytes such as cortisol or lactate are *not* waveform biosignals in the sense used here.
- A medical image is a 2-D signal $I(x, y)$; nearly every 1-D method has a 2-D analogue, taught together in this book.
- The taxonomy: deterministic vs. stochastic, stationary vs. non-stationary, continuous vs. discrete, periodic vs. transient. Stationarity comes in degrees — strict-sense, wide-sense (WSS), and local/quasi-stationary — and biosignals are usually treated as stationary only *within a window*, which motivates epoching, Welch, and HRV segments. Most biosignals are stochastic and non-stationary — hence hard.
- Processing serves measurement, monitoring, diagnosis, and control (including BCI). The ensemble-averaging $\sqrt{N}$ law is previewed: on a simulated example, $N = 100$ epochs give a $\sqrt{100} = 10\times$ amplitude SNR gain ($\approx$20 dB) — an illustrative, reproducible figure, not a clinical benchmark.

1.14 Conceptual questions & exploration tasks

1. Classify each of the following along all four taxonomy axes: a 1 kHz calibration tone; resting EEG; the ECG of a patient in atrial fibrillation; a single K-complex.
2. The EEG is 10–100 µV and the ECG is 0.5–4 mV. By roughly what factor must the EEG amplifier's gain exceed the ECG amplifier's for the two digitised signals to use the ADC's range comparably? What does this imply about noise?
3. Explain physiologically why the EOG is essential for detecting REM sleep but nearly useless for detecting stage N3.

varkonyibarnabas@gmail.com

4. A colleague proposes summarising a whole night of EEG with one power spectrum. Using the idea of non-stationarity, explain why this throws away the information a sleep scorer needs.
5. *Exploration.* In the companion notebook, change the EEG sleep stage from N2 to N3 and to REM. Describe how the dominant rhythm, and the presence or absence of spindles, changes — and connect each change to §1.4.
6. *Transfer.* A wearable records seismocardiography — the tiny chest-wall vibrations of each heartbeat — from a chest accelerometer. Identify the physiological source, the transducer, and the likely artifacts. Then classify each of these goals using the task taxonomy of §1.8 — is it detection, estimation, classification, or prediction? (a) locating individual beats; (b) estimating cardiac timing intervals; (c) flagging abnormal mechanical events; (d) predicting decompensation. Which chapters would you consult first for each goal?

1.15 Further reading

For physiology and bioinstrumentation, Webster's *Medical Instrumentation* and Rangayyan's *Biomedical Signal Analysis* give thorough, accessible treatments of the electrode, the amplifier, and the origin of each waveform. For a signals-first perspective, Semmlow's *Biosignal and Medical Image Processing* pairs 1-D and 2-D methods in the same spirit as this book. For the sleep-staging application specifically, the AASM *Manual for the Scoring of Sleep and Associated Events* (Berry et al.) is the expert-consensus reference our later chapters learn to reproduce, describing which features of EEG, EOG, and EMG define each stage. For fetal ECG extraction and analysis specifically, Sameni and Clifford's review "A Review of Fetal ECG Signal Processing; Issues and Promising Directions" surveys the abdominal-array separation problem (adaptive filtering, ICA, and beyond) that §10.5 and §10.7 return to.

varkonyibarnabas@gmail.com

Chapter 2 — Signal Acquisition, Sampling & Discretization

How does a continuous voltage at the scalp — or a slice through a moving body — become a stream of discrete numbers, and what does the electrode, the amplifier, and the scanner do to the truth along the way? Chapter 1 treated the biosignal as if it arrived clean. It never does. Between the physiology and the array $x[n]$ we finally process sits a chain of physical devices, each of which imposes limits and, if we are careless, corrupts the signal irreversibly. This chapter is about that chain — and about the two great discretisations, sampling in time and quantisation in amplitude, that turn an analog world into digital data.

2.1 Learning objectives

By the end of this chapter you should be able to:

- Describe the biomedical acquisition pipeline from sensor/transducer to ADC — plus the optional transmitter/telemetry stage of remote and wearable devices — and explain what each stage converts (L1).
- Explain the electrode-electrolyte interface: half-cell potential, the electrical double layer, and why Ag/AgCl (a non-polarizable electrode) is the clinical default (L1).
- Model the electrode-skin impedance and explain why disturbing the electrode interface is a *major physical source* of motion artifact, and why the amplifier's input impedance should far exceed the electrode impedance (L1).
- State the Nyquist-Shannon sampling theorem in one and two dimensions and predict aliasing when it is violated (L1).
- Quantify quantization noise and apply the rule SNR $\approx 6.02\,N + 1.76$ dB (L1).
- Explain spatial sampling in medical imaging — CT projection (Radon) sampling and MRI k-space sampling — and how undersampling produces streaks, moiré, and ghosting (L1).
- Describe electrical bioimpedance as an *induced* measurement modality — the tetrapolar injection principle — and its derived signals: impedance pneumography, impedance cardiography, body-composition analysis, spectroscopy, and impedance tomography (L1, L2).

2.2 The measurement pipeline

Recall the chain from Chapter 1:

$$\text{physiology} \rightarrow \text{electrode/transducer} \rightarrow \text{amplifier} \rightarrow \text{anti-alias filter} \rightarrow \text{ADC} \rightarrow x[n].$$

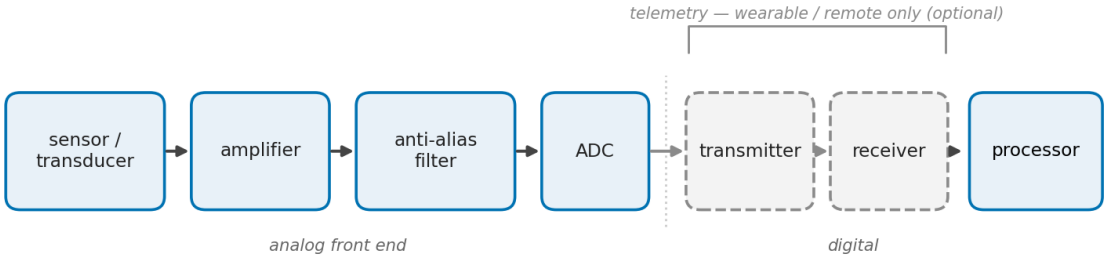
A transducer converts a physiological quantity into an electrical signal: an electrode picks up bioelectric potentials, a piezoelectric or strain-gauge element converts pressure or stretch

varkonyibarnabas@gmail.com

(a respiration belt around the chest, or the CTG TOCO over the uterus), a photodiode converts light (PPG), a microphone converts sound (PCG), and a thermistor converts temperature. Some modalities are induced rather than passive — they drive a small current into the tissue and measure its response (bioimpedance, §2.14).

Two words are used for this first stage, and it is worth being clear about them even though the distinction is often ignored. Strictly, a sensor is the element that *responds* to the physical quantity being measured (the measurand), whereas a transducer is anything that *converts energy from one form into another* — here, a physiological quantity into an electrical signal. By that reading a sensor senses and a transducer converts; but almost every real device does both at once, so in practice the two terms are used interchangeably, and this book follows that convention. An ECG electrode, for instance, is called a *sensor* and a *transducer* with equal correctness — it is at once the element that responds to cardiac potentials and the element that turns ionic current into an electronic one.

The amplifier raises the microvolt-to-millivolt signal to a level the ADC can resolve while rejecting interference. The anti-alias filter band-limits the signal before sampling. The ADC performs the two discretisations. In a bedside instrument the digitised samples then pass straight to a processor, but in a wearable, patch, or ambulatory device one further stage intervenes: a transmitter. A transmitter conditions the signal for transmission and sends it onward — most often as a wireless (Bluetooth or radio) telemetry link to a receiver or gateway that does the processing, though in older or industrial designs it may instead convert the signal to a standardized transmittable form such as a 4–20 mA current loop. The full chain, from the body to the numbers we analyse, is shown in Figure 2.1; the bracketed telemetry stage is present only when the data must travel before it is analysed (a choice we return to in Chapter 15).



The same chain underlies every biosignal; sensor and transducer name the first stage interchangeably, and telemetry is optional.

Figure 2.1. The biomedical acquisition chain. A sensor/transducer converts physiology into a voltage; an amplifier and an anti-alias filter condition it; the ADC digitises it. In a wearable or remote device an optional telemetry stage — a transmitter and a receiver (dashed) — carries the digitised signal to wherever it is processed. The words *sensor* and *transducer* are used interchangeably for the first stage.

Every stage is a design problem, and the electrode — the very first stage — is where most trouble begins.

varkonyibarnabas@gmail.com

2.3 The electrode-electrolyte interface

When a metal electrode contacts the electrolyte of skin and gel, chemistry happens at the boundary. Ions from the metal enter solution and ions from solution deposit on the metal, and this exchange settles into an equilibrium characterised by a half-cell potential — a DC offset voltage specific to the metal. At the same interface, charge separates into an electrical double layer: a sheet of charge on the metal faced by a sheet of opposite charge in the electrolyte, which behaves electrically like a small capacitor in parallel with a resistance.

Electrodes are classified by how freely charge crosses this boundary. A perfectly polarizable electrode passes no DC current — it is all capacitor — and its offset drifts badly. A perfectly non-polarizable electrode lets current cross freely, holding a stable offset. Real Ag/AgCl electrodes come close to the non-polarizable ideal, which is why they are the clinical default: their offset is stable, their drift is low, and — crucially — they tolerate the small movements that would send a polarizable electrode's baseline wandering.

2.4 Electrode-skin impedance and the interface model

The electrode does not touch the signal source directly; it sees the source *through* the skin. A useful lumped model puts, in series from tissue to amplifier: the tissue resistance, then a parallel resistor-capacitor pair for the epidermis (with its own slowly varying skin potential), then the electrode's double-layer RC and half-cell offset. The upshot is a frequency-dependent electrode-skin impedance that can range from a few kilohms (well-prepared, gelled) to hundreds of kilohms or more (dry, poorly prepared).

High electrode impedance is a problem for two reasons. First, it forms a voltage divider with the amplifier's input impedance: if the amplifier input impedance is not vastly larger than the electrode impedance, a large and variable fraction of the biosignal is lost across the electrode before it is ever amplified. This is precisely why bioamplifiers are built with input impedances in the gigaohm range — so the divider ratio stays near unity even as electrode impedance drifts. Second, mismatched impedances between the two electrodes of a differential pair convert common-mode interference (mains hum picked up equally by both leads) into a differential signal the amplifier cannot reject, degrading the effective common-mode rejection ratio (CMRR).

> Method: why input impedance must dominate. Model the electrode-skin path as a source v_s behind impedance Z_e feeding an amplifier of input impedance Z_{in}. The voltage actually seen is $v_{in} = v_s \, Z_{in}/(Z_{in} + Z_e)$. For $v_{in} \approx v_s$ we need $Z_{in} \gg Z_e$. With $Z_e = 100$ kΩ and $Z_{in} = 100$ MΩ the loss is only 0.1 %; with $Z_{in} = 1$ MΩ it is 9 % — and it *varies* as Z_e drifts with sweat and motion, which is worse than a constant loss because it is unpredictable.

2.5 How the interface creates motion artifact

Now the payoff. When the electrode moves relative to the skin, it mechanically disturbs the delicate double layer and the skin-potential layer. The stored charge redistributes, and the half-cell offset momentarily changes. Because these layers relax slowly (an RC discharge), the disturbance appears as a low-frequency transient and a wandering baseline superimposed on

varkonyibarnabas@gmail.com

the biosignal. This is a major physical source of motion artifact and baseline wander — not electrical noise, but chemistry knocked out of equilibrium.

It is worth stressing that this interface mechanism is *a* major source, not *the* single cause. Motion corrupts a recording through several coexisting pathways, and a given artifact usually mixes them: half-cell potential shifts as the metal-electrolyte equilibrium is disturbed; mechanical skin deformation that alters the epidermal skin potential; changes in electrode-skin impedance as contact area and pressure vary; cable and lead movement that flexes the conductors; triboelectric effects, where friction between moving materials (cable insulation, clothing, tape) generates stray charge; and common-mode-to-differential conversion, in which motion transiently unbalances the two electrode impedances so that interference the amplifier would otherwise reject leaks into the differential signal (see §2.4 and the CMRR discussion in §2.6). Any real mitigation strategy has to contend with all of these, which is why "just filter it out" is rarely the whole answer.

Figure 2.2 simulates exactly this on an ECG: motion events at three instants each kick the interface, producing an exponentially relaxing offset (the RC discharge of the double layer), while slow respiration- and drift-coupled wander rides underneath. The clean physiological ECG (top) is buried in the recorded trace (bottom).

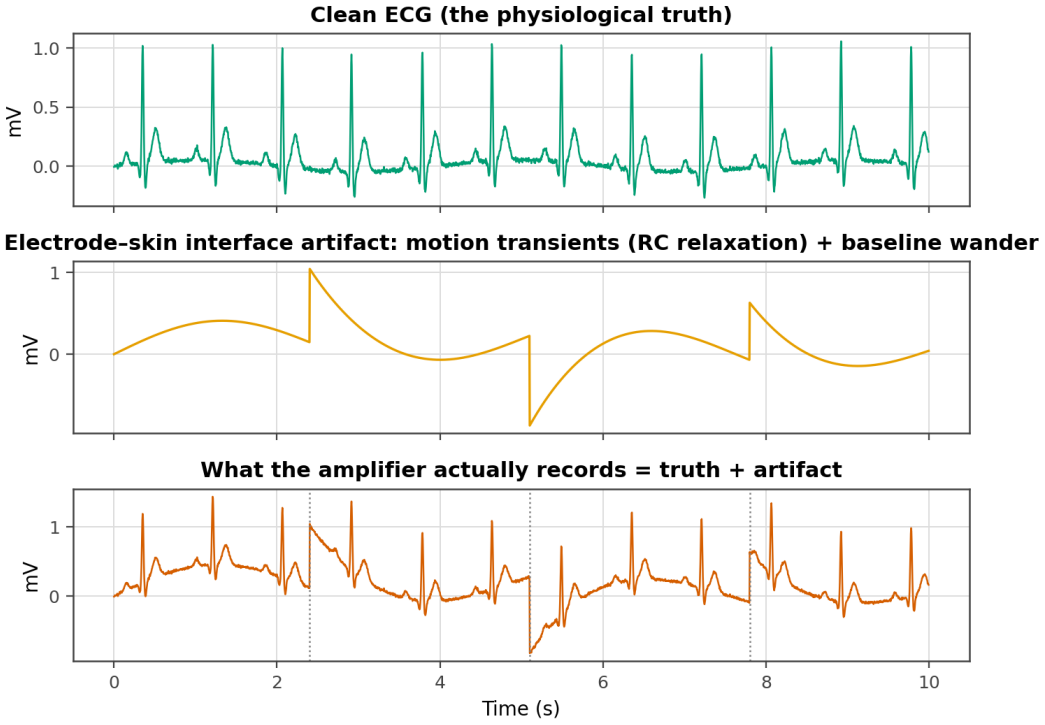


Figure 2.2. The electrode-skin interface as a source of artifact. Top: the clean physiological ECG. Middle: interface artifact — motion transients that relax like an RC discharge, plus slow baseline wander. Bottom: what the amplifier actually records, the sum of the two. Mechanically disturbing the electrode double layer injects low-frequency artifact and baseline wander — a major physical source of motion artifact (alongside cable movement, triboelectric charge, and impedance imbalance), attenuated by the noise-reduction and filtering methods of Chapters 8–10.

varkonyibarnabas@gmail.com

The mitigations follow directly from the mechanism: prepare the skin (abrade the dead epidermis, apply gel) to *lower* the impedance and stabilise the interface; use non-polarizable Ag/AgCl electrodes; and secure the leads to reduce movement. Dry electrodes trade convenience for a higher, less stable impedance and thus worse motion artifact — a real engineering trade-off, not a free lunch. Whatever remains, later chapters attenuate — by high-pass and band-pass filtering (baseline wander sits below the ECG band; see Chapter 9) and by the outlier-rejection and noise-reduction methods of Chapters 8 and 10.

2.6 Anti-alias filtering and the front end

Before the ADC, an analog anti-alias filter attenuates energy above the Nyquist frequency (§2.7) so that it does not fold back into the band of interest. The front end also fixes the gain: too little and the signal fails to fill the ADC's range (wasting bits); too much and peaks clip. And it works to reject interference: a high common-mode rejection ratio (CMRR) — often 100+ dB — lets a differential amplifier reject the mains hum that appears identically on both electrodes. It is tempting to say this "kills powerline interference at the source," but the reality is more conditional. High CMRR *reduces susceptibility* to mains pickup; it does not *remove* it. And crucially, the benefit is only realised when the two electrode impedances are balanced: an impedance mismatch converts part of the common-mode hum into a differential signal that no amount of CMRR can reject (the common-mode-to-differential conversion of §2.5). In practice a differential front end with a good CMRR and well-balanced, low-impedance electrodes suppresses much of the interference before digitisation, but a residual component almost always survives to be dealt with later — by the notch and band-pass filtering of Chapter 9.

A companion technique *actively* suppresses the common-mode interference rather than merely rejecting it. The driven-right-leg (DRL) circuit, standard in modern ECG and EEG front ends, senses the common-mode voltage on the body (essentially the average of the input electrodes), inverts and amplifies it, and drives it back into the patient through a third reference electrode. This negative-feedback loop actively pushes the body's common-mode potential toward the amplifier's reference, shrinking the mains pickup *before* the differential stage ever sees it — a *driven* reference in place of a passive ground connection. The name is historical (on an ECG the reference electrode traditionally sits on the right leg), but the principle is general and DRL works hand in hand with a high CMRR: the feedback loop lowers the common-mode voltage, and the differential amplifier rejects what remains. The drive current is deliberately limited for safety, the same patient-current constraint (IEC 60601) that governs any deliberate current path to the body (§2.14).

varkonyibarnabas@gmail.com

Montages and referencing. A channel labelled "EEG" or "ECG" is not one universal signal; it is a *difference* between two recording sites, and that choice shapes the waveform you get. Because a bioelectric amplifier measures a potential difference, every channel is defined by a derivation — which electrode is the active site and which is the reference. Change the reference and the same brain or heart activity looks different: an EEG referenced to a mastoid, to a linked-ears average, or to a common-average reference will show the same event with different amplitude, polarity, and even apparent shape, and re-referencing after the fact (an arithmetic operation on the channels) can change which features are visible. The spatial arrangement of derivations is the montage, and montages are not interchangeable. In sleep staging, for example, the scoring guidelines call for specific EEG derivations (central, frontal, and occipital sites referenced to the opposite mastoid) precisely because the graphoelements a scorer needs — spindles, K-complexes, slow waves — are clearest in those particular derivations; a different montage might obscure them. The ECG tells the same story: the twelve standard leads are twelve different derivations of one heart's activity, and an arrhythmia obvious in one lead can be nearly invisible in another. The practical lesson is that channel naming and reference choice are part of the data, not metadata to be ignored. Before comparing two recordings, or feeding channels to a model, confirm they use the same derivations — otherwise you may be teaching a classifier to exploit a referencing artifact rather than physiology.

2.7 Sampling I: temporal sampling and the Nyquist theorem

Sampling reads the continuous signal $x(t)$ at regular instants $t = n/f_s$, producing $x[n] = x(n/f_s)$, where f_s is the sampling frequency. The central result of digital signal processing is the Nyquist-Shannon sampling theorem:

$$f_s > 2f_{\text{max}},$$

where f_{max} is the highest frequency present in the signal. If this holds *and the signal is perfectly band-limited*, the continuous signal can be reconstructed exactly from its samples — this part of the theorem is a mathematical identity, not an approximation. The threshold $f_s = 2f_{\text{max}}$ is the Nyquist rate, and half the sampling rate, $f_s/2$, is the Nyquist frequency — the highest frequency a given f_s can faithfully represent. Sample an EEG band-limited to 40 Hz at $f_s = 100$ Hz and you have a comfortable guard band; sample it at 60 Hz and you violate the theorem outright — the Nyquist frequency drops to 30 Hz, so every component between 30 and 40 Hz folds irreversibly back into the 20–30 Hz band, corrupting exactly the beta activity you were trying to measure.

The exactness clause hides an engineering catch, and it is worth stating plainly. The reconstruction theorem assumes an ideally band-limited signal reconstructed with an ideal filter, and real biosignals satisfy neither assumption. No physiological signal is perfectly band-limited — there is always some energy, however small, above any chosen f_{max} — and no analog anti-alias filter has an infinitely sharp cutoff: every real filter has a finite transition band, rolling off gradually rather than dropping to zero at exactly $f_s/2$. If you sample right at the Nyquist rate, whatever energy lies in that transition band folds back as aliasing. The practical remedy is a guard band: choose f_s *comfortably* above $2f_{\text{max}}$ — a common rule of thumb is a factor of two and a half to five, not the bare minimum — so that the anti-alias filter has room to attenuate out-of-band energy to a negligible level before the Nyquist frequency. The clean

varkonyibarnabas@gmail.com

$f_s > 2f_{\text{max}}$ inequality is the *theoretical* floor; a usable acquisition system sits well above it.

> Common mistakes. Three confusions recur around sampling. (1) "Nyquist says sample at exactly $2f_{\text{max}}$." No — that is the theoretical floor, and it holds only for an ideally band-limited signal reconstructed with an ideal filter. Real signals leak energy above any cutoff and real anti-alias filters roll off gradually, so you sample *comfortably above* $2f_{\text{max}}$ with a guard band (a factor of roughly two-and-a-half to five). (2) "More bits will fix aliasing." No — aliasing is already baked in *before* the signal is quantised. It is set by the sampling rate and the anti-alias filter, both of which act ahead of the ADC's amplitude quantiser; adding bits only refines amplitude resolution and does nothing to unfold aliased frequencies. (3) Confusing the Nyquist *rate* with the Nyquist *frequency*. The Nyquist rate is a property of the *signal* — it is $2f_{\text{max}}$, the minimum sampling rate you must exceed. The Nyquist frequency is a property of a chosen *sampling rate* — it is $f_s/2$, the highest frequency that rate can represent. They are different quantities with different roles; keep them straight.

2.8 Aliasing

Violate the theorem and you get aliasing: a frequency above the Nyquist frequency is indistinguishable from — masquerades as — a lower frequency. Formally, a tone at f_0 sampled at f_s appears at the *apparent* frequency $f_{\text{app}} = |f_0 - f_s \,\text{round}(f_0/f_s)|$. The classic everyday example is the wagon-wheel effect, where a fast-spinning wheel filmed at 24 frames per second appears to rotate slowly or backwards.

Figure 2.3 makes this concrete for a 40 Hz tone. Sampled at 400 Hz (well above the 80 Hz Nyquist rate) the samples trace the true tone. Sampled at 70 Hz (below 80) the 40 Hz tone aliases to an apparent 30 Hz, and at 45 Hz it aliases to an apparent 5 Hz — a completely fictitious slow oscillation the downstream algorithm has no way to recognise as fake.

The lesson is asymmetric and unforgiving: aliasing cannot be removed after the fact, because the fast and slow components have become genuinely identical in the samples. The only defence is the analog anti-alias filter of §2.6, applied before sampling.

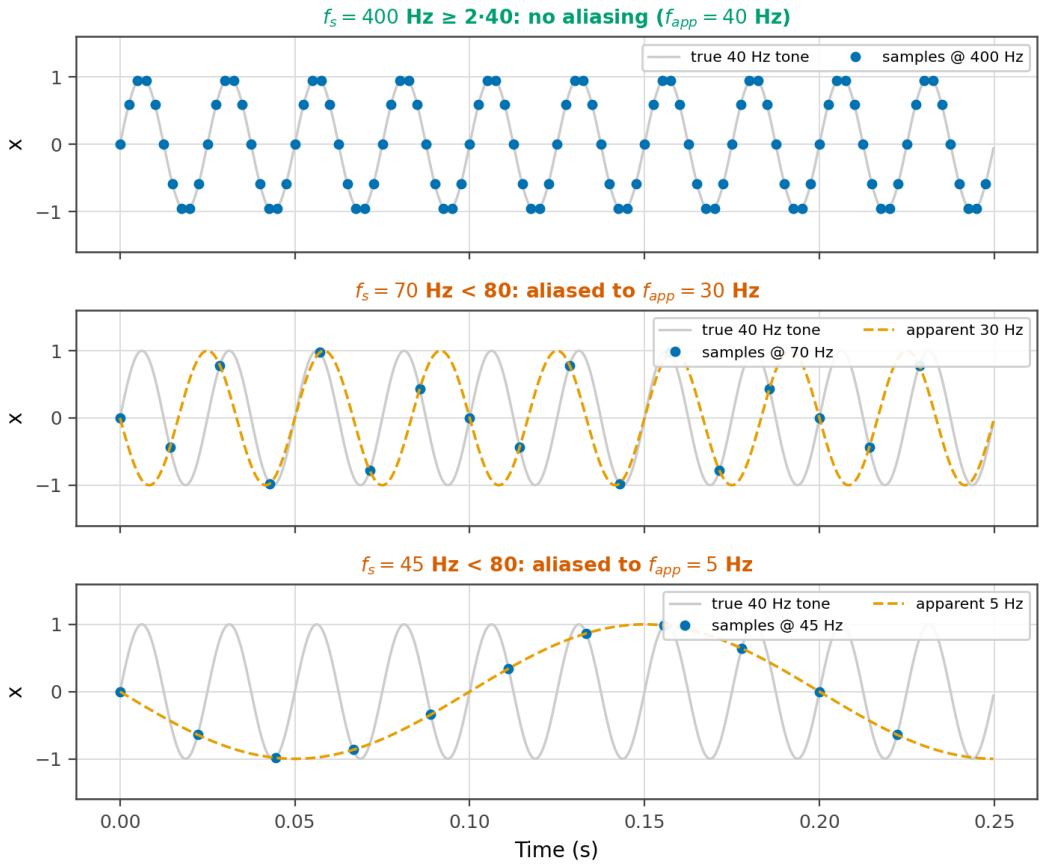
2.9 Quantization

Sampling discretises *time*; quantization discretises *amplitude*. An N-bit ADC represents the signal with 2^N levels, so the step between adjacent levels is $q = V_{\text{range}}/2^N$. Rounding each sample to the nearest level introduces a quantization error bounded by $\pm q/2$. If this error is roughly uniform and uncorrelated with the signal, it behaves like an additive noise of variance $q^2/12$, and for a full-scale sinusoid the resulting signal-to-quantization-noise ratio works out to the famous rule:

$$\boxed{\text{SNR} \approx 6.02\,N + 1.76 \text{ dB}}$$

Each added bit buys about 6 dB — one more bit doubles the number of levels and halves the quantization step. Figure 2.4 shows a synthetic sinusoid quantized at 2, 4, and 6 bits alongside the error signal; on this simulated example the measured SNR tracks the formula closely (a 6-bit conversion gives about 38 dB, illustratively — and, as the formula predicts,

varkonyibarnabas@gmail.com



Sample below the Nyquist rate (2·f_max) and a fast tone masquerades as a slow one — aliasing is irreversible.

Figure 2.3. Temporal aliasing of a 40 Hz tone. At $f_s = 400$ Hz (top) sampling is faithful. At 70 Hz and 45 Hz (below the 80 Hz Nyquist rate) the samples imply an apparent low-frequency tone (dashed) that is not really there. Aliasing is irreversible: once a fast component folds down, no later processing can undo it — which is why the anti-alias filter must act *before* the ADC.

varkonyibarnabas@gmail.com

$6.02 \times 6 + 1.76 \approx 37.9$ dB, reproducible from the companion notebook).

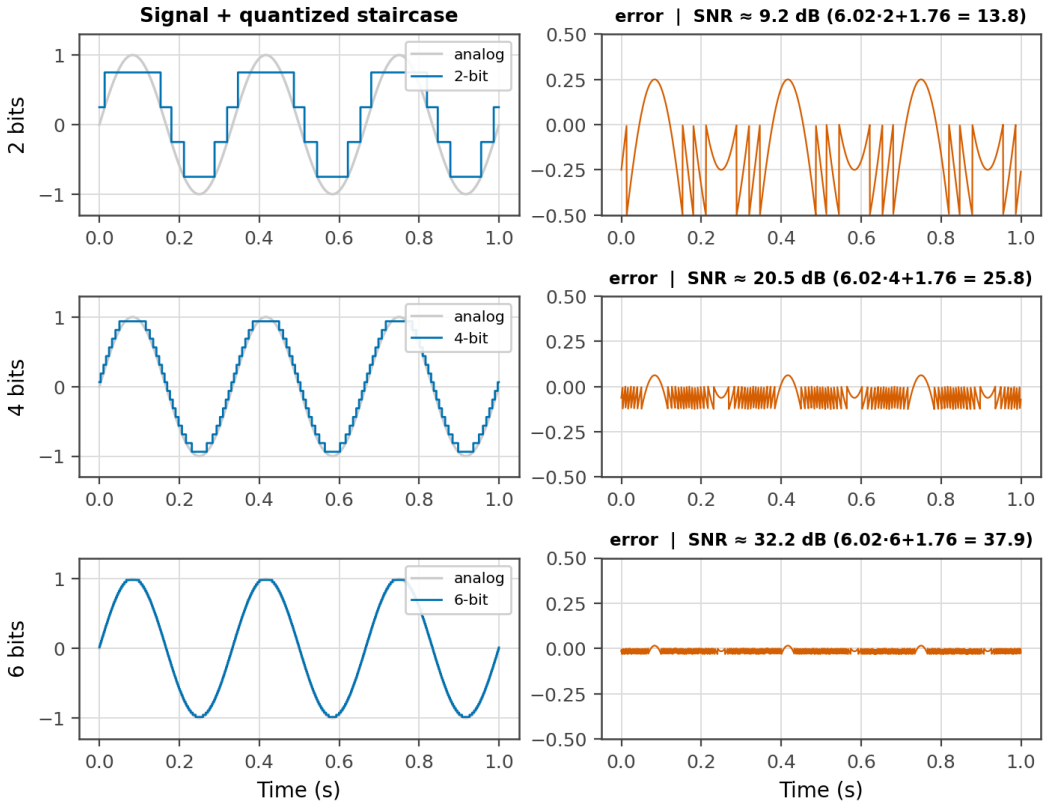


Figure 2.4. Quantization at 2, 4, and 6 bits. Left: the analog sinusoid and its quantized staircase. Right: the quantization error and the resulting SNR, which follows SNR $\approx 6.02\,N + 1.76$ dB. Every added bit buys about 6 dB; coarse quantization injects a noise floor the rest of the pipeline must live with.

That $6.02N + 1.76$ dB is a *best case*, derived for a full-scale sinusoid — one whose peaks just reach the ends of the ADC's range. Real biosignals are neither sinusoids nor full-scale. Peaky signals such as an ECG, with tall, narrow R-peaks riding on a mostly quiet baseline, have a high crest factor (the peak-to-RMS ratio): most of the signal's energy sits far below its rare peaks. To keep those peaks from clipping, you must leave headroom — deliberately scale the signal so its extremes stay inside the range — and that headroom lowers the RMS level relative to full scale. Since the SNR formula is anchored to the RMS amplitude, the *practical* quantization SNR lands typically 10–15 dB below the full-scale-sine maximum. This is one reason clinical ADCs carry more bits than the bare theoretical minimum implies: the extra bits buy back the resolution that crest factor and headroom spend.

Two practical points. First, dynamic range — the ratio of the largest to smallest resolvable signal — is set by N; an ECG with a tall R peak and subtle low-amplitude features needs enough bits to resolve both. Second, dithering (adding a tiny amount of noise before quantizing) can decorrelate the quantization error from the signal, trading a slightly higher noise floor for the removal of nasty quantization patterns — a theme revisited under the treatment of noise and artifacts in Chapter 8.

varkonyibarnabas@gmail.com

It helps to hold the two discretisations side by side, because they are dual — one acts on the time axis, the other on the amplitude axis — and each has an exact analogue in imaging.

	Sampling (time)	Quantization (amplitude)
Discretises	The time axis	The amplitude axis
Controlled by	Sampling rate f_s	Bit depth N
Governing law	$f_s > 2f_{\max}$ (Nyquist)	SNR $\approx 6.02\,N + 1.76$ dB
Failure mode	Aliasing (fast $\rightarrow$ slow)	Quantization noise floor
Reversible?	No, once aliased	No, information below q is lost
1-D example	EEG at 100 Hz vs 60 Hz	12-bit vs 16-bit ADC
2-D / imaging analogue	Pixel spacing $\rightarrow$ moiré; CT views $\rightarrow$ streaks	Gray-level depth $\rightarrow$ posterization

2.10 A/D conversion in practice

Real converters approach the ideal to varying degrees. Successive-approximation (SAR) ADCs binary-search the level in N comparisons and dominate general-purpose acquisition; sigma-delta ADCs oversample massively and shape quantization noise out of band, achieving very high resolution at modest rates — ideal for the low-bandwidth, high-precision world of biosignals. No converter delivers its full nominal bits in practice; noise and nonlinearity reduce the effective number of bits (ENOB) below N. When you read that a device is "24-bit," the ENOB is what actually governs its SNR.

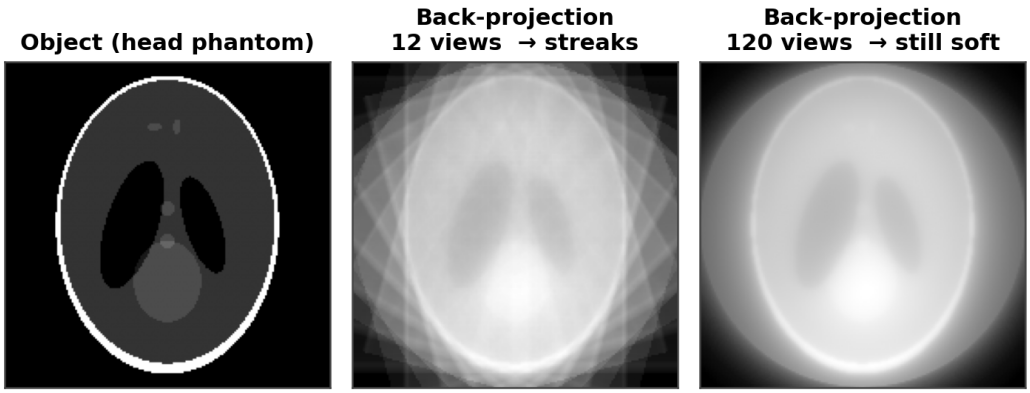
2.11 Sampling II: spatial sampling and medical imaging

Sampling is not only about time. A digital image samples a scene in *space*: the pixel spacing is the spatial sampling interval, and the two-dimensional sampling theorem says that spatial detail finer than the pixel grid can resolve will alias — appearing as moiré patterns, the spatial cousin of the wagon-wheel effect. Downsample a finely textured image and false coarse patterns emerge exactly where the fine texture beats against the coarser grid.

Medical imaging adds its own sampling geometries. CT does not sample the image directly; it acquires angular projections — line integrals of attenuation at many rotation angles — and reconstructs the cross-section from them. The set of projections is the sinogram (a Radon transform of the object), and the number of projection angles is a sampling parameter. Too few angles undersample the object and produce streak artifacts radiating across the reconstruction. Figure 2.5 shows a head phantom reconstructed by unfiltered back-projection from 12 versus 120 views: the sparse-view reconstruction is riddled with streaks; the dense-view one loses the streaks but stays soft, because unfiltered back-projection blurs the object with a $1/r$-type kernel regardless of view count. Removing that residual blur — as opposed to the streaks — needs the ramp (Ram-Lak) filter of filtered back-projection (see Further reading, §2.20, for a pointer into the imaging literature), not more views.

MRI samples yet another space: it fills k-space, the spatial-frequency plane, along a trajectory. The density and extent of k-space sampling set the image's resolution and field of view; deliberately *undersampling* k-space accelerates the scan at the cost of wrap-around/aliasing artifacts — and the mathematics of recovering a good image from that undersampled data is the foundation of accelerated and compressed-sensing MRI.

varkonyibarnabas@gmail.com



Angular undersampling adds streak artifacts on top of unfiltered back-projection's inherent blur (removed only by the ramp filter, not by more views).

Figure 2.5. Angular sampling in CT. Left: the object (a head phantom). Middle: unfiltered back-projection from only 12 projection angles — heavy streak artifacts. Right: 120 angles — the streaks are gone, but the image stays soft (unfiltered back-projection's inherent blur, not removed by more views). Spatial and angular sampling obey the same logic as temporal sampling: too few CT projection angles undersample the object and produce streaks; the residual blur is a reconstruction-filter effect, not a sampling one.

> Signals and images. The Nyquist-Shannon theorem is modality-independent. In 1-D you must sample faster than twice the highest temporal frequency; in 2-D you must sample finer than half the shortest spatial wavelength; in CT you must acquire enough angular views; in MRI you must cover enough of k-space. Undersample any of them and the penalty is the same in spirit — energy that belongs "up high" folds down and impersonates something it is not.

2.12 Motion during acquisition

Because imaging and long recordings take *time*, the scene is rarely static. Cardiac, respiratory, and bulk-body motion violate the "still object" assumption and produce blurring and ghosting in MRI and streaks in CT — the imaging analogue of a moving target being temporally undersampled. The remedies are temporal: cardiac and respiratory gating (acquiring only during a chosen phase of the cycle), triggering, breath-holds, and post-hoc motion correction. The through-line to earlier sections is worth drawing out — motion at the electrode corrupts a waveform, and motion in the scanner corrupts an image. The analogy is structural rather than mechanistic: in both cases the measurement model assumes the scene is stable during acquisition, and motion violates that assumption — though the detailed mechanisms differ (interface chemistry, impedance change, cable and common-mode effects at the electrode; acquisition-model violation and k-space/projection inconsistency in the scanner).

varkonyibarnabas@gmail.com

2.13 Non-uniform sampling and resampling

Not all sampling is uniform, and not every signal already lives at the rate you need. Some acquisitions sample irregularly by design or by accident — dropped packets, or the signal loss that punctuates external fetal monitoring (§1.4) — and reconstructing a usable series from them means estimating values the instrument never recorded. Even a perfectly uniform recording often needs its rate changed: a sleep recording that bundles EEG, EOG, and EMG at different native rates must be brought onto one common grid before any joint analysis, and two recordings from different devices must be reconciled onto the same f_s.

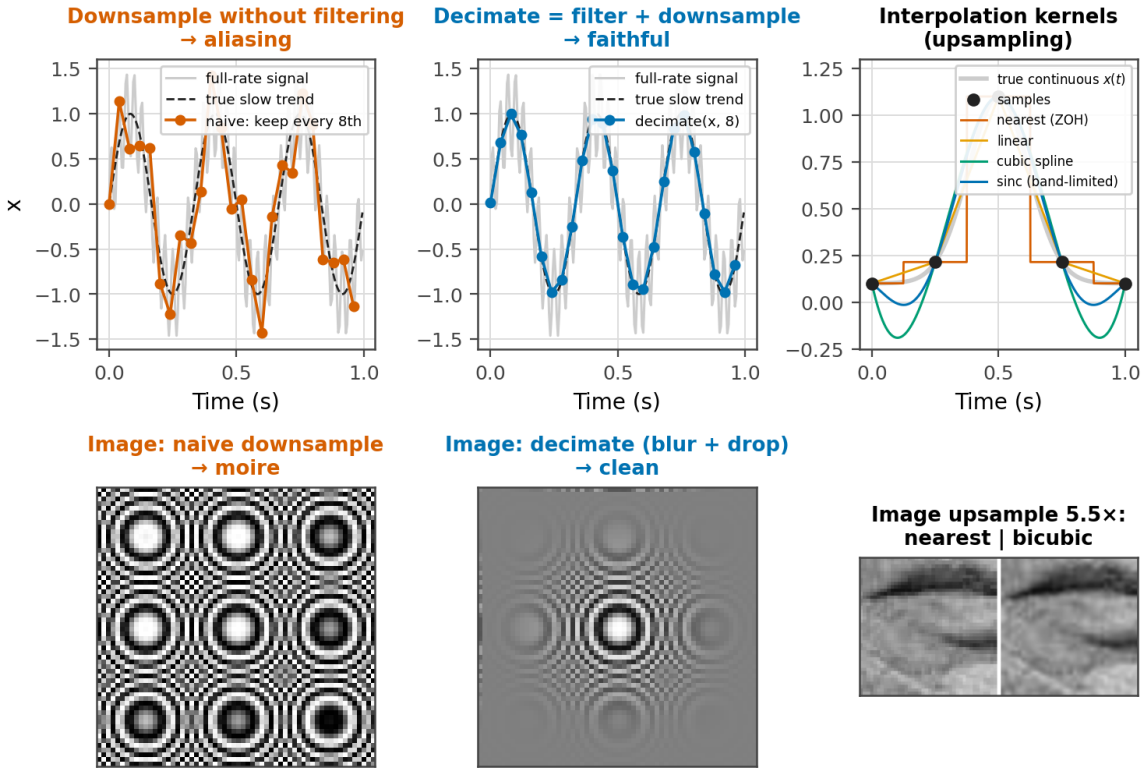
Rate change has four names, and it pays to keep them straight. Upsampling by an integer factor L raises the rate by inserting $L - 1$ zero-valued samples between each pair — on its own this adds no information and stamps unwanted spectral copies (images) into the new band. Downsampling by M is the mirror: it simply keeps every M-th sample and drops the rest. Neither is a complete, safe operation by itself. Interpolation is what makes upsampling meaningful — a low-pass ("anti-imaging") filter removes the spectral images the zero-stuffing created, leaving smoothly estimated values in place of the inserted zeros. Decimation is what makes downsampling safe — a low-pass anti-alias filter, applied *before* samples are dropped, removes content above the new, lower Nyquist frequency so that discarding samples does not fold high-frequency energy back into the band you keep (the aliasing failure of §2.8, now happening in software). The rule is symmetric: upsample, then filter; filter, then downsample.

	Rate change	Alone, it	Complete, safe version
Upsampling	$\times L$ (insert zeros)	creates spectral images	Interpolation = upsample + anti-imaging filter
Downsampling	$\div M$ (drop samples)	risks aliasing	Decimation = anti-alias filter + downsample

Figure 2.6 makes this concrete in a signal and, because the same logic governs pixels, an image. Naively keeping every 8th sample of a two-tone test signal lets a 33 Hz component alias into a spurious 8 Hz wobble that corrupts the true 3 Hz trend; `signal.decimate` (MATLAB: `decimate`) — filter first, then drop — recovers the trend cleanly. The same experiment on a *zone-plate* test image (rings that sweep from coarse at the centre to fine at the edge) shows naive pixel-dropping aliasing the fine rings into a checkerboard moiré pattern, while a 2-D blur (low-pass) before dropping pixels removes exactly the spatial frequencies the smaller image cannot represent — the concrete demonstration §2.11 promised but had not yet shown.

For interpolation technique — estimating a value the instrument never sampled, whether to fill a gap or as the filtering half of upsampling — Figure 2.6 compares the practical choices. Nearest-neighbour (zero-order hold) repeats the last known value: free, but a visible staircase. Linear interpolation draws a straight line between samples: cheap, with a small kink at every sample. Cubic-spline interpolation fits a smooth curve of continuous curvature and is the common default for physiological series — e.g. resampling an irregular RR tachogram before HRV spectral analysis (§7.11), though estimating that spectrum directly from the irregular samples via the Lomb-Scargle periodogram avoids interpolation altogether. Sinc (band-limited) interpolation is the *ideal* reconstruction the sampling theorem (§2.7) itself promises for a band-limited signal, but the ideal kernel is infinite; truncate it carelessly and it rings — visible in Figure 2.6 as the small overshoot the sinc and cubic curves show near the sparse

varkonyibarnabas@gmail.com



Upsampling/downsampling are the rate-change operations (insert/drop samples); interpolation/decimation are the complete, correct procedures that pair a rate change with the filter it needs — in 1-D and in 2-D alike.

Figure 2.6. Rate-change operations vs. the complete, safe procedures, in a signal and an image. Top-left: naive downsampling aliases a 33 Hz component into a spurious 8 Hz wobble; top-middle: `signal.decimate` (filter, then drop) recovers the true trend. Top-right: nearest, linear, cubic-spline, and sinc interpolation of sparse samples — nearest is a visible staircase, and the higher-order kernels can ring near sparse samples. Bottom-left/middle: the same naive-vs-filtered contrast on a zone-plate test image — naive downsampling aliases into moiré, pre-blurring gives a clean result. Bottom-right: one image crop enlarged 5.5× by nearest-neighbour (blocky) and bicubic (smooth) interpolation.

varkonyibarnabas@gmail.com

samples. No choice is universally best: nearest and linear are cheap and cannot overshoot; cubic is a good general default; sinc (or a windowed variant such as Lanczos) is the reference when band-limited fidelity matters. Exactly the same trade-off governs *image* resizing — a nearest-neighbour zoom looks blocky for the same reason zero-order hold does, while bilinear/bicubic/Lanczos are the image-processing names for the same interpolation filters, shown directly in Figure 2.6's final panel.

Oversample, then decimate is a front-end trick worth naming, because it is one of the most common patterns in real biosignal acquisition. A sigma-delta ADC (§2.10) samples far above what the application needs, which both pushes quantization noise into a high band the later stages can filter away (noise shaping) and — because the internal Nyquist frequency now sits so far above $f_{\max}$ — lets the *analog* anti-alias filter ahead of the ADC be gentle and cheap. A digital decimation filter, arbitrarily sharp to build in the digital domain, then removes the shaped-away noise and brings the signal down to the rate the application wants. This is why sigma-delta converters dominate low-bandwidth, high-precision biosignal acquisition: the hard part of anti-aliasing moves from an imperfect analog component to a near-ideal digital one.

Resampling puts channels on a common *rate*; a related step puts them on a common *clock*. Multi-device recordings — a polysomnography amplifier, a separate pulse oximeter, a body-worn accelerometer — start and drift independently, so their samples must be time-aligned (synchronised) before any cross-channel feature is trustworthy. Where a shared hardware trigger or a common timestamp exists, alignment is a lookup; where it does not, a cross-correlation against a shared event visible in two modalities (a QRS complex, a synchronisation pulse) recovers the fixed offset, and a slowly accumulating clock drift is corrected by re-aligning periodically. Feeding a model two channels that are a few hundred milliseconds out of register is a quiet way to destroy exactly the cross-modal timing a sleep or cardiorespiratory analysis depends on (a theme Chapter 6 returns to with correlation, and Chapter 11 with feature fusion).

Reading a real file header to discover each channel's actual, as-recorded sampling rate — not the rate the device datasheet promises — remains the unglamorous but indispensable first step of any pipeline that must resample or align channels onto a common grid.

2.14 Electrical bioimpedance: an induced measurement modality

Every modality so far has been passive — the body supplies the signal and the electrode listens. Electrical bioimpedance (EBI) is different: it is an induced measurement. We inject a small, known alternating current into the tissue and measure the resulting voltage; their ratio is the complex impedance $Z = V/I$, with a magnitude and a phase. Its physical origin — tissue as a dielectric electrolyte whose cell membranes behave as capacitors, giving the frequency-dependent β-dispersion captured by the Cole model — was introduced in §1.3. Here we treat it as a measurement chain and a family of derived signals.

The practical problem is that the electrode-skin impedance (§2.4) is itself large and variable, and it sits in series with the tissue we want to measure. The standard fix is a tetrapolar (four-electrode) arrangement: one pair of electrodes injects the current while a *separate* pair senses the voltage. Because the sensing amplifier draws almost no current, the drop across the sensing electrodes' own impedance is negligible, so the measurement reflects the tissue and not

varkonyibarnabas@gmail.com

the electrodes — a far better result than a two-electrode (bipolar) setup. Measuring at a single frequency gives one impedance value; sweeping the frequency — bioimpedance spectroscopy (BIS) — traces the whole Cole arc and separates intracellular from extracellular contributions. Throughout, the injected current must obey patient-safety limits (IEC 60601) and use charge-balanced, DC-free waveforms.

What makes EBI worth the trouble is how many signals it yields from the same principle (Figure 2.7):

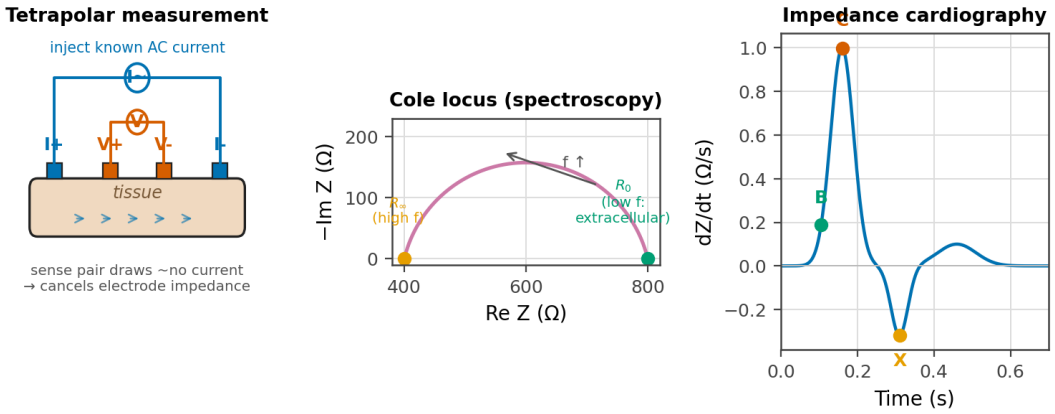
- Impedance pneumography — thoracic impedance rises and falls as the lungs fill with (insulating) air, so respiration can be read straight off the same two electrodes already used for the ECG. It is the standard apnoea/respiration channel in bedside monitors.
- Impedance cardiography (ICG) — the beat-to-beat impedance change across the thorax tracks aortic blood volume and flow; the derivative dZ/dt has landmarks (the B, C, and X points marking aortic-valve opening, peak flow, and valve closing) from which stroke volume and cardiac output are estimated non-invasively.
- Bioelectrical impedance analysis (BIA) — whole-body impedance at one or a few frequencies estimates total body water and, from it, fat-free mass and body composition, the method inside consumer body-fat scales.
- Bioimpedance spectroscopy for tissue characterization — because healthy and pathological tissue have different impedance spectra, BIS is used to characterise tissue state, including the discrimination of malignant from healthy epithelium.
- Electrical impedance tomography (EIT) — a *ring* of electrodes drives many injection/measurement pairs and an inverse problem reconstructs a cross-sectional image of conductivity. EIT is the bioimpedance cousin of the CT and MRI reconstructions of §2.11 — same spatial-sampling and inverse-problem logic — and its best-known use is bedside imaging of regional lung ventilation.
- Impedance plethysmography — segmental limb impedance changes with pulsatile and venous blood volume, used to assess blood flow and screen for venous obstruction.

> Lead-off detection is bioimpedance in disguise. The same idea protects ordinary recordings. To know whether an ECG or EEG electrode is still attached, a monitor continuously drives a tiny current (DC, or a small high-frequency tone placed outside the signal band) through each electrode and watches the return: an intact contact reads a low impedance, a detached one reads a very high impedance and raises a lead-off alarm. It is an induced impedance measurement layered on top of a passive recording — the reason a bedside monitor can tell "flatline" (a true asystole) apart from "a lead fell off."

2.15 The ML/AI bridge

Sampling and quantization set the ceiling on everything downstream. A classifier can only be as good as the data it is fed: aliased components are indistinguishable from genuine features and can mislead a model, and a coarse quantization noise floor limits the smallest real feature that can be resolved. The choice of window length — 30-second epochs in sleep scoring — is itself a sampling decision in the feature domain, trading time resolution against the frequency resolution available to the spectral features of Chapters 3–4. Getting acquisition right is not

varkonyibarnabas@gmail.com



One induced measurement — inject current, read voltage — underlies respiration, cardiac output, body composition, and imaging.

Figure 2.7. Electrical bioimpedance. Left: the tetrapolar principle — an outer pair injects a known AC current while an inner pair senses the voltage, greatly reducing (rather than eliminating) electrode-impedance error. Middle: the Cole impedance locus (a depressed circular arc in the complex plane) traced by bioimpedance spectroscopy as frequency rises through the β-dispersion. Right: one cardiac cycle of the ICG signal dZ/dt with its B/C/X landmarks, from which stroke volume is derived. One induced measurement — inject current, read voltage — underlies respiration, cardiac output, body composition, and tomographic imaging.

a preliminary to machine learning; it *is* the substrate machine learning stands on.

2.16 Companion notebook

The companion notebook makes these effects tangible and interactive. It lets you drag a sampling-rate slider and watch a tone alias; drag a bit-depth slider and watch the quantization staircase coarsen and the SNR track $6.02N + 1.76$; inspect the per-channel sample rates in a real recording header; run the electrode-interface RC demo that generates Figure 2.2; explore imaging sampling by downsampling an image into moiré and reconstructing the head phantom from a varying number of Radon projections (Figure 2.5); and compare naive downsampling against `decimate`, and the interpolation kernels of Figure 2.6, on both a signal and an image.

2.17 Case-study tie-in

This chapter is the very first step of the sleep-scoring case study. A sleep recording is a multi-channel file in which EEG, EOG, and EMG may each be stored at a *different* sampling rate, segmented into 30-second epochs. Before any analysis, the pipeline must read the file, discover each channel's f_s, confirm the anti-alias filtering was adequate, and resample channels onto a common grid. The motion-artifact and baseline-wander phenomena introduced here are exactly what the later cleaning stages must remove before features are trustworthy.

varkonyibarnabas@gmail.com

2.18 Summary & key formulas

- The acquisition chain is sensor/transducer $\rightarrow$ amplifier $\rightarrow$ anti-alias filter $\rightarrow$ ADC ($\rightarrow$ optional transmitter/telemetry in wearable/remote devices); *sensor* and *transducer* are used interchangeably, and every link imprints on the data.
- The electrode-electrolyte interface has a half-cell potential and a double-layer capacitance; Ag/AgCl (non-polarizable) is the clinical default.
- Electrode-skin impedance Z_e must satisfy $Z_{in} \gg Z_e$; disturbing the double layer by motion injects low-frequency artifact and baseline wander.
- Sampling theorem: $f_s > 2f_{\text{max}}$; violate it and get irreversible aliasing, apparent at $f_{\text{app}} = |f_0 - f_s \,\text{round}(f_0/f_s)|$.
- Quantization: $q = V_{\text{range}}/2^N$; error variance $q^2/12$; SNR $\approx 6.02\,N + 1.76$ dB ($\approx$6 dB per bit).
- Spatial sampling obeys the same theorem: CT needs enough projection angles (else streaks), MRI enough k-space coverage (else wrap/aliasing); acquisition motion causes ghosting/streaks, cured by gating.
- Electrical bioimpedance is an *induced* modality: inject a known AC current, measure voltage, $Z = V/I$; a tetrapolar setup greatly reduces (without fully eliminating) electrode-impedance error, and one principle yields respiration (pneumography), cardiac output (ICG), body composition (BIA), tissue spectroscopy, and imaging (EIT). Lead-off detection is the same idea.
- Rate change: upsampling (insert zeros) and downsampling (drop samples) are incomplete on their own; interpolation = upsampling + an anti-imaging filter, decimation = an anti-alias filter + downsampling. The same logic covers signals and image resizing alike.

2.19 Conceptual questions & exploration tasks

1. An EEG channel is band-limited to 45 Hz. What is the minimum sampling rate? If it is sampled at 80 Hz, at what apparent frequency does a stray 50 Hz mains component appear?
2. A 12-bit and a 16-bit ADC digitise the same signal. Using $6.02N + 1.76$, what is the difference in their theoretical SNR? By what factor does the quantization step shrink?
3. Explain, using the voltage-divider model, why a dry electrode with $Z_e = 500$ kΩ into a 1 MΩ amplifier input is a bad idea, and why raising the input impedance to 1 GΩ fixes it.
4. Why can aliasing never be removed by digital filtering after sampling, whereas powerline interference sometimes can?
5. *Exploration.* In the companion notebook, reconstruct the head phantom from 8, 32, and 256 projection angles. Define a quantitative adequacy criterion — e.g. the RMSE against the 256-view reconstruction dropping below a threshold you state, or streak contrast falling below a stated level — and report the number of views at which your criterion is met; relate this to the sampling theorem. Then state what this synthetic phantom *cannot* tell you: what further evidence (a task-based reader study on real anatomy, at clinical dose) would be needed before any claim that the residual streaks are *clinically* acceptable.
6. Explain why a tetrapolar bioimpedance measurement is more accurate than a two-electrode one, in terms of the electrode-skin impedance model of §2.4. Then name the physiological quantity each of impedance pneumography, ICG, and BIA extracts from

varkonyibarnabas@gmail.com

the same $Z = V/I$ measurement.

7. You need to halve a signal's sampling rate. Explain, in one sentence each, why `x[::2]` is unsafe and what a correct `decimate(x, 2)` does differently. Then explain why upsampling by zero-stuffing alone is *not* the same as interpolation, and what fixes it.

2.20 Further reading

Webster's *Medical Instrumentation* is the standard reference for the electrode interface, amplifier design, and CMRR. Oppenheim and Schafer's *Discrete-Time Signal Processing* gives the rigorous treatment of sampling, aliasing, quantization, and multirate processing (upsampling, downsampling, interpolation, and decimation). For medical imaging sampling — the Radon transform, filtered back-projection, and k-space — Prince and Links' *Medical Imaging Signals and Systems* is an excellent, physically grounded introduction, and the compressed-sensing MRI literature builds directly on the k-space undersampling ideas sketched here. For electrical bioimpedance — the tetrapolar principle, the Cole model, and applications from body composition to EIT — Grimnes and Martinsen's *Bioimpedance and Bioelectricity Basics* is the standard text.

varkonyibarnabas@gmail.com

Chapter 3 — Fourier Analysis: Series, Transform, FFT & Time-Frequency

How do we see the "notes" — the delta, theta, alpha, and spindle rhythms — hidden inside a raw EEG trace? A waveform on a screen is honest but unreadable: the rhythms that matter are tangled together in time. The single most powerful move in all of signal processing is to change representation — to re-express the same signal in a basis where its hidden structure becomes obvious. That move is the Fourier transform, and this chapter builds it from intuition to the FFT to the spectrogram, with the two-dimensional version for images travelling alongside.

3.1 Learning objectives

By the end of this chapter you should be able to:

- Explain what a transform is — a change of representation that rearranges information without adding or losing it (L1, L3).
- Decompose a periodic signal into a Fourier series and explain the Gibbs phenomenon (L3).
- State the Fourier transform and its key properties (linearity, shift, scaling, convolution) (L3).
- Explain the DFT and why the FFT's $O(N \log N)$ cost changed signal processing, including fast convolution (L1, L3).
- Distinguish the DFT bin spacing $\Delta f = f_s/N$ from the *practical* frequency resolution (set by record length, window main-lobe width, SNR, and leakage), and explain why zero-padding interpolates but does not resolve (L3).
- Read a spectrogram (STFT) and state the time-frequency uncertainty trade-off (L3).
- Interpret the 2-D Fourier transform of an image, distinguish its magnitude from its phase, and explain the inverse FFT/IDFT that reconstructs the signal exactly (L1, L3).

3.2 What a transformation is, and why we change representation

A transform re-expresses a signal in a new basis. Nothing is added and nothing is lost — the Fourier transform is invertible, so the information content is identical — but the *arrangement* changes, and arrangement is everything. In the time domain, a sum of three rhythms looks like an inscrutable tangle; in the frequency domain it is three clean peaks. We change representation to make the structure we care about *legible*.

This framing is worth stating once and reusing forever: every transform in this book — Fourier here, and later the wavelet, Hilbert, DCT, and Z-transform — is a change of basis chosen to make some particular structure obvious. Fourier chooses sinusoids, which is the right basis whenever the structure of interest is *rhythmic*.

varkonyibarnabas@gmail.com

3.3 Fourier series

Start with a periodic signal of period T. Fourier's claim, radical in 1807 and indispensable now, is that *any* reasonable periodic signal is a sum of sinusoids at integer multiples (harmonics) of the fundamental frequency $f_0 = 1/T$:

$$x(t) = a_0 + \sum_{k=1}^{\infty} \left[a_k \cos(2\pi k f_0 t) + b_k \sin(2\pi k f_0 t) \right].$$

The coefficients a_k, b_k measure how much of each harmonic the signal contains. A square wave, for instance, is built entirely from *odd* harmonics with amplitudes falling as $1/k$. Figure 3.1 shows the reconstruction growing sharper as harmonics are added — but with a stubborn wrinkle at the edges.

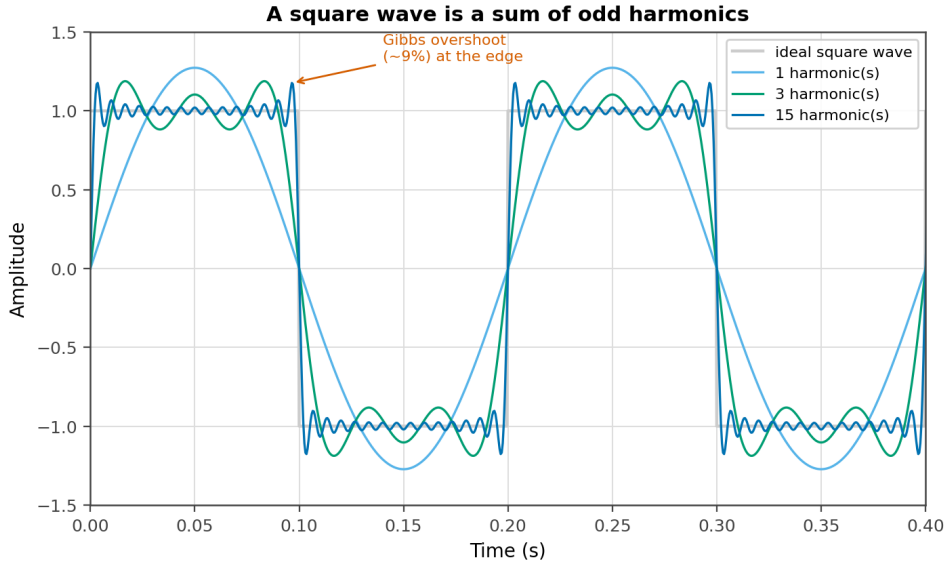


Figure 3.1. Building a square wave from its harmonics. With 1, 3, and 15 odd harmonics the partial sum approaches the ideal square wave, but a ~9% overshoot (the Gibbs phenomenon) persists at each discontinuity. Adding harmonics rebuilds the wave, yet the edge overshoot never vanishes — it only narrows.

That overshoot is the Gibbs phenomenon: near a discontinuity, a finite sum of continuous sinusoids overshoots by about 9% of the jump, no matter how many terms you add — the ripple narrows but never shrinks. Gibbs is not a numerical bug; it is the fundamental price of representing a sharp edge with smooth basis functions, and it foreshadows the leakage story of Chapter 4.

3.4 The Fourier transform

Real biosignals are aperiodic. Letting the period $T \to \infty$ turns the discrete harmonic sum into a continuous integral — the Fourier transform (if the complex exponential $e^{-j2\pi ft}$ is not

varkonyibarnabas@gmail.com

yet second nature, Appendix B builds it from Euler's formula):

$$X(f) = \int_{-\infty}^{\infty} x(t)\, e^{-j2\pi ft}\, dt, \qquad x(t) = \int_{-\infty}^{\infty} X(f)\, e^{j2\pi ft}\, df.$$

$X(f)$ is complex: its magnitude $|X(f)|$ says how much of frequency f is present, and its phase $\angle X(f)$ says how those components line up in time. The transform obeys properties that do most of the work in practice: linearity (the transform of a sum is the sum of transforms); the shift property (a delay in time is a linear phase ramp in frequency); scaling (compress in time and you stretch in frequency); and — the crown jewel — the convolution theorem:

$$x(t) * h(t) \;\longleftrightarrow\; X(f)\, H(f).$$

Convolution in time equals multiplication in frequency. This single identity is why filtering is so natural in the frequency domain, and it is about to make the FFT indispensable.

3.5 The DFT and the FFT

Computers hold finite, sampled signals, so we use the Discrete Fourier Transform (DFT), which samples the spectrum at N evenly spaced frequencies:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \qquad k = 0, 1, \dots, N-1.$$

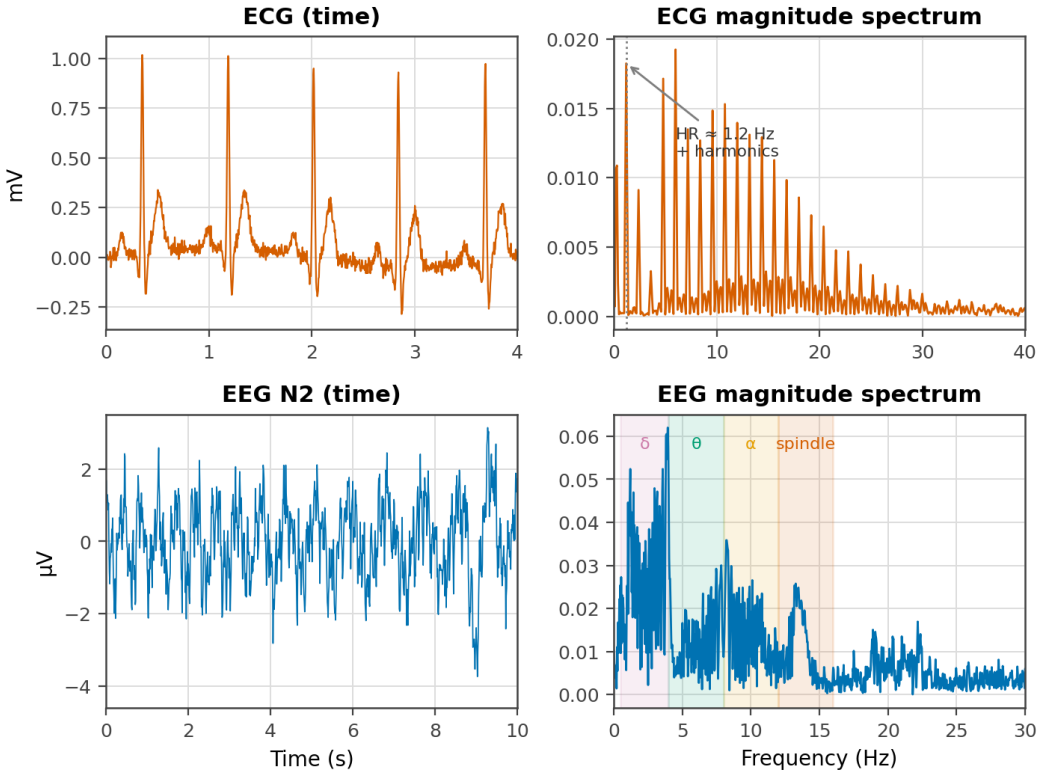
Computed directly, the DFT costs $O(N^2)$ operations — a million-point transform would need a trillion multiplications. The Fast Fourier Transform (FFT) reorganises the same computation to exploit symmetry, cutting the cost to $O(N \log N)$. For $N = 10^6$ that is a speed-up of roughly fifty thousand. It is not an exaggeration to say the FFT is *why* real-time spectral analysis exists.

Figure 3.2 shows the FFT doing its everyday job on two biosignals. The ECG's tangle of beats becomes a heart-rate line near 1.2 Hz plus a comb of harmonics; the EEG's inscrutable wiggle resolves into its rhythm bands, with the sleep spindle showing as energy near 13 Hz.

3.6 The FFT replaces convolution

The convolution theorem plus the FFT gives fast filtering. Directly convolving a length-N signal with a length-M filter costs $O(NM)$; instead, FFT both, multiply the spectra, and inverse-FFT, for a total of $O(N \log N)$ when M is comparable to N. For streaming signals longer than any single FFT, the overlap-add and overlap-save methods chop the signal into blocks, transform each, and stitch the results. This same identity underpins fast correlation (used later for template matching) and reappears whenever we apply a filter as a frequency-domain multiply. The FFT matters far beyond drawing spectra: it is the engine of fast filtering.

varkonyibarnabas@gmail.com



The FFT turns a wiggly trace into peaks: ECG shows a heart-rate line + harmonics; EEG shows its rhythm bands.

Figure 3.2. The FFT turns a wiggly trace into peaks. Top: an ECG (time) and its magnitude spectrum, showing the heart-rate line and harmonics. Bottom: an EEG N2 epoch and its spectrum, with the delta, theta, alpha, and spindle bands shaded. What is illegible in time becomes legible in frequency.

varkonyibarnabas@gmail.com

Linear vs. circular convolution: mind the wrap-around. The identity "FFT, multiply, IFFT" reproduces ordinary *linear* convolution only when the transform is long enough to hold the full result. Convolving a length-N signal with a length-M filter produces $N + M - 1$ output samples, so both inputs must be zero-padded to at least $N + M - 1$ before the FFT. Skip the padding and a plain spectral product returns circular convolution instead: because the DFT treats each sequence as periodic, the tail of the output wraps around and corrupts the beginning (wrap-around, or time-domain aliasing). For streaming signals this is handled block-by-block by overlap-add or overlap-save, which pad each block correctly and stitch the pieces so the joins are seamless. The duality demonstration in §3.12 (Figure 3.6c) uses exactly this correct zero-padding, which is why its FFT-multiply-IFFT result matches direct time-domain convolution to machine precision. The short version: match linear convolution by padding to $\geq N + M - 1$ (or use overlap-add/overlap-save); a bare spectral product gives you circular convolution.

3.7 Frequency resolution vs. zero-padding

Here is a concept students must get right, because getting it wrong wastes real experiments — but it must also be stated carefully, because the DFT bin spacing and the *practical* ability to tell two tones apart are not the same thing. The DFT bin spacing — the frequency step between adjacent DFT samples — is

$$\Delta f = \frac{f_s}{N} = \frac{1}{T_{\text{record}}},$$

set entirely by how much data you have, i.e. the record length T_{record}. It is tempting to call Δf "the smallest frequency difference the DFT can resolve," but that overstates it. Δf is the *grid spacing* of the spectrum; the practical resolution — whether two nearby tones actually appear as two peaks — depends additionally on the analysis window's main-lobe width (a taper widens it; see Chapter 4), the signal-to-noise ratio, the amount of leakage, and the estimator you use to read the spectrum. As a working rule of thumb, two equal-amplitude tones need to be separated by roughly a main-lobe width (of order Δf for a rectangular window, more for a taper) before they reliably split into two peaks. Buying resolution therefore starts with a longer record — which shrinks Δf — but the record length sets the *scale*, and the window, SNR, and estimator set how much of that scale you can actually use. To have any hope of separating two tones Δf apart you need a record at least $1/\Delta f$ seconds long; with a tapered window or low SNR you need somewhat longer.

Zero-padding — appending zeros before the FFT — is often mistaken for a shortcut. It lengthens the transform and so places *more bins* across the same frequency axis, producing a smoother, prettier curve. But those extra bins are interpolation, not information: they cannot separate two tones the data itself does not resolve. Zero-padding refines the *display* of the spectrum; it does not refine the spectrum. Figure 3.3 proves the point with two tones 1.0 Hz apart.

The broader principle — *you buy resolution with observation time* — returns in windowing (Chapter 4) and in spectral estimation, where the same record-length-versus-resolution tension governs every design choice.

varkonyibarnabas@gmail.com

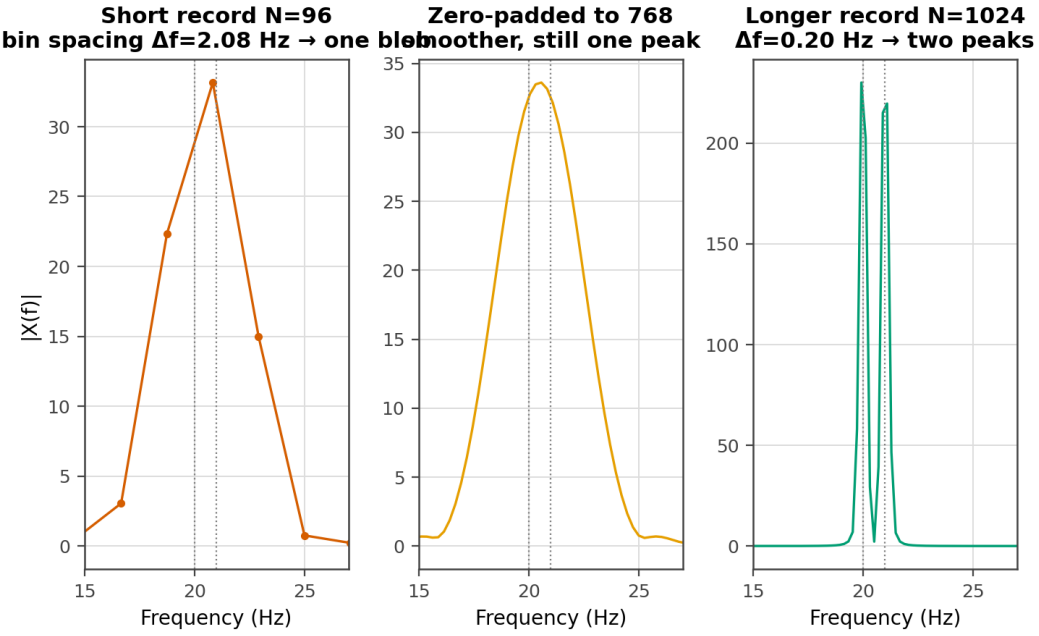


Figure 3.3. Resolution is bought with observation time, not zero-padding. Left: a short 96-sample record cannot resolve two tones 1.0 Hz apart — one blob. Middle: zero-padding that same record 8× yields a smoother curve but still one peak. Right: a longer 1024-sample record has small enough Δf to reveal both tones. Zero-padding interpolates; only more data resolves.

varkonyibarnabas@gmail.com

Common mistakes. Three errors recur around resolution and zero-padding. (1) "Zero-padding adds resolution." No — appending zeros only *interpolates* a smoother-looking spectrum onto a finer grid; it invents no new information and cannot separate two tones the data itself does not resolve. True resolution comes only from a longer *record*, which shrinks the fundamental scale. (2) "$\Delta f = f_s/N$ is the resolution." Not quite — Δf is the DFT bin spacing, the grid step, not a guarantee that two tones Δf apart will show up as two peaks. Whether they actually split also depends on the window's main-lobe width, the SNR, and the leakage; two nearby tones typically need to be separated by roughly a main-lobe width, not a single bin. (3) Forgetting the window. Analyse a finite record without a deliberate taper and the rectangular cut leaks energy into skirts, so spectral leakage can masquerade as real peaks (or bury genuine ones). Chapter 4 is devoted to this; the mistake here is treating every bump in an un-windowed spectrum as a true spectral line.

3.8 Boundary handling: zero, mirror, and periodic padding

A finite record has edges, and the DFT implicitly treats the signal as periodic — it wraps the end around to the start. If the two ends do not match, that wrap creates an artificial discontinuity that smears energy across the spectrum (a preview of leakage). How we pad the edges before transforming or filtering matters. Zero-padding assumes silence beyond the edge, which creates a discontinuity if the edge value is non-zero. Mirror (symmetric) padding reflects the signal at the boundary, greatly reducing the edge discontinuity — the usual choice for filtering signals and image borders. Periodic wrap assumes the signal repeats, which is honest only if it genuinely does. Each choice trades one artifact for another; we set the vocabulary here and apply it at filter edges and image borders later.

3.9 The short-time Fourier transform and the spectrogram

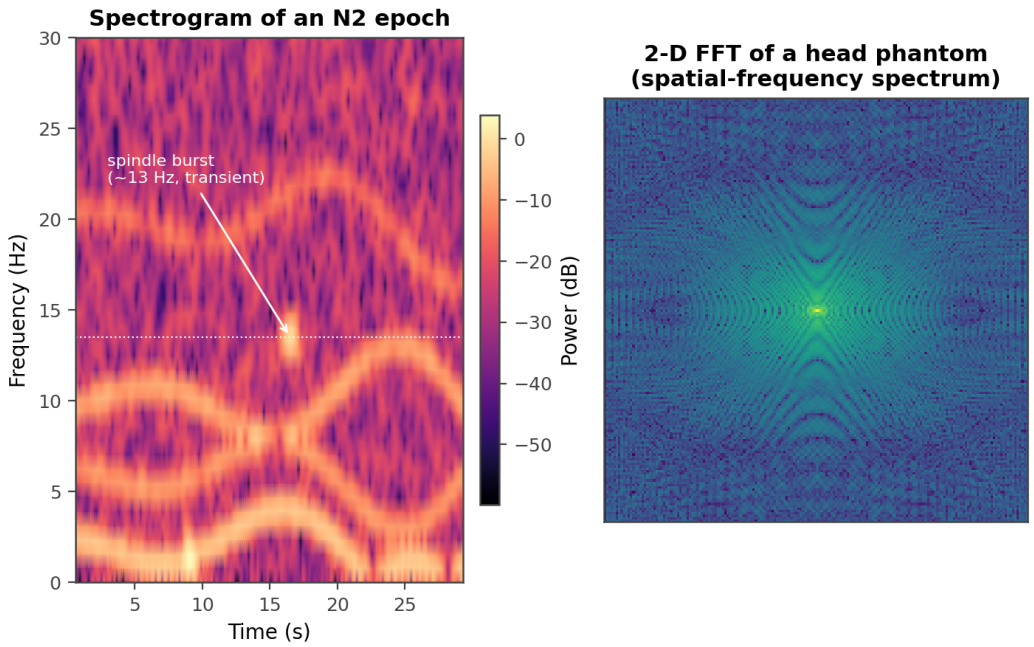
A single Fourier transform gives a *global* representation with no temporal localisation: it can perfectly well be applied to a non-stationary signal, but it reports only *which* frequencies are present across the whole record, never *when* they occur. That time-blindness is what we must fix — a single spectrum of a whole night's EEG would smear every stage together. The fix is to accept locality: slide a short window along the signal and take the FFT of each windowed segment. This is the short-time Fourier transform (STFT), and displaying its magnitude as time on one axis and frequency on the other gives the spectrogram — a picture of how the spectrum *evolves*.

Figure 3.4 (left) shows the spectrogram of an N2 sleep epoch: the background rhythms run as horizontal bands, and the sleep spindle appears as a localised bright patch near 13 Hz at the moment it occurs — visible in time *and* frequency at once, which a plain FFT could never show.

3.10 The uncertainty trade-off

The spectrogram forces a choice. A short window localises events sharply in time but, having little data, resolves frequency poorly ($\Delta f = f_s/N$ again). A long window resolves frequency finely but blurs *when* things happened. You cannot have both: the Heisenberg-Gabor uncer-

varkonyibarnabas@gmail.com



Time–frequency (STFT) localizes a transient spindle in time; the same idea in 2-D gives an image its spatial-frequency spectrum.

Figure 3.4. Time-frequency analysis. Left: the spectrogram (STFT) of an N2 epoch localises a transient sleep spindle to a specific time and frequency (~13 Hz). Right: the 2-D FFT of a head phantom — the same idea in two dimensions, revealing the image's spatial-frequency content. The STFT localises a transient in time; the 2-D FFT gives an image its spatial-frequency spectrum.

varkonyibarnabas@gmail.com

tainty principle sets a hard floor on the product of time and frequency resolution,

$$\Delta t \cdot \Delta f \;\gtrsim\; \frac{1}{4\pi}.$$

Every time-frequency method negotiates this trade-off; the STFT does it with one fixed window size, and the wavelet transform of the next chapters does it adaptively. Choosing the window is the single most consequential spectrogram parameter.

The four members of the Fourier family answer four different questions; it is worth seeing them side by side, because choosing the right one for a given signal is half the battle.

Tool	Input signal	Output domain	Answers	Biosignal use
Fourier series	Periodic, continuous	Discrete harmonics	Which harmonics build this periodic wave?	Idealised ECG/rhythm models
Fourier transform	Aperiodic, continuous	Continuous frequency	What frequencies are present overall?	Theory; transform pairs
DFT / FFT	Finite, sampled	N discrete bins	The computable spectrum of real data	Band power, whole-epoch FFT
STFT / spectrogram	Finite, non-stationary	Time $\times$ frequency	W*hen* does each frequency occur?	Spindles, seizures, evolving rhythm

3.11 The 2-D Fourier transform for images

Images have Fourier transforms too. The 2-D DFT decomposes an image into spatial-frequency components: low spatial frequencies capture broad smooth regions, high spatial frequencies capture edges and fine texture. Just as in 1-D, the 2-D transform is complex, so it carries two pieces of information at every spatial frequency. The magnitude $|F(u, v)|$ says *how much* of each spatial-frequency pattern the image contains — how strong the coarse gradients versus the fine textures are. The phase $\angle F(u, v)$ says *where* those patterns sit — how they are aligned so that edges land in the right places. In the 2-D log-magnitude spectrum (Figure 3.4, right, and Figure 3.5b), energy near the centre is the low-frequency, smooth content; energy at the periphery is the edges. This immediately gives frequency-domain image processing: a low-pass filter (keep the centre) blurs, a high-pass filter (keep the periphery) sharpens edges, and the same padding choices from §3.8 govern the image borders. Convolution-as-multiplication holds in 2-D exactly as in 1-D, which is why the FFT accelerates image filtering just as it accelerates signal filtering.

Inverting the transform. Everything the Fourier transform does, it can undo. The transform is invertible and lossless: from the full complex spectrum — magnitude *and* phase together — the inverse DFT reconstructs the original signal exactly, sample for sample. In one dimension the forward and inverse DFTs are a symmetric pair,

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \qquad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]\, e^{+j2\pi kn/N},$$

varkonyibarnabas@gmail.com

and the two-dimensional case is the same idea applied along rows and columns,

$$x[m,n] = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F[u,v]\, e^{+j2\pi(um/M+vn/N)}.$$

In code this is `np.fft.ifft` and `np.fft.ifft2` — the inverse FFT (IFFT), which shares the FFT's $O(N \log N)$ speed. The forward FFT changes representation; the inverse FFT changes it back with no loss. This round trip is what makes frequency-domain *processing* possible at all: we FFT into the spectrum, modify coefficients (multiply by a filter, zero out a band, shrink noise), and then inverse-FFT back to a usable signal or image. Every fast filter in §3.6 ends with an IFFT.

Which half of the spectrum matters? Magnitude and phase do not contribute equally to how an image *looks*. A striking experiment makes this concrete: reconstruct an image from its magnitude alone (setting all phases to zero) and, separately, from its phase alone (setting all magnitudes to one), then inverse-FFT each. Figure 3.5 does exactly this. The magnitude-only reconstruction is an unrecognisable smear, whereas the phase-only reconstruction still shows the recognisable edges and outlines of the original scene. The lesson is that phase carries most of the spatial structure — it encodes where the edges are — while magnitude mostly sets their relative strength. This is not a mere curiosity: it explains why phase must be handled carefully in image reconstruction, why many registration and matching methods key on phase, and why a full complex spectrum (not just a power spectrum) is needed to recover an image.

> Signals and images. The Fourier transform is one idea in two costumes. In 1-D it answers "which temporal rhythms are present?"; in 2-D it answers "which spatial patterns — how fine, in which direction — are present?" Fast convolution, low/high-pass filtering, the resolution-versus-record-length trade, and the invertibility (forward FFT out, inverse FFT back) all carry over unchanged. Learn it once; apply it to waveforms and images alike.

3.12 Two domains, one operation: a dictionary of dual pairs

We now arrive at the single most useful habit of thought in all of signal processing: almost every operation has two equivalent faces — one you perform in time, and its twin you perform in frequency — and they produce exactly the same result. The Fourier transform is the dictionary that translates between them, and once you can read that dictionary you can always choose whichever domain makes an operation cheapest, clearest, or easiest to reason about.

The keystone entry is the convolution theorem we met in §3.6: convolving two signals in time corresponds to multiplying their spectra in frequency,

$$(x * h)[n] \quad \longleftrightarrow \quad X(f)\, H(f).$$

One caveat carries over from the box in §3.6: with correct zero-padding (to $\geq N + M - 1$) or overlap-add/save, the FFT-multiply-IFFT computes the same linear convolution as the time-domain method; a plain spectral product instead gives circular (wrap-around) convolution.

varkonyibarnabas@gmail.com

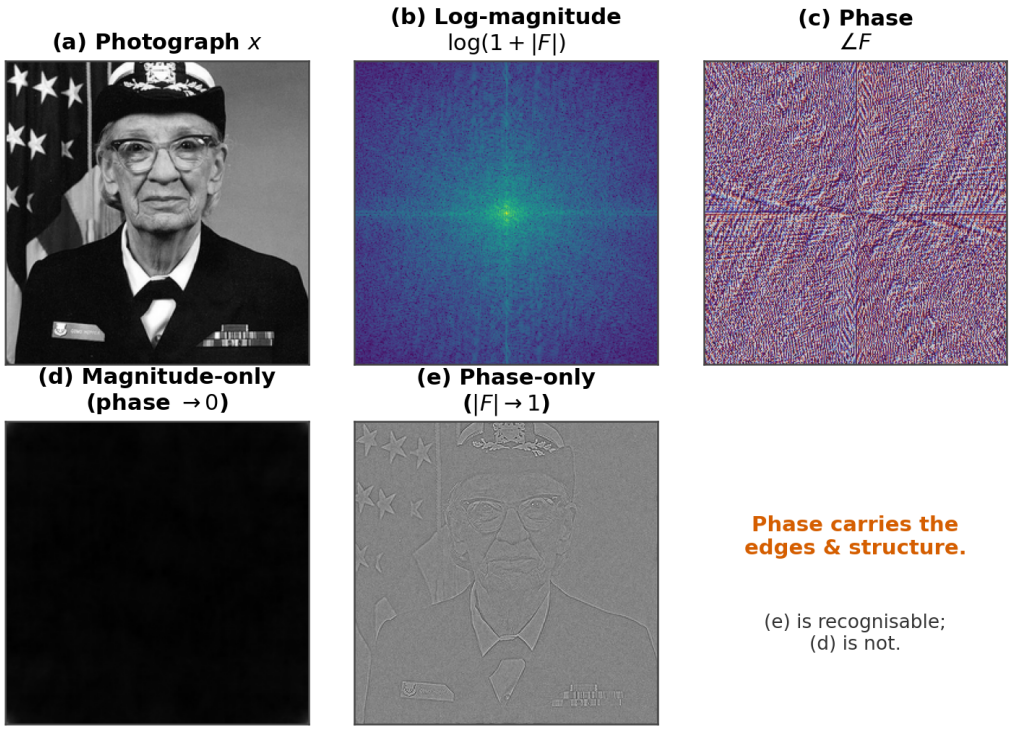


Figure 3.5. Phase carries the structure. (a) A photographic test image and (b) its 2-D FFT log-magnitude spectrum and (c) phase spectrum. (d) Reconstructing by inverse FFT from the magnitude only (phase set to zero) yields an unrecognisable image; (e) reconstructing from the phase only (unit magnitude) preserves the recognisable edges and outlines. Magnitude and phase together reconstruct the original exactly (the transform is invertible and lossless); split apart, it is the phase that holds the spatial structure. (photograph: public-domain U.S. Navy image of Grace Hopper)

varkonyibarnabas@gmail.com

Under that correct zero-padding, Figure 3.6(c) checks the identity literally — filtering a signal by direct time-domain convolution and by FFT-multiply-IFFT gives results that agree to about one part in 10^{15} (machine precision). They are not merely "similar"; they are the same computation done two ways.

Read the theorem the other way and you get its dual: multiplying two signals in time convolves their spectra in frequency,

$$x[n]\,w[n] \quad \longleftrightarrow \quad X(f) * W(f).$$

This is exactly why cutting a recording to a finite length — multiplying it by a rectangular window in time — smears every spectral line by convolving it with the window's transform. That smearing is spectral leakage, the whole subject of Chapter 4. Windowing and filtering are the *same theorem viewed from opposite sides*.

From this one idea a whole dictionary follows, and it pays enormous dividends: it lets you predict what a familiar time-domain trick does to a spectrum — and vice versa — with no new mathematics.

Smoothing is low-pass filtering. A moving average — replace each sample by the mean of its neighbours — is a convolution with a short rectangular kernel. By the convolution theorem it must be equivalent to *multiplying* the spectrum by that kernel's transform, a sinc-shaped low-pass response (Figure 3.6a–b). So "smooth the trace to knock down the noise" and "apply a low-pass filter" are two descriptions of one act. The wider the averaging window, the narrower the low-pass, and the more high-frequency detail is removed — a trade you can now see from either side.

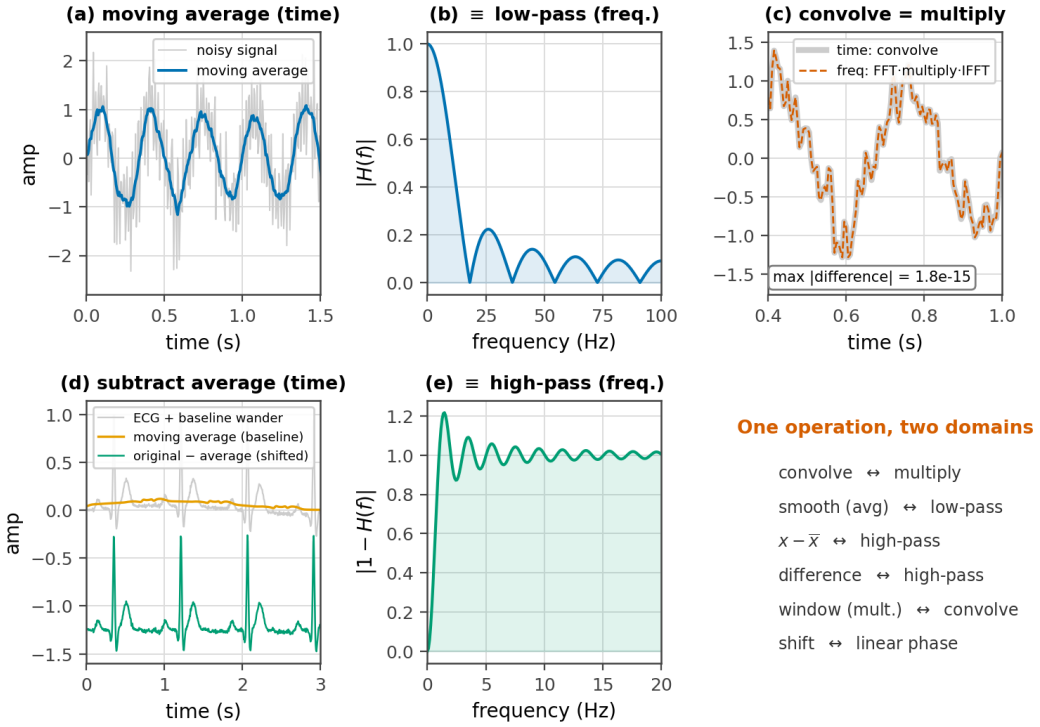
Subtracting the average is high-pass filtering. If a moving average $\bar{x}$ is the low-pass part of a signal, then the *residual* $x - \bar{x}$ is whatever the low-pass discarded — the high-pass part. In the frequency domain this is multiplication by $1 - H_{\text{LP}}(f)$, small at low frequencies and near one at high frequencies (Figure 3.6d–e). This is not an abstraction: subtracting a moving-average estimate of the slow baseline is one of the standard ways to remove baseline wander from an ECG, and it *is* a high-pass filter. The same construction in two dimensions — original minus a blurred copy — is unsharp masking, the classic image-sharpening trick, and it is a 2-D high-pass for exactly the same reason.

Differencing, delay, and modulation. Taking the first difference $x[n] - x[n-1]$ emphasises rapid changes and suppresses slow ones — a discrete derivative, and a high-pass; its opposite, the running sum, is a low-pass. Shifting a signal in time does not change its magnitude spectrum at all — a delay is invisible to $|X(f)|$ and lives entirely in the phase, as linear phase $e^{-j2\pi fk}$. Modulation — multiplying by a cosine — *shifts* the spectrum to the carrier frequency (the basis of AM radio and of nudging a biosignal band away from interference). And correlation, the engine of matched filters and template detection in Chapter 6, is multiplication by the *conjugate* spectrum, so it too can be done quickly with an FFT.

Do this in time	...and it is this in frequency	Everyday example
Convolve, $x * h$	Multiply, $X \cdot H$	apply a filter; FFT-based fast convolution

varkonyibarnabas@gmail.com

Do this in time	...and it is this in frequency	Everyday example
Multiply, $x \cdot w$	Convolve, $X * W$	windowing → spectral leakage (Ch. 4)
Moving average / smooth	Multiply by a low-pass (sinc)	denoise a noisy trace
Subtract the average, $x - \bar{x}$	Multiply by a high-pass, $1 - H_{\mathrm{LP}}$	ECG baseline-wander removal; image unsharp mask
First difference $x[n] - x[n-1]$	Multiply by ≈ high-pass	edge / derivative detector
Running sum (integrate)	Multiply by ≈ low-pass	accumulate / smooth
Shift by k samples	Multiply by linear phase $e^{-j2\pi fk}$	a pure delay (magnitude unchanged)
Multiply by $\cos(2\pi f_0 t)$	Shift the spectrum by $\pm f_0$	modulation / heterodyning
Correlate, $x \star h$	Multiply by the conjugate, $X \cdot \overline{H}$	matched filter, template match (Ch. 6)



The time-domain move and its frequency-domain twin are the SAME operation: a moving average is a low-pass; subtracting that average is a high-pass; convolving in time is multiplying in frequency.

Figure 3.6. One operation, two domains. (a) A moving average of a noisy signal in time is (b) identical to multiplying its spectrum by a low-pass (sinc) response. (c) Convolving in time equals multiplying in frequency — with correct zero-padding (to $\geq N + M - 1$) the two computations agree to machine precision (max difference $\sim 10^{-15}$); a bare product would instead give circular convolution. (d) Subtracting a moving-average baseline from an ECG in the time domain is (e) identical to a high-pass filter $|1 - H_{\mathrm{LP}}(f)|$ — the standard baseline-wander fix. The lower-right panel is the pocket dictionary: pick the domain that makes the job easiest, because the effect is the same either way.

The practical payoff is freedom of choice. A moving average is trivial to write in time; a sharp

varkonyibarnabas@gmail.com

band-pass is far easier to *specify* in frequency; convolving two long signals is dramatically faster done as an FFT-multiply-IFFT. Because the dictionary guarantees the results are identical, you may do each operation wherever it is cheapest and think about it wherever it is clearest. This "same action, two domains" habit recurs throughout the book — in windowing and leakage (Chapter 4), in filter design and 2-D image filtering (Chapter 9), in correlation and fast matched filtering (Chapter 6), and even in deep learning, where the convolution a CNN *learns* is the very same operation whose frequency-domain twin we have just written down (Chapter 14).

3.13 The ML/AI bridge

Frequency-domain representations are the raw material of features. Band power in the delta, theta, alpha, and spindle bands is the canonical sleep-scoring feature set, computed straight from the spectrum. The spectrogram is more than a diagnostic picture: fed as a 2-D image into a convolutional network, it becomes the input representation for modern deep sleep stagers — the classical time-frequency picture and the learned model literally share the same input. And because the FFT makes convolution cheap, it underlies the efficient implementation of the very convolutions a CNN learns. Chapter 3's output — spectra and spectrograms — is where hand-crafted features and learned features meet.

3.14 Companion notebook

The companion notebooks are the figure factory for this chapter: a Fourier-series demo that adds harmonics one at a time and watches Gibbs appear; an FFT-of-biosignals demo reproducing Figure 3.2 on ECG and EEG; a resolution-vs-zero-padding demo (the two-close-tones experiment of Figure 3.3) that lets you lengthen the record and watch the tones separate; a padding-comparison demo contrasting zero, mirror, and periodic edges; an STFT/spectrogram demo (Figure 3.4); and a short 2-D-FFT-of-images cell set. Each is interactive so the trade-offs become muscle memory.

3.15 Case-study tie-in

In the sleep-scoring case study, this chapter supplies the frequency-domain features. Each 30-second epoch is transformed, its band structure read off, and its spindle activity localised with a spectrogram. The record-length-versus-resolution trade-off is not academic here: the 30-second epoch fixes Δf, which in turn sets how finely the band-power features can be defined. The spectrogram of a spindle-containing epoch is the visual that motivates every frequency-domain feature the classifier later uses.

3.16 Summary & key formulas

- A transform is an invertible change of representation; Fourier chooses sinusoids, ideal for rhythmic structure.
- Fourier series: periodic signals are sums of harmonics; the Gibbs overshoot at disconti-

varkonyibarnabas@gmail.com

nuities is ~9% and never vanishes.

- Fourier transform: $X(f) = \int x(t) e^{-j2\pi ft} dt$; the convolution theorem $x * h \leftrightarrow X\,H$ is the key identity.
- DFT/FFT: $O(N^2) \to O(N \log N)$; enables fast (overlap-add/save) convolution and correlation.
- Bin spacing vs. resolution: $\Delta f = f_s/N = 1/T_{\text{record}}$ is the DFT bin spacing; the *practical* resolution also depends on the window main-lobe width, SNR, and leakage. Zero-padding interpolates the display, it does not resolve.
- STFT/spectrogram: local spectra in time; the uncertainty trade-off $\Delta t \cdot \Delta f \gtrsim 1/(4\pi)$.
- Inverse transform: the DFT is invertible and lossless; $x[n] = \frac{1}{N} \sum_k X[k]\, e^{+j2\pi kn/N}$ (2-D analogous). The inverse FFT ($O(N \log N)$) is what lets us modify a spectrum and transform back.
- 2-D FFT: spatial frequency; low = smooth, high = edges; the complex spectrum splits into magnitude (how much) and phase (where) — phase carries most of the spatial structure; filtering and fast convolution carry over.

3.17 Conceptual questions & exploration tasks

1. A recording is $T = 2$ s long at $f_s = 256$ Hz. What is Δf? What is the shortest record that could resolve two EEG tones 0.5 Hz apart?
2. You zero-pad a 1-second record to 10 seconds of FFT length. Does the spectrum now resolve tones 0.2 Hz apart? Explain in one sentence why or why not.
3. Using the convolution theorem, explain why applying a filter as an FFT-multiply-IFFT can be far cheaper than direct convolution for a long filter.
4. A colleague sets a very long STFT window to "get better frequency resolution" but then cannot locate a spindle in time. Explain the trade-off they hit and how to rebalance it.
5. *Exploration.* In the notebook, take the 2-D FFT of an image, zero out the peripheral (high-frequency) coefficients, and inverse-transform. Describe the result and connect it to low-pass filtering.
6. *Transfer.* You move from EEG rhythms to surface EMG, whose useful content is broadband (≈20–450 Hz) rather than a narrow line. Does the $\Delta f = f_s/N$ resolution argument still drive your window-length choice the same way? What changes when the feature you want is a *band's total power* rather than a *peak frequency*?

3.18 Further reading

Bracewell's *The Fourier Transform and Its Applications* is the classic, intuition-first treatment. For the discrete and computational side — DFT, FFT, overlap-add — Oppenheim and Schafer's *Discrete-Time Signal Processing* is definitive. Gonzalez and Woods' *Digital Image Processing* covers the 2-D Fourier transform and frequency-domain image filtering in exactly the paired spirit of this chapter, and Cohen's *Time-Frequency Analysis* is the reference for the STFT, the uncertainty principle, and what lies beyond the spectrogram.

varkonyibarnabas@gmail.com

Chapter 4 — Windowing & Spectral Leakage

Why does a perfectly clean sinusoid smear across neighbouring frequencies when you take its spectrum — and which window fixes the smear without hiding the sleep spindle you were looking for? Chapter 3 drew spectra as if the DFT told the whole truth. It does not, because every real record is *finite*, and cutting a signal to a finite length is itself an operation with spectral consequences. This chapter is about that operation — windowing — and the artifact it controls, spectral leakage. It is short, but it decides whether your spectral features are trustworthy.

4.1 Learning objectives

By the end of this chapter you should be able to:

- Explain the finite-length problem and why it causes spectral leakage, using the convolution-with-the-window-spectrum view (L1, L3).
- Describe the common window functions — rectangular, Hann, Hamming, Blackman, Kaiser, Tukey — and their shapes (L3).
- Define and compare window properties: main-lobe width, side-lobe level, coherent gain, equivalent noise bandwidth, and scalloping loss (L3).
- Quantify the resolution-leakage trade-off and predict which window resolves or reveals a given pair of tones (L3).
- Choose an appropriate window for ECG, EEG, and EMG analysis and justify the choice (L3).

4.2 The finite-length problem and the origin of leakage

A true sinusoid runs from $-\infty$ to $+\infty$; its spectrum is an infinitely sharp spike. But we only ever observe a finite chunk. Taking a finite chunk is mathematically the same as multiplying the infinite signal by a window function $w[n]$ that is 1 inside the observation interval and 0 outside — a rectangular window. And here the convolution theorem (Chapter 3) turns against us: multiplication in time is convolution in frequency. So the observed spectrum is not the true spectrum but the true spectrum *convolved with the window's own spectrum*:

$$x[n]\,w[n] \;\longleftrightarrow\; X(f) * W(f).$$

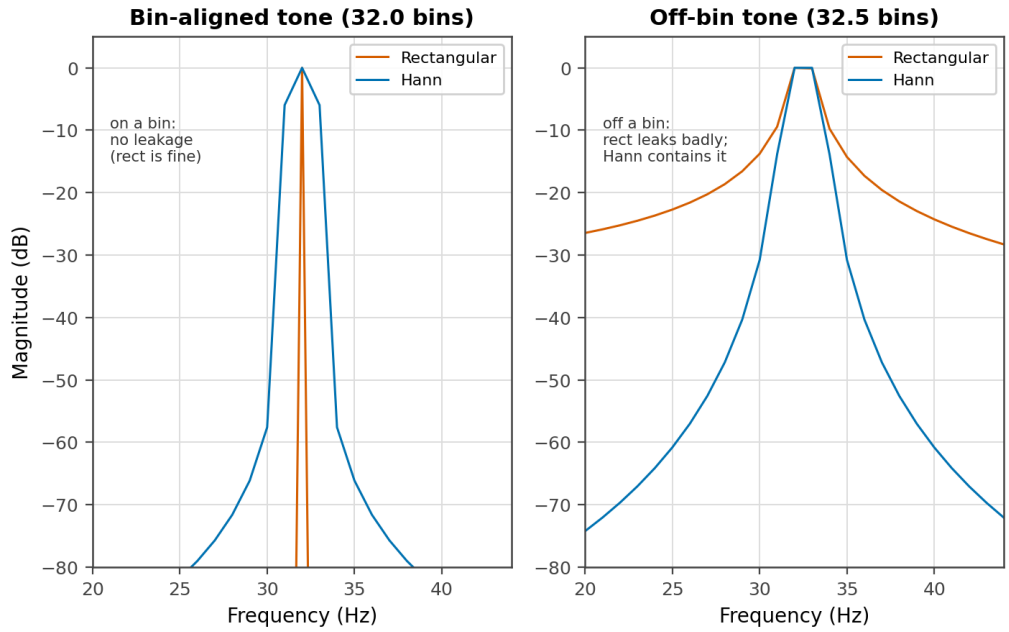
The rectangular window's spectrum is a sinc function: a narrow main lobe flanked by slowly decaying side lobes. Convolving the true spike with that sinc smears the spike into a main lobe surrounded by ripples — energy that leaks from the true frequency into neighbouring bins. That is spectral leakage, and it is not noise; it is the deterministic consequence of looking at a signal for only a finite time. This is precisely the *window ↔ convolve* entry from the dual-pairs dictionary of §3.12 — the mirror image of the convolution theorem — now working against us:

varkonyibarnabas@gmail.com

what makes fast filtering possible (multiply in one domain, convolve in the other) is the very mechanism that smears a finite spectrum.

Leakage has a subtlety worth pinning down, and it is easy to over-state. Suppose we use a plain rectangular window and the tone's frequency falls exactly on a DFT bin — an integer number of cycles fits the record. Then the rectangular window's sinc has its zeros land on all the other bins, and the DFT shows a single clean line with *no visible leakage into the other bins*. Shift the same tone half a bin off, and the sinc's side lobes are now sampled at their non-zero parts, so energy appears in every bin — leakage becomes obvious. Figure 4.1 makes this contrast dramatic.

It is important to read this correctly: the "no leakage on-bin" statement holds only for the rectangular window, sampled at the DFT bins, under these idealised conditions. A tapered window (Hann, Hamming, Blackman, ...) does *not* give a single clean line even for an on-bin tone, because its own spectrum has a wider main lobe than the rectangular sinc. A tapered window still spreads a tone's energy across neighbouring bins according to *its own* spectrum; what it buys you is far lower and faster-decaying side lobes, so the energy that spreads stays local instead of contaminating the whole axis. The rectangular window is not "leakage-free" in any general sense — it is merely the special case whose sinc zeros happen to fall on the other bins when a tone is perfectly on-bin.



Leakage appears whenever a tone falls between DFT bins — and, for a tapered window, even when it sits exactly on one.

Figure 4.1. Leakage depends on bin alignment. Left: a tone sitting exactly on a bin (32.0 bins) shows no visible leakage under a rectangular window — its sinc zeros fall on the other bins. Right: the same tone shifted half a bin (32.5 bins) leaks badly under a rectangular window, but a Hann window keeps the smear local. The on-bin "no leakage" case is a rectangular-window special case; a tapered window spreads energy per its own (wider) spectrum but with far lower side lobes.

It helps to separate three related but distinct effects, because they are often lumped together as "leakage." (i) Finite-window (side-lobe) leakage is energy from a tone spilling into distant bins through the window's side lobes — the effect that fills the whole spectrum when

varkonyibarnabas@gmail.com

a rectangular-windowed tone sits between bins. (ii) Bin misalignment, or picket-fence / scalloping loss, is the *amplitude* error suffered by a tone that falls between the DFT bins: because the DFT samples the continuous window spectrum only at the bin frequencies, an off-bin peak is read off the *shoulder* of its own main lobe rather than the top, so its measured amplitude is too low (§4.4). (iii) Main-lobe spreading is the intrinsic broadening of a single spectral line into a lobe of finite width set by the window; a tapered window trades a *wider* main lobe (this effect) for lower side lobes (less of effect i). These three are governed by the same window spectrum but answer different questions — how far energy spreads, how much peak amplitude you lose, and how wide a single line becomes.

Because a real biosignal contains a *continuum* of frequencies, essentially none of which land exactly on bins, some leakage is present in practice for any window. The usual remedy is to replace the abrupt rectangular cut with a tapered window that falls smoothly to zero at both ends, removing the sharp discontinuity that generated the tall side lobes in the first place — accepting a wider main lobe in exchange.

4.3 Window functions

A window is just a shape multiplied onto the record. The important ones, in order of increasing taper:

- Rectangular — no taper. Narrowest possible main lobe (best raw resolution) but the highest side lobes (−13 dB), so the worst leakage.
- Hann — a raised cosine falling to zero at both ends. Side lobes drop to −31 dB and roll off quickly; a superb general-purpose default.
- Hamming — a raised cosine that does *not* quite reach zero. Its first side lobe is lower (−43 dB) but the far side lobes roll off more slowly than Hann's.
- Blackman — a three-term cosine with very low side lobes (−58 dB) at the cost of a wider main lobe; excellent when you must see a weak component next to a strong one.
- Kaiser — a parameterised family (controlled by β) that tunes continuously between narrow-main-lobe and low-side-lobe behaviour; the engineer's adjustable window.
- Tukey — a flat top with cosine-tapered edges, a tunable compromise between rectangular and Hann.

Figure 4.2 shows the five shapes in time and, decisively, their magnitude spectra. The pattern is unmistakable: the more you taper the ends, the wider the main lobe grows and the lower the side lobes fall.

4.4 Window properties, quantified

Four numbers characterise a window and let you predict its behaviour:

- Main-lobe width sets the frequency resolution — how close two equal-strength tones can be and still appear as two peaks. Narrower is better *for resolution*. The rectangular window has the narrowest main lobe; heavily tapered windows have the widest.
- Side-lobe level sets the leakage floor — how far down the spurious energy sits, and

varkonyibarnabas@gmail.com

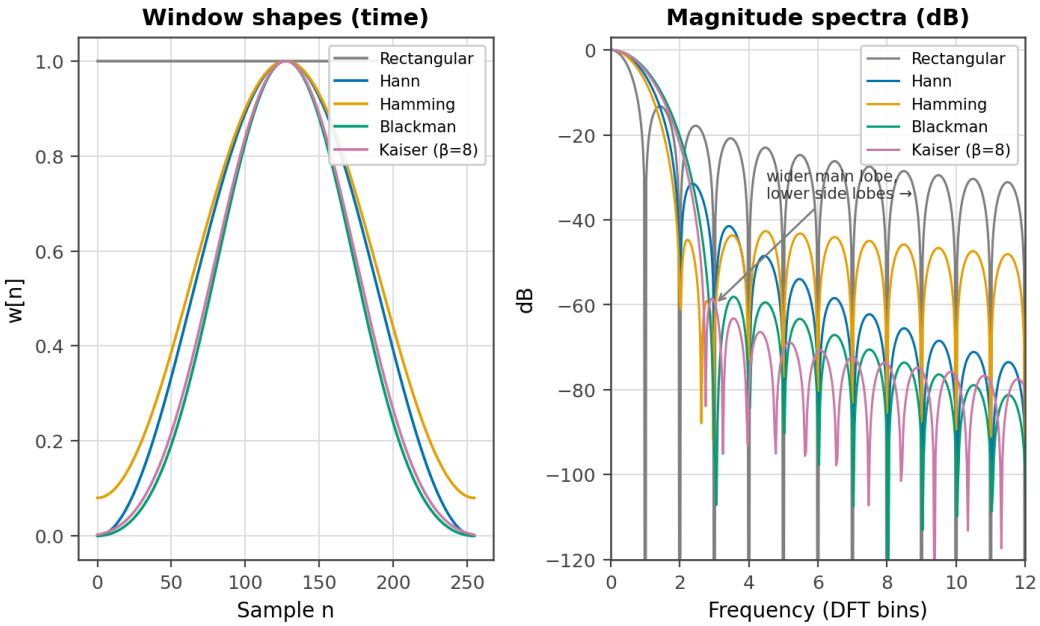


Figure 4.2. Five windows in time (left) and their magnitude spectra in dB (right). Tapering the window's ends trades a wider main lobe — worse frequency resolution — for far lower side lobes, which means much less side-lobe leakage. This single trade-off is at the heart of window design.

therefore how weak a neighbouring component can be before leakage from a strong one buries it. Lower is better *for dynamic range*.

- Coherent gain is the window's average value; it scales the measured amplitude of an on-bin tone and should be corrected for when you want true amplitudes.
- Equivalent noise bandwidth (ENBW) is the width of an ideal rectangular filter passing the same noise power as the window; it governs how the window affects the noise floor in a power-spectrum estimate.

A fifth effect, scalloping loss, is the amplitude dip suffered by a tone that falls *between* bins (up to about 3.9 dB for a rectangular window). It is worst for windows with narrow main lobes and is the amplitude-domain face of the same bin-alignment issue that caused leakage.

The table collects the headline numbers — the cheat-sheet you will actually consult.

Window	Peak side lobe (dB)	Main-lobe width (bins)	Best at	Weak at
Rectangular	−13	2	Resolving equal, close tones	Everything else (huge leakage)
Hann	−31	4	General-purpose default	Very weak neighbours
Hamming	−43	4	Low first side lobe	Slow far side-lobe roll-off
Blackman	−58	6	Weak tone near a strong one	Resolving close equal tones

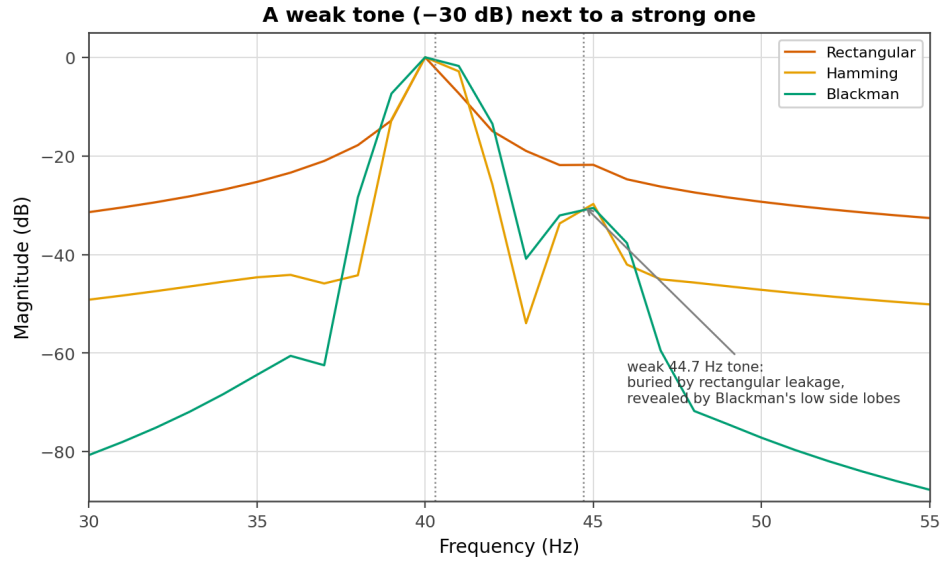
varkonyibarnabas@gmail.com

Window	Peak side lobe (dB)	Main-lobe width (bins)	Best at	Weak at
Kaiser (tunable)	set by β	set by β	Custom trade-offs	Needs a parameter choice

4.5 The resolution-leakage trade-off

There is no free window. The rectangular window gives the best possible *resolution* (narrowest main lobe) and the worst possible *dynamic range* (highest side lobes). Tapered windows reverse both. Which failure hurts depends on the question. If you must separate two tones of similar strength that sit close together, you want a narrow main lobe — lean toward rectangular or a low-β Kaiser. If you must detect a weak component beside a strong one, you want low side lobes — lean toward Blackman or a high-β Kaiser, accepting the wider main lobe.

Figure 4.3 stages the second scenario: a strong 40.5 Hz tone (deliberately off-bin, so its leakage is at its worst) with a weak 44 Hz neighbour 30 dB down (kept on-bin as a control, so any burying is the strong tone's leakage and not the weak tone's own spread). Under a rectangular window the strong tone's leakage buries the weak one entirely; under Blackman's low side lobes the weak tone stands clear. This is the trade-off made visible — and it is precisely the situation of trying to see a small spectral feature next to a dominant rhythm.



Low side lobes let a low-side-lobe window (Blackman) expose a weak neighbour that rectangular leakage would hide.

Figure 4.3. Two close tones, one weak. A strong 40.5 Hz tone (off-bin) sits next to a weak 44 Hz tone (on-bin) 30 dB below it. Rectangular-window leakage from the strong tone hides the weak one; Hamming does better; Blackman's very low side lobes expose it. Low side lobes reveal a weak neighbour that leakage would otherwise conceal.

Two threads from Chapter 3 tie in here. First, resolution is still bought with observation time — the main-lobe *width in hertz* is (window factor) $\times f_s/N$, so a longer record narrows every window's main lobe. Windowing shapes the trade-off; record length sets the scale. Second, leakage is Gibbs's cousin: both are the price of representing something with a finite, smooth-

varkonyibarnabas@gmail.com

edged basis.

> Worked example — a law and a number. Take a 30-second EEG epoch sampled at $f_s = 100$ Hz, so $N = 3000$ samples and the raw bin spacing is $\Delta f = f_s/N = 0.033$ Hz. The main-lobe *width in hertz* of each window is (main-lobe width in bins) $\times \Delta f$. For a rectangular window (2 bins) that is 0.067 Hz; for a Hann window (4 bins) it is 0.133 Hz; for Blackman (6 bins) it is 0.20 Hz. So switching from rectangular to Blackman triples the frequency you can no longer resolve — but it also drops the leakage floor from -13 dB to -58 dB, a 45 dB improvement in your ability to see a weak neighbour. Those two numbers, computed from the record length and the table in §4.4, let you *predict* before you compute whether a given window will resolve two features or bury one of them. That is the "law + number" you can reproduce in the companion notebook.

Amplitude correction is a related number worth carrying. Because a tapered window attenuates the signal, the peak in the spectrum of an on-bin tone is scaled by the window's coherent gain — about 0.5 for Hann and 0.54 for Hamming. To recover the true amplitude of a spectral peak you divide by the coherent gain; forgetting this correction makes every windowed amplitude look about half its true size, a common and avoidable error when band-power features are compared across windows.

4.6 Selection guidelines for ECG, EEG, and EMG

The right window follows from what each signal demands. For EEG spectral analysis — reading band power and hunting spindles — the Hann (or Hamming) window is the near-universal default: it suppresses leakage enough that a spindle's modest peak is not swamped by leakage from the dominant alpha or delta rhythm, while keeping resolution adequate for the 30-second epoch. When the goal is specifically to resolve a *weak* narrowband event beside a strong background, step up to Blackman or a high-β Kaiser. For ECG power spectra Hann again serves well; the R-peak comb is strong and well separated, so leakage control matters more than ultimate resolution. (Heart-rate *variability* is a different computation — the spectrum of the beat-to-beat R–R interval series, or *tachogram*, not of the ECG waveform itself; see §7.11.) For broadband EMG, where the interest is in overall spectral shape and median frequency rather than fine peaks, the exact window matters less, but Hann or Hamming remain sensible defaults that avoid rectangular leakage contaminating the band edges.

> Method: which window? Ask two questions. (1) *Are the components I care about close in frequency and similar in strength?* If yes, favour a narrow main lobe (rectangular / low-β Kaiser). (2) *Is a component I care about much weaker than a nearby one?* If yes, favour low side lobes (Blackman / high-β Kaiser). If neither dominates — the common biosignal case — Hann is the safe default. When in doubt, Hann.

varkonyibarnabas@gmail.com

Common mistakes. Three recur every time windowing is taught. First, believing that *windowing removes noise*. It does not: windowing controls leakage, a *deterministic* spreading of a signal's own energy across bins, not the *random* fluctuation we call noise. A window cannot reduce the variance of a raw spectrum — averaging (Chapter 7) does that; a window only reshapes where a component's energy lands. Second, picking a window without first stating the goal. There is no "best window" in the abstract: a narrow main lobe (rectangular) is what resolves two *close, equal-strength* tones, whereas low side lobes (Hann or Blackman) are what reveal a *weak tone sitting beside a strong one*. Choose the property the problem needs, then choose the window. Third, assuming a tapered window gives zero leakage for an on-bin tone. Only the rectangular window produces a single clean line, and only under ideal on-bin conditions where its sinc zeros fall on the other bins; any tapered window spreads a tone's energy through its own (wider) main lobe even when the tone sits exactly on a bin — what it buys you is far lower side lobes, not zero spreading.

4.7 The ML/AI bridge

Windowing quietly shapes almost every spectral feature a classifier ever sees. Power-spectral-density features are standardly computed with Welch's method, which averages the spectra of overlapping *windowed* segments; the window choice there directly sets the bias (leakage) and variance (via ENBW and overlap) of every band-power feature. A poorly chosen window leaks energy across band boundaries, blurring the very distinctions — delta vs. theta, alpha vs. spindle — that the features are meant to capture, and degrading the classifier before it ever trains. Getting the window right is therefore not cosmetic; it is feature-quality control at the source.

4.8 Companion notebook

The companion notebooks let you feel the trade-off. One plots the five windows and their spectra (Figure 4.2) and lets you sweep the Kaiser β to watch the main lobe widen as the side lobes fall. A second stages the leakage experiment (Figure 4.1): drag a tone off a bin and watch rectangular leakage bloom while Hann contains it. A third runs the biosignal windowing case — computing a Welch PSD of an EEG epoch under different windows and showing how spindle-band power estimates shift — which is exactly the feature-computation step of the case study.

4.9 Case-study tie-in

In the sleep-scoring case study, window choice is the fine print of the frequency-domain feature stage. Band-power features are computed by Welch's method on each 30-second epoch, and the window sets whether energy from the strong background rhythms leaks into the spindle band and inflates that feature spuriously. Choosing Hann for the Welch PSD — and understanding *why* — is a small decision that measurably affects the separability of N2 (spindle-rich) from neighbouring stages, and thus the accuracy of the final classifier.

varkonyibarnabas@gmail.com

4.10 Summary & key formulas

- A finite record = the true signal times a window; by the convolution theorem the observed spectrum is $X(f) * W(f)$, so some leakage is present for any finite record.
- Distinguish three effects: finite-window side-lobe leakage, picket-fence/scalloping (amplitude) loss from between-bin tones, and main-lobe spreading. The clean on-bin "no visible leakage" case holds only for a rectangular window; a tapered window (Hann, Hamming, Blackman, Kaiser) removes the edge discontinuity and suppresses side lobes, but spreads energy through its own wider main lobe.
- The core trade-off: narrow main lobe = better resolution but high side lobes (leakage); wide main lobe = worse resolution but low side lobes (dynamic range).
- Key window numbers: main-lobe width (resolution), side-lobe level (leakage/dynamic range), coherent gain (amplitude scaling), ENBW (noise), scalloping loss (between-bin amplitude dip).
- Main-lobe width in hertz $\approx$ (window factor) $\times f_s/N$ — resolution still scales with record length.
- Defaults: Hann/Hamming for EEG and ECG; Blackman or high-β Kaiser to expose a weak neighbour; rectangular only to resolve close, equal-strength tones.

4.11 Conceptual questions & exploration tasks

1. Explain, using the convolution-with-$W(f)$ view, why a tone exactly on a DFT bin shows no visible leakage under a rectangular window but a tone half a bin away leaks badly — and why a tapered window does not give a perfectly clean line even on-bin.
2. You must separate two equal-amplitude EEG tones 0.5 Hz apart in a 4-second epoch. Which window would you choose and why? Would a longer epoch help more than a different window?
3. A dominant alpha rhythm is leaking into your spindle-band power estimate and inflating it. Which window property do you need to improve, and which window would you switch to?
4. Define equivalent noise bandwidth and explain why it matters when you compare band-power features computed with different windows.
5. *Exploration.* In the notebook, place a weak tone 25 dB below a strong one and 3 Hz away. Sweep rectangular $\rightarrow$ Hann $\rightarrow$ Blackman and record the smallest weak-tone amplitude each window can still reveal. Tabulate the result against the side-lobe levels in §4.4.
6. *Transfer.* The leakage/window trade-off was taught on EEG spectra. You now compute heart-rate variability, whose LF and HF bands sit at 0.04–0.15 and 0.15–0.4 Hz on a ~5-minute RR tachogram. Which window would you pick, and why does the *record length* — not just the window shape — dominate the achievable band separation here in a way it did not for a 30 s EEG epoch?

Design brief. You must estimate the power of a weak sleep spindle (~13 Hz) sitting close to a strong alpha rhythm (~10 Hz) on a 30 s EEG epoch, and *separately* report the epoch's total delta-band power. Spec: what does each task need — resolution to separate 10 from 13 Hz, low side lobes so the strong alpha does not swamp the weak spindle, or a faithful total? Design: pick the window (rectangular / Hann / Hamming / Blackman / Kaiser) for each task and justify it against the side-lobe/main-lobe numbers of §4.4. Evaluate: which diagnostic tells

varkonyibarnabas@gmail.com

you the spindle estimate is real, not alpha leakage? Motivate: justify why the two tasks may want *different* windows. Alternatives: name a defensible different window for the spindle task and the spec change that would make it better. Rule out: name a window choice that would be *wrong* here — for instance a rectangular window whose side lobes bury the weak spindle under the strong alpha — and the requirement it breaks.

4.12 Further reading

The indispensable reference is Fredric J. Harris's 1978 paper "On the use of windows for harmonic analysis with the discrete Fourier transform," which tabulates every window property discussed here. Oppenheim and Schafer's *Discrete-Time Signal Processing* places windowing within DFT analysis and filter design, and Percival and Walden's *Spectral Analysis for Physical Applications* connects windowing to the Welch and multitaper spectral-estimation methods that turn these ideas into robust biosignal features.

varkonyibarnabas@gmail.com

Chapter 5 — Wavelets and Other Transforms

A physician glances at thirty seconds of a sleep recording and points to a single large, slow wave: "There — that's a K-complex." It lasts less than a second, it sits on top of ongoing background rhythms, and it is one of the fingerprints of stage N2 sleep. Now imagine asking a Fourier transform to find it. The Fourier spectrum will faithfully report that the epoch contains some low-frequency energy — but a single global spectrum, by construction, does not say when that energy arrived, and it effectively averages the sharp, one-off event across the whole record. For a local, transient event a global spectrum is often the wrong lens — not because Fourier is inferior, but because it answers a different question ("what frequencies, overall?") than the one we are asking ("what, and when?"). This chapter is about the transforms built to answer the second question.

5.1 Motivating question and the limits of Fourier

The Fourier transform is a magnificent instrument, and Chapters 3 and 4 earned it. But reading a *single, global* spectrum as a signal's summary quietly assumes the signal is stationary — its statistical and spectral character unchanging over the record — an assumption biosignals routinely violate. Real physiology is full of transients and regime changes — a K-complex, a burst of muscle activity, a heartbeat, an epileptic spike, an evoked response. For these, we do not want a single spectrum averaged over the whole recording; we want a time-frequency picture that says *which frequencies are present, and when.*

The short-time Fourier transform (STFT) of Chapter 3 (§3.9) was the first repair: slide a fixed window along the signal and take a Fourier transform inside each window. It works, but it forces a single compromise on you for the entire analysis. A short window pins down *time* well but blurs *frequency*; a long window sharpens *frequency* but smears *time*. This is not a shortcoming of any particular algorithm — it is the time-frequency uncertainty principle, a mathematical relative of the Heisenberg principle:

$$\Delta t \cdot \Delta f \geq \frac{1}{4\pi}.$$

You cannot make both boxes arbitrarily small. The STFT chooses one box size and uses it everywhere. The wavelet transform, the subject of the first half of this chapter, chooses a different box size at every frequency — small in time for fast events, wide in time for slow ones — which turns out to match how biological signals are actually organized.

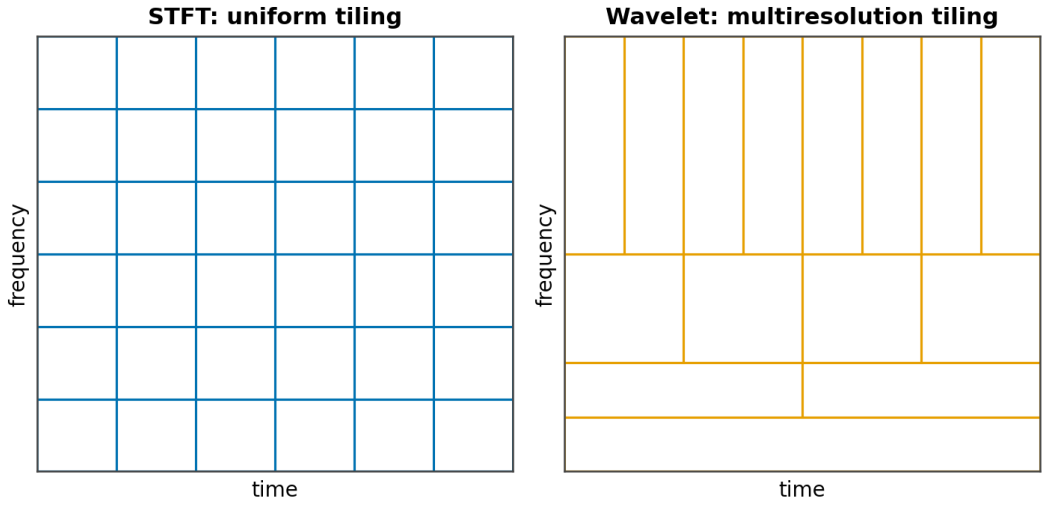
Learning objectives. After this chapter you should be able to (L1) explain *why* a single Fourier spectrum fails for transient, non-stationary biosignals; (L1) describe the continuous and discrete wavelet transforms and how decomposition levels map onto physiological frequency bands; (L1) explain multi-level (multiresolution) decomposition and the inverse wavelet transform (IDWT) / perfect reconstruction that lets you modify coefficients and rebuild the signal; (L3) apply that round trip to wavelet compression (keep-largest coefficients, as in JPEG2000) and wavelet denoising (coefficient thresholding); (L3) compute and interpret an analytic sig-

varkonyibarnabas@gmail.com

nal, its envelope, and its instantaneous frequency via the Hilbert transform; (L1) state what the discrete cosine transform, the z-transform, and empirical mode decomposition each buy you; and (L3) recognize the *same* multiresolution idea working in one dimension on a waveform and in two dimensions on an image, and use it to extract features.

5.2 Intuition first: one window size does not fit all

Picture the time-frequency plane as a floor you must tile with rectangular boxes. Each box says "I can localize energy to *this* stretch of time and *this* range of frequency." The uncertainty principle sets a floor under the area of every box — you cannot shrink below it, but you do choose its shape, and how close you cut it to that floor.



The STFT commits to one time-frequency resolution; the wavelet trades time for frequency detail scale by scale.

Figure 5.1. The STFT commits to one time-frequency resolution everywhere; the wavelet trades time resolution for frequency resolution scale by scale — tall-thin tiles for fast, high-frequency events, short-wide tiles for slow, low-frequency rhythms.

The STFT tiles the plane with identical rectangles (left panel of Figure 5.1). Whatever compromise you picked, you are stuck with it at 2 Hz and at 40 Hz alike. The wavelet transform tiles it adaptively (right panel), with frequency on the vertical axis and time on the horizontal: at high frequencies the boxes are tall and thin — narrow in time (excellent time resolution) but spanning a wide frequency band (coarse frequency resolution) — while at low frequencies they are short and wide — spanning a narrow frequency band (sharp frequency resolution) but a long stretch of time (coarse time resolution). Every tile has the same area — a consequence of how the wavelet family is built (dilating one prototype rather than sliding a fixed window) — and the uncertainty principle guarantees that area can never shrink below its floor; only the tile's shape changes with frequency. This suits much of physiology. Fast events (a spindle, a spike, a QRS complex) are brief and need precise timing; slow rhythms (delta waves, baseline drift) are sustained and need precise frequency. The wavelet's motto is *constant relative bandwidth*: each octave up the frequency axis gets twice as many tiles along the time axis.

It is worth being careful about the word "better" here. The wavelet transform is not universally superior to Fourier; the uncertainty principle puts the same floor under every tile's area in

varkonyibarnabas@gmail.com

both methods, so neither buys resolution the other cannot in principle. What differs is how each *spends* that fixed budget: the STFT commits to one tile shape everywhere, while the wavelet varies the shape with frequency. Where a signal's fast and slow structure matches the wavelet's adaptive tiling — as many biosignal transients do — the wavelet gives a more useful picture; where the signal is a steady, narrowband rhythm, a plain Fourier or STFT analysis can be clearer and easier to interpret. The right transform depends on the question, not on a ranking of methods.

5.3 The wavelet transform: CWT and DWT

A wavelet is a short, wiggly, zero-mean function — a "small wave" — that we stretch and slide across the signal, measuring the match at each position and scale. Formally, from a mother wavelet $\psi(t)$ we build a family by scaling (a) and translating (b):

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}}\,\psi\!\left(\frac{t-b}{a}\right).$$

The continuous wavelet transform (CWT) is the set of inner products of the signal with every member of this family:

$$W(a,b) = \int_{-\infty}^{\infty} x(t)\,\psi^{*}_{a,b}(t)\,dt.$$

Large scale a stretches the wavelet to catch slow features; small scale catches fast ones. The result $W(a,b)$ is a rich two-dimensional map (a *scalogram*) — beautiful for exploration but redundant and expensive.

A useful bridge back to Chapter 3 is the Gabor transform — nothing more than the short-time Fourier transform with a Gaussian window. Its atoms are Gaussian-modulated sinusoids, and among all windows the Gaussian achieves the theoretical minimum time–frequency spread (the Heisenberg–Gabor limit, §3.10), so the Gabor transform is the "best-localized" *fixed*-window analysis. The link to wavelets is direct: the Morlet wavelet used in the examples below is essentially a Gabor atom (a Gaussian-windowed complex sinusoid) turned into a *scalable* wavelet, which is why a Morlet-CWT scalogram and a Gabor spectrogram of the same spindle look so alike. The difference is exactly the one this chapter is about — the Gabor/STFT window is a fixed width at every frequency, while the wavelet's window *scales* with frequency.

In practice we use the discrete wavelet transform (DWT), which samples scale in powers of two (a *dyadic* grid) and is computed by a strikingly cheap recursive scheme. At each level the signal is passed through a matched pair of filters — a low-pass filter producing an approximation (the smooth, low-frequency content) and a high-pass filter producing a detail (the fast content) — and then downsampled by two. The approximation is fed back in and split again, so each level halves the frequency range. The whole cascade costs only $O(N)$ operations, faster even than the FFT. This is Mallat's algorithm, the *fast wavelet transform*.

The gift for biosignals is that the DWT levels line up naturally with physiological bands. For an

varkonyibarnabas@gmail.com

EEG sampled at 100 Hz, the Nyquist frequency is 50 Hz, so level-1 detail (D1) covers roughly 25–50 Hz, D2 roughly 12–25 Hz (spindle/beta territory), D3 roughly 6–12 Hz (alpha), D4 roughly 3–6 Hz (theta), and the level-4 approximation (A4) holds the slow delta content below about 3 Hz. Decompose an epoch and you have handed yourself a set of band-limited signals whose energies are ready-made features. One caveat, though: this dyadic alignment is only approximate, and a band that straddles a level boundary does not load cleanly onto a single level. The sleep-spindle/sigma band (11–16 Hz; §1.4) is the textbook case — at $f_s = 100$ Hz it spans the D2/D3 boundary (D2 $\approx$ 12.5–25 Hz, D3 $\approx$ 6.25–12.5 Hz), so spindle-band energy splits across D2 and D3. DWT band features for spindles should therefore pool the relevant levels rather than read a single one.

The standard DWT always splits the *approximation* (low-pass) branch and leaves the detail branches whole — which is why its bands are the coarse dyadic ladder just described. The wavelet packet transform (WPT) lifts that restriction: it splits *both* the approximation and the detail at every level, producing a full binary tree of sub-bands and hence a finer, more uniform frequency partition than the DWT's logarithmic one. That extra freedom lets an algorithm choose a best basis — the subset of packet nodes that most compactly represents, or most cleanly separates, a given signal, picked by an entropy or class-separability criterion. For EEG the payoff is concrete: a spindle band that straddled a DWT level boundary can be captured by a *single* packet node instead of pooled across two, and the packet energies across the tree make a richer feature set than the plain DWT levels. The cost is more coefficients to compute and — if the best basis is chosen using the class labels — the same leakage discipline the later chapters insist on (fit the basis inside the training fold).

Choosing a mother wavelet. There is no single right answer; you match the wavelet's shape to the events you care about. Daubechies wavelets (e.g., `db4`) are compact and asymmetric — a workhorse for transient detection. Symlets are near-symmetric Daubechies variants that reduce phase distortion. Coiflets add vanishing moments for smoother approximations. The Morlet wavelet, a Gaussian-windowed sinusoid, is the favourite for CWT time-frequency plots because it looks like an oscillation. As a rule of thumb, pick a wavelet that *resembles* the feature you are hunting; a K-complex, being an asymmetric slow deflection, is caught well by a Daubechies wavelet.

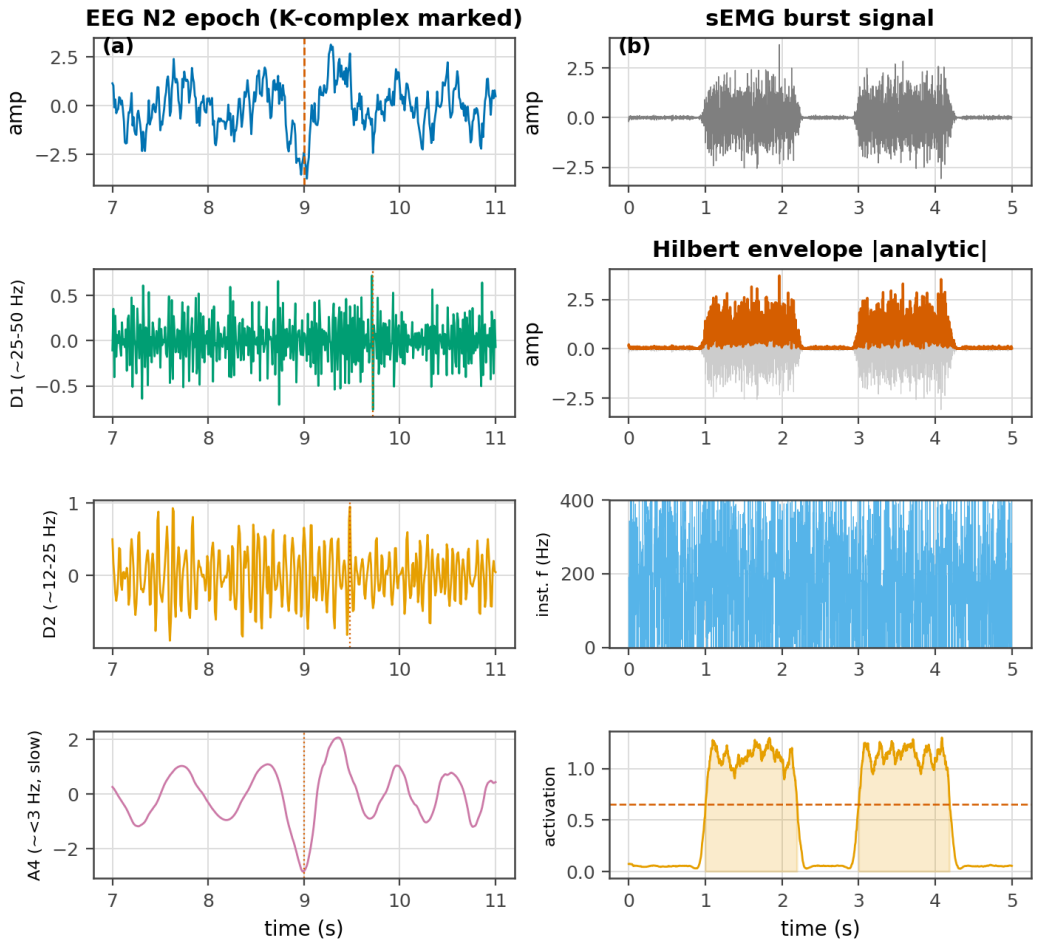
5.4 Worked example: localizing a K-complex, and an EMG envelope

Figure 5.2 puts the mechanism to work on two named signals. On the left, a 4-second window of a stage-N2 EEG epoch containing a K-complex is decomposed with `db4` to four levels. The raw epoch (top) shows the large slow deflection; the detail bands D1 and D2 stay quiet through it — the K-complex has almost no fast content — while the approximation A4 carries a clear, time-localized bump exactly where the K-complex sits. This is the payoff the Fourier transform could not give: the DWT tells us *the slow event happened here, at this second.*

A law and a number you can reproduce. Each DWT level halves the frequency span. With sampling rate $f_s = 100$ Hz, level j spans approximately $[\, f_s/2^{j+1},\ f_s/2^{j} \,]$. So D1 $\approx [25, 50]$ Hz, D2 $\approx [12.5, 25]$ Hz, D3 $\approx [6.25, 12.5]$ Hz, D4 $\approx [3.1, 6.25]$ Hz, and A4 $\approx [0, 3.1]$ Hz. A K-complex, with dominant energy below 3 Hz, must land in A4 — and it does. This is a design principle you can check against your own data with two lines of code.

The right column shows the Hilbert transform doing a different job on a different signal, which

varkonyibarnabas@gmail.com



One mechanism, two jobs: the DWT localizes the K-complex in the slow band while the Hilbert envelope turns an EMG burst into an activation trace.

Figure 5.2. One mechanism, two jobs. Left: a 1-D DWT of an N2 epoch pushes the K-complex into the slow approximation band and localizes it in time. Right: the Hilbert-transform analytic envelope turns a raw sEMG burst into a smooth activation trace with instantaneous frequency.

varkonyibarnabas@gmail.com

we develop next.

5.5 The Hilbert transform, the analytic signal, and the envelope

Some questions are not "which frequencies?" but "how strong is the activity right now, and how fast is it oscillating?" For a muscle EMG burst, we want the moment-to-moment amplitude envelope — a proxy for how hard the muscle is contracting — and perhaps the instantaneous frequency, which drops as the muscle fatigues.

The Hilbert transform builds these. Given a real signal $x(t)$, its Hilbert transform $\hat{x}(t)$ is a version with every *positive*-frequency component phase-shifted by -90° (and negative frequencies by $+90^\circ$). Pairing them gives the analytic signal:

$$z(t) = x(t) + j\,\hat{x}(t) = A(t)\,e^{j\phi(t)}.$$

The magnitude $A(t) = |z(t)|$ is the instantaneous envelope; the derivative of the phase gives the instantaneous frequency, $f(t) = \frac{1}{2\pi}\frac{d\phi}{dt}$. In the right column of Figure 5.2, `scipy.signal.hilbert` is applied to a broadband sEMG burst. The raw signal is a jittery mess; the envelope $A(t)$ (red) rides cleanly over its peaks, and a simple threshold on the smoothed envelope turns it into an activation detector — a binary "muscle on / muscle off" trace. That envelope is a feature you can hand to a classifier verbatim. The instantaneous-frequency panel shows the same idea for the frequency axis, though for broadband noise it is understandably jumpy; it becomes genuinely useful for narrowband signals such as a chirp or a fatiguing sustained contraction.

5.6 Two more transforms: DCT and the z-transform

The discrete cosine transform (DCT) is a cousin of the DFT that uses only cosines and assumes an even, real extension of the signal. Its superpower is energy compaction: for smooth signals it packs almost all the energy into a handful of low-order coefficients. This is why the DCT is the mathematical engine inside JPEG image compression (applied to 8×8 blocks) and many audio codecs, and why it doubles as a compact feature extractor — keep the first few DCT coefficients of an ECG beat and you have described its shape in a low-dimensional vector. The block-DCT versus full-image-DWT contrast is worth holding onto: JPEG tiles the image and takes a DCT of each block, while its successor JPEG2000 takes a single wavelet transform of the whole image (§5.8), trading DCT block artifacts for smoother wavelet compression.

The z-transform is the discrete-time analogue of the Laplace transform and the theoretical foundation for the digital filters of Chapter 9. It maps a sequence $x[n]$ to a function of the complex variable z:

$$X(z) = \sum_{n=-\infty}^{\infty} x[n]\,z^{-n}.$$

Its value is that a linear time-invariant system becomes a ratio of polynomials in z, whose roots — the zeros (numerator) and poles (denominator) — govern everything about the system's

varkonyibarnabas@gmail.com

behaviour. A pole near the unit circle produces a resonant peak; a causal system is stable if and only if all its poles lie *inside* the unit circle (more generally, the region of convergence must include the unit circle). When we design a notch filter to reject 50 Hz hum in Chapter 9, or an AR spectrum sprouts a sharp peak in Chapter 6, we are placing poles and zeros in the z-plane. The z-transform is the language that makes filter design and parametric spectral estimation the same subject.

A word on the Laplace transform. The z-transform's continuous-time counterpart maps a signal $x(t)$ to a function of the complex variable $s = \sigma + j\omega$: $X(s) = \int_{-\infty}^{\infty} x(t)\, e^{-st}\, dt$. It is the tool of *analog* circuit and filter design — the anti-alias front end sitting before the ADC in Chapter 2 is designed in the s-plane, not the z-plane — and its stability criterion is the mirror of the discrete-time one: a causal analog system is stable iff every pole lies in the left half-plane ($\mathrm{Re}(s) < 0$), where the z-transform's unit circle becomes the Laplace transform's imaginary axis. This book does not develop analog filter design further — by the time a biosignal reaches the methods in this chapter it has already been sampled — but the Laplace transform is why an analog Butterworth or Chebyshev prototype (Chapter 9) can be *digitized* into a discrete filter at all: the bilinear transform and impulse-invariance are the standard s-to-z maps that carry an s-plane design across to the z-plane.

5.7 Empirical mode decomposition and the Hilbert–Huang transform

Wavelets are powerful but still project the signal onto a fixed dictionary of pre-chosen shapes. Empirical mode decomposition (EMD) takes a radically data-driven route: it peels the signal into a small set of intrinsic mode functions (IMFs) derived from the signal's own oscillations, with no basis chosen in advance. Each IMF is a locally simple oscillation; applying the Hilbert transform to each IMF yields the Hilbert–Huang transform, a time-frequency representation well suited to non-linear, non-stationary signals where wavelets and the STFT struggle. The cost is that EMD is empirical — it lacks the clean mathematical guarantees of the wavelet transform and can suffer mode mixing (one IMF blending disparate scales), which ensemble EMD mitigates by averaging decompositions of many noise-perturbed copies. It is therefore best reached for when the fixed-basis transforms visibly fail.

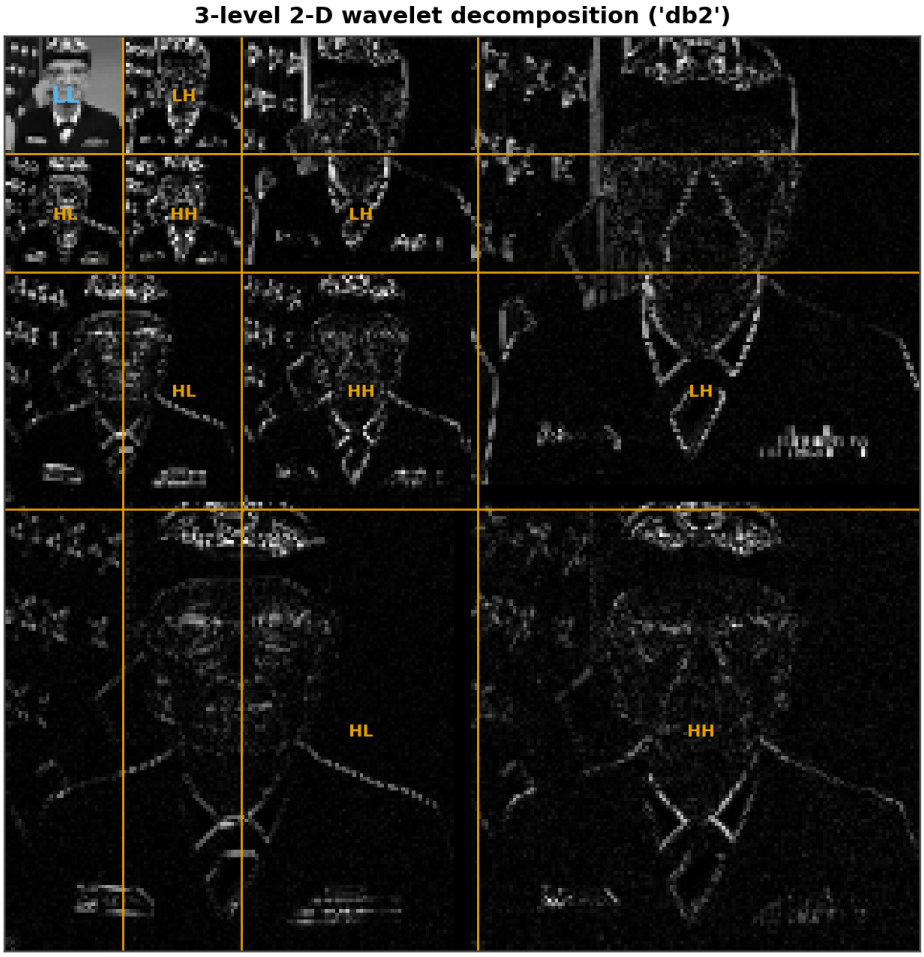
5.8 Signals-and-images box: one idea, two costumes

> The wavelet in 1-D and 2-D. The multiresolution idea is *modality-independent*: "approximation plus detail at multiple scales" is the same mechanism whether the input is a waveform or an image. In one dimension we split a signal into a smooth approximation and a detail; in two dimensions we apply the filter pair along rows *and* columns, producing four sub-images. The LL subband is a half-size blurred thumbnail (approximation); LH, HL, and HH hold horizontal, vertical, and diagonal detail — and detail, in an image, means *edges*.

Multi-level (multiresolution) decomposition. A single level splits the image once. But the LL approximation is itself an image, so we can feed it back in and split it again — and again. Repeating the split on each successive approximation gives a multi-level (or *multiresolution*) decomposition, laid out as the classic wavelet pyramid: a tiny coarse thumbnail in the top-left corner, ringed by ever-larger rings of LH/HL/HH detail as we move out to finer scales. This

varkonyibarnabas@gmail.com

is exactly Mallat's cascade of §5.3 run in two dimensions (`pywt.wavedec2`). Each level down halves the resolution and captures a coarser band of spatial frequency, so the pyramid literally *is* the image seen at several zoom levels at once — coarse structure in the centre, fine texture and sharp edges at the periphery.



Multi-level (multiresolution) 2-D wavelet transform: the tiny LL thumbnail at the top-left holds the coarse image, while LH/HL/HH detail bands at every scale light up on the photo's edges (horizontal, vertical, diagonal).

Figure 5.3. Multi-level (multiresolution) 2-D wavelet decomposition of a photographic image, arranged as the standard wavelet pyramid. The small LL thumbnail at the top-left is the coarse approximation; at every level the LH, HL, and HH detail subbands fire along the photo's horizontal, vertical, and diagonal edges. The same approximation-plus-detail idea repeats scale by scale — detail coefficients localize transients in a signal and edges in an image.

Figure 5.3 applies `pywt.wavedec2` to a photograph across three levels. Stare at the detail panels and the lesson lands instantly: the detail coefficients are near zero over the flat regions and *only* fire along the boundaries of features — precisely the edges — at every scale. This makes visually obvious what was abstract in the 1-D case: detail coefficients capture transients. In a signal, the "edge" is a K-complex or a spike; in an image, it is an anatomical boundary. This edge-catching behaviour is the seed of the two most important wavelet applications, compression and denoising, which we develop next.

varkonyibarnabas@gmail.com

Perfect reconstruction: the inverse wavelet transform. Like the Fourier transform, the DWT is invertible. Running Mallat's cascade backwards — upsampling and passing through the matched *synthesis* filters, level by level — is the inverse discrete wavelet transform (IDWT), `pywt.waverec` in 1-D and `pywt.waverec2` in 2-D. When the analysis and synthesis filters form an orthogonal (or biorthogonal) pair, the round trip is exact to numerical precision: decompose, then reconstruct, and you get the original signal back — a property called perfect reconstruction. This is what makes the wavelet transform *useful* rather than merely descriptive. Because we can go back, we are free to *modify the coefficients in between* — zero out the small ones to compress, shrink the noisy ones to denoise — and then inverse-transform to a clean, usable signal or image. The forward DWT analyses; the IDWT reconstructs; everything interesting happens to the coefficients in the middle.

Wavelet compression via energy compaction. The pyramid in Figure 5.3 is mostly black: the vast majority of detail coefficients are essentially zero, and a small minority — sitting on the edges — carry almost all the energy. This *energy compaction* is the basis of wavelet compression. Sort the coefficients by magnitude, keep only the largest handful, set the rest to zero, and inverse-transform. Because so few coefficients hold so much of the energy, the reconstruction stays faithful even after discarding the overwhelming majority. In this synthetic demo, Figure 5.4b keeps just the largest ~5% of the coefficients — a roughly 20:1 coefficient reduction — and the reconstructed photograph is barely distinguishable from the original (RMSE ≈ 0.04, illustratively). This is not a toy. JPEG2000, the successor to JPEG, replaces JPEG's block discrete cosine transform (DCT) with a full-image discrete wavelet transform — specifically the CDF 9/7 biorthogonal wavelet — precisely to exploit this energy compaction across the whole image, avoiding the blocky artifacts that JPEG's 8×8 DCT tiles produce at high compression. It is the best-known real-world wavelet application, and it is exactly the keep-the-largest-coefficients-and-invert idea running in production.

Wavelet denoising by coefficient shrinkage. The same edge-catching property gives a superb denoiser. Additive noise sprays small values across *all* the detail coefficients, whereas the real signal concentrates in a few large ones. So if we shrink every detail coefficient toward zero by a threshold — killing coefficients below it and softly pulling the rest down (*soft thresholding*) — we remove most of the noise while leaving the large, genuine edges nearly intact. Reconstructing with the IDWT then yields a cleaned image. Figure 5.4d applies soft thresholding to the noisy photograph of Figure 5.4c and, on this simulated example, cuts the RMSE roughly in half, visibly removing the grain while keeping the face and uniform edges sharp. This technique — wavelet thresholding or wavelet shrinkage, in classic forms such as VisuShrink and BayesShrink — works identically on 1-D biosignals: shrink the detail coefficients of a noisy ECG or EEG and reconstruct to suppress broadband noise without the edge-blurring a simple low-pass filter would cause. We return to it in the filtering chapter, but its logic is already complete here: *the transform separates signal from noise by magnitude, and the inverse transform puts the cleaned signal back together.*

5.9 The ML/AI bridge

Every transform in this chapter is, at heart, a feature factory. The DWT of an EEG epoch yields band-limited components whose energies (or relative energies, or entropies) become a compact feature vector — often just a handful of numbers per epoch that summarize its spectral shape far more robustly than raw samples. The Hilbert envelope of an EMG burst

varkonyibarnabas@gmail.com



Figure 5.4. Two things the invertible DWT buys you. (a) Original photograph. (b) Compression: keeping only the largest ~5% of wavelet coefficients (a ~20:1 coefficient reduction) and inverse-transforming reconstructs a near-perfect image (RMSE ≈ 0.04) — the energy-compaction principle behind JPEG2000. (c) A Gaussian-noise-corrupted version. (d) Wavelet denoising: soft-thresholding the detail coefficients and inverse-transforming roughly halves the error (RMSE 0.106 $\rightarrow$ 0.057) while preserving edges. Decompose, modify coefficients, reconstruct — the same recipe serves both jobs.

varkonyibarnabas@gmail.com

gives amplitude and duration features for activity detection. DCT coefficients compress a waveform or image patch into a few descriptive numbers. Because these transforms concentrate information, they let a downstream classifier learn from tens of features instead of thousands of raw samples — reducing overfitting and training cost. In the sleep-staging case study, DWT band-power features and the Hilbert EMG envelope are exactly the kind of hand-crafted, physiologically interpretable inputs that a model uses to separate one sleep stage from another.

5.10 Companion notebook

The companion notebook consolidates the four transforms in one place. It decomposes an EEG epoch with several mother wavelets so you can watch how the choice of wavelet changes what the detail bands capture; it computes a Hilbert envelope and instantaneous frequency on an EMG burst and lets you slide the activation threshold; it demonstrates DCT energy compaction by reconstructing an image from progressively fewer coefficients; it runs the multi-level 2-D `wavedec2` decomposition of Figure 5.3 and then the compress-and-denoise round trip of Figure 5.4, letting you dial the keep-fraction and the denoising threshold and watch the reconstruction error respond; and it runs the paired 1-D/2-D wavelet demonstration so the "one idea, two costumes" figures become something you can rebuild and perturb yourself.

5.11 Case-study tie-in

In an integrative sleep-staging case study, these transforms supply some of the most informative features. A DWT decomposition of each 30-second EEG epoch produces band powers that track the delta-heavy signature of deep sleep and the spindle activity of N2. The Hilbert envelope of a chin-EMG channel helps flag the muscle atonia of REM. Because the features are interpretable, when a model confuses two stages you can trace the confusion back to a band power or an envelope and reason about it — a diagnostic transparency that raw deep-learning inputs rarely offer.

5.12 Summary and key formulas

The Fourier transform gives one *global* spectrum with no time localisation (and its usual *stochastic* interpretation as a power spectrum assumes stationarity); biosignals are full of transients, so we need time-frequency and multiresolution tools. The wavelet transform tiles time-frequency adaptively — fine time at high frequency, fine frequency at low — and its DWT gives an $O(N)$ decomposition whose levels map to EEG bands. Repeating the split on each approximation gives a multi-level (multiresolution) decomposition, and because the transform is invertible (the IDWT gives perfect reconstruction), we can modify coefficients and rebuild the signal: keeping only the largest coefficients gives wavelet compression (the energy-compaction idea behind JPEG2000's CDF 9/7 wavelet, in contrast to JPEG's block DCT), and shrinking the detail coefficients gives wavelet denoising. The Hilbert transform builds the analytic signal, delivering an instantaneous envelope and frequency. The DCT compacts energy for compression and features; the z-transform underlies filters and parametric spectra; EMD adapts to non-linear, non-stationary signals. Above all, the

varkonyibarnabas@gmail.com

same approximation-plus-detail idea localizes transients in 1-D and edges in 2-D.

Transform	Best for	Time-frequency behaviour	Key output
STFT	mildly non-stationary signals	fixed resolution everywhere	spectrogram
CWT / DWT (wavelet)	transients, multiscale events	fine time at high f, fine f at low	scalogram / band components
Hilbert	envelope, instantaneous frequency	narrowband signals	$A(t)$, $f(t)$
DCT	compression, compact features	frequency only (real cosines)	energy-compacted coefficients
z-transform	filter/system design, stability	analytic (poles/zeros)	transfer function $H(z)$
Laplace transform	*analog* filter/system design, stability	analytic (poles/zeros)	transfer function $H(s)$
EMD / Hilbert–Huang	non-linear, non-stationary	data-driven, adaptive	intrinsic mode functions

Key formulas: wavelet family $\psi_{a,b}(t) = a^{-1/2}\psi((t-b)/a)$; analytic signal $z(t) = x(t) + j\hat{x}(t) = A(t)e^{j\phi(t)}$ with envelope $A(t) = |z(t)|$; DWT level-j band $\approx [f_s/2^{j+1}, f_s/2^j]$; z-transform $X(z) = \sum_n x[n]z^{-n}$ with stability (for a causal system) requiring all poles inside the unit circle; Laplace transform $X(s) = \int x(t)e^{-st}dt$ with stability requiring all poles in the left half-plane ($\mathrm{Re}(s) < 0$).

5.13 Conceptual questions and exploration tasks

1. Explain in one sentence why a global Fourier spectrum cannot tell you *when* a K-complex occurred, but a DWT can.
2. You sample EEG at 200 Hz and take a 5-level DWT. Which level's band contains the 11–16 Hz sleep spindle band (this book's canonical range, §1.4)? Compute the band edges.
3. Sketch the time-frequency tiling for a signal that is a slow drift for two seconds followed by a fast burst. Which transform — STFT or wavelet — represents it more efficiently, and why?
4. The Hilbert envelope of a broadband EMG burst is smooth, but its instantaneous frequency is jumpy. Why is instantaneous frequency more meaningful for a narrowband chirp than for broadband noise?
5. In the 2-D DWT of the photographic image (Figure 5.3), why are the detail subbands nearly black except along the edges of features? Relate this to wavelet denoising by coefficient thresholding.
6. A colleague proposes using the DCT of each ECG beat as a feature vector. What property of the DCT makes the first few coefficients enough, and when might this fail?
7. Wavelet compression keeps only the largest few percent of coefficients and inverse-transforms. Which property of the wavelet coefficients makes this lossy-but-faithful, and which transform step (forward or inverse) actually rebuilds the image?
8. *Exploration.* In the notebook, take a noisy signal, soft-threshold its detail coefficients, and reconstruct with the IDWT. How does raising the threshold trade noise removal against edge blurring, and why does JPEG2000 use a wavelet transform of the whole

varkonyibarnabas@gmail.com

image rather than JPEG's block DCT?

5.14 Further reading

For the definitive treatment of multiresolution analysis and the fast wavelet transform, see Mallat, *A Wavelet Tour of Signal Processing*. Daubechies' *Ten Lectures on Wavelets* is the classic source for compactly supported wavelets. For the analytic signal and instantaneous frequency, Boashash's work on time-frequency analysis is thorough. Huang et al.'s original paper introduces empirical mode decomposition and the Hilbert–Huang transform. For practical biomedical applications of wavelets, Addison's *The Illustrated Wavelet Transform Handbook* is unusually approachable and richly illustrated.

varkonyibarnabas@gmail.com

Chapter 6 — Stochastic Processes, Correlation and Time-Series Modelling

Play back ten seconds of resting EEG twice and you will not get the same waveform — not because the recording was faulty, but because the brain is not a metronome. Biosignals are partly random. Yet "random" does not mean "structureless": the intervals between heartbeats wander, but they wander within limits and with memory; EEG background looks like noise, but a noise with a preferred rhythm. This chapter gives us the vocabulary of controlled randomness — probability, stationarity, correlation — and then the models that turn that randomness into a handful of numbers a machine can learn from.

6.1 Motivating question and the random view of biosignals

Up to now we have treated signals as deterministic: given the equation, we know the value at every instant. But a resting-state EEG, the beat-to-beat variation of heart rate, and the background of every real recording are better described as realizations of a random (stochastic) process — one of infinitely many possible outcomes drawn from an underlying probability law. The engineering question is not "what is the exact next sample?" (unknowable) but "what are the *statistics* of this signal, and can I summarize them compactly?" Answering that unlocks two things at once: a principled way to describe noise (Chapter 8) and a way to build predictive time-series models whose parameters become features.

Learning objectives. You should be able to (L2) define a stochastic process, expectation, variance, and the crucial idea of stationarity and ergodicity; (L2, L3) compute and interpret the autocorrelation and autocovariance functions, and read the difference between a periodic signal and noise straight from the ACF; (L3) explain cross-correlation and coherence between two channels; (L2) state the Wiener–Khinchin theorem linking autocorrelation and power spectrum; (L3) describe AR, MA, and ARMA models and identify a suitable order from the ACF and PACF; and (L3) fit an AR model to a real epoch and read its spectral peak.

6.2 Random signals: expectation, variance, stationarity, ergodicity

A stochastic process $X[n]$ is a family of random variables indexed by time. At each instant n the value is drawn from a probability density; imagine an ensemble of many identical experiments, and at time n you have a whole distribution of possible values across the ensemble (Appendix E refreshes expectation, variance, and distributions if you want that foundation first). Its first two moments carry most of the practical information:

$$\mu_X[n] = E\{X[n]\}, \qquad \sigma_X^2[n] = E\{(X[n] - \mu_X[n])^2\}.$$

Here $E\{\cdot\}$ is the expectation — the average over the ensemble. In general these can drift with time, which would make analysis hopeless. Two assumptions rescue us.

varkonyibarnabas@gmail.com

Stationarity says the statistics do not change over time, and it comes in strengths worth distinguishing. A process is strict-sense (strictly) stationary if its *entire* probability structure — every joint distribution of any set of samples — is invariant to a shift in time. That is a very strong condition, and one we can almost never verify from data. A process is wide-sense stationary (WSS) (also called weakly stationary) under a far milder requirement: only its mean is constant and its autocorrelation depends only on the *lag* between two instants, not on their absolute position. Strict-sense stationarity implies WSS, but not the reverse; for Gaussian processes the two coincide, because a Gaussian process is fully described by its first two moments. WSS is the workhorse assumption because it is weak enough to hold approximately over a short stretch and strong enough to make the mathematics — autocorrelation, power spectrum, AR modelling — tractable.

Biosignals are rarely stationary in either sense over minutes: sleep stages shift, muscles fatigue, attention wanders. What we lean on instead is local (quasi-) stationarity — the idea that the statistics change *slowly enough* that within a short window (say a 30-second epoch, or a few seconds of EEG) the signal is *approximately* WSS. This is why the whole analysis pipeline chops recordings into short epochs and treats each as stationary: we do not claim the biosignal is globally stationary, only that it is locally well-approximated as such. Choosing the window length is therefore a judgement about how fast the physiology drifts, and every stationary-process tool in this chapter is applied under that local, approximate, within-window assumption rather than as a literal property of the signal.

Ergodicity is the bridge from theory to a single recording. In real life we do not have an ensemble of a thousand parallel brains — we have *one* EEG trace. An ergodic process is one whose time averages (along that single trace) equal its ensemble averages. Ergodicity is what licenses us to estimate the mean and autocorrelation from one long recording, which is all we ever have. Whenever you compute a statistic from a single time series and treat it as *the* statistic of the process, you are quietly assuming ergodicity.

6.3 Autocorrelation: a memory meter

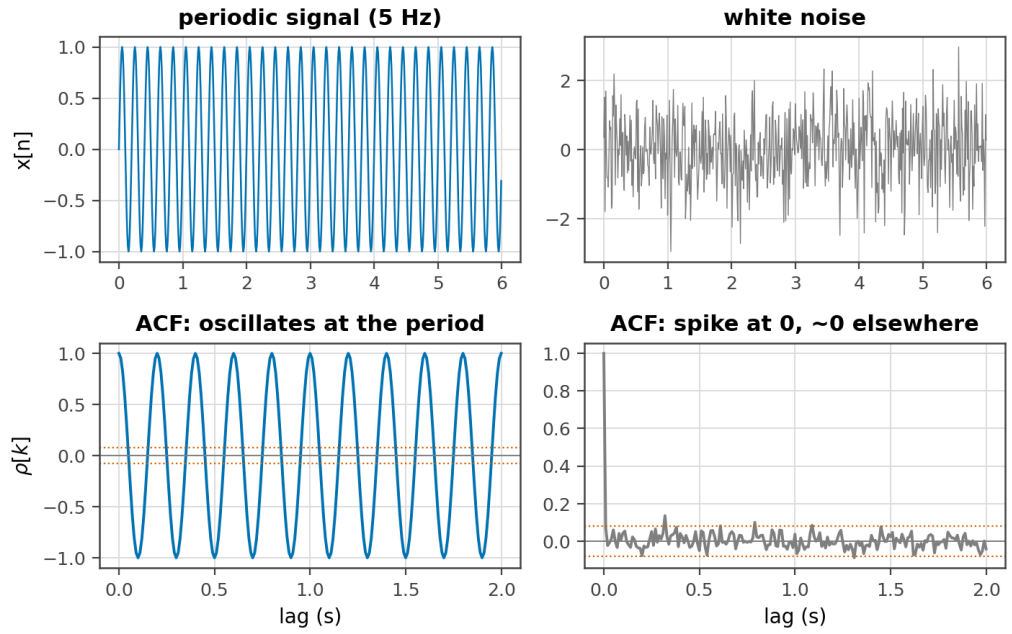
The single most useful descriptor of a random signal is its autocorrelation function (ACF) — a measure of how similar the signal is to a time-shifted copy of itself (Appendix D introduces correlation and covariance from scratch if that idea is new). For a WSS process,

$$R_X[k] = E\{X[n]\, X[n+k]\},$$

and the autocovariance subtracts the mean first. Normalizing by $R_X[0]$ (the total power) gives a correlation coefficient $\rho[k]$ between -1 and 1. Think of the ACF as a *memory meter*: $\rho[k]$ answers "knowing the signal now, how much do I know about it k steps later?"

Figure 6.1 contrasts the two extremes. A 5 Hz sinusoid (left) has an ACF that oscillates at the same 5 Hz and, in the ideal noise-free case, does not decay: a strictly periodic signal has long-range memory, and the ACF's period *equals the signal's period* — which is why the ACF is a useful and noise-robust *periodicity* detector. A word of caution, though: a strong recurring peak in the ACF is *evidence of* periodicity, not *proof of strict* periodicity. A quasi-periodic signal (a heartbeat whose interval wanders), a sum of two incommensurate tones, or a narrowband noise all produce oscillating ACFs with recurring peaks that gradually decay; only an ACF that

varkonyibarnabas@gmail.com



Autocorrelation is a memory meter: a periodic signal stays correlated at every period; white noise is correlated only with itself at lag 0.

Figure 6.1. Autocorrelation is a memory meter. A periodic signal (left) stays correlated with itself at every multiple of its period — the ACF oscillates and does not decay. White noise (right) is correlated only with itself at lag zero — a single spike, then nothing. Dotted lines are the 95% confidence band for "no correlation."

varkonyibarnabas@gmail.com

repeats undamped forever would indicate exact periodicity, and no finite real record can show that. Read the ACF peak as "there is a dominant rhythm near this period," not as "the signal is exactly periodic." White noise (right) has an ACF that is a single spike at lag zero and near zero everywhere else: to the second-order statistics each sample looks uncorrelated with every other, so the signal carries essentially no linear memory. Every real biosignal lives between these poles. An HRV series or an EEG epoch has an ACF that decays over a few lags — short-range memory — and it is precisely the *shape* of that decay that a time-series model captures.

6.4 Cross-correlation and coherence between channels

When we have two signals we can ask how one relates to the other. Cross-correlation $R_{XY}[k] = E\{X[n]\,Y[n+k]\}$ slides one against the other; its peak reveals the lag at which they best align — the basis of many time-delay estimators, from locating a source between two electrodes to synchronizing an EEG event with a stimulus. In the frequency domain a related quantity is coherence, a normalized cross-spectrum that reports, band by band, how linearly related two channels are (1 = perfectly linearly coupled at that frequency, 0 = linearly unrelated). Coherence between two EEG electrodes is a commonly used measure of functional connectivity; coherence between respiration and heart rate quantifies respiratory sinus arrhythmia.

> Dynamic time warping. Cross-correlation finds the single best *rigid* shift between two signals, but two recordings of the "same" process often differ by a *non-uniform* time distortion — two utterances of a word, two gait cycles, two heartbeats at different rates. Dynamic time warping (DTW) measures similarity under such elastic stretching: it finds the monotone alignment of the two time axes that minimizes the total distance between matched samples, computed efficiently by dynamic programming over a distance matrix. The resulting warping path is itself informative — it shows *where* one signal ran fast or slow relative to the other — and the minimum cost is a shift- and rate-tolerant similarity score. In biosignals, DTW aligns beat morphologies for template matching, compares gait or reaching trajectories across subjects, and serves as a distance for clustering or nearest-neighbour classification of variable-rate signals. Its caution mirrors correlation's: DTW is so flexible that it can align things that should not be aligned, so an unconstrained warp needs a band or slope constraint to stay physiologically sensible.

Coherence measures *linear* coupling at a single frequency, but the connectivity toolkit reaches further. Phase–amplitude coupling (PAC) — where the *phase* of a slow rhythm modulates the *amplitude* of a faster one (the classic theta-phase/gamma-amplitude coupling) — is a *cross-frequency* relationship ordinary coherence cannot see. Directed measures ask not just whether two channels are related but *which drives which*: Granger causality declares that channel X "Granger-causes" Y if X's past improves the prediction of Y beyond Y's own past, and its spectral relatives (partial directed coherence, the directed transfer function) localize that influence in frequency. These belong to the broader field of physiological connectivity, and they carry a health warning worth repeating: a statistical coupling is not a mechanistic connection, volume conduction can manufacture spurious coherence between nearby electrodes, and "causality" here is strictly *predictive*, not interventional.

These association measures are powerful and easy to over-read, so it is worth fixing, once, what each result does and does not license. The table is a claim-discipline reference for this

varkonyibarnabas@gmail.com

chapter and the connectivity methods it introduces.

Result	Supported interpretation	Common overreach
Cross-correlation peak	similar variation at a relative lag	proof that one signal *drives* the other
Coherence	frequency-specific *linear* association	direct anatomical connection or communication
Granger-style predictability	one channel's history improves prediction of another	biological *causation*
AR spectral peak	the fitted dynamics contain an oscillatory tendency	a unique physiological oscillator has been identified
DTW similarity	two sequences can be aligned under flexible timing	the timing differences are unimportant

The recurring trap is reading a *statistical* association as a *mechanistic* or *causal* one. Volume conduction alone can manufacture high coherence between two scalp electrodes that share a source; a common driver can make two signals mutually predictable without either causing the other. Establishing mechanism needs more than a number — it needs control of common sources, sensitivity analysis, and, where possible, a perturbation or an experimental design.

6.5 The Wiener–Khinchin theorem: correlation is spectrum

Here is the theorem that stitches this chapter to the next. The Wiener–Khinchin theorem states that, for a WSS process, the power spectral density is the Fourier transform of the autocorrelation function:

$$S_X(f) = \sum_{k=-\infty}^{\infty} R_X[k]\, e^{-j2\pi fk}.$$

Read it slowly, because it is one of the load-bearing ideas of the whole book. The autocorrelation (a time-domain description of memory) and the power spectrum (a frequency-domain description of energy) are two views of the same information. It explains why white noise — spike-only ACF — has a *flat* spectrum (all frequencies equally), and why a periodic signal — oscillating ACF — has a spectrum concentrated at a single line. Every spectral-estimation method in Chapter 7 is, under the hood, an attempt to estimate $R_X[k]$ well and transform it. And it is why an autoregressive model, which is a compact parametric description of the ACF, immediately yields a parametric spectrum.

6.6 Time-series models: AR, MA and ARMA

Instead of describing a signal sample by sample, we model the *rule* that generates it from past values and random shocks. Three canonical linear models cover an enormous range of biosignals.

An autoregressive (AR) model of order p predicts each sample as a weighted sum of its own recent past plus a white-noise innovation $w[n]$:

varkonyibarnabas@gmail.com

$$x[n] = \sum_{k=1}^{p} a_k \, x[n-k] + w[n].$$

This is an all-pole model — in the z-domain its transfer function is $1/A(z)$ with $A(z) = 1 - \sum_k a_k z^{-k}$ — so its spectrum is dominated by *peaks* wherever the poles sit near the unit circle. That makes AR the natural model for signals with resonant rhythms: EEG alpha, a formant, a vocal pitch.

A moving-average (MA) model of order q builds the signal from a weighted sum of past *innovations*: $x[n] = \sum_{k=0}^{q} b_k w[n-k]$. It is an all-zero model and its spectrum is dominated by *dips* (notches). The combined ARMA(p,q) model has both poles and zeros and can capture peaks and valleys together, at the cost of harder estimation.

> Two ways to model a physiological system: mechanism versus data. The AR, MA, and ARMA models are black-box (data-driven) models: we choose an order, fit the coefficients to the recording, and judge the model by how faithfully it reproduces the signal's statistics — but the coefficients a_k carry no physiological meaning of their own. You cannot read a cell's ion-channel conductance or an artery's compliance off an AR coefficient; there is in general no way to recover physiology from the fitted coefficients — though the *poles* of an AR model do carry a signal-level meaning, marking the spectral resonances (an alpha rhythm, a formant), which is a weaker thing than a physiological parameter. The opposite extreme is a mechanistic (white-box, physiological) model, written from first principles so that every parameter *is* a physical quantity — the Windkessel model of arterial blood pressure (an arterial compliance C and a peripheral resistance R), the Hodgkin–Huxley model of the action potential (membrane and ion-channel conductances), or a compartmental model of drug or gas kinetics. Because each parameter is meant to correspond to a physical quantity, a mechanistic model can be run forward — parameters in, a simulated signal out — and interpreted directly (though whether those parameters can be *uniquely* recovered from data is a separate question of identifiability, and many mechanistic models are only partly identifiable). Between the two extremes sit grey-box models, whose structure is dictated by physiology but whose parameters are still fitted to data. This is the very axis the machine-learning chapters return to: mechanistic models are the ultimate *knowledge-driven* end (few parameters, strong priors, fully interpretable), while black-box time-series models and deep networks are the *data-driven* end (many parameters, few priors, opaque). Going the other way — recovering parameters or sources *from* a recording — is the inverse problem, and it is almost always harder and more ill-posed than the forward one. That asymmetry is why so much of biosignal processing is careful bookkeeping about what can, and cannot, honestly be inferred from the data in hand.

6.7 Model identification: ACF, PACF and order selection

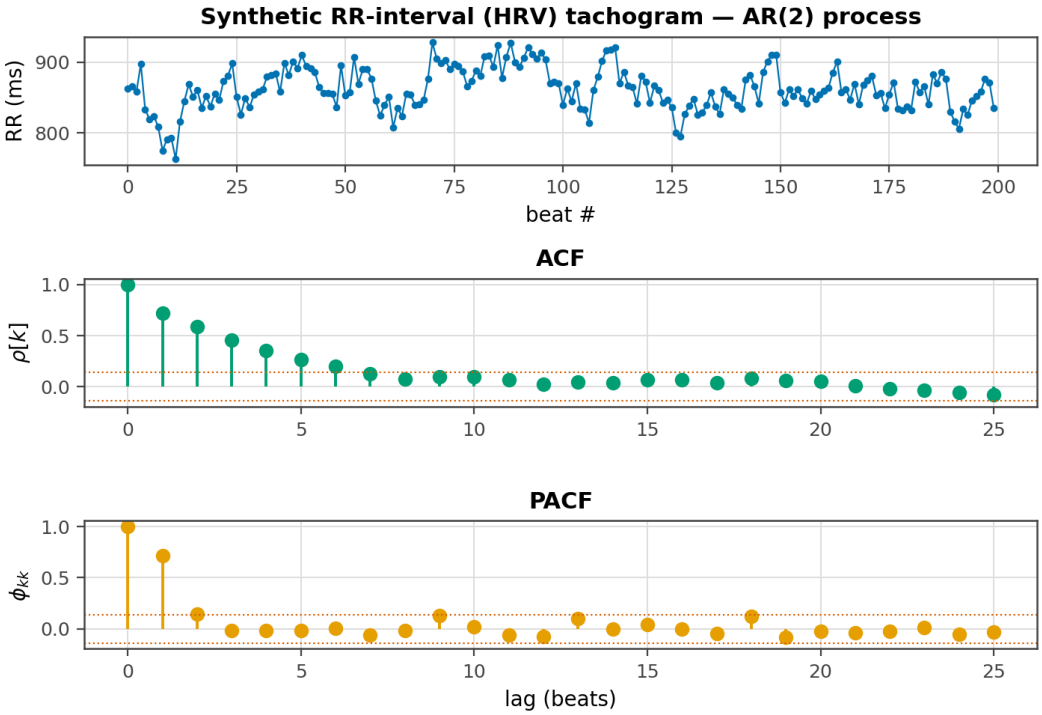
How do we choose between AR, MA, and ARMA, and what order? The classical answer reads two diagnostic plots. The ACF we already know. The partial autocorrelation function (PACF) measures the correlation between $x[n]$ and $x[n-k]$ *after removing* the influence of the intervening samples — the "direct" memory at lag k. The two behave differently for the two model families, giving a decision rule:

- A pure AR(p) process has a PACF that cuts off sharply after lag p (and an ACF that decays

varkonyibarnabas@gmail.com

gradually).

- A pure MA(q) process has an ACF that cuts off after lag q (and a PACF that decays gradually).
- An ARMA process has both tailing off.



ACF tails off while PACF cuts off after a few lags — the signature that guides AR/MA order choice for HRV.

Figure 6.2. Reading model structure from the plots. An HRV (RR-interval) series derived from ECG, with its ACF and PACF. The ACF tails off while the PACF cuts off after a few lags — the classic AR signature. Dotted lines mark the 95% significance band; spikes beyond them are "real."

Figure 6.2 walks through this on a heart-rate-variability (HRV) series. We detect R-peaks in a two-minute ECG, take the successive R-to-R intervals in milliseconds — the *tachogram* — and plot its ACF and PACF. The ACF decays gradually while the PACF shows a couple of significant spikes and then collapses inside the confidence band: the fingerprint of a low-order AR process. That is our cue to fit an AR model with a small order.

For a data-driven order choice we minimize an information criterion that trades goodness-of-fit against model complexity. The Akaike Information Criterion and the Minimum Description Length criterion both add a penalty that grows with the number of parameters p:

$$\mathrm{AIC}(p) = N \ln \hat{\sigma}_p^2 + 2p, \qquad \mathrm{MDL}(p) = N \ln \hat{\sigma}_p^2 + p \ln N,$$

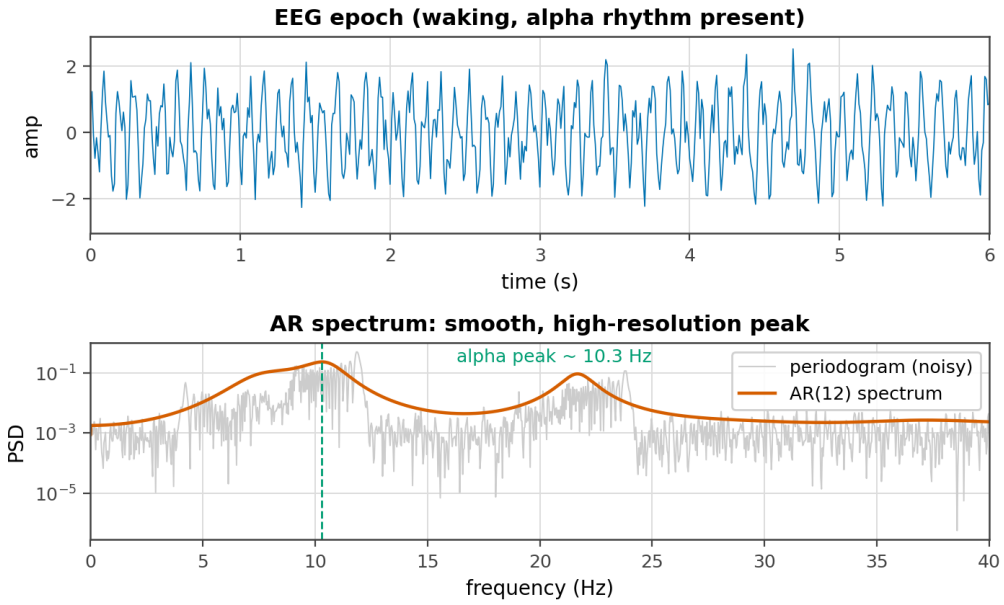
where $\hat{\sigma}_p^2$ is the residual (innovation) variance of the order-p fit. We compute the criterion over a range of orders and pick the minimum. MDL penalizes complexity more heavily than AIC and so tends to choose more parsimonious models — useful when you fear over-fitting.

varkonyibarnabas@gmail.com

Parameter estimation. Given an order, the AR coefficients follow from the Yule–Walker equations, a linear system built directly from the estimated autocorrelation — a clean consequence of Wiener–Khinchin. The Levinson–Durbin recursion solves that system in $O(p^2)$ and, as a bonus, produces the PACF values along the way. Maximum-likelihood and Burg's method (Chapter 7) are alternatives that behave better on short records.

6.8 Worked example: an AR spectrum with a real peak

Figure 6.3 fits an AR model to a synthetic waking-EEG epoch and reads its spectrum. We centre the epoch, solve the order-12 Yule–Walker equations, and evaluate the parametric spectrum $\hat{\sigma}^2/|A(f)|^2$.



An all-pole AR model turns a noisy epoch into a smooth spectrum with a sharp alpha peak — poles sit near the dominant rhythm.

Figure 6.3. An all-pole AR model turns a noisy epoch into a smooth spectrum with a sharp peak. The grey periodogram is the raw, noisy spectrum; the red AR(12) curve is smooth and resolves a clear alpha peak near 10–12 Hz, because the model's poles cluster near the dominant rhythm.

A law and a number. The raw periodogram (grey) is a jagged mess — Chapter 7 explains why its variance does not shrink as the record lengthens. On this synthetic example the AR(12) spectrum (red) is clean and shows a single dominant alpha peak near 10–12 Hz, illustratively landing in the band where waking EEG typically shows its alpha rhythm. The reason is structural: the model's poles migrate toward the unit circle at the frequency of the dominant rhythm, and a pole at radius r and angle $\theta = 2\pi f/f_s$ produces a spectral peak at frequency f whose sharpness grows as $r \to 1$. So the AR coefficients are not just numbers — they *encode the resonances of the signal*, and a handful of them reconstruct the whole spectrum. You can reproduce this: increase the order and the peak sharpens; drop it too low and the peak washes out (a preview of Chapter 7's order-selection lesson).

varkonyibarnabas@gmail.com

6.9 The ML/AI bridge

This is where randomness becomes a feature vector. A fitted AR(p) model compresses an entire epoch into just p coefficients (plus the innovation variance) that summarize its spectral shape — a dramatic dimensionality reduction from thousands of raw samples to a handful of interpretable numbers. Those coefficients, or the band powers read off the AR spectrum, are compact, low-variance inputs to a classifier. ACF-derived features (the lag of the first zero-crossing, the decay rate) quantify signal memory. Cross-correlation lags and coherence values become connectivity features when several channels are available. Because these features derive from an explicit generative model, they are interpretable: an AR coefficient that shifts tells you the resonance moved, not merely that "feature 37 changed."

6.10 Signals-and-images box

> Correlation in two dimensions. The autocorrelation idea extends directly to images: the 2-D autocorrelation measures how similar an image is to a spatially shifted copy of itself, and it reveals repeating texture and dominant orientation. Cross-correlation between two images is the engine of template matching and image registration — sliding a template across an image and finding the peak, exactly as the 1-D cross-correlation finds a time lag. The Wiener–Khinchin theorem holds here too: the 2-D power spectrum is the Fourier transform of the 2-D autocorrelation, which is why texture and periodicity in an image show up as structure in its 2-D spectrum.

6.11 Companion notebook

The companion notebook lets you build these ideas by hand. It computes ACFs for signals you dial between "pure periodic" and "pure noise" and watches the confidence band; it derives an HRV tachogram from a synthetic ECG and reads its ACF and PACF; it fits AR models of increasing order to an EEG epoch, overlays the AR spectrum on the periodogram, and plots AIC and MDL versus order so you can see the criterion find its minimum; and it demonstrates one-step prediction and residual whiteness checking so you can tell a good fit from a bad one.

6.12 Case-study tie-in

In the sleep-staging case study, AR coefficients and the band powers read from an AR spectrum are among the most compact and effective features per epoch. Because deep sleep is delta-dominated and waking/REM shift energy toward higher bands, the AR spectrum's peak location and the relative powers it implies separate stages cleanly. HRV-derived features (from the ECG channel) add an autonomic dimension that complements the EEG, and the ACF/PACF diagnostics keep the model orders honest so the features stay stable across recordings.

6.13 Summary and key formulas

Biosignals are partly random, so we describe them statistically. Stationarity comes in strengths — strict-sense (all joint distributions shift-invariant), wide-sense (constant mean,

varkonyibarnabas@gmail.com

lag-only autocorrelation), and, for real biosignals, only local/quasi-stationarity within a short window. Under (approximate, within-window) wide-sense stationarity and ergodicity, a single recording lets us estimate the mean and the autocorrelation, and the ACF acts as a memory meter that flags a dominant rhythm (oscillating, slowly-decaying ACF) versus noise (spike-only ACF) — evidence of periodicity, not proof of exact periodicity. The Wiener–Khinchin theorem makes autocorrelation and power spectrum two views of one truth. AR, MA, and ARMA models turn that memory into a compact generative rule; ACF/PACF plots and AIC/MDL identify the structure and order; Yule–Walker and Levinson–Durbin estimate the coefficients; and an AR model's poles produce a smooth spectrum whose peak marks the dominant rhythm.

Concept	Definition / rule	What it tells you
Autocorrelation $R_X[k]$	$E\{X[n]X[n+k]\}$	memory / periodicity
Wiener–Khinchin	$S_X(f) = \mathcal{F}\{R_X[k]\}$	ACF $\leftrightarrow$ PSD
AR(p)	$x[n] = \sum a_k x[n-k] + w[n]$	all-pole; spectral peaks
MA(q)	$x[n] = \sum b_k w[n-k]$	all-zero; spectral notches
ACF/PACF rule	AR: PACF cuts at p; MA: ACF cuts at q	model & order identification
AIC / MDL	$N \ln \hat{\sigma}_p^2 + \{2p,\ p \ln N\}$	order selection

6.14 Conceptual questions and exploration tasks

1. Distinguish strict-sense, wide-sense (WSS), and local/quasi-stationarity, and distinguish all three from ergodicity. Which property lets you estimate the autocorrelation from a single recording, and why do we in practice only assume stationarity *within a short window* of an EEG trace?
2. The ACF of a signal oscillates at 8 Hz and does not decay. What kind of signal is this, and what does its power spectrum look like?
3. State the Wiener–Khinchin theorem and use it to explain why white noise has a flat power spectrum.
4. You compute the ACF and PACF of an RR-interval series. The ACF tails off and the PACF cuts off after lag 2. What model and order do you fit, and why?
5. Explain why an AR spectrum has peaks and an MA spectrum has notches, in terms of poles and zeros in the z-plane.
6. AIC and MDL disagree on the order for a short EEG epoch, with MDL choosing a lower order. Which would you trust to avoid over-fitting, and what is the risk of the higher order?

6.15 Further reading

Box, Jenkins, Reinsel and Ljung's *Time Series Analysis: Forecasting and Control* is the classic reference for ARMA modelling and identification. Priestley's *Spectral Analysis and Time Series* connects the stochastic-process foundations to spectral estimation. For a signal-processing perspective on the Wiener–Khinchin theorem and random signals, Papoulis and Pillai's *Probability, Random Variables, and Stochastic Processes* is authoritative. Kay's *Modern Spectral Estimation* (also central to Chapter 7) develops Yule–Walker, Levinson–Durbin, and AR modelling

varkonyibarnabas@gmail.com

in depth, with an engineer's clarity.

varkonyibarnabas@gmail.com

Chapter 7 — Spectral Estimation

You compute the spectrum of a 30-second EEG epoch, look at it, and recoil: it is a jagged thicket of spikes, nothing like the smooth textbook curve with a clean alpha peak. You gather more data, expecting it to settle down — and it does not. It just gets jagged at finer resolution. This is not a bug; it is the central, counterintuitive fact about the raw spectrum. This chapter is the repair manual: how to turn an untrustworthy spectrum into one reliable enough to extract features from. It is a load-bearing chapter, because almost every frequency feature in the rest of the book is produced by the methods here.

7.1 Why we estimate, and the two problems

There is a subtle but crucial distinction hiding under the word "spectrum." For a deterministic signal of known form, the spectrum is a fixed thing we can compute exactly. But a biosignal is a stochastic process (Chapter 6): we observe one finite realization and want the underlying power spectral density (PSD) of the process that generated it. We are not computing a spectrum; we are estimating one from limited, noisy data — and estimation has error.

Two problems dog every estimator. The finite-observation problem: we only ever see a window of the signal, which convolves the true spectrum with the window's own spectrum and causes *leakage* and limited resolution. The variance problem: even with infinite data, the naive estimator's random error refuses to shrink. Understanding and taming these two is the whole subject.

Learning objectives. You should be able to (L2) explain why the periodogram is *inconsistent* and what leakage is; (L3) apply Bartlett's and Welch's methods and quantify how averaging cuts variance; (L2) explain the multitaper (Thomson/DPSS-Slepian) idea and why it averages tapers instead of segments; (L3) compute a parametric AR spectrum with the Yule–Walker and Burg methods and say why it wins for short segments; (L2) select an AR model order and recognize under- and over-fitting; and (L3) choose an estimator with a defensible decision framework and read biomedical band powers off the result.

7.2 The periodogram and why it is inconsistent

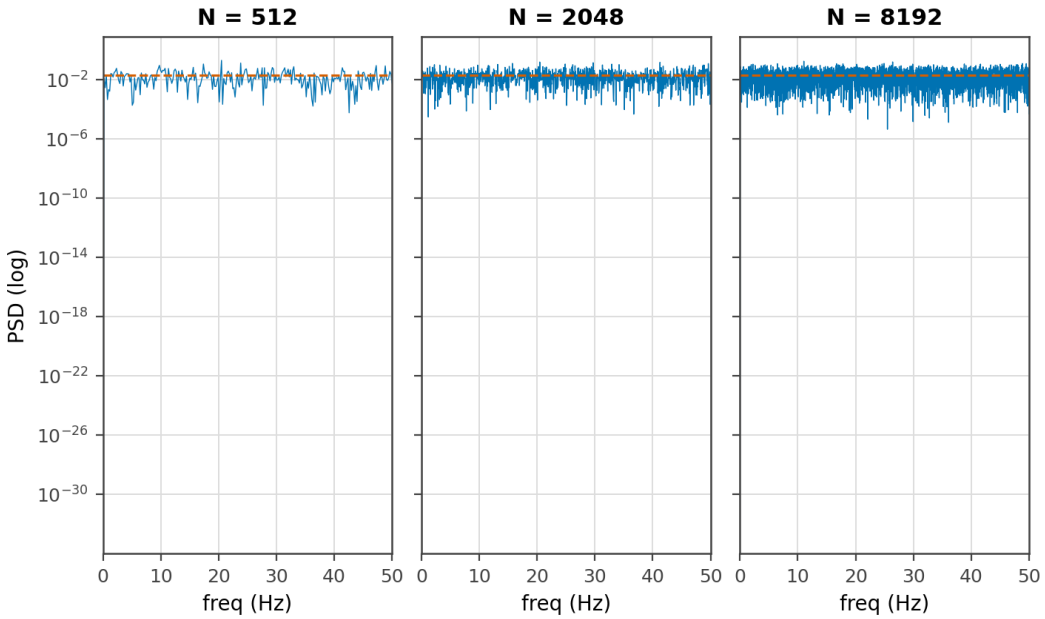
The periodogram is the obvious estimator: take the DFT of the record, and use the squared magnitude as the spectrum,

$$\hat{S}_{\text{per}}(f) = \frac{1}{N}\left|\sum_{n=0}^{N-1} x[n]\, e^{-j2\pi f n}\right|^2 .$$

Here f is a normalized frequency in cycles per sample (0 to 1), matching the DFT convention of §3.5. To read this as a physical *density* in hertz — the convention this chapter's figures,

varkonyibarnabas@gmail.com

band-power tables, and the companion notebook's `bsp.ar_psd` helper all use — divide by the sampling rate: $\hat{S}_{\text{per}}(f)/f_s$, with f now in Hz. Skipping that division is the single easiest way to be off by a factor of f_s in an absolute band-power number, so watch for it whenever a formula in this chapter and a plotted density disagree by exactly that factor. It is unbiased in the limit and cheap. It is also, as an estimator, badly behaved — and here is the punchline that surprises every newcomer:



More data does NOT calm the periodogram: the grass-like variance stays the same (dashed = true flat PSD). It is an inconsistent estimator.

Figure 7.1. More data does NOT calm the periodogram. The same white-noise process at three record lengths (N = 512, 2048, 8192); the true PSD (dashed) is flat, but the periodogram stays equally "grassy" at every length — its variance does not shrink. It is an inconsistent estimator.

Figure 7.1 shows white noise — whose true PSD is a flat line — estimated at three record lengths. As N grows, we get *more frequency bins*, but the scatter around the true value at each bin does not decrease. The variance of the periodogram at any frequency stays roughly equal to the square of the true PSD there, *no matter how long the record.* An estimator whose variance does not vanish as data accumulates is called inconsistent, and the periodogram is the standard illustration of the phenomenon. A longer record does buy finer bin spacing — the DFT samples frequency on a grid of spacing $\Delta f = f_s/N = 1/T$, so more samples means more, closer bins — but it does not buy reliability, and (as Chapter 4 stressed) bin spacing is not the same thing as *practical resolution*: whether two nearby rhythms can actually be told apart depends on the observation length, the window main-lobe width, the SNR, leakage, and the estimator itself, not merely on how densely we sample the frequency axis. This is why you cannot trust a raw spectrum for feature extraction, and why the rest of the chapter exists.

varkonyibarnabas@gmail.com

7.3 The modified periodogram: windowing and leakage

Before we fix the variance, a word on bias. Cutting a finite window out of the signal is multiplication by a rectangular window in time, which is *convolution* by that window's spectrum in frequency. The rectangular window's spectrum has a narrow main lobe but tall side lobes, so energy from a strong component leaks into neighboring frequencies, potentially burying a weak nearby peak. The modified periodogram multiplies the record by a tapered window (Hann, Hamming, Blackman) before the DFT. A taper has lower side lobes — less leakage — at the price of a wider main lobe — coarser resolution. This is the resolution-versus-leakage trade you meet whenever you pick a window: there is no free lunch, only a knob. The modified periodogram reduces bias but, on its own, does nothing for the variance problem.

7.4 Bartlett's method: average to cut variance

The cure for variance is averaging, and the intuition is exactly that of measuring a noisy quantity many times. Bartlett's method splits the record into K non-overlapping segments, computes a periodogram of each, and averages them. Because the segments are approximately independent, averaging K of them cuts the variance by roughly a factor of $1/K$:

$$\mathrm{Var}\{\hat{S}_{\text{Bartlett}}\} \approx \frac{1}{K}\,\mathrm{Var}\{\hat{S}_{\text{per}}\}.$$

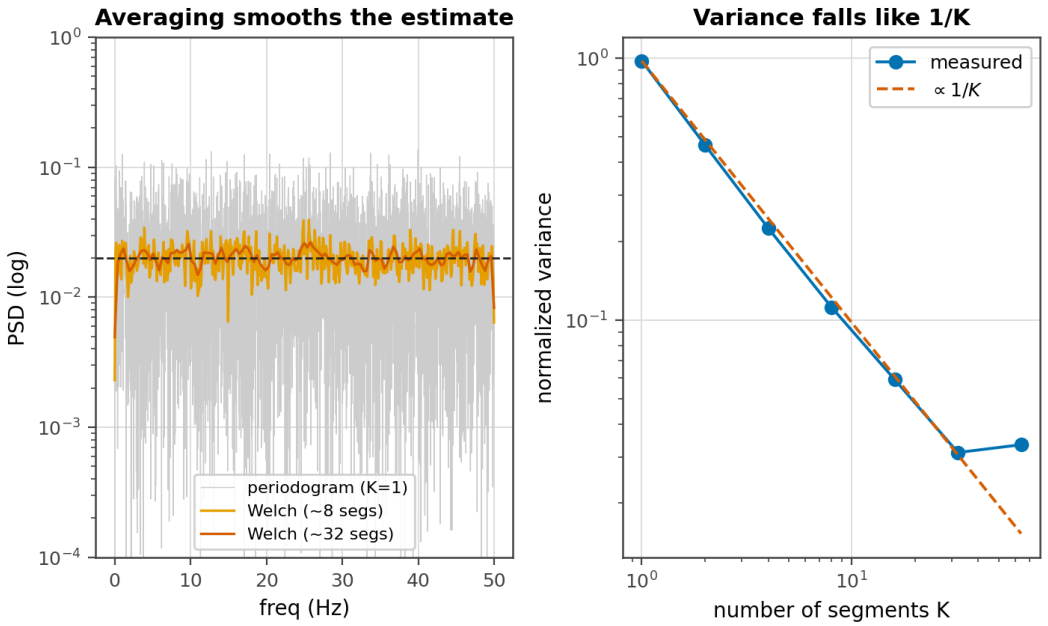
There is, inevitably, a cost. Each segment is only N/K long, so both its bin spacing and its practical resolution coarsen by a factor of K — the segment bin spacing widens to $\Delta f \approx K/T$, and because each short segment sees a proportionally wider window main lobe, genuinely close rhythms that a long record could separate now blur together. Bartlett's method converts the useless variance of the periodogram into usable smoothness, paid for in resolution. This resolution-variance trade-off is the governing law of non-parametric spectral estimation.

7.5 Welch's method: the everyday workhorse

Welch's method improves on Bartlett with two refinements: it lets the segments overlap (typically by 50%), and it windows each segment before its periodogram. Overlap recovers some of the data efficiency lost to segmentation (more segments from the same record for a given segment length), and windowing controls leakage within each segment. The averaged, windowed, overlapped result is smooth and well-behaved — which is why `signal.welch` (and MATLAB's `pwelch`) is the default spectral tool in practically every biosignal pipeline.

Figure 7.2 makes the $1/K$ law concrete. On the left, Welch estimates with more segments visibly tighten around the true flat spectrum. On the right, we measure the variance of the estimate as a function of the number of averaged segments and it falls almost exactly along the theoretical $1/K$ curve — a law and a number you can reproduce. In practice you set three knobs: `nperseg` (segment length, which sets the per-segment bin spacing $\approx f_s/$`nperseg` and, with it, the practical resolution), `noverlap` (overlap, usually 50%), and the `window` (Hann is a safe default). Longer segments give finer resolution but fewer of them to average (higher variance); shorter segments give more averaging but coarser resolution. You are always dialing

varkonyibarnabas@gmail.com



Bartlett/Welch trade frequency resolution for reliability: K averaged segments cut the variance by about 1/K.

Figure 7.2. Averaging drives variance down like 1/K. Left: Welch estimates of white noise with more segments hug the true flat PSD (dashed) ever more tightly. Right: the measured normalized variance of the estimate falls along the theoretical 1/K line as the number of averaged segments grows.

along the same trade-off.

7.6 Multitaper estimation: averaging without cutting the record

Welch buys its variance reduction by *chopping the record into shorter pieces* — and on a short epoch that is exactly the resource we cannot spare. The multitaper method, introduced by David Thomson in 1982, buys the same averaging *without shortening the record at all.* The trick is to keep the full-length signal but multiply it by several different tapers, one at a time, computing a windowed periodogram from each and averaging the results. If the tapers were arbitrary, their estimates would overlap heavily and the average would gain little. The insight is to choose a family of tapers that are mutually orthogonal and — crucially — each maximally concentrated in frequency, so that the several estimates are approximately statistically independent and can be averaged like independent measurements.

The tapers that do this optimally are the discrete prolate spheroidal sequences (DPSS), also called the Slepian sequences. Given a record length N and a chosen time-bandwidth product NW (a small number such as 3 or 4, which sets the spectral half-bandwidth W over which energy is allowed to spread), the DPSS are the sequences that pack the greatest possible fraction of their energy into the band $[-W, W]$; there are about $2NW - 1$ of them that are well concentrated, and these are the ones we use. Averaging the $K \approx 2NW - 1$ orthogonal single-taper estimates cuts the variance by roughly $1/K$, exactly as segment averaging does — but every estimate uses the whole record, so there is no loss of length. In effect, multitaper trades a modest, *controllable* smearing (the half-bandwidth W) for a variance reduction, making the same resolution-versus-variance bargain as Welch but through a different door.

varkonyibarnabas@gmail.com

That different door is what makes multitaper attractive. Because each Slepian taper has very low spectral leakage, the method controls leakage and variance *together* through the single dial NW: raise it to average more tapers (lower variance, broader main lobe), lower it for sharper resolution (fewer tapers, higher variance). It is especially valued when the record is too short to give Welch enough segments yet you still want a smooth, low-leakage, non-parametric estimate that makes no model assumption — which is precisely the situation in much of EEG and neuroscience, where multitaper spectra and multitaper spectrograms are a routine, practical choice for band power, coherence, and time–frequency displays (SciPy exposes the tapers as `scipy.signal.windows.dpss`; the MNE-Python and Chronux toolboxes wrap the full estimator). The price is a handful of extra FFTs and the need to pick NW sensibly.

7.7 Parametric AR estimation: high resolution from short records

Non-parametric methods make no assumptions but pay a resolution penalty that is brutal for short records — and physiology often hands us short records (a 30-second stationary EEG epoch, a single respiratory cycle). If we are willing to *assume a model*, we can do far better. The autoregressive (AR) model of Chapter 6 treats the signal as white noise driving an all-pole filter; fitting it gives a parametric spectrum,

$$\hat{S}_{\mathrm{AR}}(f) = \frac{\hat{\sigma}^2 / f_s}{\left| 1 - \sum_{k=1}^{p} a_k \, e^{-j 2\pi f / f_s} \right|^2},$$

with f now in hertz and the $1/f_s$ factor matching the periodogram's density convention from §7.2 — this is exactly what `bsp.ar_psd(a, sigma2, fs)` computes in the companion notebook. (Some references instead leave f normalized to cycles per sample and drop the $1/f_s$, which is equally valid but a *different* object — mixing the two conventions is the usual source of a spectrum that is a factor of f_s too large or too small.) The result is smooth by construction (it is a ratio of polynomials, not a squared random DFT) and can resolve peaks far sharper than $1/T$ because the poles can sit arbitrarily close to the unit circle. From a 30-second epoch, an AR model can produce a clean alpha peak that Welch would only blur.

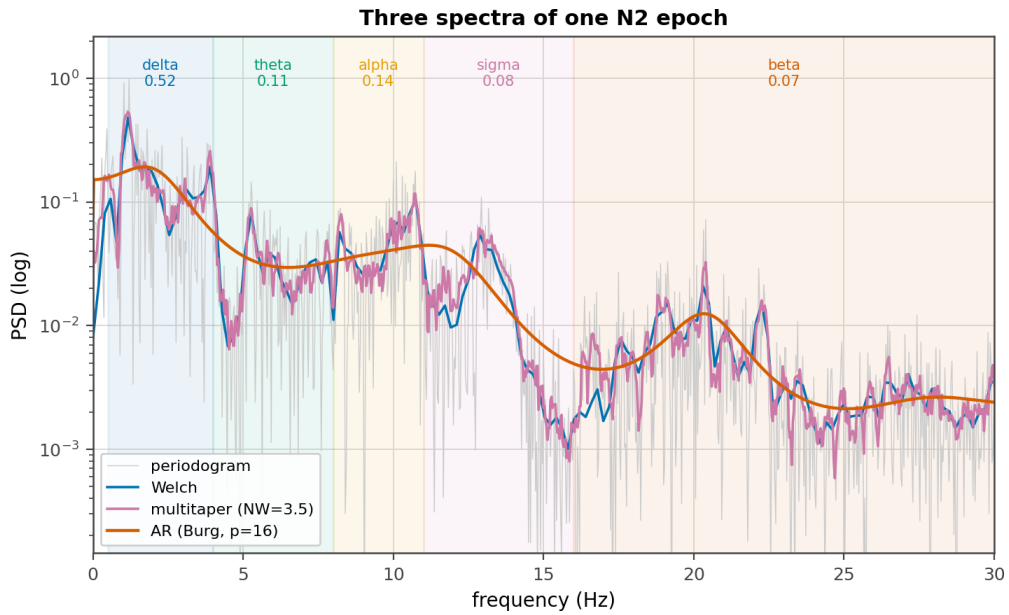
Two estimation routes matter. The Yule–Walker method builds a linear system from the estimated autocorrelation and solves it — efficiently via the Levinson–Durbin recursion ($O(p^2)$), which also yields the reflection coefficients and works its way up through every lower order for free. Burg's method takes a different tack: it minimizes forward *and* backward prediction error directly on the data (never explicitly forming the autocorrelation), which gives it better resolution and more stable estimates on short records — usually the best choice for biosignal epochs.

7.8 The three estimators on one synthetic epoch

Figure 7.3 is the comparison the chapter has been building toward: periodogram, Welch, multitaper, and Burg-AR computed on the same stage-N2 EEG epoch, with the clinical bands shaded and their powers annotated.

The visual story is immediate. The periodogram is a grey thicket; Welch tames it into a trust-

varkonyibarnabas@gmail.com



Same epoch, four lenses: the periodogram is noisy, Welch and multitaper are smooth, and Burg-AR is smoothest — all agree on the band powers that become features.

Figure 7.3. Same epoch, four lenses. The periodogram (grey) is noisy; Welch (blue) is smooth; the multitaper estimate (purple) is smooth and low-leakage while using the whole epoch; Burg-AR (red) is smoothest and sharpest. Shaded bands mark delta/theta/alpha/sigma/beta; the annotated band powers are the features that feed downstream models — and, on this synthetic epoch, the methods agree on them.

varkonyibarnabas@gmail.com

worthy curve; the multitaper estimate is comparably smooth but pays for its variance reduction in main-lobe width rather than in record length, so it keeps the full epoch's worth of resolution; the Burg-AR spectrum is the cleanest of all, tracing a smooth envelope with well-defined peaks. Crucially, the band powers — delta (0.5–4 Hz), theta (4–8), alpha (8–11), sigma/spindle (11–16), beta (16–30) — computed by integrating the PSD over each band agree across the methods on this simulated epoch. These edges follow the sleep partition of the canonical EEG band table in Chapter 1 (§1.4): that table lists sleep-alpha as 8–12 Hz, but since it also carves a sigma/spindle band (11–16 Hz throughout this book) out of low beta, alpha and sigma overlap at 11–12 Hz — so, per §1.4's own computational rule, we integrate alpha here as the non-overlapping 8–11 Hz and beta begins at 16 Hz; read any band edge here against that table. These five numbers, and their relative and log-transformed versions, are among the most heavily used features in biosignal analysis. This is the concrete payoff of everything above: *the reason we work so hard for a reliable spectrum is that we integrate it into band powers and hand those to a classifier.* The choice of method affects how clean the picture looks, but a good estimator gives band powers you can rely on.

> Common mistakes. Three errors turn a correct spectrum into a wrong number, all around PSD normalization and interpretation. First, forgetting the units and scaling. A one-sided PSD (positive frequencies only, the usual default) carries twice the height of a two-sided PSD because it folds the negative-frequency power onto the positive axis — mix the two conventions and every band power is off by a factor of two. Likewise, the taper alters the effective bandwidth of each bin: unless you account for the window's equivalent noise bandwidth, integrated band-power numbers come out wrong. A correctly normalized estimator (as `signal.welch` provides with `scaling='density'`) handles both, but you have to know it is doing so. Second, reading the *absolute height* of a raw periodogram as if it were reliable. It is not — the periodogram is inconsistent (§7.2), so any single bin's value is a noisy draw; compare *averaged* estimates (Welch, multitaper) or a *parametric* AR spectrum, which have the variance a stable comparison needs. Third, computing band power by *summing raw FFT magnitudes*. Band power is a density integrated over the band, $\int_{f_1}^{f_2} S(f)\,df$ — the PSD value at each frequency multiplied by the bin width and summed — not a bare sum of $|X(f)|$ or $|X(f)|^2$ values. Get the density and the integration right and the numbers are physically meaningful; skip them and they merely look plausible.

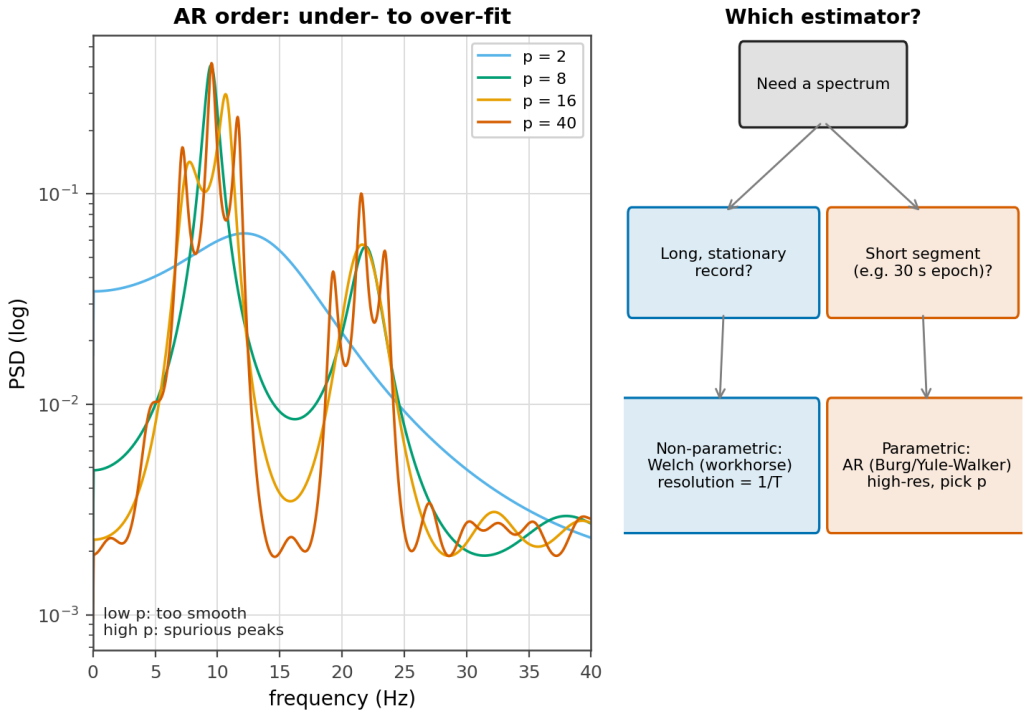
7.9 Model-order selection and a decision framework

The AR model's one dangerous knob is the order p. Choose it too low and the spectrum is over-smoothed — real peaks merge or vanish (under-fitting). Choose it too high and the model starts fitting noise, sprouting spurious peaks that are not in the true spectrum (over-fitting).

The left panel of Figure 7.4 sweeps the order on a waking EEG epoch. At $p = 2$ the spectrum is a featureless hump — it cannot resolve the alpha peak. At $p = 8$ and $p = 16$ the alpha peak near 10–12 Hz is crisp and correct. At $p = 40$ the curve breaks out in a rash of small peaks that are artifacts of over-fitting, not physiology. You choose the order the same way as in Chapter 6 — minimize an information criterion:

$$\mathrm{AIC}(p) = N \ln \hat{\sigma}_p^2 + 2p, \qquad \mathrm{MDL}(p) = N \ln \hat{\sigma}_p^2 + p \ln N,$$

varkonyibarnabas@gmail.com



Order is a dial: too low blurs peaks, too high invents them. Choose the estimator family by record length and stationarity.

Figure 7.4. Left: an AR order sweep on a waking EEG epoch — order 2 is too smooth to resolve the alpha rhythm, orders 8–16 nail it, and order 40 invents spurious wiggles. Right: a decision diagram — long stationary records favour non-parametric Welch; short segments favour parametric AR.

varkonyibarnabas@gmail.com

or the closely related FPE (final prediction error). A common rule of thumb sets p around $N/3$ at most, and often far lower for a stationary epoch. The right panel of Figure 7.4 is the decision framework in one glance: if you have a long, stationary record, use non-parametric Welch — the everyday workhorse; if you have a short segment, use parametric AR (Burg or Yule–Walker) for its resolution — and choose the order carefully. Between these sits the multitaper method of §7.6: like Welch it is non-parametric and assumption-light, but because it averages orthogonal tapers over the *whole* record it is often the better non-parametric choice on short segments where Welch runs out of segments to average — one reason it is a staple in EEG and neuroscience. MA and ARMA spectra exist too, but for biosignals the all-pole AR model captures the resonant rhythms that matter with the least fuss.

> Is that spectral peak real? A bump in a spectrum is not yet a finding. Before interpreting a peak as physiology, check that it:
>
> 1. persists across reasonable estimator choices (window type and length, segment count, AR order, taper bandwidth);
> 2. exceeds the estimator's own variability — a single periodogram is so noisy that peaks appear by chance, which is the whole reason we average;
> 3. is not explained by a known artifact or a preprocessing step (a 50/60 Hz line and its harmonics, a resampling filter, an aliasing fold);
> 4. appears in the relevant subjects, epochs, or conditions, not just one favourable segment;
> 5. is compatible with the sampling rate, record duration, and frequency resolution — you cannot resolve a 0.1 Hz splitting in a 5-second record.
>
> A peak that survives all five is worth interpreting; one that appears for a single parameter choice is a candidate for the next check, not a conclusion.

7.10 Biomedical applications

The methods here are the engine behind the field's standard frequency features. In EEG, band powers and their ratios drive sleep staging, depth-of-anesthesia monitoring, and seizure detection; the spectral edge frequency (the frequency below which, say, 95% of the power lies) and spectral entropy are Welch/AR-derived summaries. In HRV, the low-frequency (0.04–0.15 Hz) and high-frequency (0.15–0.4 Hz) band powers and their ratio index autonomic balance — and because HRV records are short and unevenly sampled, AR estimation is especially valued here. In EMG, the median frequency of the power spectrum shifts downward as a muscle fatigues, making the spectrum a fatigue monitor. Each of these is a number read off a spectrum you now know how to trust.

7.11 HRV: a worked example

Nowhere do the ideas of this chapter pay off more cleanly than in heart-rate variability (HRV) — the beat-to-beat fluctuation of the interval between heartbeats. A perfectly metronomic heart is, paradoxically, a sick one: a healthy heart is continuously nudged by the autonomic nervous system, and the *size and rhythm of those nudges* is a window onto autonomic balance.

varkonyibarnabas@gmail.com

The parasympathetic (vagal) branch slows the heart and can change it beat to beat, so it writes fast fluctuations; the sympathetic branch speeds it up and acts more slowly. Reading HRV is reading the tug-of-war between them.

The raw material is the RR-interval series (or tachogram): the sequence of intervals between successive R-peaks, obtained from the ECG. The top panel of Figure 7.5 plots our 5-minute tachogram — RR interval in milliseconds against beat time — and its restless up-and-down is exactly the variability we want to quantify.

Time-domain metrics summarize that variability directly, without a spectrum. The SDNN is simply the standard deviation of the normal-to-normal (NN) intervals, $\text{SDNN} = \sqrt{\frac{1}{N-1}\sum_i (RR_i - \overline{RR})^2}$; it captures total variability from all sources. The RMSSD is the root-mean-square of *successive* differences, $\text{RMSSD} = \sqrt{\frac{1}{N-1}\sum_i (RR_{i+1} - RR_i)^2}$; because it looks only at neighbor-to-neighbor change, it emphasizes the fast, vagally-mediated fluctuations. pNN50 is the percentage of successive intervals that differ by more than 50 ms, another short-term (parasympathetic) index. For the record in Figure 7.5 these come out to SDNN ≈ 30.7 ms, RMSSD ≈ 26.5 ms, and pNN50 $\approx 3.7\%$, at a mean heart rate of about 70 bpm (mean RR ≈ 856 ms) — unremarkable, healthy-adult numbers.

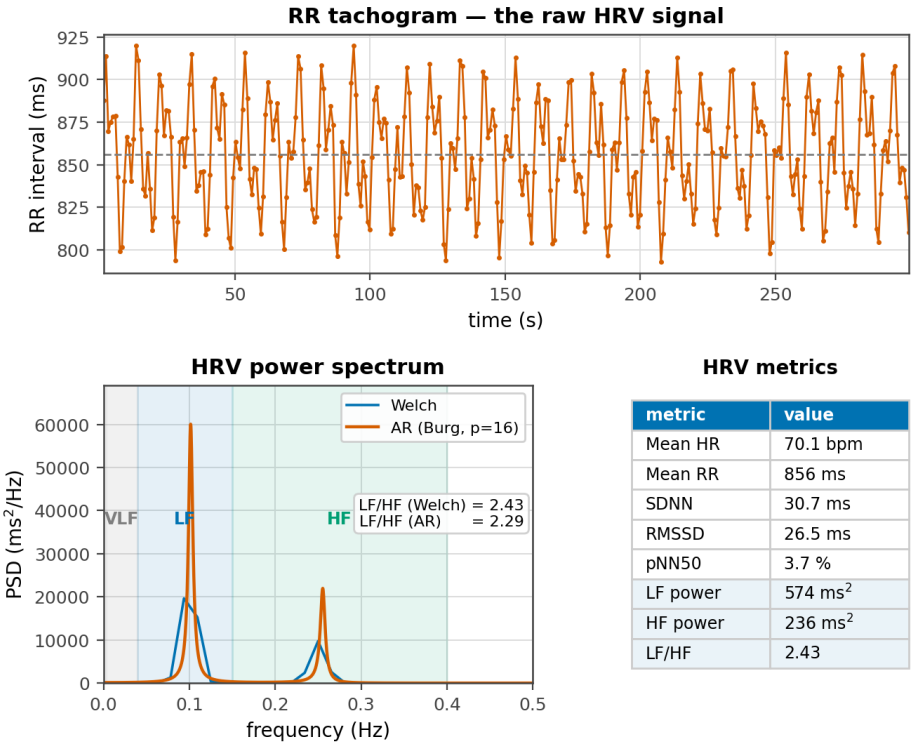
Frequency-domain metrics ask *where in frequency* the variability lives, and this is where spectral estimation enters. By convention (Task Force of ESC/NASPE, 1996) the HRV spectrum is split into three bands: VLF (0.003–0.04 Hz), LF (0.04–0.15 Hz), and HF (0.15–0.40 Hz). The HF band coincides with the breathing rhythm — respiratory sinus arrhythmia — and reflects parasympathetic (vagal) activity. The LF band is a mix of sympathetic and parasympathetic influence. The LF/HF ratio was historically read as an index of *sympatho-vagal balance*, but that interpretation is no longer considered valid as a quantitative measure of sympathetic–parasympathetic balance and is best avoided; LF and HF band powers remain useful when reported descriptively, without that mechanistic gloss. In the bottom-left panel of Figure 7.5 the spectrum shows a clear LF peak near 0.1 Hz and an HF peak near 0.25 Hz, giving LF ≈ 574 ms^2, HF ≈ 236 ms^2, and LF/HF ≈ 2.4.

There is one subtlety that makes HRV a perfect case study for this chapter. The RR series is unevenly sampled — there is exactly one value per heartbeat, and heartbeats do not arrive on a uniform grid. Every method in this chapter assumes a uniformly sampled signal, so before we can apply Welch or AR we must resample the tachogram onto a uniform time grid (here, linear interpolation to 4 Hz). Only then does "frequency in Hz" even have its usual meaning. With that done, the two estimators of this chapter do their jobs much as the theory leads you to expect. Welch needs a reasonably long record to average enough segments; on this 5-minute recording it delivers a smooth, trustworthy spectrum. The parametric AR (Burg) estimate produces a sharp, clean spectrum from a short record — which is precisely why AR is a workhorse in HRV, where clinical recordings are often only a few minutes long. Reassuringly, on this synthetic tachogram the two methods agree on the band split (LF/HF of about 2.4 for Welch versus 2.3 for AR), just as they agreed on EEG band powers in §7.8 — the non-parametric and parametric lenses see the same physiology. (As always in this book, these numbers come from the companion notebook's synthetic signals; they are illustrative and reproducible, not empirical benchmarks.)

Two caveats worth carrying forward. First, every HRV number depends on accurate R-peak

varkonyibarnabas@gmail.com

detection: a single missed or falsely-detected beat injects a spurious spike into the tachogram that can dominate RMSSD and the HF band, which is why HRV pipelines lean on a robust QRS detector such as Pan–Tompkins (Chapter 9) and on ectopic-beat correction. QRS detection is mature and reliable on clean ECG, but it remains challenging under motion, ectopy, low SNR, and wearable conditions — so this dependence is not a formality. Second, because HRV records are short and the resonances are narrow, AR-based HRV spectra are especially common — the very argument for parametric estimation from §7.7 finds one of its clearest homes here.



One HRV pipeline: RR tachogram -> uniform resample -> Welch and AR spectra agree on the LF/HF split; SDNN, RMSSD and pNN50 summarize the same variability in the time domain.

Figure 7.5. An HRV pipeline end to end. Top: the RR tachogram — the raw, unevenly-sampled HRV signal. Bottom left: after resampling to a uniform 4 Hz grid, Welch (blue) and AR/Burg (red) spectra agree on an LF peak near 0.1 Hz and an HF peak near 0.25 Hz, with the VLF/LF/HF bands shaded and LF/HF annotated. Bottom right: the matching time-domain summary (SDNN, RMSSD, pNN50) and band powers. Two lenses, one physiology.

7.12 The ML/AI bridge

Every band-power, spectral-edge, and spectral-entropy feature Chapter 11 hands to a classifier is computed from a spectrum built here — so the estimator you choose upstream is not a preprocessing footnote, it is a feature-engineering decision. A noisy, high-variance periodogram feeds a classifier noisy, high-variance features that jitter epoch to epoch, quietly inflating the apparent need for regularization or more training data; Welch's or multitaper's variance reduction is, downstream, a *cleaner label-to-feature mapping* the model does not

varkonyibarnabas@gmail.com

have to learn around. AR coefficients are their own compact feature representation — a handful of numbers standing in for the whole spectral shape — and a Burg-AR spectrum's sharp, well-resolved peaks make band-edge and peak-frequency features more stable across short epochs than periodogram-derived ones. The estimator you pick therefore has two audiences at once: the eye reading a spectrogram, and the classifier reading the band powers computed from it.

7.13 Signals-and-images box

> Spectral estimation in two dimensions. The 2-D power spectrum of an image (the squared magnitude of its 2-D DFT) suffers the same variance problem, and the same cure applies: averaging periodograms of image patches (a 2-D Welch) yields a smoother, more reliable estimate of spatial-frequency content. This matters for characterizing texture — fine repetitive texture concentrates energy at high spatial frequencies, coarse texture at low — and for detecting periodic interference patterns in medical images (for instance, structured noise from an acquisition device shows up as bright off-center spikes in the 2-D spectrum, precisely as powerline hum shows up as a line in a 1-D spectrum).

7.14 Companion notebook

The companion notebook is a spectral-estimation playground. It regenerates the periodogram-inconsistency figure so you can watch the variance refuse to shrink; it sweeps the number of Welch segments and plots measured variance against the $1/K$ line; it overlays periodogram, Welch, multitaper, and Burg-AR on a single epoch and lets you drag `nperseg`, the multitaper time-bandwidth product NW, and the AR order to feel the resolution-variance trade in real time; and it plots AIC/MDL versus order so the order-selection minimum becomes something you find rather than something you are told.

7.15 Case-study tie-in

Spectral estimation is *the* feature engine for the sleep-staging case study. Each 30-second epoch is turned into a spectrum — Welch for robustness, or AR for sharpness on the short epoch — and integrated into delta/theta/alpha/sigma/beta band powers, relative powers, spectral edge, and spectral entropy. These features carry the physiological signal that separates stages: delta dominates deep sleep, spindle (sigma) power flags N2, and higher-band activity marks wake and REM. The estimator choice is a genuine engineering decision in the pipeline, and the decision framework of Figure 7.4 is exactly how you make it.

7.16 Summary and key formulas

The raw periodogram is inconsistent: its variance does not shrink with record length, so it cannot be trusted for features. Windowing (the modified periodogram) controls leakage; segment-averaging (Bartlett, then Welch) drives variance down like $1/K$ at the cost of resolution — the governing trade-off of non-parametric estimation, with Welch the everyday

varkonyibarnabas@gmail.com

workhorse. The multitaper (Thomson/DPSS-Slepian) method reaches the same variance reduction by averaging several orthogonal Slepian tapers over the *whole* record rather than by chopping it into segments, trading a controllable half-bandwidth for low leakage and low variance — a major practical estimator, especially in EEG and neuroscience. For short segments, the parametric AR spectrum (Yule–Walker via Levinson–Durbin, or Burg) delivers smooth, high-resolution peaks, provided the order is chosen by AIC/MDL to avoid under- and overfitting. The end product — trustworthy band powers — is what feeds every downstream model.

Method	Type	Variance	Resolution	Best when
Periodogram	non-parametric	high (inconsistent)	bin spacing $\approx 1/T$	never for features; a baseline
Modified periodogram	non-parametric	high	$\approx 1/T$, less leakage	need to see a weak peak near a strong one
Bartlett	non-parametric	$\approx 1/K$	$\approx K/T$ (coarser)	long record, want simple averaging
Welch	non-parametric	$\approx 1/K$	tunable via `nperseg`	long, stationary record — the workhorse
Multitaper (Thomson/DPSS)	non-parametric	$\approx 1/K$, $K \approx 2NW-1$	full-record, set by NW	short record, want low-leakage smooth PSD (EEG/neuro)
AR (Yule–Walker/Burg)	parametric	low, smooth	very high (sub-$1/T$)	short segment, resonant rhythm

7.17 Conceptual questions and exploration tasks

1. State precisely what "the periodogram is inconsistent" means, and explain why collecting more data does not fix it.
2. You double the number of Welch segments. What happens to the variance of the estimate, and what do you pay for it?
3. A weak 11 Hz peak sits next to a strong 10 Hz peak in an EEG epoch. Would you reach for Welch or AR, and why? What role does leakage play?
4. Explain, in terms of poles, why an AR spectrum can resolve a peak sharper than the $1/T$ limit of a periodogram.
5. Sweeping AR order, you see the spectrum go from a single smooth hump to a rash of small peaks. Name the two failure modes and the criterion you would use to choose between them.
6. For a two-minute stationary RR-interval tachogram (the HRV series) you want the LF/HF ratio. Using the decision framework, which estimator do you choose, and what parameters do you set?
7. *Transfer.* You have used Welch, multitaper, and AR on EEG and HRV. For a mechanomyogram — a low-frequency muscle-vibration signal recorded through a sustained contraction that slowly fatigues — is a 30 s epoch stationary enough for any of them? What would you change, and which estimator's assumptions break first?

Design brief. For a study you must report the HRV LF/HF ratio from a 5-minute resting

varkonyibarnabas@gmail.com

tachogram and, separately, track a drifting respiratory peak across a 20-minute recovery recording. Spec: for each, state the resolution you need, the stationarity you can assume, and the record length you have. Design: choose an estimator (periodogram / Welch / multitaper / AR-Burg) and its parameters (segment length, taper count, model order) for each. Evaluate: how do you check a reported peak is real, not an AR artifact or a leakage side lobe? Motivate: justify each estimator against record length and stationarity. Alternatives: name the estimator you would switch to if the resting record were only 1 minute. Rule out: name an estimator or parameter choice that would be *wrong* — e.g. a single raw periodogram reported as *the* HRV spectrum, or an AR order so high it invents peaks — and why.

7.18 Further reading

Kay's *Modern Spectral Estimation: Theory and Application* and Marple's *Digital Spectral Analysis: With Applications* are the standard engineering references, covering the periodogram's inconsistency, Welch's method, and the full parametric machinery (Yule–Walker, Levinson–Durbin, Burg). Percival and Walden's *Spectral Analysis for Physical Applications* is excellent on the statistical foundations and multitaper methods. For the biomedical applications — HRV bands, EEG band power, EMG median frequency — the standards on heart-rate variability and Sanei and Chambers' *EEG Signal Processing* give the domain-specific detail. The definitive HRV reference is the Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology (1996), *Heart rate variability: standards of measurement, physiological interpretation, and clinical use*, Circulation 93(5):1043–1065 (also Eur. Heart J. 17:354–381), which fixes the VLF/LF/HF band definitions, the SDNN/RMSSD/pNN50 metrics, and the resampling and AR-modeling conventions used above.

varkonyibarnabas@gmail.com

Chapter 8 — Noise and Artifacts: Properties, Sources and Identification

A clinician looks at a messy ECG and says, in one breath, "that's mains hum, that's a loose electrode, that's the patient shivering." Three different problems, three different cures — and the diagnosis came first. That is the discipline of this chapter. Before you filter anything, you name it. Every kind of contamination carries a fingerprint in time, in frequency, or in space, and learning to read those fingerprints is what tells you whether to reach for a notch filter, a high-pass, or the median filter of the next chapter. Removing noise you have not identified is how you accidentally remove signal.

8.1 Motivating question and the value of a diagnosis

"Clean the signal" is not a well-posed instruction. Powerline hum, baseline sway, muscle bursts, eye blinks, and electrode pops each demand a different tool, and a tool aimed at the wrong target does damage: a filter tuned to remove muscle noise from an ECG will also erase the sharp QRS complex you were trying to protect. So the first move in any preprocessing pipeline is identification — characterize the contamination, then choose the remedy. This chapter is the field guide to the noises and artifacts of biosignals, organized so that each one's signature points directly at its cure.

A useful distinction up front: noise is broadly random, often additive, and describable statistically (thermal noise, quantization noise); an artifact is a specific, often structured intrusion with a physical or physiological cause (a blink, a step change from a moving electrode, mains interference). The line blurs, but the instinct is right — noise you characterize, artifacts you recognize.

Learning objectives. You should be able to (L2) characterize a contamination by its time-domain, frequency-domain, and — for images — spatial signature, and quantify it with SNR; distinguish white from coloured (pink, brown) noise by PSD slope; recognize powerline interference and its harmonics; identify impulsive/outlier noise in 1-D and its salt-and-pepper twin in 2-D; trace baseline wander and motion artifacts to the electrode–skin interface; and spot physiological cross-contamination (EOG/EMG in EEG). You should also (L2) run a first-pass quality-control screen that detects and flags unusable data before any filtering. The unifying skill is *naming the noise before choosing the tool.*

8.2 Quality control before preprocessing

The first decision about an unwanted pattern is not *how to remove it.* It is *what it is*: removable contamination, another physiological source riding along, a clinically meaningful event in its own right, or evidence that the measurement has failed and should be discarded rather than cleaned. Reaching for a filter before answering that question is how a real seizure gets "denoised" away, or how a disconnected electrode gets band-passed into a plausible-looking

varkonyibarnabas@gmail.com

trace. And before applying any correction, state which waveform morphology, timing, amplitude relationship, or uncertainty must remain interpretable afterwards — the point of cleaning is to *preserve* the signal you care about, not merely to make the trace look tidy.

Before any of the identification-and-filtering that follows, there is an earlier, humbler step that real biosignal work lives or dies by: screening the raw data for segments that are not merely noisy but broken. A filter assumes there is a signal underneath the contamination to recover. When there is not — when an electrode has fallen off, an amplifier has saturated, or a channel is simply missing — filtering does not repair the record; it quietly manufactures a plausible-looking trace out of garbage, and every downstream number inherits the fiction. The discipline is to detect and exclude or flag bad data first, so that it is never silently filtered and passed on as if it were physiology.

The screen is a checklist of failure modes, most of them cheap to test for. Flatlines — long stretches of (near-)constant value — signal a dead channel, a saturated buffer, or a disconnected lead; a simple test is a rolling variance that drops below a floor. Clipping and saturation show up as samples pinned at the ADC's maximum or minimum for many consecutive points, a flat-topped waveform where the true peak has been cut off; count samples at the rail. Disconnected or "railing" electrodes drift to a rail or float wildly, often with an impossibly large or impossibly small amplitude and a broadband or DC-dominated spectrum. Improbable amplitudes — values far outside the physiological range for that modality (an "EEG" swinging by millivolts, an "ECG" of tens of millivolts) — indicate a scaling error, a units mismatch, or an artifact, not a real signal. Missing channels must be detected explicitly, because a downstream feature extractor that expects a fixed montage will otherwise compute nonsense (or crash) on the absent lead. Sampling-rate mismatches between channels, sessions, or devices silently corrupt every frequency measurement — a spectrum computed at the wrong f_s puts every peak at the wrong hertz — so the recorded rate must be checked against the assumed one. And annotation or time-alignment errors — event markers off by a fixed offset, a trigger channel that lags the data, epochs mislabeled — poison supervised learning at the root, because the model is then taught the wrong answer.

None of these are the "noise" of the rest of the chapter, and none is fixed by a notch or a high-pass. They are *data-integrity* problems, and the appropriate response is to exclude the affected segment, flag it for review, or repair it deliberately (re-reference a missing channel, resample to a common rate, re-align to a corrected trigger) — not to run it through the same filter as good data and hope. In practice this screen is automated into a small battery of checks that runs first and writes a per-segment quality label. This is not a fussy preliminary; it is central to trustworthy biosignal analysis and to the reproducibility of everything built on top of it. It also connects directly forward: the same integrity checks are the front end of the data-curation and dataset-hygiene discipline of Chapter 12, where we will see that a model is only as trustworthy as the data it was fed.

varkonyibarnabas@gmail.com

Quality control before preprocessing. Before you filter anything, screen the raw record for data that is broken rather than merely noisy. Test for: flatlines (rolling variance near zero → dead/disconnected channel), clipping/saturation (many samples pinned at the ADC rail → true peaks lost), disconnected/railing electrodes (drift to a rail, impossible amplitude), improbable amplitudes (values outside the physiological range → scaling/units error), missing channels (absent leads a fixed montage expects), sampling-rate mismatches (wrong f_s → every spectral peak at the wrong frequency), and annotation/time-alignment errors (offset markers, lagging triggers, mislabeled epochs). The rule: bad segments should be detected and excluded or flagged, not silently filtered. These checks are the front end of the data-curation material in Chapter 12.

8.3 Characterizing noise: signatures and SNR

Three lenses reveal a contamination. The time-domain view shows its shape and dynamics: is it a steady oscillation (hum), a slow drift (baseline wander), a brief spike (electrode pop), or a burst (muscle)? The frequency-domain view — a Welch spectrum from Chapter 7 — shows *where* its energy sits: a sharp line (powerline), a low-frequency hump (drift), a broad high-frequency shelf (muscle), or a characteristic slope (coloured noise). For images, a third spatial view shows its pattern: scattered dots (salt-and-pepper), a smooth grain (Gaussian), or structured stripes (interference).

We quantify contamination with the signal-to-noise ratio, usually in decibels:

$$\mathrm{SNR}_{\mathrm{dB}} = 10 \log_{10} \frac{P_{\mathrm{signal}}}{P_{\mathrm{noise}}},$$

where P is average power (variance for zero-mean signals). SNR sets expectations: at 20 dB the signal dominates and light filtering suffices; near 0 dB signal and noise are comparable and aggressive, possibly adaptive, methods are needed. But SNR alone is not enough — *where* the noise sits relative to the signal band decides whether it is removable at all, which is why the frequency signature matters as much as the ratio.

Two relatives of the SNR round out the vocabulary. When the concern is a system's own distortion — not external noise but harmonics the hardware *generates* — the total harmonic distortion (THD) measures it. In the usual amplitude convention, $\mathrm{THD}_F = \sqrt{V_2^2 + V_3^2 + \cdots} \,/\, V_1$ — the root-sum-square of the harmonic amplitudes over the fundamental amplitude V_1 — quoted as a percentage; the *power* ratio is its square, and dB conventions must be stated. A saturating amplifier, a poorly biased sensor, or an over-driven ADC shows rising THD, and the spectrum tells distortion from noise at a glance — noise lifts the floor everywhere, whereas distortion plants *discrete extra lines* at integer multiples of the signal frequency. For images, the natural analogue of SNR is the peak signal-to-noise ratio (PSNR), referenced to the *maximum possible* pixel value rather than the signal's own power: $\mathrm{PSNR} = 10 \log_{10}(\mathrm{MAX}^2/\mathrm{MSE})$, where MSE is the mean-squared error to a clean reference. PSNR is the standard yardstick for how much a denoising or compression step has degraded a medical image — reported alongside perceptual measures such as SSIM, because a high PSNR does not always mean the result *looks* right.

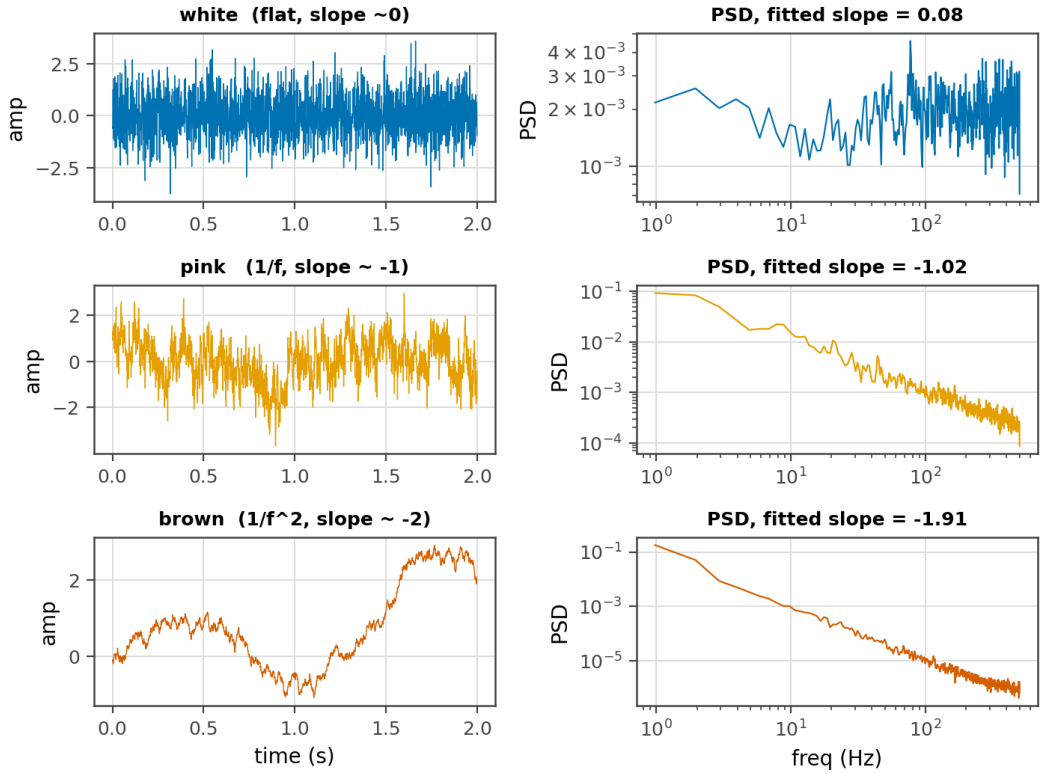
varkonyibarnabas@gmail.com

8.4 White noise

White noise has, by definition, a flat power spectrum — equal power at every frequency — and, by the Wiener–Khinchin theorem of Chapter 6, an autocorrelation that is a single spike at lag zero: successive samples are uncorrelated. Physically it arises from thermal (Johnson–Nyquist) agitation of electrons in resistors and from shot noise in semiconductors — the irreducible electronic floor of any amplifier. Because white noise spreads its energy everywhere, some of it always overlaps the signal band and cannot be filtered out without touching the signal; the only clean weapons against it are averaging (Chapter 7) and improving the acquisition hardware.

8.5 Coloured noise: pink and brown

Not all noise is flat. Coloured noise has a sloped spectrum, and the slope is its fingerprint.



Noise has a colour you can hear in its spectrum: the log-log PSD slope (0, -1, -2) fingerprints white, pink and brown noise.

Figure 8.1. Noise has a colour you can read off its spectrum. Time traces (left) and log-log PSDs (right) for white, pink, and brown noise. The fitted PSD slopes — about 0, -1, and -2 — are the definitive signatures: flat, 1/f, and 1/f-squared.

Figure 8.1 lines up the three canonical noises. Pink (1/f) noise has a PSD that falls off as $1/f$, a slope near -1 on a log-log plot — measured here at -1.02. It is astonishingly common in nature and biology: neural firing, heart-rate fluctuations, and many biological time series show 1/f-like scaling, so pink noise is often not merely contamination but a signature of the

varkonyibarnabas@gmail.com

physiology itself. Brown (Brownian, random-walk) noise falls off as $1/f^2$, a slope near -2 — measured here at -1.91 — and is the noise of integration and drift: it is what you get by accumulating white noise, and it dominates the very low frequencies of slow baseline movement. The time traces make the difference visible even without the spectrum: white is a fuzz with no structure, pink has gentle undulations, and brown wanders far from its origin like a drunkard's walk. A law and a number: identify a coloured noise by fitting a straight line to its log-log PSD and reading the slope — 0 means white, -1 pink, -2 brown.

Because 1/f-type scaling is so pervasive in physiology, there is a standard time-domain tool for quantifying it directly: detrended fluctuation analysis (DFA). DFA measures how the signal's fluctuations grow with the size of the window over which they are examined, after removing a local trend in each window, and summarizes that growth as a single scaling exponent α — a robust alternative to reading a PSD slope, and one that tolerates the mild non-stationarity of real recordings. It is a routine descriptor of long-range correlations in HRV and EEG (where, roughly, $\alpha \approx 0.5$ indicates white-like behaviour, $\alpha \approx 1$ the 1/f regime, and $\alpha \approx 1.5$ Brownian drift).

8.6 Powerline interference

The most recognizable artifact of all is powerline interference — the hum coupled into recordings from the electrical mains through capacitive and inductive pickup. Its fingerprint is unmistakable: a razor-thin spectral line at the mains frequency (50 Hz in most of the world, 60 Hz in the Americas) plus harmonics at integer multiples.

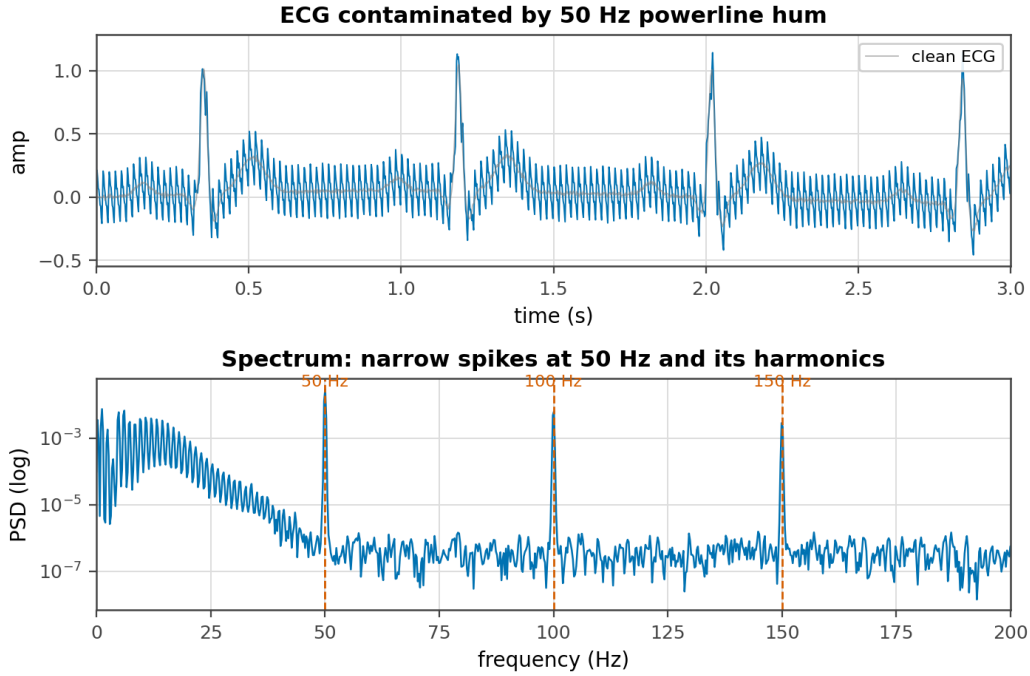
Figure 8.2 adds 50 Hz hum with second and third harmonics to a clean ECG. In the time domain the ECG rides on a fine buzz; in the spectrum the hum announces itself as sharp spikes at exactly 50, 100, and 150 Hz, standing well apart from the ECG's own energy (largely below 40 Hz). This separability is what makes powerline interference the *friendliest* artifact: because it is narrow and its frequency is known in advance, a notch filter (a narrow band-stop) at 50 Hz and its harmonics removes it while barely touching the signal — the recipe developed in Chapter 9's filter-design box. Prevention helps too: good grounding, shielding, and twisted leads reduce pickup at the source, which is usually better than filtering after the fact.

8.7 Impulsive and outlier noise — the 1-D/2-D twin

Some contamination is not a steady process but a scatter of extreme, isolated outliers. In a 1-D recording these are electrode pops and motion spikes: a momentary change in the electrode–skin contact produces a huge, brief step or spike, wildly out of scale with the signal. In an image the exact same statistical creature appears as salt-and-pepper noise — pixels driven randomly to pure white or pure black. This is one of the book's deliberate "one idea, two costumes" pairings.

Figure 8.3 shows them side by side, with the salt-and-pepper case on a photographic image so the damage to real texture and edges is unmistakable — the same specks would riddle a CT or MRI slice in exactly this way. Why introduce them together, and why here? Because their shared statistical character dictates their shared cure. Impulsive outliers are not removable by a linear (averaging) filter: a mean filter, faced with one pixel driven to pure white among

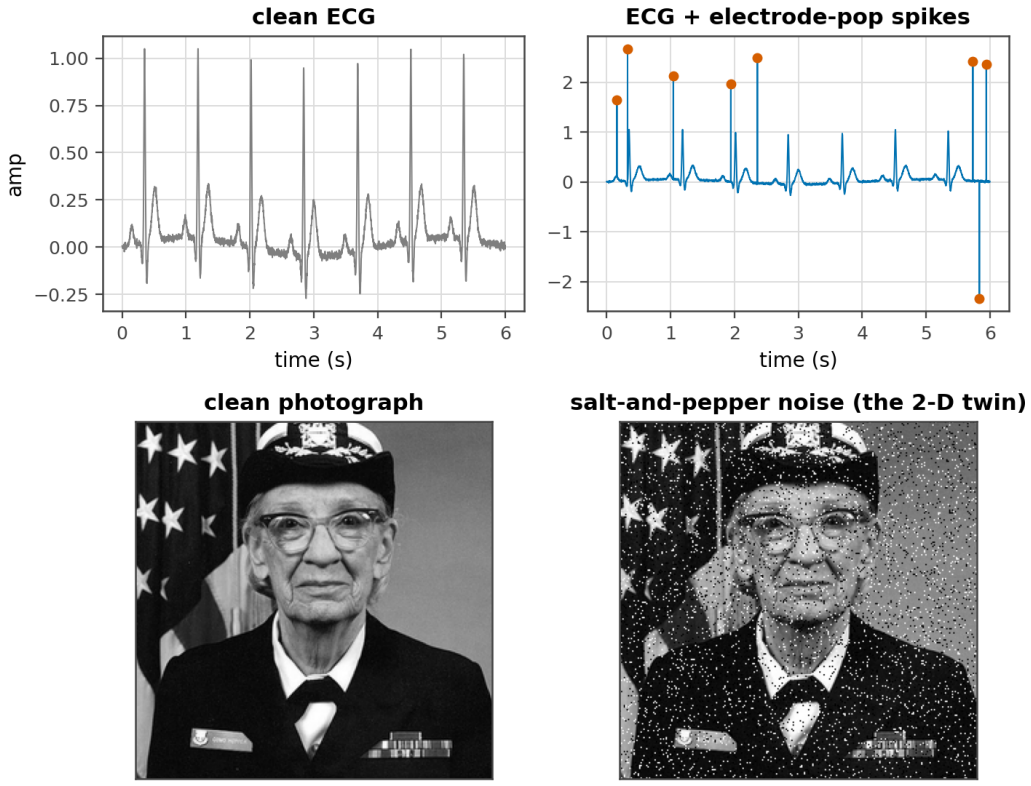
varkonyibarnabas@gmail.com



Powerline interference is the easiest artifact to name: razor-thin lines at 50 Hz and multiples — a job for a notch filter.

Figure 8.2. Powerline interference is the easiest artifact to name. An ECG contaminated with 50 Hz hum plus harmonics (top); the spectrum (bottom) shows narrow spikes at 50, 100, and 150 Hz. Narrow, predictable, and off the signal's main energy — a textbook job for a notch filter.

varkonyibarnabas@gmail.com



The same impulsive noise in two guises — 1-D electrode pops and 2-D salt-and-pepper. Both wreck a mean/linear filter and set up the median filter of Chapter 9.

Figure 8.3. The impulsive-noise twin. Left: electrode-pop spikes on an ECG (red markers) — a few samples driven far out of range. Right: salt-and-pepper noise on a photographic image — pixels flipped to pure black or white, an effect far more visible on real image texture and edges than on a smooth phantom. The identical pattern appears on medical images such as a CT or MRI slice. Both are impulsive outliers; both wreck a mean/linear filter and set up the median filter of Chapter 9.

varkonyibarnabas@gmail.com

neighbors of moderate grey, simply smears the outlier into a blur — it lets the outlier vote. The right tool is the non-linear median filter, which replaces each sample with the *median* of its neighborhood; a lone extreme value cannot move a median, so the spike is rejected outright while edges are preserved. This chapter names the noise; Chapter 9 delivers the median filter and contrasts it head-to-head with the mean. Meeting the 1-D pop and the 2-D salt-and-pepper as twins here makes the later lesson land as a single idea rather than two.

8.8 Baseline wander and motion artifacts

Baseline wander is the slow, low-frequency drift of the signal's baseline — the whole trace sagging and rising beneath the features of interest. Its origin is physical and traces straight back to the electrode–skin interface: as the subject breathes, sweats, or moves, the half-cell potential and impedance of that interface shift, dragging the DC level with them. Respiration alone imposes a drift near 0.15–0.3 Hz on ECG; sweat and slow motion push even lower. Because its energy sits mostly below the signal's band, baseline wander is usually removed with a high-pass filter — but the cutoff must be chosen with care, and the right value depends on the task. For monitoring (rhythm and heart-rate) a cutoff around 0.5 Hz is acceptable, but a diagnostic ECG must preserve content down to about 0.05 Hz: too aggressive a high-pass distorts the low-frequency ST segment and T wave and can manufacture artificial ST shifts that mimic ischaemia. (This is the same 0.05–150 Hz diagnostic band quoted in §1.4.) Motion is a major physical source of this and related contamination, but it is not the whole story: the electrode–skin interface can be disturbed by half-cell potential shifts, skin deformation and stretch, impedance change, cable movement, triboelectric (friction) effects, and common-mode-to-differential conversion, each of which can inject drift or transients. Motion artifacts proper are the broader, more violent cousins of slow wander: gross electrode movement produces large transients and steps that can overlap the signal band, which is why they are among the hardest artifacts and often demand adaptive methods (Chapter 10) or, when a reference of the motion exists, adaptive cancellation.

8.9 Physiological cross-contamination

Sometimes the "noise" is another perfectly good biosignal intruding where it is not wanted. EOG in EEG: eye movements and blinks generate large corneo-retinal potentials that swamp the microvolt-scale EEG, especially at frontal electrodes near the eyes. EMG in EEG: muscle activity from the scalp, jaw, or neck adds a broadband high-frequency shelf that contaminates the EEG's beta and gamma bands. EMG in ECG: trunk or limb muscle activity intrudes on the ECG. These cross-contaminations are distinctive because they are *structured* — a blink has a stereotyped shape, EMG a characteristic band — and often a reference channel is available (a dedicated EOG electrode), which is exactly the setup that adaptive noise cancellation (Chapter 10) exploits to subtract the interference while keeping the EEG.

8.10 Instrumentation and quantization noise

Finally, the measurement chain adds its own noise. Amplifier and electronic instrumentation noise is the white/thermal floor of Section 8.4. Quantization noise is the rounding error of

varkonyibarnabas@gmail.com

the analog-to-digital converter (a callback to Chapter 2): representing a continuous voltage with N bits introduces an error bounded by half a quantization step, well modeled as additive noise and giving the familiar rule of thumb, for a full-scale sinusoidal input,

$$\mathrm{SNR}_{\mathrm{dB}} \approx 6.02\,N + 1.76,$$

so every extra bit of ADC resolution buys about 6 dB of SNR — though, per Chapter 2's crest-factor caveat, a peaky biosignal that cannot fill the full range in practice gets less. This is why bit depth is a design choice, not an afterthought — under-resolving a small signal wastes it in quantization noise before any processing begins.

8.11 A diagnostic decision framework

Identify before you filter. Run the three-lens check on any suspicious contamination. *Time domain:* steady oscillation → powerline; slow drift → baseline wander; brief spikes → electrode pops; bursts → muscle. *Frequency domain:* sharp line at 50/60 Hz and multiples → powerline (notch it); energy below the signal band → baseline wander (high-pass it); broadband high-frequency shelf → EMG; a fitted PSD slope of 0/−1/−2 → white/pink/brown noise. *Spatial* (*images*): scattered black/white dots → salt-and-pepper (median filter); smooth grain → Gaussian (mean/Gaussian filter); stripes/off-center spectral spikes → structured interference. The rule that ties the chapter together: the noise's signature chooses the tool. Match a narrow, known-frequency artifact to a notch; low-frequency drift to a high-pass; broadband in-band noise to averaging or adaptive filtering; and impulsive outliers to a median — not the reverse.

8.12 The ML/AI bridge

Noise identification is not just a cleanup step — it shapes features and labels. First, the same measurements that identify noise become features: the PSD slope quantifies noise colour; power at 50 Hz flags mains contamination; the count and amplitude of outliers score signal quality; a high-frequency-band power index detects EMG intrusion. Second, an automated signal-quality index built from these lets a pipeline reject or down-weight bad epochs before they poison a classifier — a hugely consequential step, since a model trained on artifact-laden epochs learns to detect artifacts rather than physiology. Naming the noise, in other words, is the first act of quality control that makes downstream machine learning trustworthy.

8.13 Signals-and-images box

Every 1-D noise has a 2-D twin. The correspondences run right through this chapter. Additive white noise in a signal ↔ Gaussian grain in an image. Impulsive electrode pops ↔ salt-and-pepper pixels. Powerline hum (a spectral line) ↔ periodic structured interference (off-center spikes in the 2-D spectrum). Baseline wander (a low-frequency trend) ↔ smooth illumination gradient across an image. Recognizing the twin means the cure transfers: a notch removes a periodic pattern from an image just as it removes hum from a trace, and a median filter rejects salt-and-pepper pixels just as it rejects electrode pops. The modality changes; the diagnosis and the remedy do not.

varkonyibarnabas@gmail.com

8.14 Companion notebook

The companion notebook is a noise line-up you can operate. It generates white, pink, and brown noise and classifies each by fitting a slope to its log-log PSD; it injects 50 Hz hum with harmonics into an ECG and locates the spectral spikes; it adds electrode pops to a 1-D trace and salt-and-pepper to an image so you can see the twin, then previews how a mean filter fails and a median filter succeeds on both; and it drops EOG-like blinks into a frontal EEG channel so you can practice spotting cross-contamination by its stereotyped shape.

8.15 Case-study tie-in

In the sleep-staging case study, this chapter motivates the entire preprocessing stage. Before any features are computed, the EEG is high-passed to remove baseline wander, notched at the local mains frequency to kill powerline hum, and screened for EOG and EMG contamination — with frontal channels watched for blinks and high-frequency shelves watched for muscle. Epochs that fail a signal-quality check are flagged so they do not corrupt the trained model. Every one of these choices is a direct consequence of having first *named* the noise, and each sets up a specific tool from Chapters 9 and 10.

8.16 Summary and key formulas

Before you name the noise, screen the data: a quality-control pass detects flatlines, clipping, disconnected or railing electrodes, improbable amplitudes, missing channels, sampling-rate mismatches, and annotation/time-alignment errors, so broken segments are excluded or flagged rather than silently filtered. Then name the noise before you filter it. Characterize contamination through three lenses — time, frequency, and (for images) space — and quantify it with SNR, remembering that *where* the noise sits matters as much as how much there is. White noise is flat and irreducible in-band; pink and brown noise are identified by PSD slopes of -1 and -2; powerline hum is a sharp line at 50/60 Hz and harmonics (notch it); baseline wander is low-frequency drift from the electrode–skin interface (high-pass it); impulsive pops and their salt-and-pepper twin are outliers (median-filter them); and physiological cross-contamination (EOG/EMG in EEG) is structured and often has a reference channel for adaptive cancellation. The signature chooses the tool.

Noise / artifact	Time signature	Frequency signature	Typical cure
White (thermal)	structureless fuzz	flat PSD (slope 0)	averaging; better hardware
Pink (1/f)	gentle undulation	slope ≈ -1	often physiology, not removed
Brown (random walk)	slow wandering	slope ≈ -2	high-pass / detrend
Powerline	fine steady buzz	line at 50/60 Hz + harmonics	notch filter
Baseline wander	slow drift	very low frequency	high-pass filter
Electrode pop / spike	brief huge outlier	broadband transient	median filter
Salt-and-pepper (2-D)	random black/white pixels	broadband	median filter
EOG / EMG in EEG	stereotyped shape / bursts	large low-f / high-f shelf	adaptive cancellation

varkonyibarnabas@gmail.com

Key relations: $\text{SNR}_{\text{dB}} = 10\log_{10}(P_{\text{signal}}/P_{\text{noise}})$; PSD slope $0/-1/-2 \rightarrow$ white/pink/brown; ADC $\text{SNR}_{\text{dB}} \approx 6.02N + 1.76$.

8.17 Conceptual questions and exploration tasks

1. You are handed a spectrum with a flat floor and a single razor-sharp spike at 60 Hz. Name the two components and the tool for each.
2. Explain why a mean (averaging) filter fails on electrode-pop noise but a median filter succeeds. Connect this to the salt-and-pepper image case.
3. Given three noise traces, you fit slopes of 0.0, -1.0, and -2.0 to their log-log PSDs. Identify each and name a physical source.
4. Baseline wander in an ECG traces to the electrode–skin interface. What cure do you apply, and what is the danger of setting its cutoff too high?
5. A frontal EEG channel shows large, stereotyped deflections not present at occipital channels. What is the likely contamination, and what property of the setup would let you cancel it adaptively?
6. Your ADC has 8 bits and the recorded small signal looks steppy. Estimate the quantization-limited SNR, and state how many extra bits you would add to gain 12 dB.

8.18 Further reading

For a thorough, practical catalog of biosignal noise and artifacts, Rangayyan's *Biomedical Signal Analysis* is a standard reference and organizes the material by physiological source. Sörnmo and Laguna's *Bioelectrical Signal Processing in Cardiac and Neurological Applications* is excellent on the electrode–skin interface, powerline interference, and baseline wander. For the statistical and spectral characterization of coloured noise, Percival and Walden (also cited in Chapter 7) is authoritative. On image noise models — Gaussian and salt-and-pepper — and the median filter that motivates Chapter 9, Gonzalez and Woods' *Digital Image Processing* gives the clearest treatment.

varkonyibarnabas@gmail.com

Chapter 9 — Digital Filter Design for 1-D Signals & 2-D Images

How do we keep the spindles and drop the hum — and why is the trick for erasing "snow" from an image the same trick that erases spikes from an ECG?

Every recording of a living body arrives contaminated. A scalp electrode faithfully reports the brain's 13 Hz sleep spindle, but it reports the building's 50 Hz power hum just as faithfully. A wrist sensor tracks the pulse and the tremor of a moving arm with equal enthusiasm. A chest lead captures the QRS complex and, superimposed on it, the electrical bark of a shifting electrode. The instrument does not know which part we want. Deciding what to keep and what to discard — and doing it without mangling the very features a later classifier depends on — is the job of a *filter*. This chapter builds filtering from its foundation, the linear time-invariant system, up through the practical recipes you will reuse in every preprocessing pipeline in this book. Along the way we meet the single most important operation in all of signal processing, convolution, and a deceptively simple lesson — *when to average and when to take the median* — that turns out to be the difference between a clean signal and a ruined one.

9.1 Learning objectives

By the end of this chapter you should be able to:

- Explain what a linear time-invariant (LTI) system is and why its behaviour is *completely* described by its impulse response (L1, L2).
- Interpret convolution as "the impulse response slid along the signal," and connect it to the transfer function and frequency response (L2, L3).
- Contrast FIR and IIR filters and choose an IIR family (Butterworth, Chebyshev, elliptic) from a specification (L3).
- Apply zero-phase filtering with `filtfilt`, and reason about padding and edge transients (L3).
- Distinguish linear (mean/Gaussian) from non-linear (median) filtering, and know that impulsive, outlier-type noise demands the median (L3, L4).
- Extend all of the above to two dimensions: spatial convolution kernels, Gaussian smoothing, and Sobel edge detection on images (L3).

9.2 Intuition first: a system is a machine, a filter is its fingerprint

Picture a signal-processing "black box": a signal goes in, a signal comes out. If the box obeys two modest rules, everything we do in this chapter follows. The first rule is linearity: if you double the input, you double the output, and the response to a sum of inputs is the sum of the individual responses (superposition). The second is time-invariance: the box behaves the same today as tomorrow — delay the input by half a second and the output is simply delayed

varkonyibarnabas@gmail.com

by half a second, unchanged in shape. A box obeying both is a linear time-invariant (LTI) system, and the astonishing fact is this: an LTI system's entire behaviour is captured by a single measurement — its response to one infinitely brief "poke."

That poke is the unit impulse $\delta[n]$ (a 1 at $n = 0$, zeros elsewhere), and the box's reaction to it is the impulse response $h[n]$. Why does one measurement suffice? Because any input signal can be written as a sum of scaled, shifted impulses — sample by sample. By linearity, the output is the same sum of scaled, shifted *impulse responses*. Stack them up and you have computed the output for *any* input. That summation has a name, and it is the engine of this whole book.

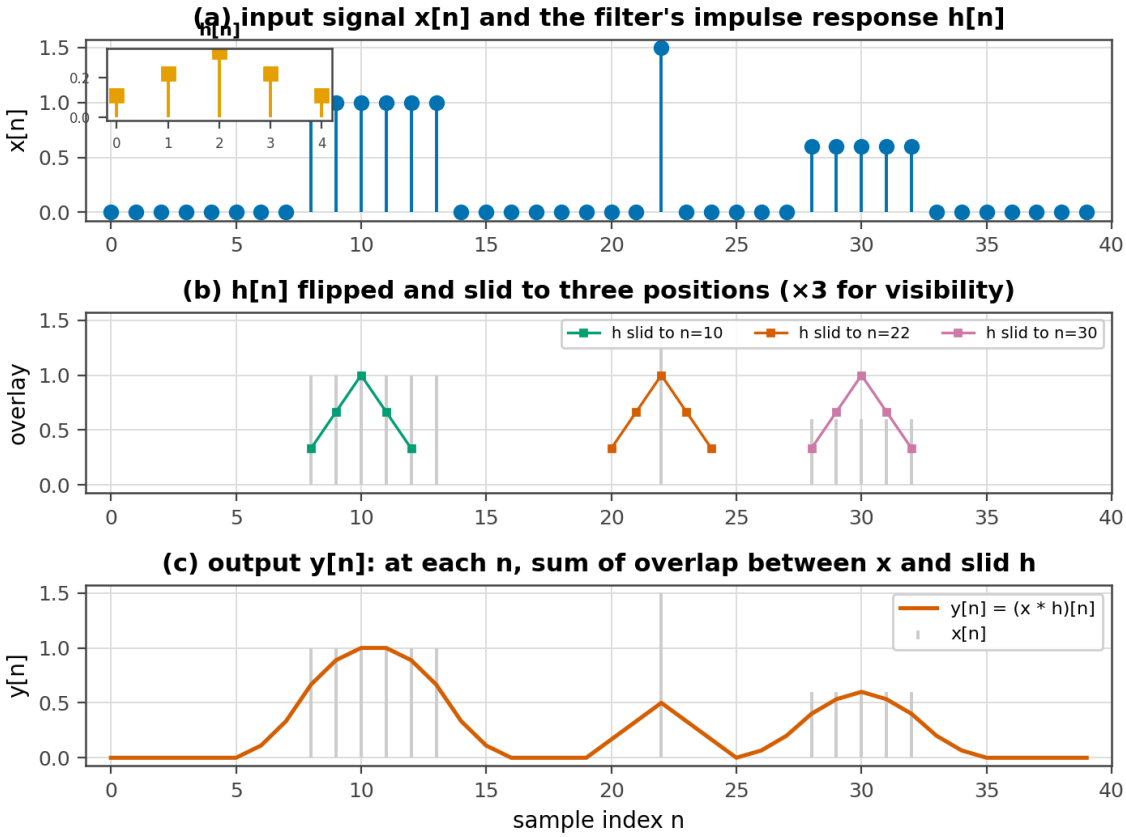


Figure 9.1. Convolution is not an abstraction: it is the impulse response $h[n]$ flipped, slid along the signal $x[n]$, and at each position multiplied-and-summed against the overlap. The smoothing kernel widens the two rectangular bumps gently but tames the lone spike only partially — a preview of why impulsive noise needs a different tool.

Figure 9.1 shows the mechanics with a five-tap smoothing kernel. In panel (a) we have a signal with two flat bumps and one tall spike, plus the kernel $h[n]$ inset. In panel (b) the kernel is flipped and parked at three positions. In panel (c) the output $y[n]$ is drawn: at every index, it is the sum of the products of overlapping samples. Where the signal is flat, the kernel averages the neighbourhood and reproduces the level; at edges it rounds the corners; at the spike it spreads the energy over several samples but cannot remove it. This picture is worth memorising: a linear filter is its impulse response, and filtering is that impulse response slid along the data. The same mental image will return in Chapter 14, where a convolutional neural network learns its own $h[n]$ instead of us designing it.

varkonyibarnabas@gmail.com

9.3 Theory: impulse response, convolution, and the frequency response

The convolution sum makes the intuition precise (Appendix C walks through it step by step if you want the mechanics in isolation first). For a discrete signal $x[n]$ and impulse response $h[n]$,

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k]\, h[n-k].$$

Read it literally: to get output sample n, take the impulse response h, reverse it and shift it to position n ($h[n-k]$ as a function of k), multiply point-by-point with the input, and sum. Slide n forward one step and repeat. Convolution is commutative ($x * h = h * x$), associative, and distributive — which is why cascading two filters is equivalent to convolving their impulse responses into one.

An LTI system also satisfies a difference equation relating output to input,

$$y[n] = \sum_{i=0}^{M} b_i\, x[n-i] \;-\; \sum_{j=1}^{N} a_j\, y[n-j],$$

whose coefficients b_i (feed-forward) and a_j (feedback) *are* the filter design. Taking the Z-transform (Chapter 5) turns convolution into multiplication and yields the transfer function

$$H(z) = \frac{\sum_i b_i z^{-i}}{1 + \sum_j a_j z^{-j}} = \frac{B(z)}{A(z)}.$$

Note the sign on a_j: the AR model of §6.6 writes $x[n] = +\sum_k a_k\, x[n-k] + w[n]$ with $A(z) = 1 - \sum_k a_k z^{-k}$, whereas the filter convention here (and SciPy's `b, a` convention) puts the minus sign on the difference equation itself and $A(z) = 1 + \sum_j a_j z^{-j}$. The two a vectors differ by a sign — feeding one chapter's coefficients to the other's formula flips every pole through the origin, so convert deliberately rather than by habit.

The roots of $B(z)$ are the zeros (frequencies the filter suppresses) and the roots of $A(z)$ are the poles (frequencies it emphasises). Evaluating $H(z)$ on the unit circle, $z = e^{j\omega}$, gives the frequency response $H(e^{j\omega})$, whose magnitude tells you how much each frequency is scaled and whose phase tells you how much each is delayed. When we plot "the filter's response," we are plotting exactly this. Two further properties fill out the picture (linearity and time-invariance were already established in §9.2): causality ($h[n] = 0$ for $n < 0$; the output cannot anticipate the input — mandatory for real-time monitoring, optional for offline analysis) and stability (a bounded input yields a bounded output, guaranteed when every pole sits *inside* the unit circle).

varkonyibarnabas@gmail.com

Box A — Linear filtering methods. A *linear* filter obeys superposition, so its action is a convolution and its effect is fully described in the frequency domain by $H(e^{j\omega})$. The workhorses are the moving-average / FIR smoother (equal or tapered weights), the Gaussian smoother (a bell-shaped kernel that is a smooth, ring-free low-pass; whether it is *optimal* depends entirely on the assumptions you make about the signal and noise, and for a plain Gaussian kernel those assumptions are rarely stated), and the recursive IIR filter (feedback gives a sharp response at low cost). If you do want the *optimal* linear estimator under stated second-order/stationary assumptions — minimising mean-squared error given the signal and noise power spectra — the right object is the Wiener filter, not a fixed Gaussian; the Gaussian smoother is best read as a convenient, well-behaved low-pass rather than an optimum. Linear filters are the right tool when signal and noise occupy *different frequency bands* and the noise is well-behaved (stationary, roughly Gaussian). They fail badly on *outliers*, because averaging a spike into its neighbours smears it rather than deletes it — the lesson of §9.7.

9.4 FIR versus IIR, and the IIR families

Filters come in two architectures. A finite impulse response (FIR) filter has no feedback ($a_j = 0$): its impulse response is exactly the list of b_i coefficients and dies after $M + 1$ samples. FIR filters can be designed to have *exactly linear phase* — every frequency is delayed by the same amount, so no *phase* distortion: the passband is passed with its waveform *shape* and relative timing intact (the magnitude response still shapes whatever it attenuates — see §9.5). That property matters enormously when the *shape* of a QRS complex or a K-complex is diagnostic. They are unconditionally stable. Their cost is order: a sharp cutoff may need hundreds of taps.

FIR filters are designed in two main ways. The window method (`scipy.signal.firwin`) starts from the ideal brick-wall impulse response — a sinc — and multiplies it by one of the window functions of Chapter 4 to tame the truncation ripple, trading transition width against stopband depth exactly as those windows did. The Parks–McClellan algorithm (`scipy.signal.remez`) instead solves for the *optimal* taps directly: given the band edges and the relative weight you place on passband versus stopband error, it finds the equiripple filter that minimises the *maximum* deviation from the ideal (the Chebyshev/minimax criterion), spreading the error evenly rather than letting it pile up near the transition. For a fixed tap count, Parks–McClellan meets a tighter specification than the window method — which is why it is the standard choice for a demanding anti-alias or narrow-band filter — while the window method wins on simplicity and when a specific window shape is wanted.

An infinite impulse response (IIR) filter uses feedback, so its impulse response rings on forever. Feedback buys a very sharp transition for a very low order (a 6th-order IIR can rival a 100-tap FIR), but at the price of non-linear phase and the need to keep poles inside the unit circle for stability. IIR filters are the standard choice for biosignals when phase can be repaired afterwards — which, offline, it can (§9.5).

One practical caveat on realising an IIR filter: a high-order design should be implemented as a cascade of second-order sections (SOS, or "biquads") rather than as a single high-order transfer function in the b/a form of the transfer function above. Packing many poles and zeros into one long numerator and denominator makes the coefficients numerically fragile — small rounding errors can shift a pole across the unit circle and destabilise the fil-

varkonyibarnabas@gmail.com

ter — whereas the SOS form spreads them across independent, well-conditioned 2nd-order blocks. In practice you design with `scipy.signal` using `output='sos'` and filter with `sosfilt`/`sosfiltfilt`. This matters most under the fixed-point arithmetic of embedded deployment (Chapter 15), where coefficient quantisation makes the single-transfer-function form especially unsafe. Each biquad is itself wired in a standard realisation structure — most often Direct Form II, which shares its delay elements between the feed-forward and feedback halves to use the minimum memory; the same difference equation can be realised several equivalent ways that differ only in numerical behaviour, not in their ideal response. The analogous efficiency trick for *multirate* filters is the polyphase structure, which computes a decimator or interpolator (§2.13) without ever forming the samples it is about to throw away.

Among IIR designs, the classical families trade the same currency — passband flatness, stopband depth, transition sharpness, and phase — in different proportions. Figure 9.2 designs three at matched 6th order and a 30 Hz cutoff.

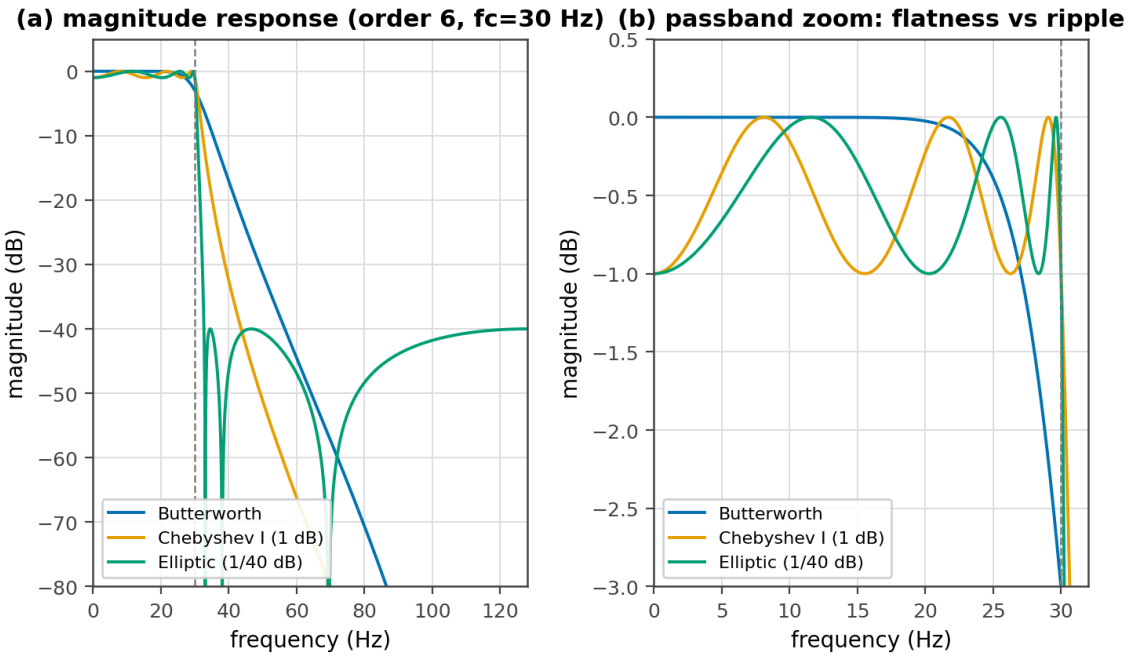


Figure 9.2. The same order buys different things. Butterworth is maximally flat in the passband but rolls off gently; Chebyshev I trades equiripple passband error for a steeper skirt; the elliptic filter, allowing ripple in both bands, achieves the sharpest transition of all. The passband zoom (right) makes the flatness-versus-ripple bargain literal.

Family	Passband	Stopband	Transition	Phase	Typical biosignal use
Butterworth	maximally flat	monotone	gentle	fairly linear	general-purpose band-pass; when passband fidelity matters
Chebyshev I	equiripple	monotone	steeper	worse	sharper cutoff, ripple tolerable outside features

varkonyibarnabas@gmail.com

Family	Passband	Stopband	Transition	Phase	Typical biosignal use
Chebyshev II	flat	equiripple	steeper	worse	flat passband + guaranteed stopband floor
Elliptic (Cauer)	equiripple	equiripple	steepest	worst	tight bands, lowest order (e.g., anti-alias)
Bessel	flat, gentle roll-off	gentle	gentlest	maximally flat group delay	when waveform shape must be preserved (near-constant delay)

The engineering reading of Figure 9.2: if you must not distort the passband magnitude, choose Butterworth; if you must preserve the *waveform shape* — because a filter's group delay (the delay it imposes as a function of frequency) can smear a transient when it varies across the band — choose Bessel, whose defining virtue is a *maximally flat group delay* (near-constant delay, hence minimal phase distortion) rather than any special magnitude sharpness; if you must reject an interferer very close to the passband edge at minimal order, choose elliptic and accept the ripple; Chebyshev sits between. There is no universally best filter — only the best filter *for a stated specification.*

> Box B — How to design a filter, step by step. (1) Write the specification from the signal's band structure: passband and stopband edges, allowed passband ripple, required stopband attenuation, transition width. (2) Choose FIR vs IIR and a family (linear phase → FIR; sharp cutoff at low order → elliptic IIR; flat passband → Butterworth). (3) Choose the order — use `scipy.signal.buttord/ellipord` to get the minimum order meeting the spec. (4) Realise and apply with zero-phase `filtfilt`. (5) Validate on real data: plot the magnitude/phase response *and* the before/after spectra. We work this recipe once here for an EEG band-pass + 50 Hz notch (§9.6) and reuse it thereafter.

9.5 Zero-phase filtering, padding, and edge transients

An IIR filter's non-linear phase shifts different frequencies by different amounts, which can, for example, move the peak of a QRS complex relative to its onset — corrupting a measurement that depends on timing. The offline cure is elegant: filter the signal forward, reverse it, filter again, and reverse back. The two passes apply the phase shift and then its negative, cancelling it. The result is zero-phase filtering, implemented as `scipy.signal.filtfilt`. What `filtfilt` actually guarantees is worth stating carefully: it cancels phase delay, so it preserves timing and phase alignment — features are not shifted relative to one another — but it is *not* shape-preserving in general. The forward–backward pass squares the magnitude response (so a -3 dB point becomes -6 dB, and the effective attenuation is doubled — design with that in mind), and it can still attenuate clinically relevant components or introduce edge and ringing transients wherever the magnitude response bites. The right slogan is: `filtfilt` preserves *timing*; you must still verify *shape*. The price on top of that is that `filtfilt` is inherently non-causal, so it is for recorded data, not for a real-time monitor.

varkonyibarnabas@gmail.com

At the *ends* of a finite record the filter has no data to reach into, and how you fill that void — the padding choice — shapes the start-and-end transient. `filtfilt`'s `padtype` offers `'odd'` (odd/antisymmetric extension, the default, which continues the slope smoothly), `'even'` (mirror/reflect, which continues the level), `'constant'`, or `None` (no padding — the filter starts from a cold state, injecting a startup step and ringing; note that `padtype=None` means *no* extension, not zero-padding). For a signal with baseline wander, odd extension usually minimises the edge artifact; for one that returns to baseline, reflection is fine. This is the applied payoff of the boundary-handling ideas introduced as boundary handling in §3.8 and windowing in Chapter 4: the same "what happens at the edge?" question, now answered for filters.

9.6 Worked example: an EEG band-pass + 50 Hz notch (a law and a number)

What must survive this filter? Before choosing a single cutoff, name what the downstream task cannot afford to lose: for sleep-staging EEG that is the delta-band content (0.5–4 Hz) that defines deep sleep, the spindle morphology (~11–16 Hz), and the timing and relative amplitude of graphoelements. A filter that cleans the trace but flattens delta or smears a spindle has failed the task even if the plot looks tidier. Design to the specification below with those survivors in mind.

Raw EEG carries three unwanted guests: slow baseline drift below ~0.5 Hz, broadband muscle and instrumentation noise above the cortical bands, and a sharp 50 Hz (or 60 Hz) powerline hum. We will fix a single, coherent, application-appropriate specification and design to it. For sleep-staging EEG — where the slow delta rhythm (0.5–4 Hz) is a load-bearing feature of deep sleep and must be preserved — the appropriate answer is a high-pass at 0.5 Hz (removing baseline drift while sparing delta), a low-pass at about 35–40 Hz (keeping delta through beta while rejecting muscle and instrumentation noise), and a 50 Hz notch to delete the powerline hum. Following Box B, we realise the band-pass as a 4th-order Butterworth (0.5–40 Hz here) and the line rejection as a narrow IIR notch (`iirnotch`, quality factor $Q = 30$) at 50 Hz, applied with zero-phase `filtfilt`. This worked example samples at 200 Hz (comfortably above the 100 Hz baseline used to keep earlier chapters' arithmetic simple), so the 50 Hz line sits well below the 100 Hz Nyquist frequency and the notch is a genuine narrow band-stop rather than a degenerate filter at the Nyquist edge. One caveat from §9.5 bites precisely here: because `filtfilt` runs the filter twice, it squares the magnitude response, so the *effective* high-pass attenuation at the 0.5 Hz corner is doubled (−3 dB becomes −6 dB) and reaches a little way into the 0.5–4 Hz delta band. If preserving delta amplitude exactly matters, design the high-pass to a slightly lower corner (or verify the two-pass response across 0.5–4 Hz) rather than trusting the single-pass 0.5 Hz specification.

The cutoffs are a deliberate choice, not a default, because they are application-specific. A more aggressive high-pass — say at 1 Hz — noticeably attenuates the delta band and would quietly weaken exactly the feature that distinguishes deep sleep, so it is the wrong choice *here* even though it is common for other EEG tasks that do not care about delta. The same caution applies across modalities: an ECG high-pass set too high distorts the low-frequency ST segment that clinicians read for ischemia, which is why diagnostic ECG uses a very low high-pass cutoff. The lesson is that a cutoff is chosen against the feature you must protect, not copied from another pipeline.

varkonyibarnabas@gmail.com

Figure 9.4 (bottom row) shows the result on synthetic N2 EEG with an added 50 Hz hum. In the time trace the ragged high-frequency fuzz vanishes and the underlying rhythm emerges; in the power spectrum the tall spike at 50 Hz (dashed line) is driven down into the noise floor while the delta-through-beta content below ~40 Hz is left intact. The reproducible number (on this synthetic example): isolating the notch's own contribution — the PSD at 50 Hz just before the notch versus just after it — the notch attenuates the line by roughly 48 dB (a factor of about 250 in amplitude) on top of the roughly 23 dB the 0.5–40 Hz band-pass already removes there on its own (its low-pass edge reaches into the line), for a measured ~71 dB total from raw to filtered; the notch's own half-power bandwidth is only about $50/Q \approx 1.7$ Hz wide. It is worth being precise about what Q controls here: Q sets the notch's bandwidth and selectivity, not its depth. For an ideal notch the zero sits exactly *on* the unit circle, so the null is essentially bottomless — the depth is not a function of Q at all. The "~48 dB" is a *measured residual* on this synthetic mains line (limited by finite arithmetic, the pole placed just inside the zero, and the exact line frequency), whereas $Q \approx 30$ is what makes the notch only ~1.7 Hz wide. Read the two numbers separately: one is how deep the null happens to bite on this example, the other is how narrow Q makes it. That is the whole art of the notch: deep and narrow, so it deletes the interferer without eating its immediate neighbours. Note that with a 40 Hz low-pass in place, any activity at or above ~48 Hz — including much of the gamma band — is already strongly attenuated by the band-pass itself; if a study genuinely needed gamma above 40 Hz, the low-pass and notch would have to be redesigned together, and one could not simultaneously keep a 48 Hz component and low-pass at 40 Hz.

> Common student mistakes. Filter design goes wrong most often through phase, not magnitude. First, judging a filter by its magnitude response alone. A filter that passes exactly the right band can still wreck a signal if its phase (equivalently, its group delay) varies across that band, because different frequency components of a transient then arrive at different times — enough to smear a QRS complex or a K-complex out of the shape a clinician or classifier reads. Always inspect phase and group delay alongside magnitude. Second, treating zero-phase `filtfilt` as a free lunch. It cancels phase delay, but the forward–backward pass doubles the attenuation (the magnitude response is squared) and needs careful edge handling (§9.5) or it injects start-and-end transients — it fixes timing, not every side effect. Third, reaching for too-aggressive cutoffs. A high-pass placed at 1 Hz "to remove drift" quietly eats the EEG delta band that defines deep sleep; an ECG high-pass set too high distorts the low-frequency ST segment that clinicians read for ischemia. Set each cutoff against the feature you must protect, not by habit.

9.7 The outlier-removal gateway: mean versus median (taught with images first)

Not all noise is created equal, and the frequency-domain worldview of §9.2–9.6 has a blind spot: impulsive, outlier-type noise. The clearest picture of it is an image corrupted by salt-and-pepper noise — random pixels flipped to pure black or pure white, like snow on an old television. Figure 9.3 (top row) shows a photographic image hit with 10% salt-and-pepper, and on a real photograph — with its fine texture, smooth tones, and sharp edges — the lesson is unmistakable in a way a smooth phantom could never make it. Now compare two repairs. A mean (averaging) filter replaces each pixel with the average of its 3×3 neighbourhood — a *linear* filter. Because averaging is sensitive to outliers, each white speck drags its neigh-

varkonyibarnabas@gmail.com

bourhood upward: the specks are only diluted, not deleted, and every edge and texture in the photograph is blurred into mush (mean-squared error falls only modestly). A median filter replaces each pixel with the *median* of its neighbourhood — a *non-linear* filter. A lone extreme value cannot be the median of nine, so the speck is simply discarded outright, and because the median of a locally uniform region is that region's value, the photograph's edges and texture stay razor-sharp. In Figure 9.3 the 3×3 median drives the error down more than threefold versus the mean (MSE 0.007 → 0.002) and returns an image that looks almost untouched.

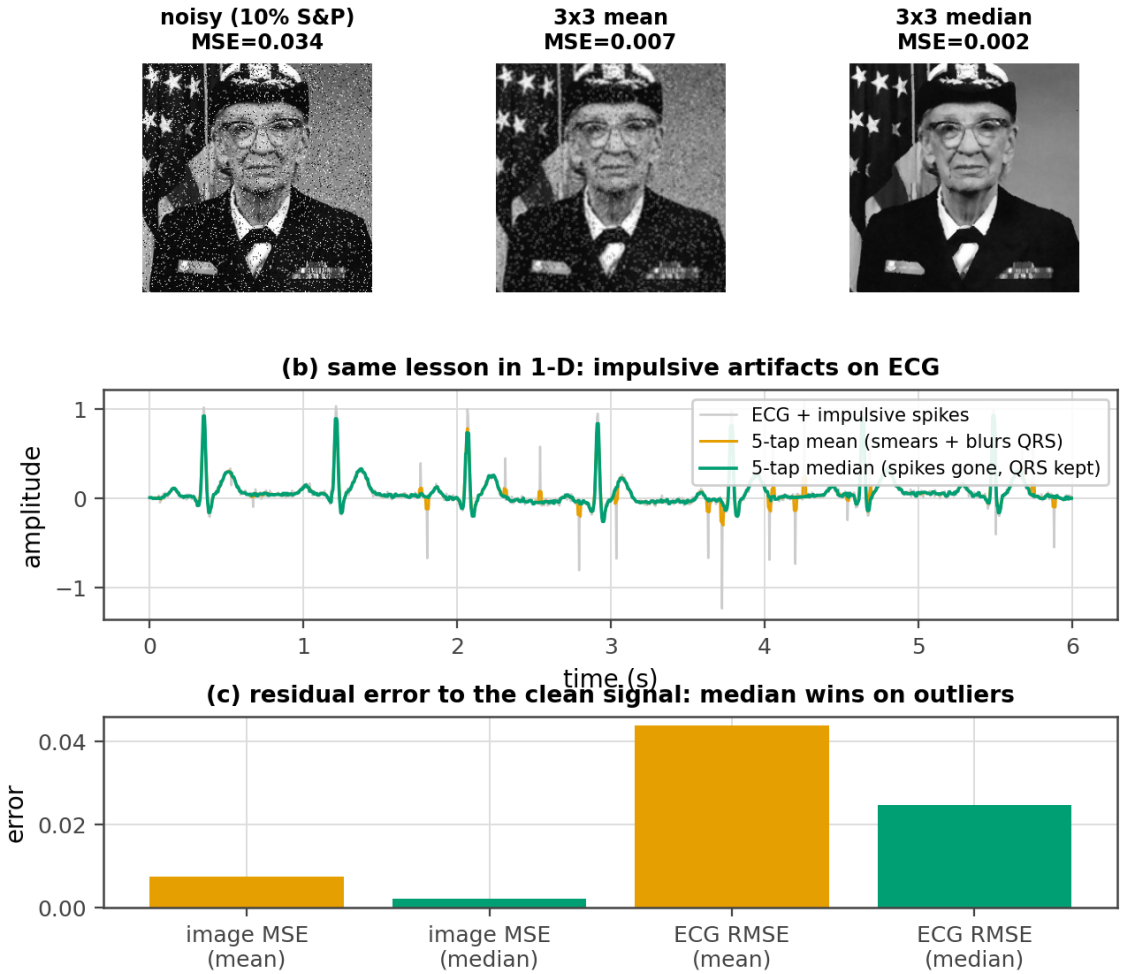


Figure 9.3. The same lesson in two dimensions and one, shown on a photographic image so the effect on real edges and texture is obvious. Top: a photograph with 10% salt-and-pepper noise; the 3×3 mean only dilutes the specks while blurring away every edge and texture, whereas the 3×3 median removes the specks while keeping edges sharp (note the far lower MSE) — as long as the window stays smaller than the features it must preserve. Middle: impulsive electrode "pops" on an ECG — the 5-tap mean blurs the QRS while barely denting the spikes, the 5-tap median erases the spikes and preserves the QRS. Bottom: residual error to the clean signal, median winning in both modalities. Averaging is linear and outlier-sensitive; the median is non-linear, outlier-robust, and edge-preserving.

Here is the payoff promised by design principle 3 — *build the intuition on an image, then cash it in on a signal.* Motion and electrode contact produce exactly the 1-D analogue of salt-and-pepper: brief, extreme spikes ("pops") riding on an otherwise clean ECG. Figure 9.3 (middle) adds such spikes and applies the same two operators in 1-D. The 5-tap mean smears each

varkonyibarnabas@gmail.com

spike across five samples *and* rounds off the sharp QRS peak we most want to keep. The 5-tap median (`scipy.signal.medfilt`) removes the spikes cleanly while leaving the QRS complex — which is never a lone outlier — essentially intact. The lesson is a rule you will apply for the rest of your career:

- Gaussian, band-limited, stationary noise that occupies a different band from the signal → a linear filter (mean, Gaussian, Butterworth).
- Impulsive, outlier, spike, or salt-and-pepper noise → a non-linear median filter, because the median ignores extremes that no linear average can. This also fixes the order when a signal carries both kinds of noise (say powerline hum *and* electrode pops): remove the impulses with a median pass *first*, because a spike is broadband and a linear filter (a notch or band-pass) cannot delete it — it only smears the spike across a wide neighbourhood — so impulse removal must precede any linear filtering.

> Savitzky–Golay: smoothing that keeps the peak. Between the blurring mean filter and the outlier-robust median sits a third tool worth knowing. The Savitzky–Golay filter slides a short window along the signal and, at each position, fits a low-order polynomial (typically quadratic or cubic) to the samples by least squares, then reports the fitted value at the window's centre. Because a parabola can follow a peak that a flat average cannot, Savitzky–Golay smooths broadband noise while largely preserving the height, width, and area of peaks — the property a mean filter destroys — provided the window is kept shorter than the peak it must follow; too long a window, or too low a polynomial order, still flattens a narrow feature. That makes it a favourite for ECG and optical spectroscopy, where the amplitude and shape of a peak *are* the measurement; it is also one of the few smoothers that returns a smoothed derivative for free (differentiate the fitted polynomial analytically), handy for slope-based feature detection. It is still a *linear* filter, so, like the mean, it does not reject true outliers — pair it with a median pre-pass when you need both smoothing and spike removal. In SciPy it is `savgol_filter(x, window_length, polyorder)`.

9.8 Time–frequency duality and 2-D linear filtering

Two threads close the chapter. First, a duality worth internalising: convolution in time equals multiplication in frequency. This is the §3.12 dictionary of dual pairs, now cashed out in filter design. Smoothing (convolving with a low-pass kernel) multiplies the spectrum by a low-pass window; differencing ($y[n] = x[n] - x[n-1]$, convolution with $[1, -1]$) multiplies by a high-pass response, which is why differencing sharpens edges and amplifies high-frequency noise. A moving average *is* a low-pass filter, and subtracting a moving average is a high-pass filter — which is exactly why the simplest ECG baseline-wander fix (subtract a smoothed estimate of the slow baseline) is a high-pass, no filter-design machinery required. Every filter has these two faces; choose whichever domain makes the design and the reasoning easiest.

This duality is also how filtering is implemented *fast*: for a long signal and a long kernel it is cheaper to take the FFT of each, multiply, and inverse-transform than to slide the kernel sample by sample. But there is a trap.

varkonyibarnabas@gmail.com

FFT convolution: circular vs linear. Multiplying two DFTs and inverse-transforming does not by itself give the ordinary (linear) convolution you designed the filter for — it gives circular convolution, in which the tail of the output wraps around and contaminates the beginning (as if the signal were periodic). The result equals true linear convolution only if you zero-pad both sequences to at least $N + M - 1$ samples (signal length N, kernel length M) before the FFT, or use a block method — overlap-add or overlap-save — that stitches padded blocks together correctly. A plain DFT product with no padding will corrupt the edges with wrap-around artifacts. Library routines such as `scipy.signal.fftconvolve` handle the padding for you; if you ever roll your own FFT-based filter, pad first.

Second, everything generalises to images by moving from a 1-D kernel slid along a line to a 2-D kernel slid across a plane — spatial convolution. Figure 9.4 (top row) applies two classic kernels to an image. A Gaussian kernel blurs gently, suppressing fine noise (the 2-D counterpart of 1-D Gaussian *smoothing*; the exact twin of a 1-D moving average is instead a uniform box mean kernel, and the Gaussian is its smoother, ring-free cousin — both are low-pass, but the box kernel's frequency response has sinc-shaped sidelobes that ring, whereas the Gaussian's is smooth). The Gaussian is also separable, so it can be applied as two cheap 1-D passes. A Sobel kernel approximates the spatial derivative and, taking the gradient magnitude, lights up edges — the 2-D analogue of 1-D differencing. These are the payoff of the convolution concept from §9.2: the operation that filtered our ECG also smooths an MRI slice and finds the boundary of a tumour, and it is the same operation a CNN will later *learn*.

9.9 Worked example: Pan–Tompkins QRS detection

Almost everything a clinician or an algorithm wants to know from an ECG begins with one deceptively simple question: *where are the heartbeats*? The tall, spiky QRS complex marks the ventricles firing, and its sharp peak — the R-peak — is the natural fiducial for the whole beat. Find the R-peaks and you have the instantaneous heart rate (the reciprocal of the R–R interval), the raw material for heart-rate variability (HRV) analysis, the triggers that let you ensemble-average many beats into a clean P–QRS–T template, and the beat-by-beat morphology that arrhythmia detectors classify. R-peak detection is therefore the base layer beneath a large fraction of ECG processing: get it wrong and every downstream number inherits the error. The difficulty is that the raw trace is hostile to naïve thresholding — a tall T wave, a wandering baseline, or a burst of muscle noise can all rise above a fixed level, while a low-amplitude beat can hide beneath it. The classic answer, published by Pan and Tompkins in 1985 and still a workhorse today, is not a single clever filter but a short pipeline that transforms the signal until a beat becomes an unmistakable bump, then applies an adaptive threshold. Each stage is a filter you already understand, and each maps directly onto the time/frequency dual-pairs dictionary of §3.12.

Figure 9.5 runs the whole pipeline on ten seconds of synthetic ECG (250 Hz) carrying realistic beats, heart-rate variability, and baseline wander. Stage 1 is a band-pass at roughly 5–15 Hz (a zero-phase Butterworth). This is the frequency-domain scalpel of §9.6 aimed at the QRS: most of the QRS energy lives in this band, whereas slow baseline wander and the broad P and T waves sit below it and muscle noise sits above it. Panel (b) shows the payoff — the P and T waves collapse, the drift is gone, and the QRS stands out as a clean biphasic wiggle. Stage 2 is a five-point derivative, and here the dictionary is explicit: *differencing is a high-pass* (§3.12),

varkonyibarnabas@gmail.com

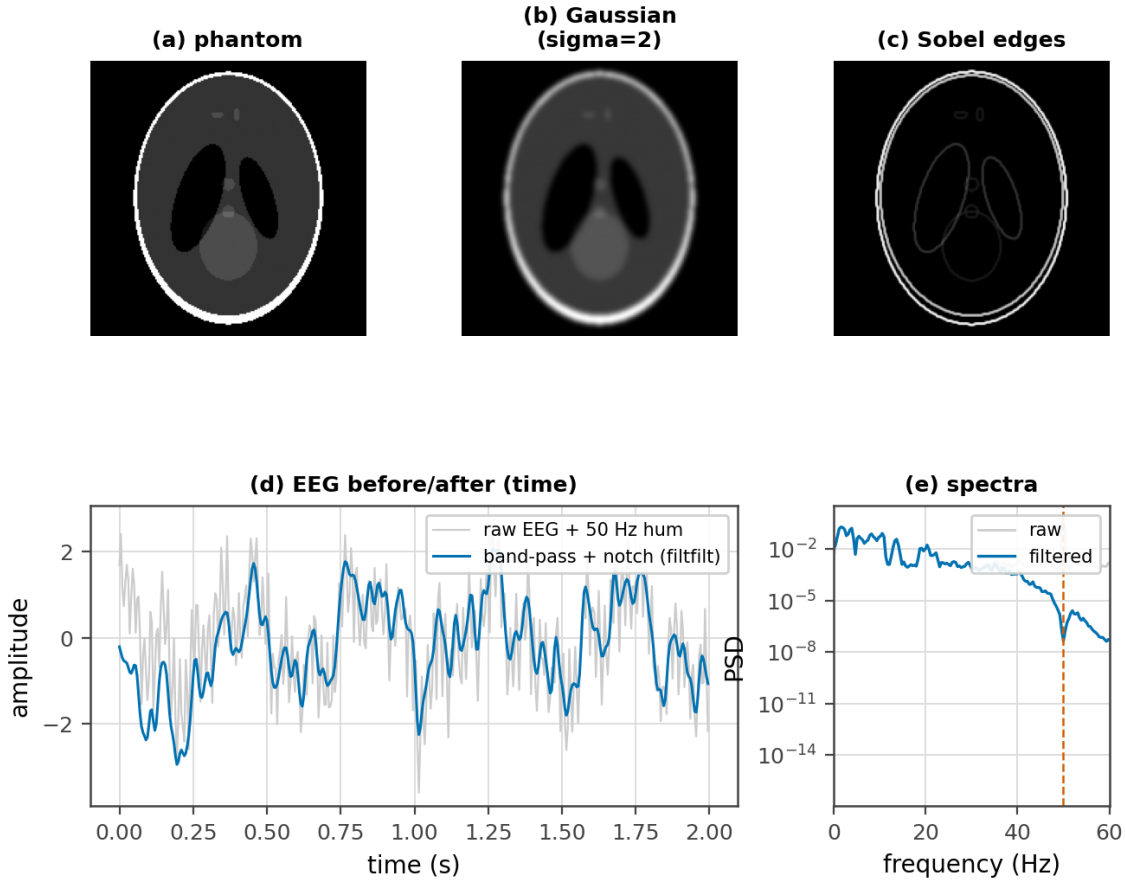


Figure 9.4. Two-dimensional linear filtering is one-dimensional filtering with a slid 2-D kernel. Gaussian smoothing (b) is the image analogue of 1-D low-pass smoothing — a box mean filter is the exact twin of a moving average, the Gaussian its ring-free cousin; Sobel edge magnitude (c) is the image twin of differencing. Bottom: the §9.6 sleep-EEG band-pass (0.5–40 Hz) + 50 Hz notch, before and after, in time (d) and spectrum (e) — the same convolution machinery, applied to a 1-D biosignal.

varkonyibarnabas@gmail.com

so the derivative responds to how *fast* the signal is changing and thus to the QRS's characteristically steep slopes, while ignoring the flatter P and T waves that survived the band-pass. Stage 3 squares the derivative pointwise. Squaring is a non-linearity, not a convolution, and it does two useful things at once: it makes every sample positive (so upstrokes and downstrokes both count as "activity") and, because it is quadratic, it amplifies the large derivatives of the QRS far more than the small ones of residual noise, improving the contrast between beat and background (panel d). Stage 4 is moving-window integration — convolution with a rectangular window about 150 ms wide, the width of a QRS. A moving average *is a low-pass* (§3.12), and its job here is to merge the several sharp, multi-peaked spikes that squaring produced within a single QRS into one smooth bump per beat (panel e). The window width matters: too short and the QRS fragments into several detections, too long and adjacent beats fuse.

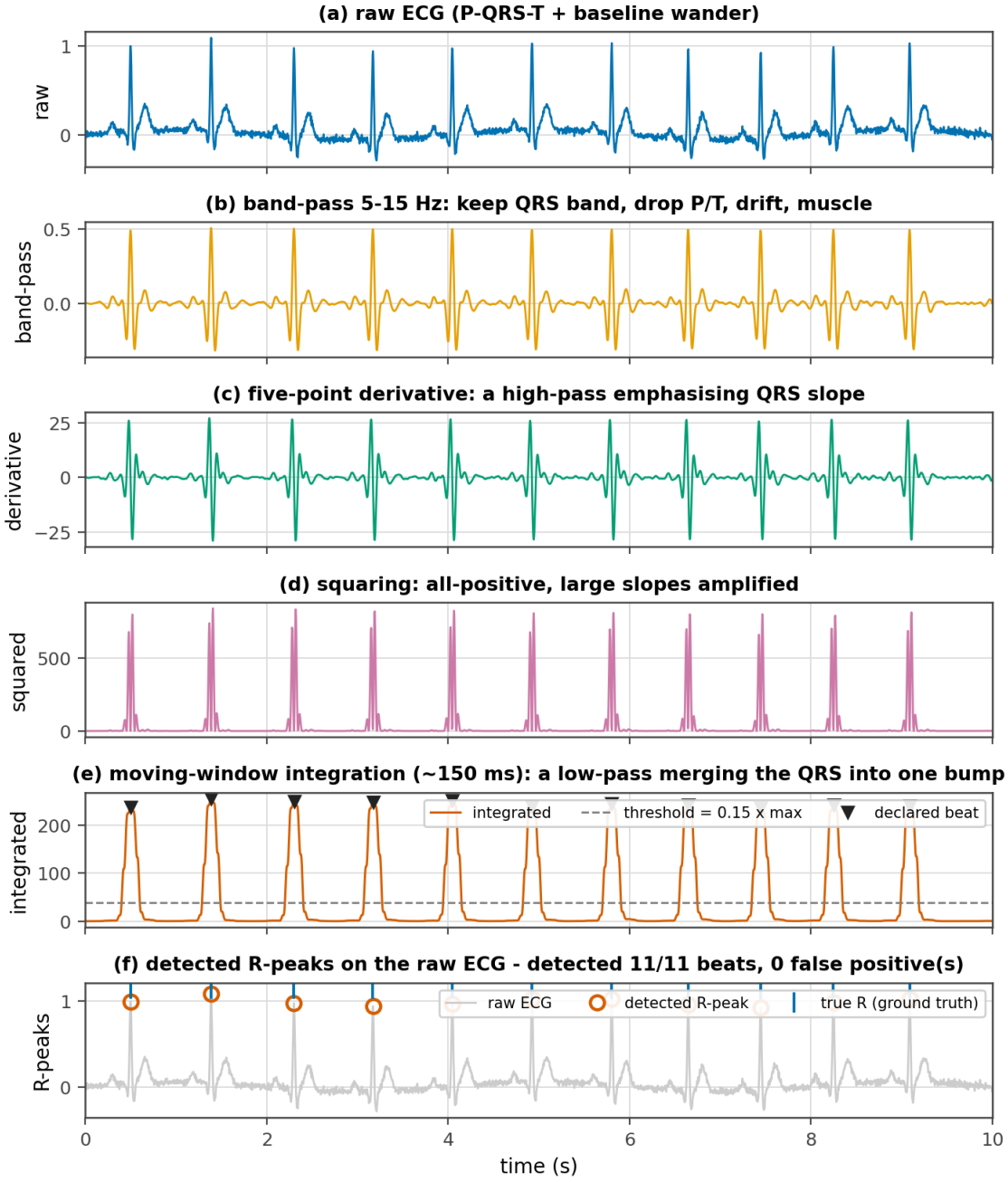
After these four filtering stages a beat is a broad, isolated hump, and Stage 5 declares it with a simple adaptive threshold. We place a level at a fraction of the integrated signal's scale (here 15 % of its maximum, a stand-in for Pan–Tompkins' running noise/signal estimates) and take every peak above it, enforcing a refractory period of about 300 ms so that a single QRS or a tall T wave is not counted twice — a deliberately conservative, *adult* example value (the original algorithm uses ~200 ms, i.e. a 300 bpm ceiling). The right value is population- and task-dependent and should be shortened for neonates or suspected tachyarrhythmias, where beats legitimately arrive faster. Panel (f) marks the detected R-peaks on the raw ECG, having localised each one to the tallest raw deflection near its integrator bump. The law is the pipeline itself — *band-pass*, *differentiate* (*high-pass*), *square*, *integrate* (*low-pass*), *threshold with a refractory guard* — and the number is the score against the ground-truth R-peak times the generator provides: on this synthetic record the detector finds 11 of 11 beats with zero false positives, and the marked peaks sit within a couple of milliseconds of truth. This illustrative result should not be read as a benchmark: on clean, well-conditioned ECG, Pan–Tompkins and its descendants are mature and reliable, and on the standard annotated databases published QRS detectors report very high sensitivity — which is why a pipeline first described in 1985 is still a common starting point. But R-peak detection is *mature, not solved*: it remains genuinely challenging under motion artifact, ectopic beats, low SNR, and the noisier conditions of neonatal and wearable recordings, where naïve thresholds miss beats or fire on artifact. The right stance is that this is a well-understood tool that works dependably in easy settings and still needs care in hard ones.

The detected R-peaks are not the end of the story but the beginning of several. Their spacing is the R–R interval, whose reciprocal is heart rate and whose beat-to-beat fluctuation is the HRV signal analysed spectrally in Chapter 7 — the low- and high-frequency band powers of the R–R tachogram that index autonomic balance depend entirely on accurate R-peak timing. The same peaks serve as the alignment triggers for the ensemble beat-averaging of Chapter 10, which sums many R-locked beats into a low-noise P–QRS–T template. A robust QRS detector is thus the small, sharp tool on which a surprising amount of cardiology software is built.

9.10 The ML/AI bridge

Filtering is the first stage of every pipeline in this book, and its downstream role is twofold. First, it conditions the features: band powers, spectral entropy, and Hjorth parameters (Chapter 11) are only meaningful once the line noise and drift are gone, so a classifier's accuracy often rides on the quality of the preprocessing far more than on the model. Second, convolution *is*

varkonyibarnabas@gmail.com



Pan-Tompkins chains a band-pass, a derivative (high-pass), squaring, and a moving-window integrator (low-pass) so a simple adaptive threshold marks every QRS: 11/11 beats, 0 false.

Figure 9.5. The Pan–Tompkins pipeline (Pan & Tompkins, 1985), stage by stage on 10 s of synthetic ECG. (a) raw trace; (b) a 5–15 Hz band-pass keeps the QRS energy band and rejects P/T waves, baseline wander, and muscle noise; (c) the five-point derivative is a high-pass that emphasises the QRS's steep slopes; (d) squaring makes everything positive and amplifies large slopes; (e) moving-window integration is a low-pass that merges each multi-peaked QRS into a single bump, with the adaptive detection threshold drawn; (f) the resulting R-peaks marked on the raw ECG — detected 11/11 beats with 0 false positives, within milliseconds of the ground truth.

varkonyibarnabas@gmail.com

the model in modern deep learning: a convolutional neural network is a stack of learnable filters whose impulse responses are found by gradient descent rather than by the design recipe of Box B. Understanding the fixed filters of this chapter is the prerequisite for understanding the learned ones of Chapter 14 — same operation, different way of choosing h.

> Signals-and-images. The 1-D/2-D correspondence is exact and worth carrying forward: a time-domain moving average ↔ a 2-D box / mean filter (with the smooth-kernel, ring-free version being a spatial Gaussian blur); 1-D differencing ↔ a Sobel edge detector; a 1-D median spike remover ↔ a 2-D median salt-and-pepper cleaner; a low-pass anti-alias filter ↔ image down-sampling with a pre-blur. Whatever you learn to do to a signal, you can do to an image by sliding the kernel in one more dimension.

9.11 Companion notebook

The companion notebook walks the whole chapter interactively. It builds the convolution animation of Figure 9.1 tap by tap; lets you design Butterworth, Chebyshev, and elliptic filters and watch the magnitude response deform as you drag the order and cutoff; and runs the paired outlier demo — salt-and-pepper on an image and impulsive spikes on an ECG, cleaned side by side by mean versus median (with a Savitzky–Golay smoother added as a third contender, to show it preserving a peak the mean flattens) — so the lesson of §9.7 is something you *do*, not just read. A further cell reproduces the EEG band-pass + notch of §9.6 end to end, printing the measured line attenuation so you can confirm the ~48 dB notch-stage number (and the ~71 dB total) yourself. A final cell runs the Pan–Tompkins pipeline of §9.9 stage by stage on an ECG with known R-peaks, letting you watch each filter reshape the signal and read off the detection rate against ground truth.

9.12 Case-study tie-in

In the integrative sleep-staging case study, this chapter *is* the preprocessing module. Every 30-second epoch of raw EEG, EOG, and EMG first gets a median pass to strike brief electrode pops — per §9.7, impulse removal must precede any linear filter, or the notch/band-pass would only smear the spike across a wide neighbourhood instead of deleting it. Only then is the epoch high-pass filtered to remove baseline wander, notch-filtered at the local line frequency, and band-split into the bands the feature stage will summarise — all with the zero-phase `filtfilt` recipe of Box B, so the spindle and K-complex shapes that distinguish N2 survive intact. Get this stage wrong and every later stage inherits the damage; get it right and the classifier starts from clean, comparable epochs.

9.13 Summary, questions, and further reading

Key formulas. Convolution $y[n] = \sum_k x[k]h[n-k]$; difference equation $y[n] = \sum_i b_i x[n-i] - \sum_j a_j y[n-j]$; transfer function $H(z) = B(z)/A(z)$ with zeros from B, poles from A; frequency response $H(e^{j\omega}) = H(z)\big|_{z=e^{j\omega}}$; stability ⇔ all poles inside the unit circle; zero-phase filtering squares the magnitude and nulls the phase; notch bandwidth $\approx f_0/Q$.

varkonyibarnabas@gmail.com

Key ideas. An LTI system is fully described by its impulse response; convolution is that response slid along the signal; FIR gives linear phase, IIR gives sharpness cheaply; filter families trade passband flatness against transition sharpness; linear filters average and so smear outliers, the non-linear median deletes them; and every one-dimensional operation has a two-dimensional image twin.

Conceptual questions. (1) Why does one impulse-response measurement determine an LTI system's response to *every* input? (2) A colleague applies a causal Butterworth band-pass and complains the R-peak has shifted — what happened and what should they use instead? (3) You have a signal with both 60 Hz hum and occasional electrode pops. In what order do you apply a notch and a median filter, and why does order matter? (4) Sketch what happens to a step edge under a mean filter and under a median filter, and explain the difference in one sentence.

Exploration tasks. Design an elliptic and a Butterworth low-pass to the same specification and compare the minimum order each needs. Take a clean ECG, add both Gaussian noise and impulsive spikes, and show that no single linear filter cleans both — then fix it with a median-then-band-pass cascade. Apply a Gaussian and a Sobel kernel to a medical image slice and relate each to its 1-D counterpart.

Design brief. A wearable single-lead ECG must (i) raise a *real-time* arrhythmia alarm from R-peak timing on a low-power microcontroller and (ii) store a trace a cardiologist will later read for ST-segment changes. Work the five moves. Spec: list what the filter chain must preserve versus may remove for *each* use, and the hard constraints (latency, causal vs. zero-phase, compute). Design: specify the filter(s), type, order, and cutoffs — the two uses may need *different* chains. Evaluate: name the diagnostic that shows the ST segment survived and that QRS timing is unbiased. Motivate: justify causal filtering for the alarm versus zero-phase `filtfilt` for the stored review, against the latency budget. Alternatives: give one defensible *different* design and the spec change that would make it the better choice. There is no single correct answer — you are judged on the fit between spec and design and on the honesty of the trade-off. Rule out: name one filter choice that would be *wrong* for this spec — not merely worse — and the requirement it breaks (for example, one that erases the low-frequency ST segment, or a non-causal `filtfilt` placed in the real-time alarm path where its look-ahead is impossible).

Further reading. Oppenheim & Schafer, *Discrete-Time Signal Processing* (LTI systems, convolution, filter design). Proakis & Manolakis, *Digital Signal Processing* (IIR families and specification). Gonzalez & Woods, *Digital Image Processing* (spatial filtering, median filtering, Sobel edges). Pan & Tompkins, "A Real-Time QRS Detection Algorithm," *IEEE Transactions on Biomedical Engineering* 32(3):230–236, 1985 (the band-pass / derivative / squaring / integration / adaptive-threshold pipeline of §9.9). The SciPy `signal` and `ndimage` reference documentation for the exact design and application functions used throughout this chapter.

varkonyibarnabas@gmail.com

Chapter 10 — Noise Reduction & Signal Enhancement

When a filter alone can't separate signal from noise because they overlap in frequency, what else works?

Chapter 9 gave us the frequency-domain scalpel: if the signal lives in one band and the noise in another, a well-designed filter cleanly separates them. But nature is rarely so tidy. An evoked cortical response and the ongoing EEG that buries it occupy *the same* frequencies. A motion artifact drifts through the pulse band and changes second by second. An eye blink smears broadband energy across every EEG channel at once. In each case a fixed frequency filter is helpless, because there is no band to cut that would not also delete the signal. This chapter assembles the second toolbox — methods that exploit *structure other than frequency*: repetition (ensemble averaging), correlation with a reference (adaptive filtering), sparsity (wavelet denoising), and statistical independence (ICA/PCA). The unifying question is always the same: what does the signal have that the noise does not, and how do we cash that difference in?

10.1 Learning objectives

By the end of this chapter you should be able to:

- State and *derive* the $\sqrt{N}$ law for synchronous averaging and invert it to choose N for a target SNR (L2, L3).
- Name the assumptions that make $\sqrt{N}$ hold and the real-world effects (jitter, amplitude variation) that break it (L3, L4).
- Explain the adaptive noise-cancellation structure and implement a simple LMS filter (L3).
- Describe wavelet denoising by thresholding and choose hard versus soft thresholds (L3).
- Explain how ICA and PCA remove artifacts by unmixing sources rather than by filtering frequency (L2, L3).
- Select a denoising method from the character of the noise, and quantify the improvement (L4).

10.2 Intuition first: the signal repeats, the noise does not

The oldest and most powerful idea in noise reduction needs no filter at all. Suppose an event repeats — a heartbeat locked to its R-peak, a cortical response locked to a stimulus flash — and each repetition contains the *same* signal plus a *different*, *independent* dollop of noise. Line the repetitions up on their trigger and add them. The signal, identical every time, grows in direct proportion to the number of trials N. The noise, random and independent each time, grows only as $\sqrt{N}$ (random walks add in quadrature, not linearly). Divide by N to average, and the signal is preserved while the noise shrinks by $\sqrt{N}$. This is ensemble (synchronous)

varkonyibarnabas@gmail.com

averaging, and its governing law is short enough to memorise: averaging N aligned epochs improves the signal-to-noise ratio by $\sqrt{N}$.

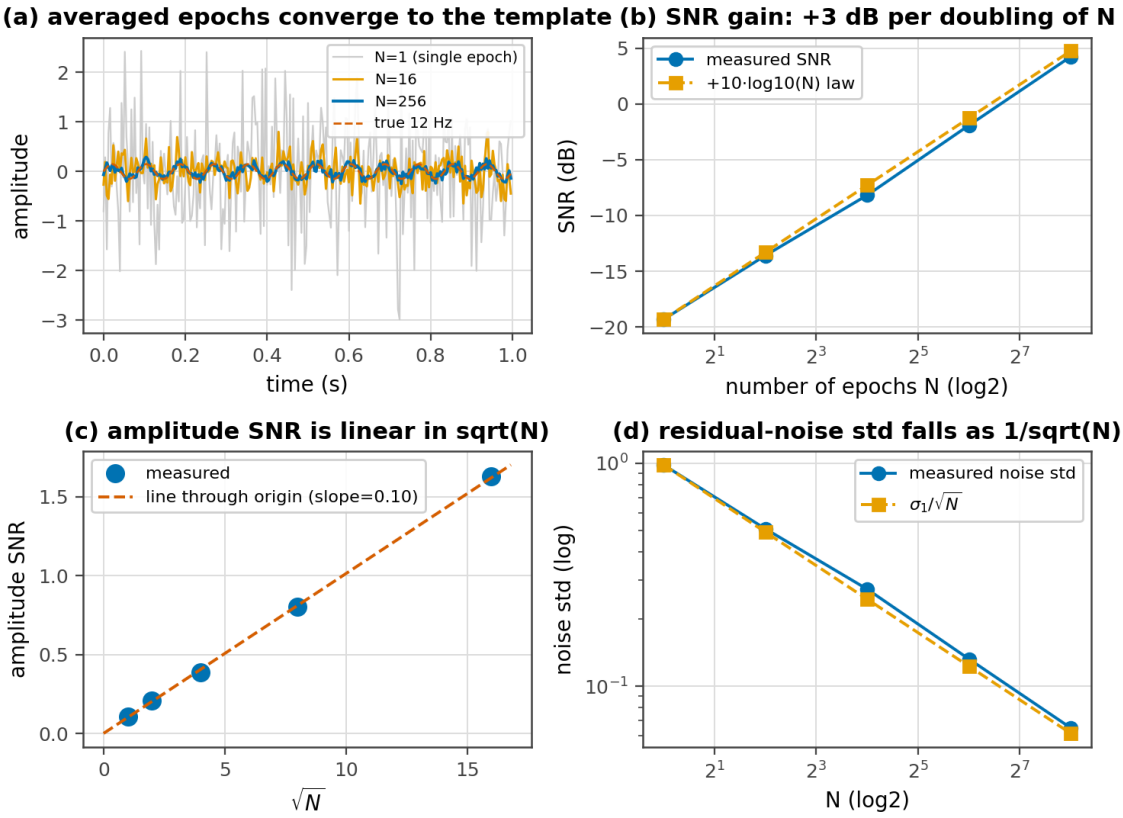


Figure 10.1. The $\sqrt{N}$ law, four ways. (a) Averaging more stimulus-locked SSVEP epochs pulls the 12 Hz response out of the noise. (b) SNR in dB rises by exactly 3 dB per doubling of N, tracking the $+10\log_{10} N$ line. (c) Amplitude-domain SNR is *linear* in $\sqrt{N}$ — a straight line through the origin. (d) The residual-noise standard deviation falls as $1/\sqrt{N}$. One law, confirmed by direct measurement.

10.3 Theory: the $\sqrt{N}$ law, and the assumptions that make it true

Model the i-th aligned epoch as a fixed signal $s[n]$ plus zero-mean noise $v_i[n]$ of variance σ^2:

$$x_i[n] = s[n] + v_i[n].$$

The ensemble average over N epochs is

$$\bar{x}[n] = \frac{1}{N}\sum_{i=1}^{N} x_i[n] = s[n] + \frac{1}{N}\sum_{i=1}^{N} v_i[n].$$

The signal term is untouched. The averaged noise term has variance σ^2/N when the v_i are

varkonyibarnabas@gmail.com

uncorrelated across trials, so its standard deviation is $\sigma/\sqrt{N}$. Since amplitude SNR is signal amplitude over noise standard deviation,

$$\mathrm{SNR}_N = \sqrt{N}\,\mathrm{SNR}_1, \qquad \mathrm{SNR}_N^{\mathrm{dB}} = \mathrm{SNR}_1^{\mathrm{dB}} + 10\log_{10} N.$$

The consequences are worth having at your fingertips: $N = 4$ buys $2\times$ (+6 dB), $N = 100$ buys $10\times$ (+20 dB), $N = 1000$ buys $\approx 31\times$ (+30 dB). To *invert* the law — the practically useful direction — solving for N gives $N = (\mathrm{SNR}_{\mathrm{target}}/\mathrm{SNR}_1)^2$: doubling your SNR costs four times the trials, so the returns diminish quadratically. Figure 10.1(b) confirms the $+3$ dB-per-doubling law directly, panel (c) shows the amplitude SNR rising on a straight line through the origin against $\sqrt{N}$, and panel (d) shows the residual noise decaying as $1/\sqrt{N}$ — the same law seen three ways.

The law is only as good as its assumptions, and knowing when it breaks is as important as knowing the rule. It requires: (i) noise that is zero-mean, stationary, and *uncorrelated across trials* — correlated noise (a persistent 50 Hz line) averages *coherently* and does not shrink; (ii) a signal that is *stable* trial to trial in shape and amplitude; and (iii) *accurate alignment* to the trigger. The dominant real-world spoiler is latency jitter: if the signal's timing wanders from trial to trial, averaging blurs and *attenuates* it (each trial's peak lands in a slightly different bin), so beyond some point adding trials sharpens the noise floor but no longer sharpens the signal — jitter *caps* the achievable gain. Trial-to-trial amplitude variation and habituation (the response shrinking with repetition) violate the stable-signal assumption similarly. In practice you average until the marginal trial stops helping, and you fix jitter with better triggering, not with more trials.

10.4 Worked example: SSVEP averaging for a BCI (a law and a number)

Before the numbers, a framing point that keeps two paradigms distinct. Ensemble (time-locked) averaging is the *classic* technique for transient evoked potentials: a *discrete* stimulus — an oddball tone, a click, a flash — triggers a stereotyped, time-locked transient that averages up by $\sqrt{N}$. The canonical cases are the P300 in an oddball task, the auditory brainstem response (ABR), and transient visual evoked potentials; the whole point of averaging is to pull that fixed-shape transient out of the noise in the time domain. An SSVEP is a different animal — a *steady-state* response to a *continuous* flicker rather than a discrete event — and it is usually detected in the frequency domain: the FFT amplitude at the flicker frequency, or canonical correlation analysis (CCA) against sinusoidal templates. It is used here because *phase-locked* averaging of SSVEP epochs also improves SNR by $\sqrt{N}$ — the same law, cleanly demonstrated — but treat the transient ERP as the textbook case for time-domain averaging and SSVEP as its steady-state cousin, so you do not conflate transient (time-domain averaging) with steady-state (frequency-domain) analysis.

A steady-state visual evoked potential (SSVEP) BCI flickers a target at a fixed frequency; the brain responds with EEG energy at that same frequency, but a single one-second epoch is hopelessly noisy — the response is a fraction of the ongoing EEG. Figure 10.1(a) averages $N = 1, 4, 16, 64, 256$ stimulus-locked epochs of a 12 Hz SSVEP. At $N = 1$ the trace is dominated by noise; by $N = 16$ the 12 Hz rhythm is visible; by $N = 256$ it matches the true template almost exactly. The reproducible number: because each doubling of N adds 3 dB, going from a single

varkonyibarnabas@gmail.com

epoch to 256 epochs (2^8) adds $8 \times 3 = 24$ dB of SNR — a factor of ~16 in amplitude — which is precisely the measured jump in Figure 10.1(b). Inverting the law gives the BCI designer a *first-order* estimate of how many flicker cycles a reliable command needs: if a single epoch gives SNR_1 and the decoder needs $4\times$ that, roughly $4^2 = 16$ epochs. It is only an estimate, because it assumes the idealised conditions of §10.3 — uncorrelated trial-to-trial noise and a stationary, perfectly time-locked response; real EEG is temporally correlated and non-stationary, and the decoder, trial variability, and latency budget have the final say. The same machinery, aligned to the R-peak instead of a stimulus, averages many heartbeats into a clean P–QRS–T template for high-resolution and fetal ECG, and aligned to a rare stimulus recovers the microvolt ERP waveforms of cognitive neuroscience.

10.5 Adaptive filtering: LMS noise cancellation

Ensemble averaging needs repetition. When the interference is *not* repetitive but you have a separate reference measurement correlated with it, adaptive noise cancellation applies. The structure has two inputs: a primary channel carrying signal-plus-artifact, and a reference channel carrying a correlated copy of the artifact alone (a second electrode near the noise source, an accelerometer sensing motion, an antenna picking up the mains). An adaptive filter reshapes the reference to best match the artifact in the primary, then subtracts it. Crucially, because the filter *learns* from the data, it tracks interference that drifts over time — exactly where the fixed filters of Chapter 9 fail.

The least-mean-squares (LMS) algorithm updates the filter weights w one sample at a time to minimise the squared error. With reference vector $\mathrm{x}[n]$, filter output $y[n] = \mathrm{w}^\top \mathrm{x}[n]$, and error $e[n] = d[n] - y[n]$ (where $d[n]$ is the primary),

$$\mathrm{w}[n+1] = \mathrm{w}[n] + 2\mu\, e[n]\, \mathrm{x}[n].$$

The error signal $e[n]$ is the cleaned output: whatever in the primary correlates with the reference is subtracted away, leaving the uncorrelated signal. The step size μ trades convergence speed against stability — too large diverges, too small crawls; a rough stability bound is $0 < \mu < 1/(L \times \text{input power})$, where L is the number of filter taps (12 in Figure 10.2) — the single-tap input power alone overstates the safe range by a factor of L. Its faster cousin, recursive least squares (RLS), converges in far fewer samples at higher computational cost by tracking an estimate of the input correlation matrix.

> Box C — Adaptive filters. When the noise changes over time or overlaps the signal band, a fixed filter fails and an adaptive one succeeds — *provided you have a reference correlated with the noise but not with the signal.* The primary-plus-reference structure, driven by LMS or RLS, is the workhorse for two canonical biosignal problems: cancelling 50 Hz powerline from an ECG (reference = a mains pickup) and removing ECG contamination from a trunk EMG (reference = a chest ECG lead). The adaptive filter forms the best linear estimate of the artifact from the reference and subtracts it; the residual error is your clean signal.

varkonyibarnabas@gmail.com

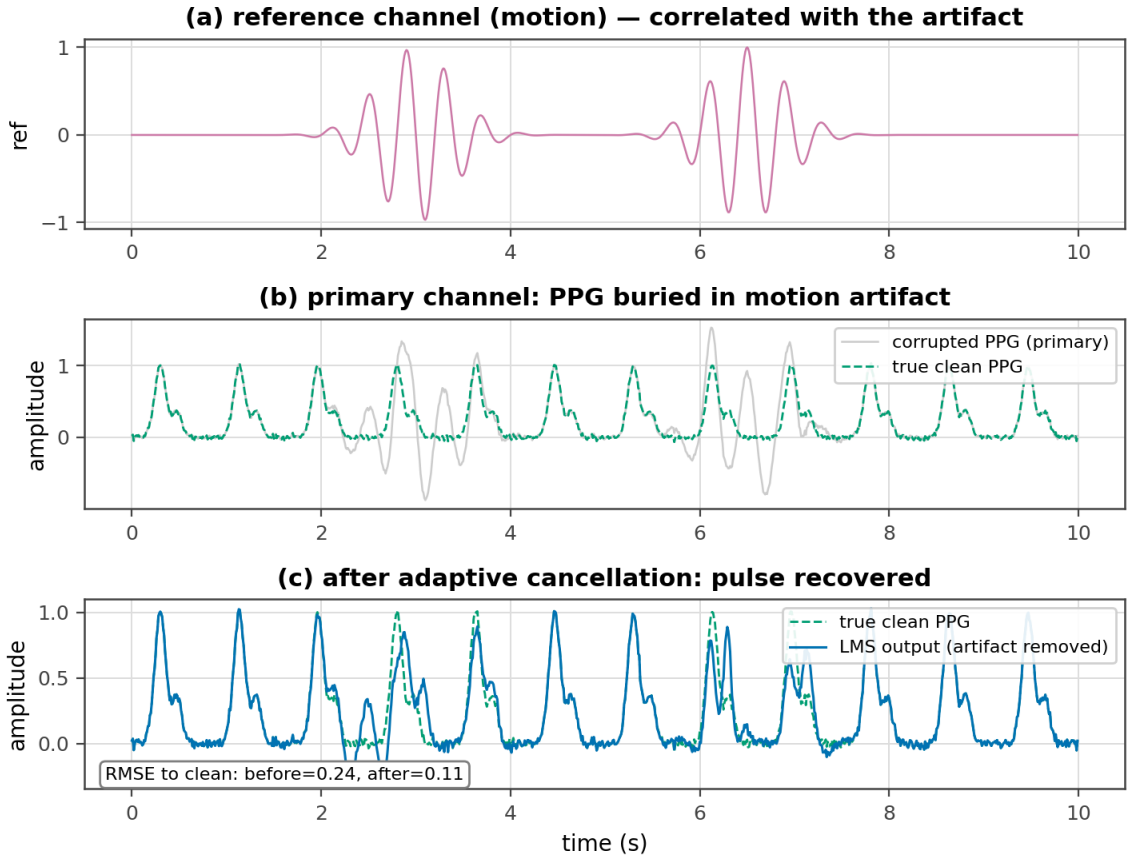


Figure 10.2. Adaptive noise cancellation recovers a pulse buried in motion. (a) The reference channel senses the motion artifact. (b) The primary PPG is corrupted where motion occurs. (c) A simple 12-tap LMS filter, driven by the reference, subtracts the correlated artifact — the error output tracks the true clean pulse, cutting the RMSE to the clean signal by more than half. The fixed filter of Chapter 9 could not do this, because the artifact overlaps the pulse band and drifts over time.

varkonyibarnabas@gmail.com

Box D — The Kalman filter: recursive state estimation. LMS and RLS adapt a filter to *cancel* an interference; the Kalman filter is their close relative for *tracking a slowly changing quantity* — a hidden state you want to estimate as it drifts. It assumes a linear-Gaussian state-space model: an unobserved state evolves with known dynamics plus process noise, and each measurement is a noisy, linear glimpse of it. The filter then alternates *predict* (roll the state forward) and *update* (correct with the new measurement), maintaining a running estimate and its uncertainty. Under those linear-Gaussian assumptions it is the optimal (minimum-mean-square-error) linear estimator, and it is recursive, so it runs in real time on a stream. In biosignals it is the natural tool for tracking a *drifting parameter*: ECG baseline wander, instantaneous heart rate from noisy RR intervals, respiratory rate, or sensor-fusion orientation from an IMU (gyroscope-plus-accelerometer). When the dynamics are non-linear, the extended and unscented Kalman filters extend the idea. We flag it here as the state-estimation companion to LMS/RLS and point to it as further reading rather than deriving it in full.

10.6 Wavelet denoising: exploiting sparsity

A third kind of structure is sparsity. Many biosignals — an ECG especially — are compact in the wavelet domain: a handful of large coefficients capture the sharp QRS and the smooth waves, while broadband noise spreads *thinly* across *all* coefficients. This asymmetry is the opening. Wavelet denoising decomposes the signal (Chapter 5), then *thresholds* the detail coefficients: coefficients below a threshold are assumed to be noise and shrunk, while the few large ones carrying the signal survive. Two shrinkage rules exist. Hard thresholding zeros everything below the threshold and leaves the rest unchanged — it keeps sharp features but can leave residual "blips." Soft thresholding additionally shrinks the survivors toward zero, giving a smoother result at the cost of slightly attenuating true peaks. A robust default threshold is the universal threshold $\lambda = \sigma\sqrt{2\ln n}$, where σ is estimated from the finest detail coefficients via the median absolute deviation ($\sigma \approx \mathrm{MAD}/0.6745$), which resists contamination by the signal's own large coefficients.

The mother-wavelet choice matters: a wavelet whose shape resembles the feature you want to keep (a Symlet or Daubechies wavelet for the QRS, say) concentrates the signal into fewer, larger coefficients and denoises better. The pay-off over a linear low-pass is that thresholding is *adaptive* — it removes noise where the signal is quiet without blurring the sharp transients where the signal is loud.

10.7 ICA and PCA: unmixing sources

The last structure is statistical independence. Multichannel EEG records a *mixture*: each electrode sees a weighted sum of brain sources plus artifacts (eye blinks, ECG, muscle). If the underlying sources are statistically independent and non-Gaussian, independent component analysis (ICA) can recover them from the mixture alone — no reference channel needed. The eye blink, being independent of the brain rhythms and highly non-Gaussian, separates into its own component; zero that component, remix the rest, and the EEG is cleaned with minimal loss of brain signal. This is *blind source separation*: it unmixes by independence, a property invisible to any frequency filter. The identical trick, on an entirely different signal, recovers a non-invasive fetal ECG from an array of electrodes on the maternal abdomen (§1.4):

varkonyibarnabas@gmail.com

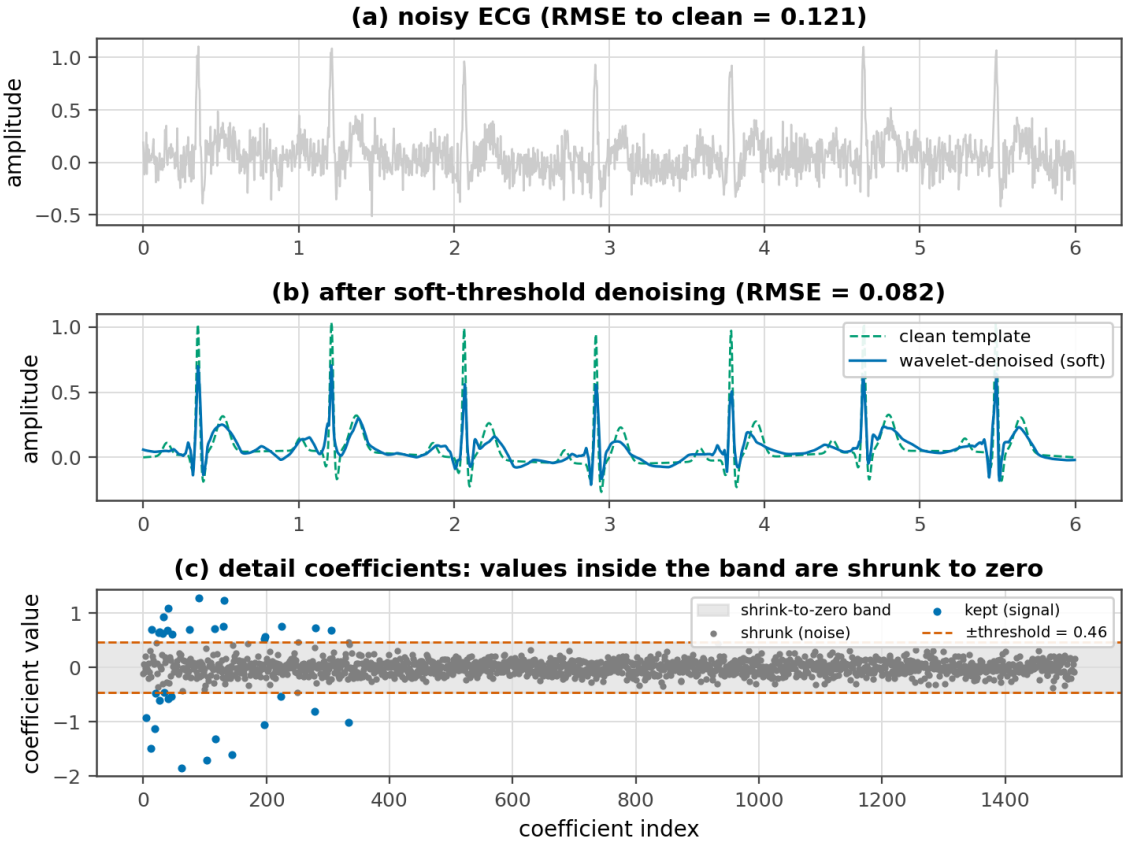


Figure 10.3. Wavelet soft-threshold denoising of a noisy ECG. (a) The noisy input. (b) After sym4 decomposition, universal-threshold soft shrinkage, and reconstruction, the RMSE to the clean template drops by roughly a third — but the diagnostic that matters most (§10.8) is the QRS peak itself: soft shrinkage measurably attenuates it here, by about 30% on this example, precisely the kind of feature-distorting cost a good RMSE number can hide. A linear low-pass would round the peak off worse still; a level-dependent or hard threshold on the finest scales trades back some RMSE gain for a sharper peak when peak fidelity matters more than overall RMSE. (c) The detail coefficients (value vs. index): the dashed threshold shrinks the dense scatter of small (noise) coefficients to zero while sparing the few large (signal) ones outside it.

varkonyibarnabas@gmail.com

the mixture is now a strong maternal ECG plus a much weaker fetal ECG plus noise, and either an adaptive filter subtracting a maternal-ECG template (§10.5) or an ICA unmixing of the multi-electrode recording separates the two — the fetal component, invisible in any single raw channel, falls out as one of the recovered sources. Principal component analysis (PCA) is the linear cousin that decorrelates rather than fully separates: it finds orthogonal directions of maximal variance and is ideal for multichannel denoising when the signal lives in a low-dimensional subspace and the noise fills the rest — dropping the small-variance components denoises, and it bridges directly to the dimensionality reduction of Chapter 13.

10.8 Method-selection guidelines

The methods are not interchangeable; each exploits a different property, and choosing wrongly wastes it.

- Ensemble averaging — when the event *repeats* and you can *trigger* on it (evoked potentials, averaged heartbeats). Gain: $\sqrt{N}$, capped by jitter.
- Adaptive (LMS/RLS) — when the noise is *non-stationary* or *overlaps the band* but a *correlated reference* exists (motion, powerline, ECG-in-EMG).
- Wavelet thresholding — when the signal is *sparse/transient* and the noise is *broadband* (a single noisy ECG with no repetition and no reference).
- ICA — when *multiple channels* record a *mixture* of *independent* sources (eye-blink and ECG removal from EEG).
- PCA — when *multichannel* signal occupies a *low-dimensional subspace* and you want decorrelated denoising or compression.

Always quantify the result: report SNR improvement (dB), RMSE to a clean reference, and — critically — a check that the method did not *distort* the feature you care about (a denoiser that flattens the QRS has failed even if its SNR number looks good).

Method	Exploits	Needs	Typical target	Cost
Ensemble averaging	repetition	a reliable trigger	ERP, SSVEP, ECG template	trials (time)
LMS / RLS adaptive	reference correlation	a reference channel	motion, 50 Hz, ECG-in-EMG	low / moderate
Wavelet thresholding	sparsity	one channel	noisy ECG transients	low
ICA	independence	many channels	eye-blink/ECG from EEG; fetal ECG from maternal abdomen	moderate
PCA	low-rank structure	many channels	multichannel denoising	low

10.9 The ML/AI bridge

Denoising sits between raw data and features, and its quality propagates. A classifier trained on epochs cleaned by ensemble averaging or ICA sees tighter, more separable feature distri-

varkonyibarnabas@gmail.com

butions than one trained on raw data, so the same model scores higher for reasons that have nothing to do with the model. But there is a subtler point that Chapter 12 will sharpen, and it is easy to state too bluntly, so let us state it carefully.

The governing rule is not "never fit anything before the split." It is that your evaluation pipeline must mirror deployment — whatever the model will have available at inference time is fair to use during evaluation, and whatever it will *not* have available is leakage. That distinction cuts the denoising methods of this chapter into two cases. Unsupervised, per-recording preprocessing that would also run at inference is *not* leakage: if, in deployment, you will fit an ICA unmixing matrix (or a PCA basis, or a wavelet-noise estimate) on each incoming recording on its own and apply it to that same recording, then doing exactly that to each test recording during evaluation is faithful, not cheating — the test recording's own statistics are legitimately available to it at inference. What *does* leak is anything learned from information the model will not have: fitting a supervised feature selector or a scaler on pooled train+test data, standardising with a mean and variance computed over the whole dataset, choosing an ICA/PCA basis on the pooled set so that test subjects influence the transform applied to training, or — worst of all — letting the test labels shape any part of the pipeline. Anything estimated from the test-label distribution, or from pooled train-plus-test subjects, can leak; per-test-record unsupervised preprocessing that is genuinely available at inference generally does not.

So the honest practice is not a blanket "fit on train only," but a matching principle: reproduce the deployment pipeline exactly, fit anything *cross-recording* or *supervised* on the training split alone, and let per-recording unsupervised steps see only their own recording (Chapter 12 works through the full checklist). Denoising is not a free lunch you apply once globally over train and test together; it is a step inside the pipeline that must respect the split it will actually face in the field.

10.10 Companion notebook

The companion notebook makes the $\sqrt{N}$ law tangible: it averages growing numbers of SSVEP epochs (and, separately, R-peak-aligned heartbeats), measures the SNR at each N, and plots it against $\sqrt{N}$ so you watch the straight line appear — then it injects latency jitter and shows the line bend and saturate, making the assumption failure visceral. A second section implements the LMS filter of §10.5 from scratch and cancels a reference-correlated artifact, letting you vary μ to see convergence and instability. A third section wavelet-denoises a noisy ECG (hard versus soft, several mother wavelets) and runs FastICA to lift an eye-blink component out of a multichannel EEG mix.

10.11 Case-study tie-in

In the sleep-staging case study this chapter is the artifact-removal stage that follows preprocessing. EOG and EMG contamination bleed into the EEG channels, and left in place they corrupt the very band-power and entropy features that distinguish REM from wake. The pipeline offers three escalating options: simple regression against the EOG reference, adaptive (LMS) cancellation when the contamination drifts, and ICA when a blink component must be surgically removed from many channels at once. The $\sqrt{N}$ idea appears too, in building clean per-

varkonyibarnabas@gmail.com

subject templates. The evaluation habit from §10.8 carries straight in: every artifact-removal choice is judged not by how clean the trace *looks* but by whether it improves downstream staging without distorting the physiological features.

10.12 Summary, questions, and further reading

Key formulas. Ensemble average $\bar{x} = s + \frac{1}{N}\sum v_i$ with residual-noise std $\sigma/\sqrt{N}$; $\mathrm{SNR}_N = \sqrt{N}\,\mathrm{SNR}_1$ and $+10\log_{10} N$ dB; invert as $N = (\mathrm{SNR}_{\text{target}}/\mathrm{SNR}_1)^2$; LMS update $\mathrm{w}[n{+}1] = \mathrm{w}[n] + 2\mu\, e[n]\,\mathrm{x}[n]$ with cleaned output $e[n]$; universal threshold $\lambda = \sigma\sqrt{2\ln n}$, $\sigma \approx \mathrm{MAD}/0.6745$.

Key ideas. Averaging exploits repetition, adaptive filtering exploits a correlated reference, wavelets exploit sparsity, ICA exploits independence, PCA exploits low rank — none of which a frequency filter can see. The $\sqrt{N}$ law is exact under stationary, uncorrelated, well-aligned conditions and is *capped by jitter*. Adaptive filters track drifting, in-band noise. Always quantify improvement *and* check for feature distortion.

Conceptual questions. (1) You average 100 evoked responses and gain only 10 dB instead of the predicted 20 dB — name two assumptions that may have failed. (2) Why does a persistent 50 Hz line *not* shrink under ensemble averaging? (3) In LMS cancellation, why must the reference be correlated with the *noise* but not with the *signal*? (4) When would you prefer soft over hard wavelet thresholding? (5) Why does ICA succeed at eye-blink removal where a band-pass filter fails?

Exploration tasks. Verify the $\sqrt{N}$ law on averaged heartbeats and then break it by adding random per-trial latency jitter; plot the saturating SNR. Implement LMS and cancel a 50 Hz line from an ECG using a synthetic mains reference, sweeping μ. Wavelet-denoise the same ECG with three mother wavelets and rank them by RMSE while checking QRS amplitude preservation. Run ICA on a two-source mixture and recover the sources.

Design brief. Three cleanups land on your desk: (i) remove 50 Hz powerline from an EEG whose gamma band must be kept; (ii) suppress ocular (blink) artifact from a multi-channel EEG; (iii) recover a weak evoked response buried in noise across many repeated trials. Spec: for each, name what must survive and what structure the noise has — narrowband? spatially structured? repeatable? Design: pick the method — notch, adaptive filter, ICA, wavelet denoising, or ensemble averaging — for each. Evaluate: the diagnostic that shows you removed the artifact without eating the signal. Motivate: justify why ensemble averaging fits (iii) but not (ii). Alternatives: for (ii), name a defensible alternative to ICA and its cost. Rule out: name a method that would be *wrong* for one of these — e.g. ensemble-averaging a one-off single-trial artifact, or running ICA with far fewer channels than sources — and the assumption it violates.

Further reading. Widrow & Stearns, *Adaptive Signal Processing* (LMS/RLS, noise cancellation). For the Kalman filter and its non-linear extensions (Box D), Simon's *Optimal State Estimation* is a clear, practical starting point. Donoho & Johnstone on wavelet shrinkage (thresholding theory). Hyvärinen & Oja, *Independent Component Analysis* (FastICA). Sörnmo & Laguna, *Bioelectrical Signal Processing in Cardiac and Neurological Applications* (ensemble averaging, evoked potentials). The SciPy, PyWavelets, and scikit-learn documentation for the implementations

varkonyibarnabas@gmail.com

used here.

varkonyibarnabas@gmail.com

Chapter 11 — Feature Engineering & Selection for Biosignals & Images

What numbers summarise a 30-second epoch well enough for a classical model to name its sleep stage — and when might a model instead learn from the waveform itself?

A machine-learning model does not necessarily see a signal the way you do. Hand a classical, feature-based model 30 seconds of EEG — 3,000 or more raw samples — and, unless you first summarise it, it faces a point in a 3,000-dimensional space, most of whose axes are noise, redundancy, or phase accident. Two epochs of the *same* sleep stage, differing only by a random shift, can look very different in that raw space. Feature-based classical models usually require fixed-length feature vectors; deep models can learn directly from raw or minimally processed waveforms, and some classical methods use time-series kernels or shapelets. Feature engineering is the translation used throughout the classical pipeline: distilling each epoch into a short, fixed-length vector of numbers that tend to be *stable* within a class and *different* across classes. This is a hinge of the book — the point where signals meet classical learning. Get the features right and a simple classifier often succeeds; get them wrong and extra model sophistication rarely rescues you. Hand-crafted features are especially valuable when data are limited and when interpretability matters, since each feature has a name a clinician can reason about. This chapter builds the feature vocabulary (time, frequency, time–frequency, and image features), then confronts the opposite danger — *too many* features — with the tools of feature selection.

11.1 Learning objectives

By the end of this chapter you should be able to:

- Explain why classical feature-based models usually need fixed-length feature vectors, when a model might instead learn from the raw waveform, and what makes a good feature (L2, L4).
- Compute the core time-domain features, including the Hjorth parameters and entropy measures (L4).
- Compute frequency-domain features — band powers, relative powers, ratios, spectral edge, spectral entropy — and time–frequency (wavelet) features (L4).
- Describe the main image features (intensity, texture/GLCM, edges) and where hand-crafted features give way to learned ones (L2, L4).
- Scale features correctly and recognise the leakage trap that scaling can introduce (L4).
- Rank and select features by mutual information, F-test, and model importance, and explain the curse of dimensionality (L4).

varkonyibarnabas@gmail.com

11.2 Intuition first: from a raw epoch to a feature vector

A feature is a single number that summarises some property of an epoch. The mean absolute amplitude is a feature; the fraction of power in the delta band is a feature; the rate at which the signal crosses zero is a feature. Collect a fixed list of them and you have a feature vector $\mathbf{x} \in \mathbb{R}^d$ — one point per epoch, the same length regardless of how the raw waveform wiggled. That fixed length is what lets a *classical* classifier compare epochs directly, since most such models expect an input of constant dimension.

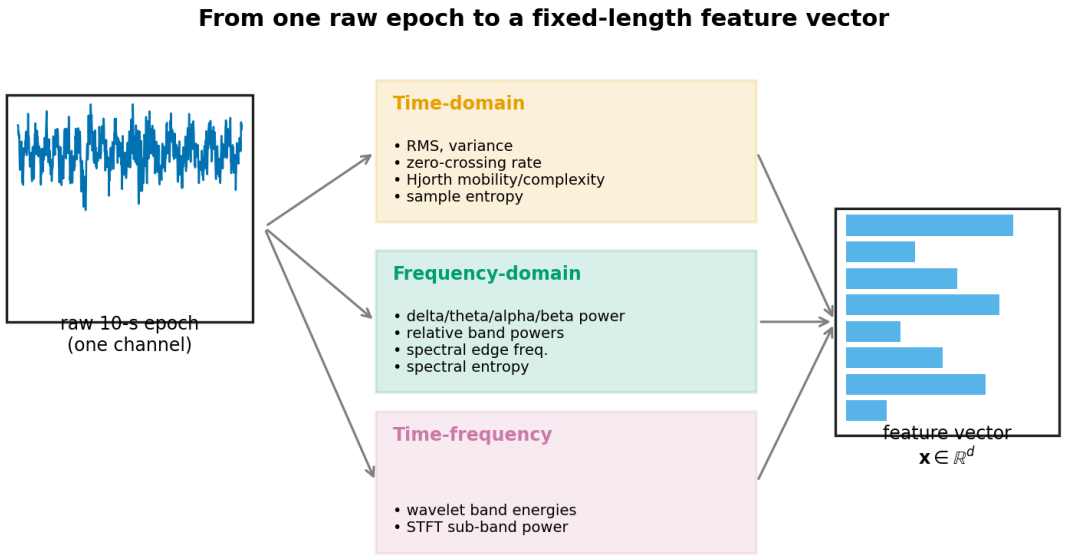


Figure 11.1. Feature extraction is a funnel: one variable-looking raw epoch enters, and a fixed-length vector leaves. The features fall into natural groups — time-domain (amplitude, variability, complexity), frequency-domain (band powers and spectral shape), and time–frequency (wavelet/STFT sub-band energies) — each capturing a different facet of the same 30 seconds. A classical classifier reads this vector rather than the raw waveform; a deep model (Chapter 14) can instead learn its own summary from the waveform.

Good features tend to share three virtues. They are discriminative (they take different values in different classes — delta power is high in deep sleep, low in wake). They are robust (small nuisance changes barely move them — a good feature largely ignores an irrelevant DC offset). And, as a *set*, they are non-redundant (two features that always move together carry the information of one). Much of feature engineering is chasing the first two; much of feature selection is fixing the third.

11.3 Theory: the feature families

Time-domain features are computed directly from the samples and describe amplitude, variability, and shape. The statistical moments — mean, variance, skewness, kurtosis — summarise the amplitude distribution. Energy and root-mean-square (RMS) amplitude measure signal power. The zero-crossing rate is a cheap proxy for dominant frequency (a fast signal crosses zero often). The Hjorth parameters deserve special mention because they capture

varkonyibarnabas@gmail.com

spectral shape *without an FFT*, from time-domain variances alone. With σ_x^2 the variance of the signal, $\sigma_{x'}^2$ of its first difference, and $\sigma_{x''}^2$ of its second:

$$\text{Activity} = \sigma_x^2, \quad \text{Mobility} = \sqrt{\frac{\sigma_{x'}^2}{\sigma_x^2}}, \quad \text{Complexity} = \frac{\sqrt{\sigma_{x''}^2/\sigma_{x'}^2}}{\text{Mobility}}.$$

Activity is total power, mobility estimates the mean frequency, and complexity estimates the bandwidth (how far from a pure sinusoid). This plain-language gloss ("mobility $\approx$ mean frequency, complexity $\approx$ bandwidth") is a useful *approximation*, not an identity: the correspondence is exact only for idealised spectra, and the Hjorth parameters are defined by the variance formulas above, which is what you actually compute. Finally, sample entropy and approximate entropy quantify signal *regularity* — a highly regular alpha rhythm has low entropy, a disorganised waking EEG has high entropy — capturing nonlinear structure the moments miss. These entropy measures are the everyday members of a broader family of non-linear / complexity features, motivated by the fact that biosignals such as HRV and EEG arise from non-linear dynamical systems rather than simple linear generators. The same family supplies Poincaré-plot descriptors (SD1, SD2) for HRV, detrended fluctuation analysis (DFA) scaling exponents (introduced beside 1/f noise in Chapter 8), and — with the explicit caution that they are highly sensitive to noise, record length, and parameter choices — the correlation dimension and Lyapunov exponents.

Frequency-domain features are built on the power spectrum (Chapter 7). Band powers — the integrated power in the delta (0.5–4 Hz), theta (4–8 Hz), alpha (8–13 Hz), and beta (13–30 Hz) bands — are the backbone of EEG analysis, because sleep stages are *defined* by which rhythms dominate. For the sleep case study we also add the sigma (spindle) band and switch to the *sleep-analysis* partition of the canonical band table (§1.4): because sleep-alpha (8–12 Hz) and sigma (11–16 Hz) overlap at 11–12 Hz, a band-power feature vector must use non-overlapping edges — we integrate alpha as 8–11 Hz and sigma as 11–16 Hz (or, alternatively, start sigma at 12 Hz, which departs from this book's canonical 11–16 Hz sigma-band convention stated in §1.4) so no frequency bin is double-counted. Relative band powers (each band as a fraction of total) normalise away overall amplitude, making them robust across recordings. Power ratios (e.g., delta/beta) sharpen contrasts. The spectral edge frequency (the frequency below which, say, 95% of the power lies) is a quantile of the cumulative power distribution — where the spectrum's mass runs out, not where it balances — and spectral entropy measures how flat versus peaked the spectrum is — a single dominant rhythm gives low spectral entropy, broadband activity gives high. We use it here in its normalised form, $H = -\sum_k p_k \ln p_k \,/\, \ln N$, over the power spectrum turned into a probability distribution ($p_k = S_k / \sum_j S_j$, N the number of frequency bins): dividing by $\ln N$ makes the feature dimensionless and bounded in $[0, 1]$ rather than reported in nats or bits, and makes the log-base choice truly irrelevant (it cancels top and bottom). The trade-off is that, unlike a raw nats/bits entropy, this normalised value depends on how many frequency bins the spectrum was computed with (`nperseg`) — so a "reproducible number" here also means fixing that. The peak frequency names the single loudest rhythm, and the spectral centroid — the power-weighted mean frequency $\sum_k f_k S_k / \sum_k S_k$ — is a compact one-number summary of where the spectrum's mass sits (a "brightness" measure borrowed from audio analysis), a close cousin of the Hjorth mobility and the spectral edge.

varkonyibarnabas@gmail.com

Time–frequency features capture signals whose spectrum *changes within the epoch*. Wavelet decomposition (Chapter 5) yields the energy in each sub-band at each scale; STFT gives sub-band power over short windows. These matter when a transient — a spindle, a K-complex — occupies only part of the epoch and would be diluted by a whole-epoch spectrum.

Morphological (fiducial) features are indispensable wherever the *shape* of a repeating waveform is the diagnosis — above all the ECG. Rather than summarise statistics over a whole window, they first locate named fiducial points on each beat — the P wave, the Q/R/S deflections of the QRS complex, and the T wave — and then read clinically defined amplitudes and intervals from them: the PR interval (atrioventricular conduction time), the QRS duration (ventricular depolarisation), the QT interval and its heart-rate-corrected form QTc (repolarisation and arrhythmia risk), the ST-segment level (elevated or depressed in ischemia), and P- and T-wave amplitudes and polarities. These are exactly the measurements a cardiologist makes by eye, and they are the backbone of automated ECG interpretation; the same idea gives fiducial landmarks on a PPG pulse (systolic peak, dicrotic notch) or an ABP waveform. Their prerequisite is reliable delineation — the Pan–Tompkins-style detector of §9.9 finds the R peak and P/T delineation extends it — so a fiducial-feature pipeline is only as trustworthy as its beat detector. (The ECG landmarks these name are the ones first shown in Figure 1.2 and detected stage by stage in §9.9.)

Combining channels: feature fusion. The families above describe features of *one* signal, but the sleep case study — like most real problems — records several at once (EEG, EOG, EMG). There are three points at which their information can be merged, and the choice matters. Early (feature-level) fusion concatenates the per-channel feature vectors into one long vector before a single model sees it; it preserves all cross-channel information and lets the model learn interactions, at the price of a higher-dimensional input and a demand that the channels be well synchronised (§2.13) and comparably scaled. Late (decision-level) fusion trains a separate model per channel and combines their outputs — by voting, averaging, or a small meta-model — which is modular, tolerant of a missing channel, and interpretable, but blind to interactions the separate models cannot each see. Intermediate fusion meets in the middle, combining mid-level representations (learned embeddings, or selected features per channel) before a final stage; deep multimodal networks (Chapter 14) usually fuse this way inside the architecture. No level is universally best — early fusion when channels are tightly coupled and clean, late fusion when they are heterogeneous or unreliable — and the sleep pipeline in practice uses feature-level fusion of the three signals, because their joint pattern (spindles in EEG, atonia in EMG, rapid movements in EOG) is exactly what defines a stage.

11.4 Worked example: features that separate sleep stages (a number to reproduce)

Do these features actually distinguish sleep stages? Figure 11.2 extracts the time- and frequency-domain set from many synthetic epochs of four stages (Wake, N2, N3, REM), standardises each feature, and shows the mean per stage as a heatmap. The pattern mirrors the physiology: N3 loads heavily on delta power (deep, red, across every delta-related feature) and low on the higher bands, because deep sleep *is* slow-wave sleep. Wake loads on the higher-frequency features (alpha, beta), because relaxed eyes-closed wakefulness is dominated by the alpha rhythm; REM, despite being physiologically "activated," is instead

varkonyibarnabas@gmail.com

theta-dominated in this feature set — real REM is low-amplitude, mixed-frequency sleep with *less* alpha than wake, not more, so it separates from N3 for a different spectral reason than Wake does. N2 sits in between. The reproducible number (on this simulated example): relative delta power roughly *doubles* from N2 to N3 ($\approx$0.53 $\rightarrow$ $\approx$0.92) while relative beta power falls by roughly a factor of four ($\approx$0.06 $\rightarrow$ $\approx$0.01) — a contrast large enough that even a linear rule tends to separate deep sleep from lighter stages. This illustrates the "law and a number": the features here are not arbitrary; they encode the defining rhythms, and their *differences across stages* are what a classifier will learn.

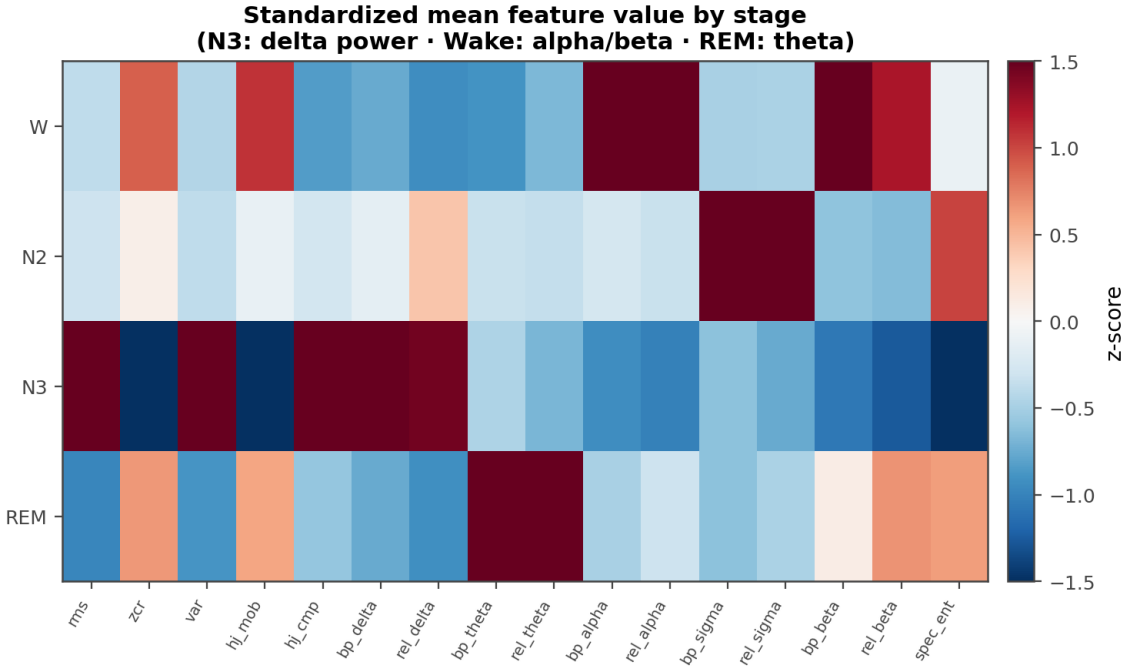


Figure 11.2. Standardised mean feature value by stage. Each column is a feature, each row a stage; red is high, blue is low. The block of delta-related features glows red for N3 and blue for Wake/REM, while the beta/alpha features do the reverse — visual evidence that the features encode the physiology that defines the stages.

11.5 Image features — and where hand-crafting ends

The same programme runs on images, and it foreshadows a turning point. Intensity features summarise the pixel-value histogram (mean, variance, skew — bright versus dark, uniform versus textured). Texture features capture spatial pattern; the classic tool is the grey-level co-occurrence matrix (GLCM), which counts how often pairs of grey levels occur at a given offset, from which contrast, homogeneity, energy, and correlation are derived — powerful for distinguishing, say, healthy from diseased tissue in a scan. Edge and shape features (from the Sobel and gradient operators of Chapter 9) describe boundaries. But here hand-crafted image features often hit a ceiling: a fixed recipe of histograms and GLCMs tends to struggle to capture the hierarchical structure of a tumour or a retinal lesion as well as learned features do. This is the bridge to Chapter 14: a convolutional network *learns* the filters — the equivalent of designing a problem-specific feature extractor from data, rather than picking

varkonyibarnabas@gmail.com

from a human-made menu. For 1-D biosignals the hand-crafted features of this chapter often remain competitive and interpretable; for many image tasks, learned features tend to win, though hand-crafted descriptors can still be a strong baseline when data are scarce.

The same "learn the filter" idea also appears *across channels* rather than across pixels. In multichannel EEG — motor-imagery brain–computer interfaces especially — the workhorse spatial feature extractor is Common Spatial Patterns (CSP): a *supervised* spatial filter that learns a weighting of the electrodes so that the resulting combination has maximal variance (band power) for one class while minimal variance for the other, sharpening the very contrast a classifier needs. Because CSP is supervised — it uses the class labels to fit the filter — it must be fit inside the training fold, exactly like the scaler and selectors of this chapter; fitting it on pooled train-plus-test data leaks label information and inflates accuracy (the leakage caution of Chapter 12).

> Signals-and-images. The feature mindset transfers cleanly. A signal's amplitude histogram ↔ an image's intensity histogram; a signal's band powers ↔ an image's spatial-frequency energy; a signal's zero-crossing rate ↔ an image's edge density; a signal's entropy ↔ an image's texture entropy. Whatever property you summarise along a time axis, you can summarise across a spatial plane — and in both, learned features (Chapter 14) eventually rival hand-crafted ones.

11.6 Feature scaling — and the leakage it can hide

Features arrive on wildly different scales: band power might be thousands, a relative power a small fraction, an entropy near 1. Distance-based methods (kNN, SVM) and gradient-based ones tend to be dominated by the largest-scale feature unless you standardise — subtract the mean and divide by the standard deviation so every feature has comparable spread. This step is essential for those methods, but it hides a trap that Chapter 12 (Section 12.4) will make central: the mean and standard deviation used to scale should be estimated *from the training data only*. Fit the scaler on the whole dataset — including the test epochs — and information about the test set leaks into training, tending to inflate your reported accuracy. Scaling is a *learned* transformation, and like every learned step it should live *inside* the training partition. We flag it here and dissect it in Section 12.4.

11.7 Feature selection and the curse of dimensionality

More features are not automatically better. As dimensionality grows, the data tend to become sparse — points drift apart in high-dimensional space, distances lose meaning, and a model may need many more samples to fill the space. This is the curse of dimensionality, and in practice it means that a *pruned* set of good features often beats a *bloated* set that includes redundant or noisy ones. A full biosignal pipeline might generate roughly 15–30 features per channel (a dozen or so time-domain, a comparable count of frequency- and time–frequency-domain), which across several channels gives a pipeline total on the order of a hundred or more; the goal is usually to keep the few dozen that matter.

Selection methods come in three flavours. Filter methods score each feature against the label independently of any model: the variance threshold drops near-constant features; cor-

varkonyibarnabas@gmail.com

relation pruning removes one of each redundant pair; the ANOVA F-test ranks features by between-class versus within-class variance; and mutual information measures, without assuming linearity, how much knowing a feature reduces uncertainty about the label — capturing nonlinear relationships the F-test misses. Wrapper methods (recursive feature elimination) score subsets by an actual model's performance — more accurate, more expensive. Embedded methods get selection for free from the model: Random-Forest importance and L1-regularised (Lasso) coefficients rank features as a by-product of training.

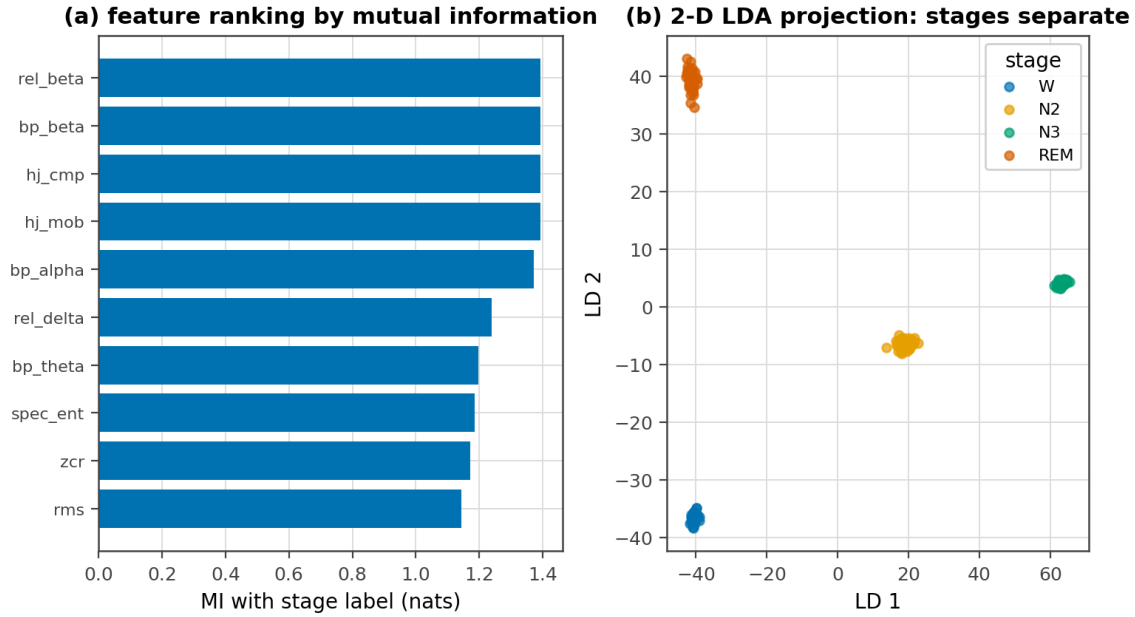


Figure 11.3. Selecting and visualising discriminative features. (a) A mutual-information leaderboard ranks the computed features by how much each tells us about the sleep stage — the beta/alpha band-power and Hjorth features rise to the top. (b) Projecting all features onto two supervised (LDA) discriminant axes shows the four stages forming distinct clusters: the features carry enough class information to separate the stages before the *final* classifier is trained. Because LDA uses the labels, a real pipeline must fit it inside the training fold, like any supervised step — this illustrative figure fits it on the full synthetic set purely to visualise class separation.

Figure 11.3 puts two of these to work. Panel (a) is a mutual-information leaderboard: the features are ranked by their MI with the stage label, and the band-power and Hjorth-complexity features top the list — exactly the physiologically meaningful ones. Panel (b) projects the full feature set to two dimensions with linear discriminant analysis (LDA) — itself a *supervised* projection that uses the labels, so, like the scaler and selectors above, it must be fit inside the training fold — and shows the stages forming separated clusters. That separation is the whole point of this chapter made visible: if the classes cluster in feature space *before the final classifier is trained*, much of the hard work is already done — the model mainly draws the boundaries. (For a genuinely label-free view, use an unsupervised projection such as PCA instead.) If they do not, a classifier is unlikely to rescue you, and the productive move is usually to return to feature engineering.

varkonyibarnabas@gmail.com

Method	Type	Captures nonlinearity?	Uses a model?	Cost	Best for
Variance threshold	filter	—	no	trivial	dropping dead features
Correlation pruning	filter	linear only	no	low	removing redundancy
ANOVA F-test	filter	linear only	no	low	quick linear ranking
Mutual information	filter	yes	no	moderate	nonlinear relevance ranking
Recursive elimination	wrapper	via model	yes	high	squeezing a specific model
Random-Forest importance	embedded	yes	yes	moderate	ranking + selection in one

Common student mistakes. Three recur around raw waveforms versus engineered features. First, believing that a classifier can *only* consume hand-crafted features. Classical models (SVMs, random forests, logistic regression) usually do need a fixed-length feature vector, which is why this chapter exists — but deep models can learn directly from raw or minimally processed waveforms, spectrograms, or learned embeddings (Chapter 14), and some time-series methods use shapelets or kernels rather than a hand-picked vector. Engineered features are one good option, not the only one. Second, computing features per epoch but leaking global statistics into them. Fitting a scaler, a feature selector, or any statistic on the *whole* dataset — test epochs included — lets information about the test set seep into training and inflates reported accuracy; fit every learned transform on the training folds only (cross-referenced in Chapter 12). Third, adding ever more features in the belief that accuracy will climb monotonically. It will not: the curse of dimensionality means redundant or noisy features can *hurt*, thinning the data across a larger space and confusing distance- and variance-based models. A pruned set of good features usually beats a bloated one.

11.8 The ML/AI bridge

This chapter *is* the bridge — its output, the feature matrix, is the literal input to every model in Chapters 12 and 13. Two disciplines carried from here into those chapters keep the bridge sound. First, selection should live inside cross-validation: rank and pick features using only each fold's training data, or the reported accuracy tends to be optimistically inflated (the failure-case clinic in Section 12.4 makes this concrete). Second, interpretability is an asset: because these features are named and physiological, a clinician can be told *why* an epoch was scored N3 ("dominant delta power"), an explanation that learned features often cannot offer as directly — the trade-off that Chapter 14 revisits.

11.9 Companion notebook

The companion notebook runs the full pipeline end to end on a recording: it slices the signal into epochs, computes the complete time-, frequency-, and time–frequency-feature set into a feature matrix, and standardises it for the visualisations that follow — and, because scaling is a learned transform, notes that in a real predictive pipeline the scaler must be fit on the training

varkonyibarnabas@gmail.com

fold only, never the whole set (Chapter 12). It then builds the mutual-information leaderboard of Figure 11.3, prunes correlated features, and projects the result to two dimensions with PCA and LDA so you can *see* class separation improve as junk features are removed. A further cell delineates an ECG beat and reads off its morphological/fiducial features (PR, QRS, QT, ST), and a short experiment contrasts early- versus late-fusion of the three sleep channels so the fusion trade-off of §11.3 is a number you measure, not just read. A final section computes intensity and GLCM texture features on a medical image, making the signal–image parallel of §11.5 concrete.

11.10 Case-study tie-in

In the sleep-staging case study this chapter builds the `feature_extraction` and `feature_selection` modules directly. Each 30-second epoch of every channel is turned into its feature vector — band powers and relative powers for the defining rhythms, Hjorth parameters and entropy for shape and regularity, spindle-band energy for N2. The full set runs to over a hundred features; the selection stage prunes it to the few dozen that carry stage information, ranked by mutual information and Random-Forest importance *inside* the cross-validation loop. The heatmap of Figure 11.2 is, in miniature, the entire rationale of the case study: the features encode the physiology, and their differences across stages are what the classifier learns.

11.11 Summary, questions, and further reading

Key formulas. Hjorth mobility $\sqrt{\sigma_{x'}^2/\sigma_x^2}$ and complexity $\sqrt{\sigma_{x''}^2/\sigma_{x'}^2}/$mobility; relative band power $P_{\text{band}}/P_{\text{total}}$; spectral entropy $H = -\sum_k p_k \ln p_k \,/\, \ln N$ over the normalised spectrum $p_k = S_k/\sum_j S_j$ (dimensionless, bounded in $[0, 1]$; depends on the bin count N, not on log base); standardisation $(x - \mu)/\sigma$ with μ, σ from *training data only*; mutual information $I(X; Y)$ as label-relevance score.

Key ideas. Classical feature-based models usually need fixed-length feature vectors; feature engineering translates a waveform into a vector that is discriminative, robust, and non-redundant (deep models, by contrast, can learn from the raw waveform — Chapter 14). Time features capture amplitude/variability/complexity, frequency features capture the defining rhythms, time–frequency features capture transients. Scaling is essential for distance-based methods but must respect the split. More features can eventually *hurt* (curse of dimensionality); select the few dozen that matter, and do it inside cross-validation. Class separation in feature space is the goal — often visible before any model is trained.

Conceptual questions. (1) Why does a classical, feature-based classifier usually need a fixed-length feature vector rather than the raw 3,000-sample epoch, and what kind of model can instead learn from the waveform directly? (2) What does a *low* spectral entropy tell you about a signal? (3) Why prefer mutual information over the ANOVA F-test for ranking? (4) How can adding a feature *reduce* test accuracy? (5) Why should feature scaling and selection be fit on training data only?

Exploration tasks. Extract the full feature set from one recording, rank by mutual information,

varkonyibarnabas@gmail.com

and remove features one at a time from the bottom while tracking a classifier's accuracy — find the point of diminishing returns. Compute Hjorth parameters and verify that mobility tracks the dominant frequency on synthetic sinusoids. Project features to 2-D with PCA and LDA and compare the class separation. Compute GLCM texture features on two image classes and test their discriminative power.

Design brief. Choose the feature set for a wearable sleep stager that runs *on-device* from a single frontal EEG channel under a tight compute and memory budget. Spec: what must the features capture (which stages, which graphoelements such as spindles), and what are the on-device constraints? Design: pick a compact set — which band powers with which (non-overlapping) band edges, which time-domain and entropy features — and state how many features you can afford. Evaluate: how will you check the set does not leak and that each feature earns its place (mutual information, an ablation)? Motivate: justify dropping an expensive feature against its marginal value. Alternatives: name a richer set you would use if compute were free and a leaner fallback for a microcontroller, and the spec point at which you would switch between them. Rule out: name one feature choice that would be *wrong* here — one that cannot see the spindle the spec says must survive (e.g., a single wide 8–13 Hz band, or amplitude-only features) — and say why no amount of tuning or honest validation would rescue it.

Further reading. Guyon & Elisseeff, "An Introduction to Variable and Feature Selection" (the selection landscape). Hjorth's original 1970 paper (the parameters). Richman & Moorman on sample entropy. Haralick, Shanmugam & Dinstein on GLCM texture features. Bishop, *Pattern Recognition and Machine Learning* (the curse of dimensionality). The scikit-learn feature-selection documentation for the estimators used here.

varkonyibarnabas@gmail.com

Chapter 12 — The Machine-Learning Pipeline: Foundations, Validation & Evaluation

How do we know a 90%-accurate sleep scorer isn't fooling us — and will it work on a patient it has never seen?

There is a special kind of disappointment in machine learning for biosignals: the model that scored 95% in the lab and 60% in the clinic. Almost always, the lab number was a lie the pipeline told itself — a subtle leak that let the model peek at the answer, an imbalance that let it win by guessing, a test set spent so many times it stopped being a test. This chapter is about *not being fooled*. We build the honest machine-learning pipeline from the ground up — the workflow, the train/validation/test discipline, the bias–variance trade-off, and the cross-validation schemes that biosignals demand — and then we run the failure-case clinic: a series of demonstrations that *show* a score inflating, *diagnose* the cause, and *fix* it. The recurring lesson is that in biomedical machine learning the hardest problem is not building the model; it is measuring it truthfully.

12.1 Learning objectives

By the end of this chapter you should be able to:

- Frame a biosignal problem as supervised learning and distinguish it from unsupervised and reinforcement learning (L2).
- Explain overfitting, underfitting, and the bias–variance trade-off in terms of model complexity (L2, L4).
- Choose a cross-validation scheme whose unit of splitting matches the deployment claim, and explain why subject-independent splits (such as Leave-One-Subject-Out or GroupKFold) matter for new-subject generalisation in biosignals (L4).
- Identify and fix the canonical data-leakage failures (L4).
- Handle class imbalance and select honest evaluation metrics — confusion matrix, macro-F1, balanced accuracy, Cohen's kappa — explain why accuracy alone misleads, and why kappa is chance-corrected yet not immune to prevalence (L4).
- Treat labels as noisy measurements with inter-scorer variability, and read metrics in light of that label-reliability context (L2, L4).
- Assess calibration (reliability, Brier score), use an abstention/reject option, and put confidence intervals on the metrics (L4).
- Explain why subject-independent CV is necessary but not sufficient, and where external (dataset/device/site) validation fits before a clinical claim (L4, L5).
- Run the failure-case clinic: measure an inflated score, apply the fix, measure the honest score, and report the gap (L4).

varkonyibarnabas@gmail.com

12.2 Intuition first: learning is fitting, and fitting can go wrong

Traditional programming writes rules by hand: a human expert states, in code, that "if delta power is high and the epoch is quiet, call it N3." Machine learning inverts this — it is handed examples of epochs *already labelled* N3 and infers the rule itself, tuning its internal parameters until its predictions match the labels. The payoff is that the model can discover combinations of features too subtle or too numerous for a human to hand-code; the price is that it will just as happily learn spurious patterns if the data or the evaluation let it. In supervised learning — the setting of this book — we give the model feature vectors (Chapter 11) paired with correct labels (the sleep stage) and it learns the mapping. (Unsupervised learning finds structure without labels, as in the clustering and PCA of Chapters 10 and 13; reinforcement learning learns from rewarded actions, as in adaptive controllers — neither is our main path here.) The supervised workflow is a fixed sequence: extract features, split the data, scale, select features, train, tune, and evaluate — and *every* arrow in that sequence is a place a mistake can quietly inflate your score. The sleep-staging problem frames cleanly in these terms: the features are the per-epoch vectors, the labels are the expert-scored stages, and the *target* is a model that predicts an unseen sleeper's stages as well as a human scorer would.

The central tension is fitting. A model too simple cannot capture the pattern — it underfits, with high *bias*, wrong on training and test data alike. A model too complex memorises the training data, including its noise — it overfits, with high *variance*, perfect on training data and poor on anything new. Between them lies the sweet spot. Figure 12.1 shows this exactly: as a kNN classifier's complexity rises (from many neighbours to one), training error falls to zero while validation error traces a U — dropping as the model learns the real pattern, then rising as it starts memorising noise. The minimum of the validation curve is the model you want; the gap between the training and validation curves is the overfitting you must watch.

12.3 Theory: splitting, validation, and matching the split to the claim

Before splitting, state the claim: what population, device, site, time period, and clinical use should the reported performance represent? The validation split must be built to match that claim — everything below is how.

To estimate how a model will perform on unseen data, we hold data out. The classical scheme is a three-way split: a training set to fit parameters, a validation set to tune hyperparameters and choose models, and a test set — touched *once*, at the very end — to report final performance. Because a single split is noisy, we use cross-validation: partition the data into k folds, train on $k-1$ and validate on the held-out one, rotate, and average. Stratified k-fold preserves each class's proportion in every fold — important under the class imbalance of sleep data.

There is a subtler leak the moment you *both* cross-validate and tune. If you choose hyperparameters (an SVM's C and γ, a forest's depth) by maximising the very cross-validation score you then report, that score is optimistic — you have fit the knobs to the validation folds. The remedy is nested cross-validation: an *inner* loop selects hyperparameters within the training folds while an *outer* loop, which the tuning never sees, estimates performance. For biosignals both loops must be subject-grouped (GroupKFold/LOSO inside and out), so no subject informs both the tuning and the estimate. Whenever a later exercise asks you to sweep a hyperparameter and read off the best validation score, that best score is a *biased* estimate of

varkonyibarnabas@gmail.com

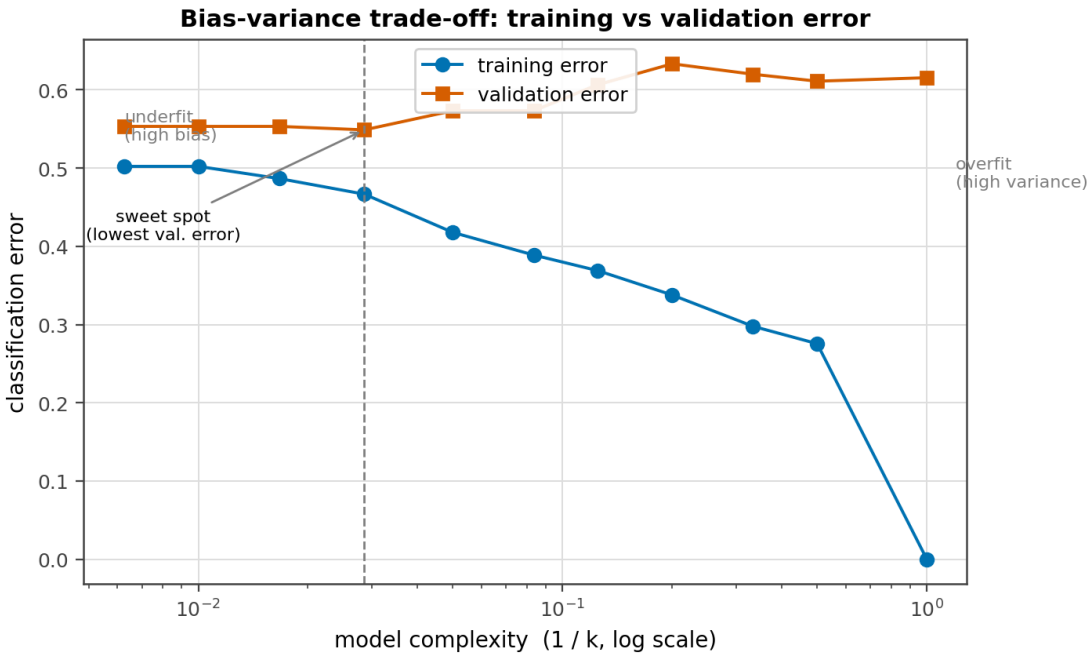


Figure 12.1. The bias–variance trade-off, drawn from a kNN classifier. At low complexity (many neighbours, left) both errors are high — underfitting, high bias. At high complexity (k=1, right) training error hits zero but validation error climbs — overfitting, high variance. The sweet spot is the minimum of the validation curve, and the ever-widening gap between the two curves is the signature of overfitting.

generalisation unless the sweep sits inside a nested, subject-grouped loop.

For biosignals, one principle organises the rest: the unit of splitting should match the deployment claim. The question you are really asking of a validation score is "generalise to *what*?", and the answer dictates what must be held out. If the claim is that the model will work on a *new subject*, then subjects must be kept whole across the split, because a night of sleep yields hundreds of epochs from one person, and epochs from the same night share that person's idiosyncratic physiology — electrode placement, skull thickness, baseline rhythms. If a random split scatters one night's epochs across both train and test, the model can "recognise the subject" instead of learning the stage, and accuracy soars — then collapses on a genuinely new patient. For this new-subject claim, use subject-independent splits such as Leave-One-Subject-Out (LOSO) cross-validation — train on all subjects but one, test on the held-out subject, rotate through all — or its generalisation GroupKFold, which keeps each subject's data entirely within one fold.

The same logic scales to bolder claims. A claim of working across *sites or devices* is not tested by subject-level splits alone; it needs site- or device-level splits that hold out whole hospitals, amplifiers, or montages, because those introduce distribution shifts a within-site subject split never sees (Section 12.13). A *longitudinal* claim — that the model will keep working on the same person weeks later — calls for future-session splits that train on earlier sessions and test on later ones, so drift over time is exposed rather than hidden. Subject-independence is therefore not a universal ritual but a conditional principle: it is the right choice precisely when the claim is new-subject generalisation, which is the common case for a sleep scorer, and it should be strengthened to site-, device-, or session-level splitting whenever the claim reaches further. What all these schemes share is the refusal to let the evaluation leak the very

varkonyibarnabas@gmail.com

thing the claim promises to generalise across.

> Common student mistakes. A quick recap of the ways a split silently inflates a score. (a) *Splitting the wrong unit.* Randomly splitting epochs so the same subject or recording lands in both train and test makes accuracy look excellent and then collapse on a genuinely new person — the model has learned to recognise the sleeper, not the stage. Split by the independent unit that matches the deployment claim: subject or recording (via LOSO or GroupKFold) for a new-subject claim, site or device for a multi-site claim, and a future session for a longitudinal claim. (b) *Learning a transform before splitting.* Running feature selection, scaling, or resampling on the whole dataset before the split lets the training partition peek at the test partition; every learned transform must be fit *inside* the training partition and only then applied to the held-out data. (c) *Tuning on the test set.* Each peek at the test score to adjust something spends a little of its independence — it is spent once, at the very end.

12.4 The failure-case clinic: how to fool yourself, and how to stop

The rest of this chapter is a clinic. Each failure is a way pipelines routinely inflate their own scores; we show the inflation, diagnose it, and fix it. Meet them once here and you will spot them everywhere.

Failure 1 — scaling before the split. Standardising features (Chapter 11) using the mean and variance of the *whole* dataset lets the training set peek at the test set's statistics. The fix is mechanical: fit the scaler on the training fold only, then apply it to the test fold. The same rule governs *any* learned transformation — PCA bases, imputation values, denoising matrices — all fit on train, applied to test.

Failure 2 — epoch-level splitting instead of subject-level. This is the big one, and Figure 12.2 measures it. A naïve random split of epochs (the same night appearing in both train and test) reports far higher accuracy than an honest LOSO split. In this synthetic demonstration the leaky split scores 0.51 while the honest LOSO split scores 0.38 — a 13-point gap that is pure illusion (the exact numbers are illustrative, from simulated signals; the *direction* is the point). The naïve number would evaporate the moment the model met a new patient. The fix, when the claim is new-subject generalisation, is a subject-independent split (LOSO or GroupKFold).

Failure 3 — window-overlap leakage. Overlapping epochs that straddle the split share raw samples, a milder version of Failure 2; the fix is to assign whole overlapping groups to one side. Failure 4 — feature selection outside the cross-validation loop. Ranking features on the *whole* dataset before splitting lets the selection see the test labels; selection should be re-run inside each fold.

Failure 5 — class imbalance hidden by accuracy is important enough for its own section, which follows immediately (Section 12.5): under imbalance a lazy majority-guesser can post a high accuracy while having learned nothing, so the fix is to read the confusion matrix, macro-F1, and Cohen's kappa instead of accuracy alone.

Failure 6 — tuning on the test set. Every time you look at the test score and adjust something, you spend a little of the test set's independence; after enough peeks it is just a second validation set. The test set is spent *once*.

varkonyibarnabas@gmail.com

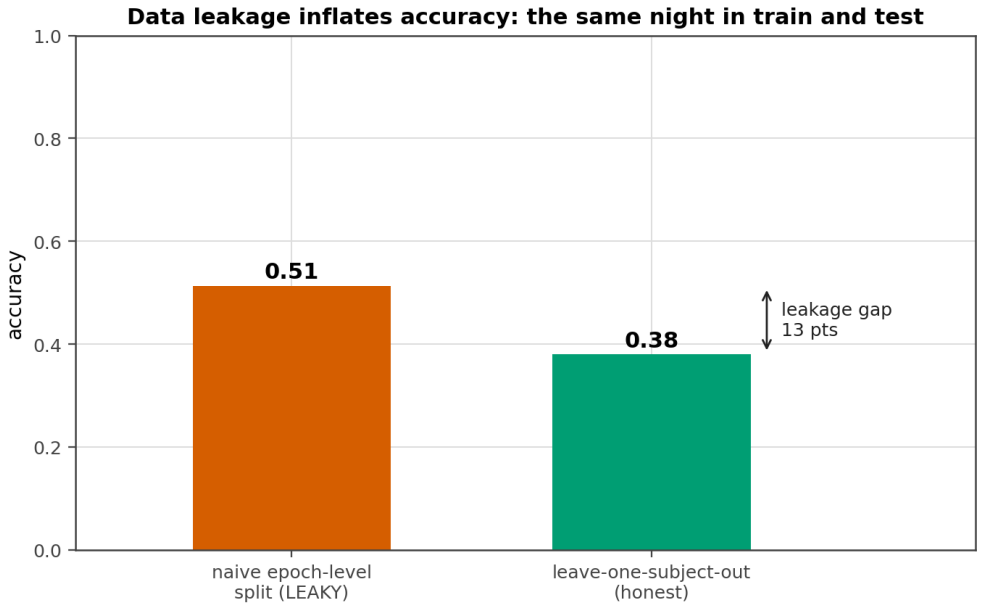


Figure 12.2. Data leakage made visible (synthetic demo; numbers illustrative). A naïve epoch-level split lets the same subject appear in training and test, so the model recognises the night rather than the stage and reports an inflated accuracy. Splitting by subject (leave-one-subject-out) removes the leak — and the accuracy drops to its honest value. The size of the gap is the size of the illusion.

12.5 Class imbalance and honest metrics

Sleep is imbalanced: N2 alone is roughly 45% of a night, while N1 might be 5%. A lazy classifier that predicts "N2, always" achieves ~45% accuracy having learned *nothing*. Accuracy, therefore, is a treacherous headline metric under imbalance. The honest evaluation starts with the confusion matrix — a table of true versus predicted classes whose off-diagonals reveal *which* stages are confused for which. From it come precision (of the epochs called N3, how many were N3), recall (of the true N3 epochs, how many we caught), and their harmonic mean, the F1 score. Averaging F1 across classes gives macro-F1, which weights every stage equally and so *punishes* ignoring the rare ones — unlike weighted averaging, which lets the majority class dominate.

Above all sits Cohen's kappa, which measures agreement corrected for chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

where p_o is observed agreement (accuracy) and p_e is the agreement expected by chance given the class frequencies. Kappa is 0 for a chance-level guesser *no matter how high its raw accuracy*, and 1 for perfect agreement. It is a common headline metric in sleep scoring precisely because it is *less easily fooled than raw accuracy* — but chance-correction is not the same as immunity. Kappa is still sensitive to class prevalence and to systematic bias between raters, and two datasets with the same underlying agreement but different class balance can yield different kappas (the so-called kappa paradoxes). So kappa should be read *alongside*, not instead of, per-class recall, macro-F1, balanced accuracy, the confusion matrix, and clinical summaries — the same panel that guards against the accuracy illusion also guards against over-trusting a

varkonyibarnabas@gmail.com

single kappa. (The Matthews correlation coefficient, MCC, plays a similar chance-corrected role and is popular in binary problems.)

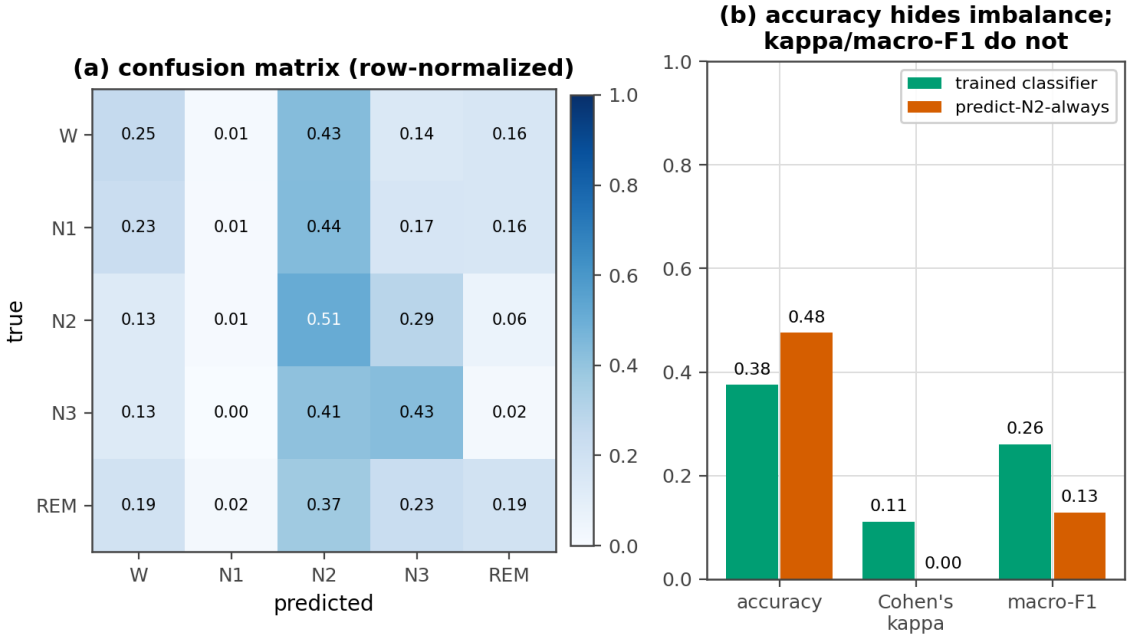


Figure 12.3. Why accuracy misleads under imbalance (synthetic demo; numbers illustrative). (a) The row-normalised confusion matrix shows the classifier's errors concentrating on the rare stages — much of N1 and N3 is swallowed by the dominant N2. (b) The metrics tell the true story: a lazy "always-N2" baseline actually beats the real classifier on *accuracy* (0.48 vs 0.38), yet its Cohen's kappa is 0.00 and its macro-F1 is far lower — because it has learned nothing. Accuracy rewards the majority guess; kappa and macro-F1 do not.

The metrics differ in exactly what they reward, and choosing the wrong one is how honest-looking numbers deceive:

Metric	What it measures	Chance-corrected?	Rewards majority guess?	Best for
Accuracy	fraction correct overall	no	yes — dangerous	balanced classes only
Precision / recall	per-class hit/miss trade-off	no	no	inspecting one class
F1 (per class)	harmonic mean of precision & recall	no	no	a single class of interest
Macro-F1	unweighted mean of per-class F1	no	no — punishes ignoring rare classes	imbalanced multiclass
Weighted F1	frequency-weighted mean of F1	no	partly	when majority matters most
Cohen's kappa	agreement above chance	yes	no — 0 for any chance guesser	a common headline for sleep scoring (report with a metrics panel)
MCC	correlation of true vs predicted	yes	no	binary / imbalanced binary

varkonyibarnabas@gmail.com

Figure 12.3 delivers the lesson with a jolt. In this synthetic demo the trained classifier reaches accuracy 0.38, kappa 0.11, and macro-F1 0.26. The "always-N2" baseline reaches accuracy 0.48 — *higher* — yet kappa 0.00 and macro-F1 far lower (the values are illustrative; the *contrast* is the teaching point). Read by accuracy alone, the baseline "wins"; read by kappa and macro-F1, it is exposed as a rule that only ever predicts the majority class. This is Failure 5 made undeniable, and it is why an honest report of a sleep scorer tends to lead with the confusion matrix and a metrics panel, not accuracy alone.

Common student mistakes. A quick recap of the ways a metric misleads under imbalance. (a) *Reporting accuracy on imbalanced classes.* When one stage dominates, a majority-guesser scores high while having learned nothing — accuracy rewards the majority guess. (b) *Treating Cohen's kappa as immune to imbalance.* Kappa is chance-corrected, so it is far harder to fool than raw accuracy, but it is *not* immune: it is still sensitive to class prevalence and to systematic rater bias, and two datasets with the same underlying agreement but different balance can yield different kappas. Report it *alongside* macro-F1, balanced accuracy, per-class recall, and the confusion matrix, never as a lone headline. (c) *Reading one headline number instead of the matrix.* A single scalar hides *which* stages are confused for which; the confusion matrix and the clinical summaries tell the story the headline cannot.

Labels are measurements too. It is tempting to treat the reference labels as perfect truth and every disagreement as a model error. But sleep-stage labels are *expert annotations*, not ground truth handed down by nature. The American Academy of Sleep Medicine (AASM) manual provides the scoring standard, yet even trained human scorers do not agree perfectly with one another: reported inter-scorer epoch agreement is often around 80–83%, with Cohen's kappa well below 1 and lowest for the transitional stages (N1 in particular). Note that these two numbers live on *different scales* and must not be conflated: an 82% *raw* agreement is not a kappa of 0.82. Concretely, with an observed agreement $p_o = 0.82$ and a chance agreement $p_e = 0.30$, $\kappa = (0.82 - 0.30)/(1 - 0.30) = 0.52/0.70 \approx 0.74$ — so an 82% raw agreement is a kappa of about 0.74, not 0.82 (illustrative numbers). Because kappa subtracts the agreement expected by chance, a given kappa always corresponds to a *higher* raw percent agreement — so the "reputable scorers reach $\kappa \approx 0.8$" figure quoted in Section 12.9 sits above roughly 80–83% raw agreement, not at it. (Published multi-centre AASM runs report five-stage inter-scorer kappa around 0.68–0.80; see Rosenberg & Van Hout, 2013.) Reading a percent agreement as though it were a kappa is exactly the scale confusion this chapter fights. This *inter-scorer variability* has two consequences for how we read metrics. First, it provides a practical reference for model performance: judged against a single noisy human rater, a model cannot reliably appear to exceed the consistency of those labels, so a kappa near the human–human kappa may already be "as good as this target allows," not a failure. This is *label-reliability context*, not a universal mathematical ceiling — a model evaluated against a better-defined target (consensus, adjudicated, or repeated-rating labels) can legitimately exceed pairwise human agreement. Second, it means some of a model's apparent errors are really cases where the experts themselves would have disagreed; a confusion matrix concentrated on N1↔N2 or N2↔REM boundaries often mirrors exactly where human scorers disagree. The honest move is to compare a model's agreement with an expert against the agreement *between* experts on the same data, and to treat the label as a noisy measurement with its own reliability, quantified and reported like any other measurement in this book.

varkonyibarnabas@gmail.com

Failure-case clinic summary. (1) *Scale before the split* → fit the scaler on train only. (2) *Epoch-level split* → use LOSO / GroupKFold. (3) *Window-overlap leakage* → keep overlapping groups on one side. (4) *Selection outside CV* → select inside each fold. (5) *Imbalance hidden by accuracy* → read kappa, macro-F1, and the confusion matrix. (6) *Tuning on the test set* → touch the test set once. Each failure inflates a score; each fix restores the honest one; the gap between them is what you must report.

From multiclass to binary detection: ROC, PR, and the clinical metric cluster

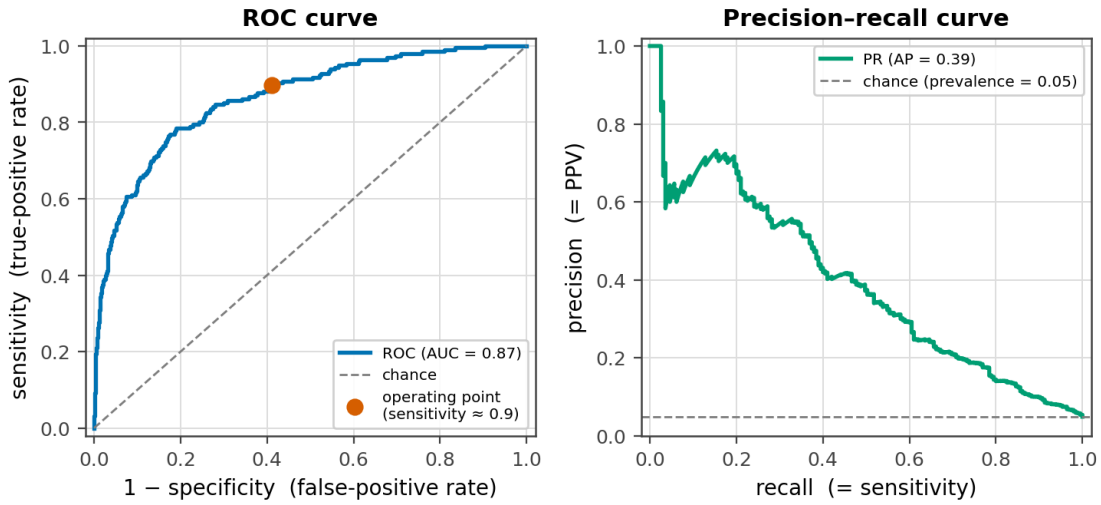
The metrics above suit multiclass staging, but many biosignal tasks are binary detection — apnea vs none, seizure vs background, this beat abnormal or not — and the binary setting has its own vocabulary, the one a clinical audience expects. The four cells of a binary confusion matrix generate a cluster of paired rates. Sensitivity (= recall = true-positive rate) is the fraction of real positives the detector catches; specificity (true-negative rate) is the fraction of real negatives it correctly clears. Their predictive counterparts read the matrix the other way: positive predictive value (PPV) — precision — is the fraction of positive *alarms* that are real, and negative predictive value (NPV) the fraction of negative calls that truly are. The crucial and often-missed point: sensitivity and specificity are properties of the *test* and do not depend on how common the condition is, whereas PPV and NPV depend heavily on prevalence — a 99%-specific apnea detector still floods a low-prevalence night with false alarms, so its PPV can be poor even when its specificity looks superb. Report the pair the clinical question needs.

Each of these numbers is fixed only *once a decision threshold is chosen*, yet most detectors output a continuous score, so two curves sweep the threshold and summarise the whole trade-off. The receiver operating characteristic (ROC) plots sensitivity against 1 − specificity as the threshold moves; a perfect detector hugs the top-left corner, the diagonal is chance, and the area under the curve (AUC) condenses it to a single threshold-free number (0.5 = chance, 1.0 = perfect). ROC is the default for *comparing* detectors, but it has a blind spot under heavy imbalance: because the false-positive *rate* has a huge negative denominator, a detector can look excellent on ROC while raising far more false than true alarms. The precision–recall (PR) curve — precision against recall — exposes exactly that, which is why it is the honest companion whenever positives are rare (the usual case in event detection). Figure 12.4 shows both for one imbalanced detector: the same model that looks strong on ROC reveals a mediocre precision on the PR curve, and the operating point you would actually deploy is the one that meets the clinical constraint — a minimum sensitivity for a screening tool, a minimum PPV for one that triggers an intervention — not the one that maximises accuracy.

12.6 Domain metrics: from epochs to sleep architecture

Per-epoch metrics answer "how often does the model agree with the expert on a single 30-second epoch?" But a clinician reading a sleep study cares about the *shape of the night*, and that is summarised by measures derived from the whole-night hypnogram (the sequence of stages plotted against time). The core set: total sleep time (TST, the minutes actually asleep), sleep-onset latency (SOL, how long to fall asleep), REM latency (time from sleep onset to the first REM period, a marker in depression and narcolepsy), wake after sleep onset (WASO, fragmentation), and sleep efficiency (TST as a fraction of time in bed). These are the numbers that

varkonyibarnabas@gmail.com



Under heavy imbalance the same detector looks strong on ROC (high AUC) yet has modest precision (PPV) on the PR curve — pick the operating point from the clinical constraint, not from accuracy.

Figure 12.4. Threshold-sweeping curves for a binary detector on imbalanced data (synthetic; numbers illustrative). (a) The ROC curve (sensitivity vs 1 − specificity) sits well above the chance diagonal with a high AUC, so the detector *looks* strong. (b) The precision–recall curve, on the same scores, tells the harder truth: because positives are rare (5% prevalence), precision (PPV) falls as recall rises, so many alarms are false — a weakness the ROC hides. Read ROC/AUC to compare detectors, PR when positives are rare, and choose the operating point (dot) from the clinical constraint, not from accuracy.

drive a diagnosis.

The subtlety is that a scorer can post a respectable epoch-level kappa yet still mis-estimate these summaries — because errors that scatter randomly across epochs cancel in a *count* like TST but errors that cluster (say, systematically calling early-night N1 as wake) bias it. So agreement is also assessed at the *night* level, comparing the model's hypnogram-derived measures against the expert's with a Bland–Altman plot: the difference between the two estimates plotted against their mean, revealing both a systematic bias and the limits of agreement (roughly ± 1.96 standard deviations of the difference). A Bland–Altman plot says something a correlation coefficient cannot — *how far off*, and *in which direction*, the model's clinical summaries run. Statistical performance (kappa, macro-F1) and clinical usefulness (accurate architecture measures) are related but not identical, and a complete evaluation reports both, because a model can be statistically decent and clinically misleading, or vice versa.

12.7 Uncertainty and calibration

Every metric so far reports a *point* — a single kappa, a single macro-F1 — as though the model were equally sure of every epoch and the score itself carried no error bar. For clinical decision support, both of those silences are dangerous, and closing them is as important as the headline number.

The first gap is calibration: does a predicted probability mean what it says? A model that outputs "0.9 probability N3" is *well calibrated* if, across all epochs it scores 0.9, roughly 90% truly are N3. Many high-accuracy models are badly calibrated — confidently wrong — and

varkonyibarnabas@gmail.com

confidence that does not track correctness is worse than no confidence at all. Calibration is checked with a reliability diagram (predicted probability on the x-axis, observed frequency on the y-axis; the diagonal is perfect) and summarised by the Brier score, the mean squared difference between predicted probability and outcome, which rewards being confident *and right* and punishes being confident and wrong. When a model is miscalibrated, its probabilities can be post-hoc corrected (Platt scaling, isotonic regression) — fit, like every learned transformation, on a held-out calibration split, never the test set.

Calibrated confidence unlocks a second, clinically vital option: the abstention or reject option — knowing *when not to score*. Rather than forcing a label on every epoch, a model can decline the ambiguous ones (low-confidence epochs, artifact-riddled segments, transitions the experts themselves dispute) and route them to a human. This trades coverage (the fraction of epochs the model scores) against selective accuracy (its accuracy on the epochs it does score), and the trade-off is drawn as a *risk–coverage curve*. For clinical decision support the ability to say "I am not sure — please review this epoch" is often more valuable than a marginal gain in overall accuracy, because a confident wrong answer on a critical epoch can be worse than a flagged uncertainty.

The third gap is uncertainty around the *metrics themselves*. A kappa of 0.66 computed on a handful of subjects is not the same evidence as a kappa of 0.66 across hundreds, yet the point estimate looks identical. Report metrics with confidence intervals — a bootstrap over subjects for kappa, macro-F1, and the clinical summaries, or the spread across cross-validation folds — so the reader can see whether a difference between two models is real or within noise. A number without an interval invites over-reading; a number with one is honest about how much it actually knows.

> Comparing two models: use a paired test. Confidence intervals put an error bar on *one* model; deciding whether classifier A genuinely beats B is a different question, and comparing their point estimates (or even their overlapping intervals) is not enough. Compare the two models *on the same folds or subjects* with a paired test that exploits that pairing. For per-epoch predictions, McNemar's test works on the counts of epochs where the two models disagree — where A is right and B wrong versus the reverse. One caveat carries the same subject-dependence lesson as the rest of the chapter: pooling *per-epoch* discordant pairs treats epochs as independent, but many epochs come from each subject, so the discordant counts are over-dispersed and the test becomes anti-conservative — it reports differences as "significant" too easily. Aggregate to the subject level (a paired test across each subject's accuracy or kappa) or use a cluster-robust variant before believing a p-value. For cross-validated scores, the naïve paired t-test is over-optimistic because CV folds share training data and are not independent; the corrected resampled t-test of Nadeau & Bengio inflates the variance estimate to account for that dependence and is the honest choice. Either way, a small measured gap between two models can vanish once the pairing and the fold dependence are taken seriously.

12.8 External validation

Subject-independent cross-validation (Section 12.3) is *necessary* for a new-subject claim, but it is not *sufficient* for a clinical one. LOSO or GroupKFold still draws its train and test folds from the *same* dataset — the same recruitment, the same devices, the same site, the same labelling protocol — so an honest within-dataset score certifies generalisation to new *subjects* drawn

varkonyibarnabas@gmail.com

from that one distribution, and no further. The next layer of evidence is external validation: evaluating the frozen model, with no refitting, on an *independent* dataset — a different hospital, a different device or electrode montage, a different population, ideally labelled by a different scoring team. It is common for a model with a strong internal kappa to lose several points externally, because external data expose the covariate, prior, and concept shifts of Section 12.13 that internal folds cannot. This is why any clinical claim should rest on external, not merely internal, evidence, and why regulators and reviewers ask for it.

> The clinical-validation ladder. External validation is one rung of a longer evidence ladder that a method must climb before it reaches patients, and it is worth seeing the whole ladder so an internal cross-validation number is read for what it is — the *first* rung, not the last. (1) Proof of concept / analytical validation — the retrospective, within-dataset performance this chapter has been about (LOSO/GroupKFold, the honest metric panel): does the method work at all, on curated data? (2) External / clinical validation — the frozen model evaluated prospectively or on independent data from other sites, devices, and populations: does it still work in the wild? (3) Clinical utility / comparative-effectiveness study — ideally a randomised comparison: does *using* the method actually change decisions and improve outcomes, not merely predict a label? (4) Post-market surveillance — continuous monitoring after deployment for performance drift (Section 12.13), rare failure modes, and harms the trials were too small to see. Each rung answers a strictly harder question than the last, and a strong number on rung 1 says nothing about rung 3. The regulatory pathways of Chapter 15 are, in effect, the paperwork that certifies a method has climbed the rungs relevant to its risk class.

External validation is also where the clinical-summary metrics earn their keep. Beyond epoch-level agreement, an external evaluation compares whole-night architecture — total sleep time (TST), sleep efficiency, REM latency, wake after sleep onset (WASO) — and overall hypnogram agreement, and it reports the systematic bias and limits of agreement with Bland–Altman analysis (Section 12.6) on the new cohort. A model can hold its epoch kappa across sites yet drift on TST or REM latency in a way that changes a diagnosis, so both statistical and clinical agreement are re-checked externally. The capstone (Chapter 16) puts the honest-evaluation machinery to work end to end: it freezes the tuned pipeline and reports the epoch metrics and clinical-summary agreement plots on a held-out set — nights from the same study left untouched during every iteration. That held-out evaluation is a genuine test of generalisation, but it is still an *internal* one, drawn from the same dataset and distribution; true external validation — an *independent* cohort, device, or site, ideally labelled by a different scoring team — is the *next* evidence layer, beyond what the case study demonstrates, and it is what a clinical claim ultimately rests on. The point here is that this external layer sits *after* subject-independent CV and after an internal held-out check, not as a substitute for either.

12.9 Data curation and quality control

Before a single model is trained, the dataset itself must be made trustworthy, because no validation scheme can rescue a model fed corrupted or mislabelled examples. Garbage in, garbage out is not a slogan here; it is the dominant term in final model quality, and it swamps almost every choice of algorithm. A *curated* dataset is one deliberately cleaned and documented rather than merely collected. Curation has several parts. First, consistent preprocessing: every recording passes through the same filtering, resampling, and normalisation, so a spurious

varkonyibarnabas@gmail.com

50 Hz notch applied to half the data does not become a feature the model latches onto. Second, artifact and epoch rejection: segments corrupted by motion, saturation, or disconnection (Chapter 8) are removed by a fixed, reproducible rule, not by eye. Third, label auditing: the labels themselves are checked. In sleep scoring two experts rarely agree perfectly, and their agreement is quantified by Cohen's kappa — reputable scorers typically reach $\kappa \approx 0.8$ (Rosenberg & Van Hout, 2013, report five-stage inter-scorer kappa of roughly 0.68–0.80), and a dataset whose inter-rater reliability falls far below that is teaching the model an inconsistent target. Remember that this $\kappa \approx 0.8$ is *chance-corrected* and so is not the same scale as the raw 80–83% inter-scorer agreement of the "Labels are measurements too" box in Section 12.5: the same scorers post a raw agreement higher than 82% and a kappa near 0.8, because kappa first removes the agreement expected by chance. Comparing a percent agreement directly against a kappa is the scale error this chapter warns against. Fourth, de-duplication: exact or near-duplicate recordings are removed, since a duplicate straddling a split is a leak (Section 12.4) and a duplicate within the training set silently reweights it. Fifth, documentation: the dataset ships with a *datasheet* — who was recorded, on what device and montage, how labels were produced, and what was excluded — plus a documented train/test provenance so anyone can verify the test subjects were never seen during development. A curated dataset is auditable; an uncurated pile of files is not, and the difference surfaces months later as an unexplained clinical failure.

12.10 Interpolation of gaps

Real biosignals arrive with holes. An electrode disconnects, a cardiotocography (CTG) probe loses the fetal heart-rate (FHR) trace as the fetus shifts, a wireless sensor drops packets. The question is not *whether* to fill the gaps but *when filling is honest*. The tools form a ladder of assumptions. Linear interpolation draws a straight line across the gap — cheap, safe for very short holes, but it flattens curvature and injects sharp corners at the endpoints. Spline or cubic interpolation fits a smooth polynomial through the surrounding samples, preserving curvature and looking physiologically natural over slightly longer gaps. Model-based interpolation goes further, using a signal model (an autoregressive predictor, Chapter 6, or a learned generator) to synthesise plausible samples — powerful but capable of *inventing* structure that was never measured. That danger sets the golden rule: interpolation may reconstruct a gap short enough that the underlying physiology cannot have changed much, but it must never bridge a gap so long that it invents physiology that was never recorded. A concrete FHR example makes the boundary vivid. At a normal fetal heart rate near 140 beats per minute, a dropout shorter than 0.5 s spans at most a beat or two; the trace on either side pins the value tightly, and cubic interpolation gives a plausible fill — but even a short gap can mask a brief deceleration, so a filled sample is an estimate, not evidence. A 30 s loss is a different animal: an entire deceleration — the very event that signals distress — could begin and end inside that window, so interpolating it would paint a reassuring smooth line over a possible emergency. The short gap is interpolated; the long gap is flagged and excluded, and its *duration* recorded, because as the next section argues the loss itself may be informative. A useful rule of thumb: interpolate gaps up to a small fraction of the shortest event you care about — often on the order of one second or a few percent of the signal — and flag anything longer for exclusion and human review.

varkonyibarnabas@gmail.com

12.11 Missing data — is it missing at random?

Interpolation asks *how* to fill a gap; the deeper question is *why the gap is there at all*, because the mechanism that produced the missingness decides whether filling it is safe or actively harmful. Statisticians distinguish three mechanisms, and confusing them is one of the most consequential mistakes in clinical machine learning.

Missing Completely At Random (MCAR) means the probability of a value being missing is independent of everything — of both the observed data and the missing value itself. A sensor battery that dies at a random moment, or a lab result lost to a clerical filing error, is MCAR. Here the observed data are an unbiased sample of the whole, so dropping the missing rows or imputing them is comparatively safe. Missing At Random (MAR) is the more subtle and more common case: missingness depends on *observed* variables but not, once those are accounted for, on the missing value itself. A blood gas is measured more often in patients already flagged as sick, so its presence correlates with observed severity; a photoplethysmography (PPG) trace drops out more often in colder or darker-skinned patients, but skin temperature and skin tone are themselves observed covariates. Because the dependence runs through variables we *can* see, imputation that conditions on those covariates — regression or multiple imputation — can be valid. Missing Not At Random (MNAR) is the dangerous case: missingness depends on the *unobserved value itself*. The canonical example is CTG. Fetal-monitor signal loss often coincides with fetal movement or fetal distress, so the very fact that the FHR is missing carries risk information — the sickest segments are disproportionately the missing ones. Dropping or naïvely imputing those segments does not merely lose data; it systematically *hides the sickest cases* and biases the model toward optimism. An EHR analogue is a test that was never ordered precisely because the clinician judged the patient too unwell to move, or conversely too obviously well to bother — the absence encodes the clinician's hidden judgement.

Mechanism	Missingness depends on	Biosignal / EHR example	Safe to impute?
MCAR	nothing (independent of all data)	sensor battery dies at random; lab value lost to clerical error	Yes — drop or impute; observed data unbiased
MAR	*observed* variables only	blood gas measured more in already-flagged sicker patients; PPG dropout tied to observed skin tone / temperature	Yes, *if* imputation conditions on those observed covariates
MNAR	the *unobserved* value itself	CTG signal loss coincides with fetal movement / distress — missingness itself carries risk; test not ordered because patient judged too unwell (or too well)	No — imputation alone biases the model; add a missingness indicator

The practical upshot is a short discipline. Always ask *why* a value is missing before you impute it, and report the assumed mechanism as part of the dataset's documentation. Crucially, MCAR, MAR, and MNAR are *assumptions* about the missingness process, not properties you can read off the observed data alone: an observed association between CTG signal loss and poor outcome makes the loss *plausibly* MNAR, but does not by itself prove that missingness

varkonyibarnabas@gmail.com

depends on the unobserved FHR value — so where the distinction matters, complement the model with a sensitivity analysis across mechanisms and a clinical inquiry into the cause, and remember that a missingness indicator lets the model *use* the informativeness of absence but does not *remove* the underlying MNAR bias. When MNAR is plausible, do not try to erase the missingness — *encode* it: add a missing-indicator feature (a binary flag, and often the gap's duration) so the model can learn that absence itself predicts risk, because for a genuinely MNAR signal imputation alone can never recover what the absence was telling you. Concretely, the move is to *engineer the absence of data as a predictive feature*: append a binary missingness-flag channel alongside the signal, together with summary statistics such as the missing fraction and the longest-gap duration, to the downstream feature set. A classifier can then learn that, for example, CTG signal dropout is *itself* predictive of fetal distress, rather than treating the missingness purely as data loss to be imputed away. And whatever you impute, fit the imputation model — its means, regressions, or generators — on the training partition only and apply it to the test partition; imputing with statistics computed over the whole dataset is exactly the leakage of Section 12.4, letting the test set's values leak backward into training. The standard reference for doing this properly is Van Buuren's *Flexible Imputation of Missing Data*, which develops multiple imputation and the MCAR/MAR/MNAR framework in depth.

12.12 Data augmentation

When labelled biosignal data are scarce — and they usually are, because expert labelling is expensive — data augmentation expands the training set with *label-preserving* transformations of the examples already in hand. The point is to teach the model the invariances the physiology genuinely has, so it stops keying on accidents of a particular recording. For one-dimensional signals the standard repertoire is: time-shifting or jittering an epoch by a few samples; amplitude scaling to mimic gain and electrode-impedance differences; time-warping to stretch or compress locally, imitating natural rate variation; adding realistic, physiological noise (baseline wander, muscle activity, powerline hum from Chapter 8) rather than white noise; and mixup, which forms convex combinations of two examples and their labels to smooth the decision boundary. For images — spectrograms, ultrasound frames, histology — the analogues are flips, small rotations, and elastic deformations. Two cautions decide whether augmentation helps or quietly poisons the model. First, every augmentation must be physiologically plausible: a horizontal time-flip may be fine for a texture but inverting an ECG top-to-bottom produces a waveform that does not occur in nature, and labelling it as the same class teaches an impossibility. The transformation must map a real signal to another *real* signal of the same class. Second — and this connects straight back to the leakage clinic — augmentation must be applied inside the training fold only. Augmenting the full dataset *before* the split scatters transformed copies of a test example into the training set, so the model effectively trains on its own test data and the score inflates exactly as in Failure 2. Split first, augment the training partition, evaluate on the untouched original test data.

12.13 Domain shift and dataset shift

A model can pass every check in this chapter — subject-independent validation, honest metrics, no leakage — and still fail the moment it meets data from a new source. The reason is dataset shift: the distribution the model is deployed on differs from the distribution it was

varkonyibarnabas@gmail.com

trained on, so a validation estimate that was honest *for the training distribution* no longer predicts field performance. Shift comes in three flavours. Covariate shift is a change in the input distribution $P(x)$ while the input-to-label rule $P(y \mid x)$ stays fixed: a different device, electrode montage, hospital, or patient population, or simply one night to the next. A sleep stager trained on one polysomnography (PSG) montage often stumbles on another montage that places electrodes differently; an ECG model trained on one manufacturer's amplifier can misread another's filtered output. Label or prior shift is a change in the class balance $P(y)$: a model trained on a *clinical* population, thick with pathology, is miscalibrated when deployed as a *screening* tool on a mostly healthy stream, because the base rates it implicitly assumed no longer hold. Concept drift is the deepest: the input-to-label mapping $P(y \mid x)$ itself changes over time, as clinical definitions, protocols, or populations evolve, so yesterday's correct rule becomes today's error. Shift is detected by monitoring performance on incoming data and by distribution tests that compare deployment inputs against the training distribution and raise an alarm when they diverge. It is mitigated up front by diverse, multi-site training data and by subject- and site-independent validation — the LOSO discipline of Section 12.3, extended to leave out whole sites so the reported number reflects a genuinely new hospital — and after deployment by recalibration and domain adaptation that re-tune the model to the new distribution. Because shift often develops gradually as populations and equipment change, its slow-onset form, model drift, together with the monitoring and retraining that answer it, is treated in the deployment chapter (Chapter 15).

12.14 The ML/AI bridge

This chapter is the discipline that makes every model in this book trustworthy. Whatever the classifier — the kNN and SVM here, the ensembles of Chapter 13, the deep networks of Chapter 14 — the validation and evaluation logic is the same: split so that the held-out unit matches the deployment claim (subjects for a new-subject claim), fit every learned step inside the training partition, and judge by imbalance-aware metrics read as a panel rather than a single number. A more powerful model does *not* relax these rules; if anything, a higher-capacity model overfits and leaks *more* eagerly, so the honest-evaluation harness matters more as the models grow. The pipeline, not the algorithm, is what earns clinical trust.

12.15 Companion notebook

The companion notebook is a validation-and-metrics dashboard built around a LOSO harness. It runs each failure case live: it scales before and after the split and prints the inflated-versus-honest accuracy; it splits by epoch and by subject and tabulates the leakage gap of Figure 12.2; it selects features outside and inside the CV loop and shows the difference. It then trains a classifier on subject-independent folds and produces the full metrics panel of Figure 12.3 — confusion matrix, per-class precision/recall/F1, macro-F1, balanced accuracy, and Cohen's kappa with a bootstrap confidence interval — alongside the "always-majority" baseline so the imbalance illusion is unmissable. For a binary event-detection task it also sweeps the decision threshold to draw the ROC and precision–recall curves of Figure 12.4 and reads off sensitivity, specificity, PPV, and NPV at a chosen operating point, making the imbalance blind-spot of the ROC something you watch happen. It adds a reliability diagram and Brier score to check calibration and demonstrates an abstention (reject-option) threshold with a risk–coverage

varkonyibarnabas@gmail.com

curve. A final section derives hypnogram measures and draws a Bland–Altman agreement plot, previewing the external-validation workflow of Chapter 16.

> Hands-on: a trustworthy-cross-validation toolkit. The splitting discipline of this chapter — subject/group-grouped, temporal (with *purging/embargo* for time series so a window's neighbours cannot straddle the split), nested, with automatic leakage checks and clinical confidence intervals — is packaged in TrustCV (`pip install trustcv`; Abtahi & Karbalaie, 2025), a companion cross-validation library. It extends the familiar scikit-learn splitter interface with grouped and purged temporal splitters, a `DataLeakageChecker` that scans for shared-subject, temporal, feature-statistical, and near-duplicate leakage, bootstrap sensitivity/specificity/PPV/NPV, and a `RegulatoryReport` for FDA/CE documentation (Chapter 15). Its 14 interactive notebooks and worked demonstrations (`ki-smile.github.io/trustcv`) are a fast self-check on exactly the pitfalls of §12.3–12.4 across ICU time series, multi-site, patient-level, and imaging scenarios — work through them and see which you would catch.

12.16 Case-study tie-in

In the sleep-staging case study this chapter defines the *rules of evaluation* that all iterations obey. Because the deployment claim is new-subject generalisation, performance is reported under Leave-One-Subject-Out (or GroupKFold) cross-validation from the very first iteration rather than an epoch-level split, so the reported number predicts performance on a genuinely new sleeper even at the deliberately weak baseline. Every learned step, from the feature scaler to the feature selector to any denoiser, is fit inside the training folds. And the headline metric is Cohen's kappa read alongside the full confusion matrix and macro-F1, not accuracy, so a model cannot win by exploiting the roughly 45%-N2 imbalance — with the reminder that the expert labels themselves carry inter-scorer variability that bounds how high kappa can honestly go. The failure-case clinic is not an academic aside here; it is the checklist that separates a case study that generalises from one that merely looks good on paper, and a genuinely clinical claim would go further still, to the external validation of Section 12.8.

12.17 Summary, questions, and further reading

Key formulas. Bias–variance: total error decomposes into bias^2 + variance + irreducible noise; Cohen's kappa $\kappa = (p_o - p_e)/(1 - p_e)$; F1 $= 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$; macro-F1 = unweighted mean of per-class F1; LOSO error = mean over subjects of held-out-subject error.

Key ideas. Model error trades bias against variance; validation curves locate the sweet spot and reveal overfitting. The unit of splitting should match the deployment claim — subject-independent splits (LOSO/GroupKFold) for new-subject generalisation, site/device-level splits for multi-site claims, future-session splits for longitudinal ones. Every learned transformation lives inside the training partition. Under imbalance, accuracy misleads; read the confusion matrix, macro-F1, balanced accuracy, and Cohen's kappa together — kappa is chance-corrected but still prevalence-sensitive, not immune. Labels are noisy measurements with their own inter-scorer variability, which provides label-reliability context for performance against human annotations — not a universal ceiling. Report calibration, an abstention option, and confidence intervals on the metrics; and remember that subject-independent

varkonyibarnabas@gmail.com

CV is necessary but not sufficient — external validation is the next evidence layer. The six canonical failures each inflate a score, each have a fix, and the honest report is the *gap* between inflated and corrected.

Conceptual questions. (1) A model scores 98% training accuracy and 62% validation accuracy — diagnose it and prescribe a fix. (2) Why does an epoch-level random split inflate accuracy for a subject-varying signal? (3) A classifier reports 65% accuracy but kappa 0.05 — what has it likely learned? (4) Where in the pipeline should feature scaling be fit, and why? (5) Why is the test set "spent once"? (6) Given a validation claim, name the splitting scheme it requires: new subjects; a new hospital; the same patients weeks later. (7) Two experts agree on ~82% of epochs — how should that bound your reading of a model's kappa? (8) A model has high accuracy but a poor reliability diagram — what is wrong, and how could an abstention option help? (9) Why is a strong LOSO kappa still not enough to support a clinical claim? (10) Transfer. Your leakage-aware LOSO pipeline was built for sleep EEG. A colleague reuses it for seizure detection from continuous multi-hour scalp EEG, where each patient contributes many time-adjacent windows and seizures cluster in time. Name the correct splitting unit, and one leakage risk that continuous within-patient recording introduces that per-night sleep epochs did not.

Exploration tasks. Take a labelled feature set and measure the leakage gap: report accuracy under a naïve epoch-level split and under LOSO, and tabulate the difference. Repeat with feature selection moved inside versus outside the CV loop. Build a confusion matrix on an imbalanced problem and compare accuracy, macro-F1, and kappa against an "always-majority" baseline. Sweep a model's complexity and plot the training/validation curves to locate the sweet spot.

Design brief. Design the validation scheme for a sleep scorer that will ship first as a *new-patient home device* and later be claimed to work *across three hospitals*. Work the five moves for *each* claim. Spec: state precisely what population, site, and device the reported number must represent. Design: choose the splitting unit and cross-validation scheme (and whether tuning needs a nested loop). Evaluate: which metric and which held-out set report honestly for that claim? Motivate: justify the split against the deployment claim. Alternatives: name the *wrong-but-tempting* scheme a hurried analyst would use, and show exactly what it would silently reward. The two specs should produce two different, each-defensible designs. Rule out: for either claim, name a validation choice that would be *wrong* — one that reports a number the claim cannot support (e.g., a random-epoch split for a new-patient claim) — and show exactly what it silently rewards.

CV design scenarios (choose the split, name the wrong one). For each case state the splitting unit you would use, *why*, and what a plain random k-fold would *silently reward* — then check yourself against the interactive quiz and worked demonstrations at `ki-smile.github.io/trustcv`. (a) Diabetes risk from 8 hospitals (5000 patients) that must work at a *ninth* hospital. (b) Sepsis prediction from ICU vitals sampled every 5 minutes over 72 hours. (c) A cardiology model from 800 patients with 3–8 visits each. (d) A pneumonia detector from chest X-rays with several images per patient. In which of these is plain k-fold not merely suboptimal but *wrong*, and exactly which dependency (site, time, subject) does each ignore?

Further reading. Hastie, Tibshirani & Friedman, *The Elements of Statistical Learning* (bias–variance, cross-validation, evaluation). Cohen's 1960 paper on kappa. Kaufman et al. on

varkonyibarnabas@gmail.com

leakage in data mining. Saeb et al., "The need to approximate the use-case in clinical machine learning" (subject-level validation). Van Buuren, *Flexible Imputation of Missing Data* (the MCAR/MAR/MNAR framework and multiple imputation). Quiñonero-Candela et al., *Dataset Shift in Machine Learning* (covariate, prior, and concept shift). Guo et al., "On Calibration of Modern Neural Networks" (reliability, temperature scaling) and the classic Platt/isotonic calibration work. Rosenberg & Van Hout, "The American Academy of Sleep Medicine inter-scorer reliability program: sleep stage scoring," *Journal of Clinical Sleep Medicine* 9(1):81–87 (2013), and related studies on inter-scorer reliability in sleep staging (five-stage inter-scorer kappa $\approx$ 0.68–0.80; why human agreement provides label-reliability context rather than a universal ceiling, and why percent agreement and chance-corrected kappa are different scales). The AASM sleep-scoring manual for the scoring standard and hypnogram-derived architecture measures, and the scikit-learn model-selection, calibration, and metrics documentation for the tools used here. TrustCV (Abtahi & Karbalaie, 2025; MIT-licensed) — a framework-agnostic trustworthy-cross-validation toolkit with grouped, temporal, and purged splitters, an automatic `DataLeakageChecker`, clinical metrics with bootstrap confidence intervals, and regulatory reporting, plus 14 interactive notebooks and clinical demonstrations at `ki-smile.github.io/trustcv`.

varkonyibarnabas@gmail.com

Chapter 13 — Classical Learning Algorithms & Unsupervised Analysis

Given a clean feature matrix, which learner should score the sleep — and can any structure be found without a single label at all?

Chapter 12 built the honest scaffolding: how to split, how to validate, how to measure without fooling ourselves. But it left a hole in the middle of the pipeline shaped like the word *classifier*. This chapter fills it. We meet the workhorse supervised learners of biomedical signal analysis — k-nearest neighbours, support vector machines, decision trees, and the random forest that so often wins — and we learn not just how each draws its decision boundary but *when* to reach for it. Then we turn the lights off. Unsupervised methods — k-means, hierarchical clustering, and principal-component analysis — ask a different and quietly profound question: is there structure in the feature space *before* any expert has labelled a thing? The two halves meet in one lesson that runs through the whole book: reach for the simplest model that fits the problem, prove it works with the discipline of Chapter 12, and treat the powerful models as a step you *earn*, not a place you start.

13.1 Learning objectives

By the end of this chapter you should be able to:

- Explain how k-NN, SVM, decision trees, and random forests each form a decision boundary, and match each to a problem's shape and size (L2, L4).
- Reason about the bias–variance and interpretability trade-offs among the classical learners (L4).
- Apply the maximum-margin idea and the kernel trick, and set the SVM's C and γ sensibly (L4).
- Explain bagging, feature randomness, and out-of-bag error in a random forest, and read a feature-importance ranking (L4).
- Run k-means and hierarchical clustering and interpret their output against known labels (L4).
- Use PCA for visualisation and dimensionality reduction, and distinguish it from feature *selection* (L4).
- Adopt the classical-baseline mindset: fit the simplest capable model first, then justify any move to something heavier (L5).

13.2 Intuition first: a boundary through a feature cloud

Every supervised classifier does the same job — it carves the feature space into regions, one per class, and labels a new point by the region it lands in. What differs is *how* each draws the fence. Figure 13.1 makes this literal: the same four sleep-like classes, in the same two-

varkonyibarnabas@gmail.com

dimensional feature plane (a slow-band-power axis against a fast-band-power axis), handed to three different learners.

The k-nearest-neighbour classifier is the most direct idea imaginable: to label a new epoch, find its k closest neighbours in the training set and take a majority vote. It draws no boundary in advance — the boundary is implicit, a jagged mosaic that follows the local density of points. The support vector machine does the opposite: it finds one smooth surface that separates the classes with the *widest possible margin*, ignoring all but the handful of points nearest the frontier. The random forest grows hundreds of decision trees, each a cascade of axis-aligned "is feature X above threshold T?" questions, and averages their votes — producing a boxy, staircase boundary that can bend into almost any shape.

Look at what happens to the three training accuracies printed on the panels (from this synthetic example). k-NN and the SVM score about 0.90 and 0.89 on the training data; the random forest scores a perfect 1.00. It would be a beginner's mistake to conclude the forest is best. That perfect score is the forest memorising every training point — the overfitting Chapter 12 warned about. The honest question is not "who fits the training data?" but "who generalises?", and answering it requires the cross-validation of Chapter 12, not the picture. Hold that thought; Section 13.4 settles it.

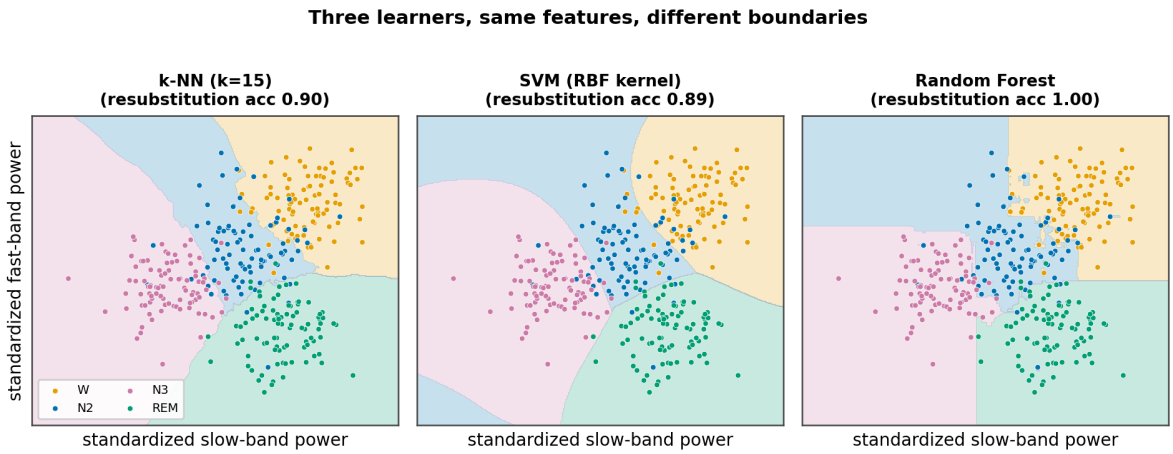


Figure 13.1. Three learners, one feature space, three boundaries (synthetic example). k-NN grows a jagged, density-following mosaic; the SVM lays down one smooth maximum-margin surface; the random forest builds a boxy staircase flexible enough to memorise the training set (note its perfect resubstitution accuracy of 1.00 on this demo). A perfect training score is a warning, not a trophy: only cross-validation (Chapter 12) tells us which boundary generalises.

13.3 Theory: four ways to draw a fence

13.3.1 k-Nearest Neighbours

k-NN stores the training set and classifies by proximity. Given a query point x, it finds the k training points nearest under a distance metric — usually Euclidean, $d(\mathrm{x}, \mathrm{x}') = \sqrt{\sum_i (x_i - x'_i)^2}$ — and returns their majority label. Two design choices dominate. First, the value of k trades bias against variance directly: $k = 1$ follows every point (low bias, high

varkonyibarnabas@gmail.com

variance, a wildly jagged boundary), while a large k smooths the boundary toward the global class proportions (high bias, low variance). Second — and this is the trap — k-NN is exquisitely sensitive to feature scaling. Because distance sums the squared differences across features, a feature measured in large units (say, absolute EEG power in μV^2) will dominate one measured in small units (a spectral-edge ratio between 0 and 1), and the "nearest" neighbour will be chosen almost entirely by the loud feature. Standardising every feature to zero mean and unit variance (Chapter 11), *fit on the training fold only* (Section 12.4), is essential for k-NN to behave sensibly. k-NN also tends to suffer the curse of dimensionality: in high-dimensional feature spaces points become roughly equidistant from one another, and "nearest" loses much of its meaning — a reason to reduce dimensionality first (Section 13.5.2).

13.3.2 Support Vector Machines

An SVM seeks the separating surface that maximises the margin — the perpendicular distance from the boundary to the closest training points of each class. Those closest points are the support vectors; they alone define the boundary, which is why SVMs are memory-efficient and robust to points far from the frontier. For a linearly separable two-class problem with labels $y_i \in \{-1, +1\}$ the decision function is $f(\mathrm{x}) = \mathrm{w}^\top \mathrm{x} + b$. Merely asking every point to sit on the correct side ($y_i(\mathrm{w}^\top \mathrm{x}_i + b) > 0$) is too weak — it is satisfied trivially, even by $\mathrm{w} = 0$, and says nothing about the margin. The fix is a scale convention: since w and b can be rescaled freely without moving the boundary, we adopt the canonical hyperplane, fixing the functional margin of the closest points to exactly 1. This turns the requirement into the margin constraint $y_i(\mathrm{w}^\top \mathrm{x}_i + b) \geq 1$ for every point i, and because the geometric margin then equals $2/\|\mathrm{w}\|$, maximising the margin is equivalent to minimising $\frac{1}{2}\|\mathrm{w}\|^2$ subject to that ≥ 1 (not just > 0) constraint — a well-posed problem with a unique solution. Real data are not cleanly separable, so a soft margin introduces non-negative slack variables ξ_i that let a point violate the ≥ 1 margin constraint (relaxing it to $y_i(\mathrm{w}^\top \mathrm{x}_i + b) \geq 1 - \xi_i$), penalising the total slack by a cost parameter C: small C tolerates violations for a wider, smoother margin (more bias, less variance); large C insists on separating the training data (less bias, more variance). The SVM's genuine magic is the kernel trick. A kernel $K(\mathrm{x}, \mathrm{x}')$ computes an inner product *as if* the data had been lifted into a much higher-dimensional space, letting a linear boundary there correspond to a curved boundary here — all without ever computing the lifted coordinates. The radial-basis-function (RBF) kernel, $K(\mathrm{x}, \mathrm{x}') = \exp(-\gamma\|\mathrm{x} - \mathrm{x}'\|^2)$, is the default: γ sets the reach of each support vector — small γ gives broad, smooth influence; large γ makes each point's influence local and the boundary wiggly. Like k-NN, the SVM *requires* feature scaling, for the same distance-based reason. For the multiclass sleep problem, scikit-learn fits several binary SVMs one-versus-one and votes.

13.3.3 Decision trees and random forests

A decision tree classifies by asking a sequence of threshold questions — "is delta power above 40?" then "is the spindle count below 2?" — each splitting the data into purer subsets until a leaf holds (nearly) one class. Each split is chosen to maximise purity, measured by the Gini impurity $G = 1 - \sum_c p_c^2$ or the entropy $H = -\sum_c p_c \log_2 p_c$, where p_c is the fraction of class c in the node. Trees are prized for interpretability — the path to a leaf reads as a rule a clinician can inspect — and they need no feature scaling, since each split concerns one feature at a time. Their curse is variance: a single deep tree overfits savagely, memorising noise (that

varkonyibarnabas@gmail.com

perfect training score again).

The random forest tames that variance by bagging — bootstrap aggregating. It trains many trees, each on a different bootstrap resample of the data, and *also* restricts each split to a random subset of features. This double randomness decorrelates the trees, so averaging their votes cancels the idiosyncratic errors while preserving the shared signal — the classic variance-reduction of an ensemble. Two gifts fall out for free. Out-of-bag (OOB) error: each tree is tested on the roughly one-third of samples its bootstrap left out, giving a validation estimate at no extra cost — but with a caveat that is decisive for biosignals. OOB leaves out *samples*, not *subjects*: when one recording contributes many epochs, a held-out epoch's near-twins from the same subject stay in the bag, so OOB measures only *sample-level* generalisation and can badly overstate the *subject-level* performance this book cares about (Chapter 12). Read OOB as a quick internal diagnostic, never as a substitute for a leave-one-subject-out estimate. Feature importance: how much each feature reduced impurity across the forest ranks the features, a fast, if imperfect, guide to which features carry the signal (compare with the selection methods of Chapter 11). Random forests are the perennial strong baseline for tabular biosignal features: they handle mixed feature scales, ignore irrelevant features gracefully, resist overfitting far better than a lone tree, and need little tuning. Class weights — up-weighting the rare stages — help against the N2-dominant imbalance of sleep.

13.3.4 Boosting, and the wider ensemble family

Bagging builds its trees *in parallel* and averages them to cut variance. Boosting builds them *in sequence* to cut bias: each new weak learner (a shallow tree) is trained to fix the errors the current ensemble still makes, and the models are summed with learned weights. Gradient boosting frames this as gradient descent in function space — each tree fits the negative gradient of the loss — and its modern implementations, XGBoost and LightGBM, are, on tabular biosignal features, frequently the *strongest* off-the-shelf classifier, often edging out the very random forest this book tells you to beat. The price is that boosting is more sensitive to noisy labels and needs more tuning (learning rate, tree depth, number of rounds, regularisation) than a forest, and it can overfit if run too long — so it is validated exactly like everything else in Chapter 12.

Bagging and boosting are two members of a small family of ensemble strategies worth naming together. Voting (or averaging) simply combines the predictions of several *different* model types — say a forest, an SVM, and a boosted tree — by majority vote or averaged probability, and often beats any single member because their errors are partly independent. Stacking goes further, training a small *meta-model* to learn the best way to combine the base models' outputs — under one non-negotiable rule: the meta-model must be fitted on base predictions generated *out of fold* (each base model scoring samples it never trained on), and for biosignals those folds must be subject-grouped. Fit it on in-sample base predictions and it merely learns to trust whichever base model overfit hardest — leakage dressed up as cleverness — and the final test predictions must never be used to fit it. The unifying idea across all four — bagging, boosting, voting, stacking — is that a chorus of diverse, individually-imperfect learners is more reliable than any soloist, *provided their mistakes are not identical.*

Two more single learners round out the standard toolkit. Naive Bayes applies Bayes' rule under the (usually false but often harmless) assumption that the features are conditionally independent given the class; it is fast, needs little data, and makes a reasonable probabilistic

varkonyibarnabas@gmail.com

screening baseline. Logistic regression draws a *linear* decision boundary but outputs probabilistic scores that are *often* reasonably well calibrated — because it optimises a proper scoring rule — and coefficients whose exponential (e^{β}, per unit change in the feature) a clinician can read as an odds ratio, which is why it remains a favourite for interpretable clinical risk scores. Calibration is not *guaranteed*, though (regularisation, misspecification, or dataset shift can spoil it), so the probabilities should still be checked with a reliability diagram and the Brier score, and recalibrated if needed.

13.4 Worked example: three learners on the same folds

The honest comparison is not the training-accuracy picture of Figure 13.1 but a cross-validated one. Trained on an eight-dimensional synthetic feature set (four informative axes plus four pure-noise axes, a realistic mix) and evaluated with five-fold cross-validation using Cohen's kappa (Chapter 12 — plain stratified k-fold is appropriate *here* because this synthetic set has no subject structure to leak; on real biosignals you would use subject-grouped GroupKFold/LOSO), the three learners show no clear difference on this example: k-NN at $\kappa = 0.66 \pm 0.05$, the RBF SVM at $\kappa = 0.66 \pm 0.03$, and the random forest at $\kappa = 0.66 \pm 0.06$. (Reading overlap off the mean $\pm$ SD is only suggestive; a firm claim of equivalence would need a paired test across the shared folds — which are not independent — or a Bayesian practical-equivalence analysis.) This is the single most important point in the chapter, and it is deliberately *undramatic*. The forest that scored a perfect 1.00 on the training set generalises no better here than the humble k-NN. The law: resubstitution accuracy is not generalisation, and a more flexible model's advantage on training data is largely the overfitting that cross-validation strips away. The reproducible number (illustrative, from simulated features): on this feature set, moving from k-NN to a random forest buys essentially *zero* kappa — the gains, when they come, tend to come from better features and honest validation, not from a fancier learner on the same inputs. The table below is the map for *choosing*, since raw accuracy will not choose for you.

Learner	Boundary shape	Scaling needed?	Interpretable?	Key knobs	Shines when
k-NN	jagged, density-following	yes (critical)	poor	k, distance metric	small, low-dim, well-scaled data
SVM (RBF)	one smooth curved margin	yes (critical)	poor	C, γ, kernel	medium data, clear margins, high-dim
Decision tree	axis-aligned boxes	no	excellent	depth, min-leaf, criterion	when a readable rule matters
Random forest	many boxes, averaged	no	moderate (importances)	n-trees, max-features, class-weight	tabular features, default strong baseline

Method: scale before k-NN and SVM. Both classifiers judge "closeness" by summed squared feature differences, so an unscaled feature in large units silently dominates. Standardise every feature to zero mean and unit variance, and — per Chapter 12 (Section 12.4) — fit the scaler on the *training* fold only, then apply it to the held-out fold. Trees and forests are essentially insensitive to per-feature scaling, because each split concerns a single feature and depends only on the *order* of its values, not their units; that insensitivity is one reason forests are such a low-friction default.

varkonyibarnabas@gmail.com

13.5 Unsupervised learning: structure without labels

Everything above needed labelled epochs. But labels are expensive — a night of sleep takes an expert scorer an hour — and sometimes we simply want to know whether the feature space *has* structure at all. Unsupervised methods answer that.

13.5.1 k-means and hierarchical clustering

k-means partitions the points into k clusters by an iterated two-step dance: assign each point to the nearest cluster centre, then move each centre to the mean of its assigned points; repeat until nothing moves. It minimises the within-cluster sum of squared distances, $\sum_j \sum_{\mathrm{x} \in C_j} \|\mathrm{x} - \mu_j\|^2$. It is fast and scalable but assumes roughly spherical, equally sized clusters, needs k chosen in advance (the *elbow* of the within-cluster-distance curve, or the silhouette score, helps), and — being distance-based — needs scaled features. Crucially, its cluster labels are *arbitrary*: cluster 2 is not "N2", it is just a group; matching clusters to meaning is a separate, human step.

Hierarchical (agglomerative) clustering builds a tree instead of a flat partition. It starts with every point as its own cluster and repeatedly merges the two closest, recording the merge distances as a dendrogram — a tree whose branch heights show how far apart the merged groups were. Cutting the dendrogram at a chosen height yields any number of clusters, and its shape reveals *nested* structure a flat k-means cannot. Figure 13.2 runs both on the sleep-like features: k-means centres (panel a) land squarely inside the true stage clouds where the classes are well separated (Wake, N3) but blur where the physiology genuinely overlaps (N2 versus REM) — a faithful mirror of how sleep stages themselves shade into one another. The dendrogram (panel b) recovers the coarse grouping, merging same-stage epochs at short distances and joining the deep-sleep and wake extremes only near the root.

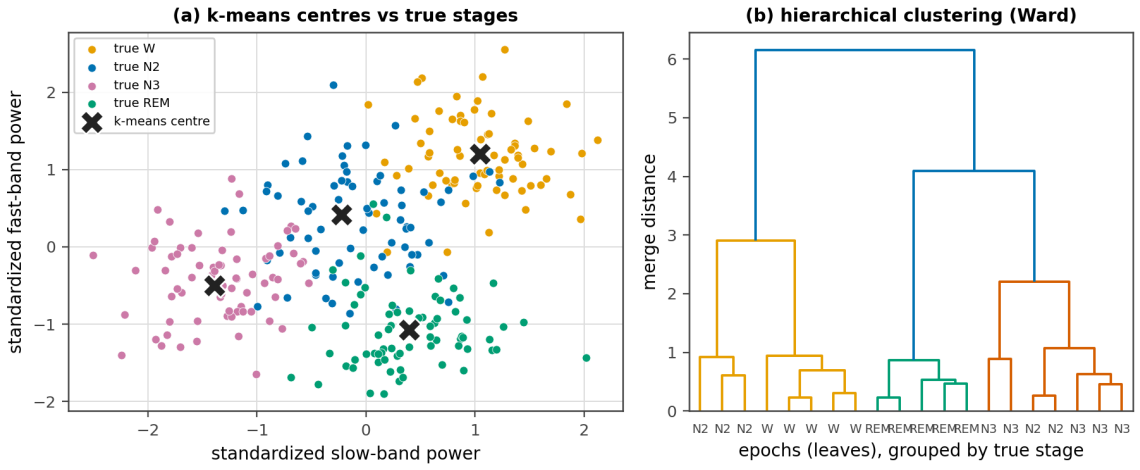


Figure 13.2. Finding structure with no labels. (a) k-means (black crosses = cluster centres) recovers the well-separated stages — Wake and N3 fall cleanly — but its centres blur exactly where the physiology overlaps, between N2 and REM. (b) A Ward-linkage dendrogram merges same-stage epochs first and joins the extremes only near the root. Unsupervised structure tracks the real physiology: clusters are clean where stages are distinct and muddy where they truly overlap.

varkonyibarnabas@gmail.com

13.5.2 Multivariate analysis: PCA, ICA, and non-linear embeddings

Feature spaces are usually high-dimensional and redundant — many features are correlated (mean and median frequency move together; several band-power ratios say almost the same thing). Principal-component analysis (PCA) finds a new set of orthogonal axes — the principal components — ordered so the first captures the most variance in the data, the second the most of what remains, and so on. Mathematically the components are the eigenvectors of the feature covariance matrix, and each eigenvalue is the variance along its axis. Projecting onto the first two or three components gives a picture of a many-dimensional cloud, and keeping only the top components that together explain, say, 90% of the variance compresses the feature set with little loss. Figure 13.3 shows both uses: a scatter of the first two components coloured by true stage (panel a), where the stages already separate visibly along axes *no one designed*, and the scree curve (panel b) of variance explained per component. On this eight-dimensional synthetic set the first two components carry about 39% of the variance, and it takes seven components to pass the 90% line (six reach only ~86%) (numbers illustrative, from this demo) — a direct readout of how much redundancy the features hold on this example.

Two cautions and two cousins. PCA maximises *variance*, not *class separation*: its top component points along the data's biggest spread, which need not be the direction that best splits the classes — so PCA is a superb visualiser and compressor but not, by itself, a classifier. And PCA is not feature *selection*: selection keeps a subset of the *original*, interpretable features (Chapter 11), whereas PCA builds *new* features that are blends of all originals, trading interpretability for compactness. Its cousin independent-component analysis (ICA) seeks statistically independent rather than merely uncorrelated components, which is why it excels at *unmixing* sources — separating an EOG artifact from the EEG it contaminates (Chapter 10) rather than at ranking variance. Finally, when structure is non-linear, embeddings such as t-SNE and UMAP map high-dimensional points to two dimensions for visualisation by preserving local neighbourhoods; they make striking cluster pictures but distort global distances, so their layouts should not be read as a faithful map of true separation — a visualisation aid, not a metric.

13.6 The ML/AI bridge

This chapter is the join between the features of Chapter 11 and the deep networks of Chapter 14. The classical learners here consume the hand-crafted feature matrix directly, and — evaluated honestly under Chapter 12's rules — they set the *baseline a heavier model should be asked to beat*. That baseline matters more than it looks. A random forest on good features, cross-validated by subject, is often within a few points of much more elaborate models, trains quickly, can run on modest hardware (Chapter 15), and can explain itself through feature importances. The unsupervised methods feed the bridge from the other side: PCA and clustering are how you *look* at a feature space before modelling — spotting redundant features, mislabelled epochs, or subgroups of subjects — and PCA-style compression is a preprocessing step that can stabilise a downstream classifier. Two cautions come with it: PCA is scale-sensitive, so standardise the features first (an unscaled feature in μV^2 would dominate every component, the same trap that hurts k-NN, SVM, and k-means); and because the PCA basis is *learned from data*, it must be fit on the training fold only and then applied to the held-out data — fitting it on the whole set before the split is leakage (Ch. 12, Failure 1 lists "PCA bases" among the transforms that leak). When Chapter 14 replaces hand-crafted features with *learned* ones, the yardstick it aims to

varkonyibarnabas@gmail.com

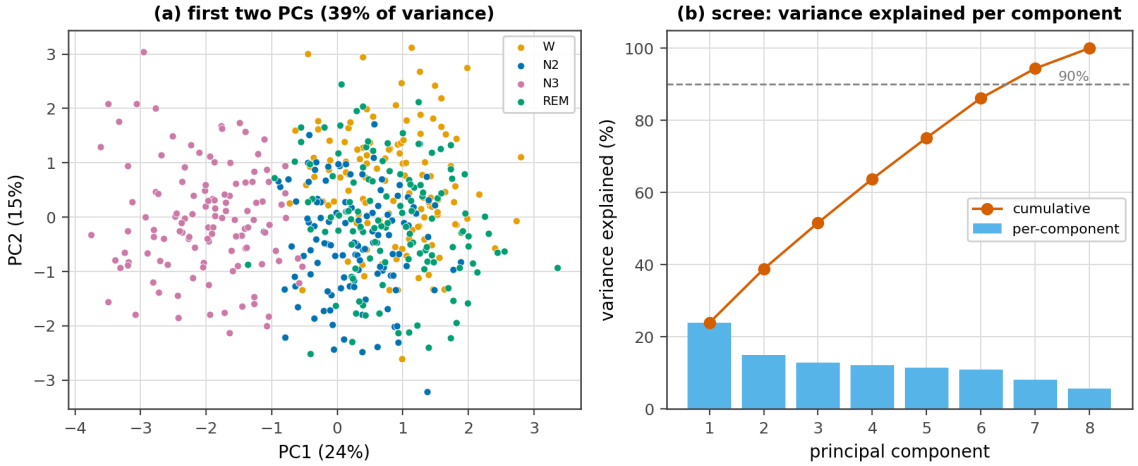


Figure 13.3. PCA as compressor and lens (synthetic example; numbers illustrative). (a) Projecting the eight-dimensional feature set onto its first two principal components — axes discovered from variance alone, not designed — already pulls the stages apart. (b) The scree curve: the first two components carry ~39% of the variance and it takes seven components to reach 90%, quantifying how much redundancy this feature set holds. PCA reveals and compresses structure but optimises variance, not class separation — it is a lens, not a classifier.

clear is the honest kappa of the random forest built here.

13.7 Signals-and-images box

> The 2-D analogue. Nearly every method in this chapter carries over to image features with little change. A random forest on texture and edge descriptors (Chapter 11) is a strong classifier for many medical-image tasks and often a hard-to-beat baseline before a CNN. k-means is a classic tool for image segmentation by intensity — cluster the pixels of a scan into tissue classes with no labels — and PCA on image patches yields the "eigen-patch" bases that compress and denoise images, the direct visual sibling of the eigen-features here. The lesson transfers broadly: cluster to find structure, project to see it, and let a forest set the bar a deep model should aim to clear.

13.8 Companion notebook

The companion notebook trains all three classifiers on one labelled feature matrix under identical folds (stratified k-fold, per §13.4's note on this synthetic set), prints the per-learner kappa with its spread, and reproduces the decision-boundary panels of Figure 13.1 so the reader can *see* how k, C, and γ reshape each fence in real time. A short extension adds a gradient-boosting classifier and a voting ensemble of the three learners to the same folds, so the "chorus beats the soloist" claim of §13.3.4 is a kappa you measure rather than take on faith. It then switches off the labels: it runs k-means (with an elbow/silhouette diagnostic for choosing k), overlays the clusters on the true stages, draws the dendrogram, and finishes with a PCA scatter and scree curve plus a side-by-side of PCA compression versus feature selection — so the difference between *new* features and a *subset* of features is unmistakable.

varkonyibarnabas@gmail.com

13.9 Case-study tie-in

In the sleep-staging case study this chapter supplies the classifiers for every iteration. A natural progression starts with k-NN as the simplest capable learner, moves to a scaled SVM for a smoother boundary, then to a random forest with class weights against the N2 imbalance — each swapped into the identical subject-independent (LOSO/GroupKFold) harness of Chapter 12 so the comparison is fair. Read that ladder (Section 16.3) alongside this section's controlled result, not as a repeat of it: there, the model and the feature set grow together at every rung, so the rising kappa is the combined payoff of both levers, not an isolated test of the model. Section 13.4's own comparison is where the model-alone question is actually settled — same features throughout, only the learner swapped — and there the biggest jumps did *not* come from the model. The unsupervised methods appear as an *exploratory* pass — a PCA scatter to sanity-check that the feature space separates the stages at all, and a k-means overlay to spot subjects whose physiology clusters oddly — before a single label is trusted.

13.10 Summary, questions, and further reading

Key formulas. Euclidean distance $d = \sqrt{\sum_i (x_i - x'_i)^2}$; SVM margin objective, minimise $\frac{1}{2}\|\mathrm{w}\|^2$ subject to $y_i(\mathrm{w}^\top \mathrm{x}_i + b) \geq 1$ (geometric margin $2/\|\mathrm{w}\|$); RBF kernel $K = \exp(-\gamma\|\mathrm{x} - \mathrm{x}'\|^2)$; Gini impurity $G = 1 - \sum_c p_c^2$; entropy $H = -\sum_c p_c \log_2 p_c$; k-means objective, minimise $\sum_j \sum_{\mathrm{x} \in C_j} \|\mathrm{x} - \mu_j\|^2$.

Key ideas. Every classifier carves the feature space; they differ in boundary shape, scaling sensitivity, and interpretability. Scale before k-NN and SVM; forests need no scaling. A perfect training score is overfitting, not skill — judge by cross-validated kappa, where on this synthetic example the three classical learners tie at ~0.66 (illustrative). Random forests are the strong tabular default via bagging, feature randomness, OOB error, and importances. Unsupervised methods find structure with no labels: k-means partitions, hierarchical clustering builds a dendrogram, and PCA reveals and compresses variance. PCA is not feature selection — it makes new blended axes optimised for variance, not class separation.

Conceptual questions. (1) Why does forgetting to scale features cripple k-NN but not a decision tree? (2) A random forest scores 1.00 on training data — is it the best model? What number would settle it? (3) What do C and γ each control in an RBF SVM, and which way does raising each move the bias–variance balance? (4) Your k-means clusters do not align with the true sleep stages — give two innocent reasons this can happen even with a good feature set. (5) When would you prefer PCA over selecting a subset of the original features, and what do you lose? (6) Transfer. You chose a random forest on hand-crafted features for sleep staging. For PPG-based atrial-fibrillation screening on a wearable — few labels, strong class imbalance, and a need for a *calibrated probability* rather than a hard label — which learner of this chapter would you reach for first, and what one change to the evaluation (versus sleep staging) does the screening use-case force?

Exploration tasks. Train k-NN, SVM, and a random forest on one feature matrix under LOSO and tabulate their kappa and its spread — then improve the features and watch which change moves the number more. Sweep k in k-NN and C, γ in the SVM and plot validation kappa to locate each sweet spot. Run k-means for a range of cluster counts and pick k by the elbow and

varkonyibarnabas@gmail.com

the silhouette; overlay the result on the true stages. Compute a PCA scree curve, decide how many components reach 90% variance, and compare a classifier trained on the compressed features against one trained on a selected feature subset.

Design brief. Choose the classifier for a PPG-based atrial-fibrillation screener on a wearable: few labelled subjects, strong class imbalance (AF is rare), a need for a calibrated probability (not a hard label), and on-device interpretability preferred. Spec: list the constraints — data size, imbalance, calibration, interpretability, compute. Design: choose the learner (logistic regression / SVM / tree / random forest / boosting / k-NN) and how you handle imbalance and calibration. Evaluate: which metric and which validation (Chapter 12) report honestly here, and how do you check calibration? Motivate: justify the learner against the small-data-plus-calibration need. Alternatives: name the learner you would switch to given 100$\times$ more labelled subjects. Rule out: name a learner or metric that would be *wrong* — e.g. reporting plain accuracy on a 1%-AF cohort, or training a from-scratch deep network on 20 subjects — and why.

Further reading. Hastie, Tibshirani & Friedman, *The Elements of Statistical Learning* (all four learners, PCA, clustering, with the mathematics). Breiman's original random-forest paper (bagging, OOB, importance). Cortes & Vapnik on support vector machines and the kernel trick. Bishop, *Pattern Recognition and Machine Learning* (a probabilistic view of the same methods). The scikit-learn user guide for the classifiers, clustering, and decomposition tools used here.

varkonyibarnabas@gmail.com

Chapter 14 — Deep Learning & Time-Series Foundation Models for Biosignals

What if the model learned its own features — and what if it had already "read" millions of hours of physiological signals before it ever saw ours?

For thirteen chapters we have built features by hand. We filtered, transformed, and measured; we chose band powers and Hjorth parameters and morphology descriptors; and we handed that curated feature matrix to a classifier. Deep learning proposes something radical: skip the hand-crafting. Feed the raw signal to a network deep enough to *discover its own features*, tuned by nothing but the pressure to predict the label well. This chapter is a careful, honest tour of that idea for biosignals — what a neural network is, why a convolutional network is really just Chapter 9's convolution with the kernel set free, how sequence models carry context across a night, and what a *foundation model* is and why it might let a small clinical dataset punch far above its weight. It is a forward-looking chapter, and a cautious one: deep learning is powerful and data-hungry in equal measure, and one of its most valuable lessons is knowing *when the classical forest of Chapter 13 still wins*.

14.1 Learning objectives

By the end of this chapter you should be able to:

- Contrast hand-crafted and learned representations, and state what each gains and loses (L2, L4).
- Describe a neural network — neurons, layers, loss, backpropagation — at a working level (L2).
- Explain why the convolution in a CNN is the convolution of Chapter 9 with a *learned* kernel, and apply CNNs to 1-D signals and 2-D spectrograms/images (L4).
- Describe how sequence models (RNN/LSTM and the transformer's attention) carry temporal context across epochs (L2, L4).
- Explain self-supervised pretraining and what a foundation model is, and how transfer learning / fine-tuning helps a small labelled dataset (L4).
- Judge, using evaluation parity with Chapter 12, when deep learning beats classical methods and when it does not (L4, L5).
- State the core responsible-AI concerns for clinical deep learning (L5).

14.2 Intuition first: from designed features to learned ones

A hand-crafted feature is a human decision written in code — "measure delta-band power", "count the spindles". It is interpretable, cheap, and grounded in physiology, but it is also a *bottleneck*: the model can only ever be as good as the features a person thought to compute. Deep learning removes the bottleneck by making the features themselves *trainable*. A deep

varkonyibarnabas@gmail.com

network is a stack of simple transformations whose parameters are adjusted, layer by layer, until the whole stack maps raw input to correct label; the intermediate layers, left to their own devices, learn representations — edges, then rhythms, then stage-specific motifs — that a human might never have hand-coded. The gain is capacity: given enough data, the network can capture patterns too subtle or too numerous to design by hand. The loss is interpretability and data-thrift: the learned features are opaque, and learning them from scratch demands far more labelled data than fitting a forest to good features. The whole chapter lives in that trade.

Figure 14.1 draws that trade as a ladder, because it is not really a binary choice between hand-crafted features and end-to-end learning but a continuum. At the knowledge-rich, data-thrifty bottom sits classical machine learning on physiological features; at the data-hungry top, deep networks on raw waveforms; and in between the pragmatic hybrids that most biomedical work actually uses — deep learning on lightly pre-processed inputs, or on the time–frequency *images* of §14.9. The rung you can afford is set by how many labelled examples you have: the information needed to solve the task has to come from *somewhere*, and whatever you do not supply as prior knowledge you must pay for in data (and in model capacity). Because labelled clinical data is scarce and expensive, useful systems have long sat toward the knowledge-rich end — which is exactly why the self-supervised foundation models of §14.6 matter, since they let you buy representational power with *unlabelled* data and spend your few precious labels only on fine-tuning. The extreme knowledge end of the same axis is the mechanistic physiological model of Chapter 6, which learns almost nothing from data at all.

It is worth being precise about what climbing the ladder actually does, because it is easy to mistake "learned features" for "no assumptions." Moving toward raw-input learning does not *remove* domain knowledge; it *relocates* it — out of the hand-crafted feature and into the dataset you assemble, the pretraining objective you choose, the augmentations you apply, the architecture you pick, the labels you trust, and the evaluation you design. A network that discovers features automatically still depends, entirely, on human choices about what data and what errors matter. The trade is real, but it is a trade in *where* the knowledge lives, not a way to do without it.

> The result supports / does not establish. When a deep or foundation model reports strong numbers, keep the claim inside its evidence. Supports: predictive utility on the tested data under the stated evaluation design. Does not establish: a physiological mechanism, that the representation will transfer to a new sensor or population, fairness across subgroups, or clinical benefit. Each of those is a separate claim needing its own evidence — a fair split (Chapter 12), a subgroup analysis, an external cohort, or a prospective study.

14.3 Neural-network basics

A neural network's atom is the neuron: it takes a weighted sum of its inputs, adds a bias, and passes the result through a non-linear activation function (the rectified linear unit, $\mathrm{ReLU}(z) = \max(0, z)$, is the default for convolutional and feed-forward hidden units, while transformers typically use its smooth cousins GELU or Swish/SiLU; the point — a simple nonlinearity that makes deep networks trainable — is the same whichever you pick). A single neuron computes $a = \sigma(\mathbf{w}^\top \mathbf{x} + b)$. Stack neurons side by side into a layer, stack layers front to back, and the

varkonyibarnabas@gmail.com

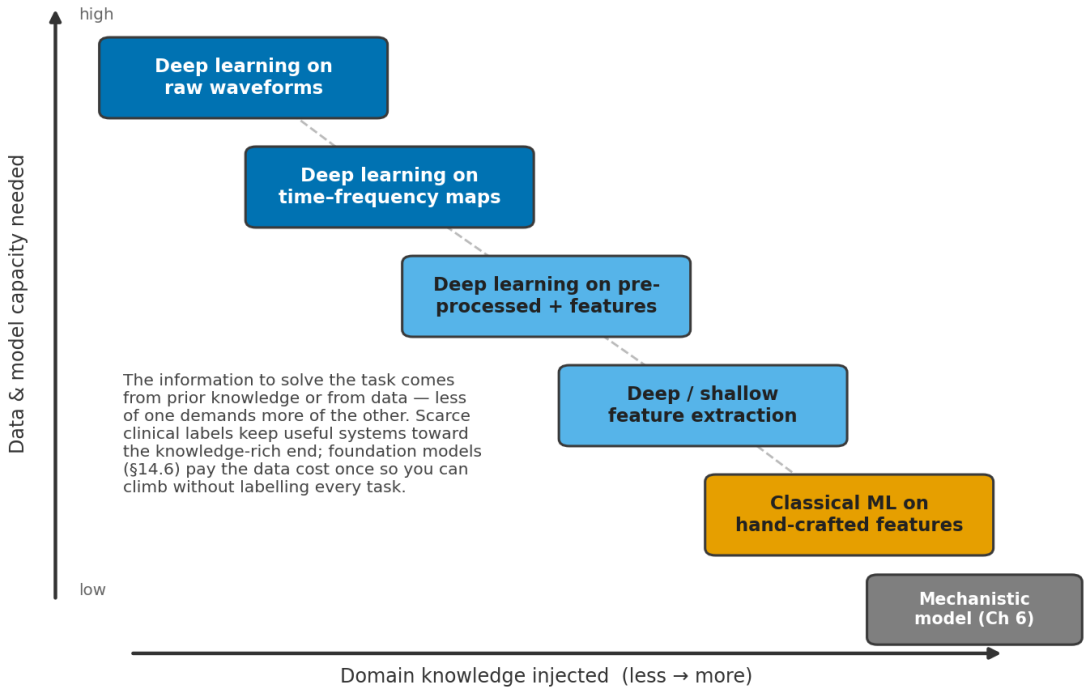


Figure 14.1. The knowledge–data trade-off, drawn as a ladder. Climbing the staircase trades domain knowledge for data: each higher rung injects less hand-built structure — features, preprocessing, a chosen representation — and so demands more labelled data and more model capacity to learn the rest for itself. Classical machine learning on hand-crafted features sits at the knowledge-rich, data-thrifty corner; end-to-end deep learning on raw waveforms at the data-hungry corner; and a mechanistic physiological model (Chapter 6) is the extreme knowledge end. Self-supervised foundation models (§14.6) let a system climb the ladder without paying the labelling cost for every new task.

varkonyibarnabas@gmail.com

network becomes a flexible function approximator: with enough neurons it can represent essentially any mapping from input to output. Training means finding the weights. We define a loss — for classification, the cross-entropy between predicted class probabilities and the true label — and we descend it by gradient descent: nudge every weight a little in the direction that most reduces the loss. The gradients are computed efficiently by backpropagation, the chain rule applied backward through the layers, and applied in small mini-batches by an optimiser such as Adam. The practicalities that make or break training are unglamorous but decisive: standardise the inputs, initialise the weights sensibly, choose a learning rate that is neither too timid nor divergent, and — above all on small clinical datasets — regularise hard (dropout, weight decay, early stopping, data augmentation) to fight the overfitting that a high-capacity model courts by default.

14.4 CNNs: the convolution you already know, learned

Here is the chapter's keystone, and the single clearest bridge from classical signal processing to deep learning. In Chapter 9 we designed a digital filter by choosing its impulse response $h[k]$ — the filter *taps* — and we convolved it with the signal:

$$y[n] = \sum_{k} h[k]\, x[n-k].$$

A convolutional layer in a CNN performs *exactly this operation*. It slides a short kernel across the signal and computes the same weighted, shifted sum. The only difference — at the level of this sliding weighted sum — is that the kernel's taps $h[k]$ are not designed by a human; they are learned by backpropagation. Where a Chapter 9 band-pass filter has taps we computed to pass 11–16 Hz spindles, a CNN's first layer holds a *bank* of such filters whose taps are adjusted during training until they respond to whatever patterns help predict the sleep stage — and the network typically *discovers* band-pass-like, edge-detecting, and rhythm-selective filters on its own, because those are what the data reward. The equivalence is exact at the level of the sliding weighted sum, but a full convolutional *layer* wraps a little more around it: it adds a bias and applies a pointwise nonlinearity (the ReLU of §14.3) to each output, it usually pools or strides to downsample, and each filter mixes across all input channels rather than one signal at a time — so a layer maps a stack of input maps to a stack of output maps. And as Appendix C notes, deep-learning frameworks actually implement *cross-correlation* — the slide *without* the flip — and simply call it convolution; because the taps are learned, the missing flip changes nothing. None of this weakens the bridge; it just says the learned filter bank is the *core* of the layer, not the whole of it. Figure 14.2 draws this equivalence directly: a learned kernel sliding over an EEG segment to produce a feature map (panel a), which is the very picture of convolution from Chapter 9, and the same operation stacked into a compact CNN — convolution, pooling, more convolution, a dense layer, a stage decision (panel b).

Two more ideas complete the CNN. Pooling downsamples each feature map — taking the maximum or average over small windows — which shrinks the representation, grants a little translation invariance (a spindle counts wherever in the epoch it falls), and lets deeper layers see a wider context. Depth stacks the pattern: early layers learn simple, local motifs; deeper layers combine them into stage-specific structure. The idea generalises across dimensions without changing its nature. A 1-D CNN runs the kernel along a time axis — natural for raw EEG or

varkonyibarnabas@gmail.com

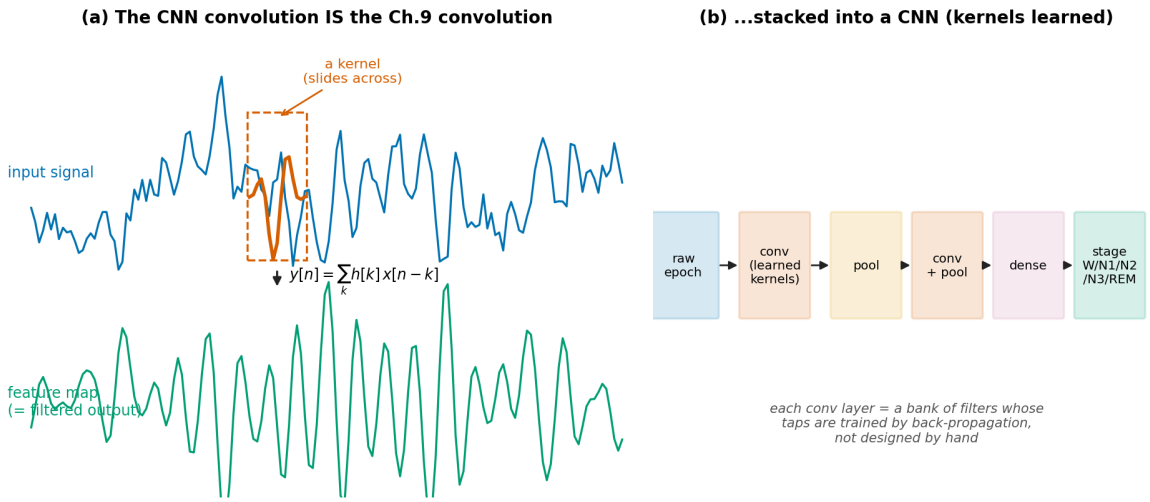


Figure 14.2. The single clearest bridge from classical DSP to deep learning. (a) A CNN's convolutional layer slides a short kernel across the signal and computes the identical weighted-shifted sum $y[n] = \sum_k h[k]\,x[n-k]$ we met in Chapter 9 — the feature map is just a filtered signal. (b) Stacked into a CNN, each convolutional layer is a bank of such filters. The one difference from Chapter 9: the kernel taps are learned by backpropagation, not designed by hand.

ECG epochs. A 2-D CNN runs the kernel across a plane — natural for spectrograms (Chapter 5), where a sleep epoch becomes a time–frequency *image* and a CNN can learn to spot the ridge of a spindle, and equally natural for medical images (the image thread's payoff), where the same 2-D convolution learns edge and texture detectors that classical feature engineering approximated by hand.

14.5 Sequence models: context across the night

A sleep epoch is not an island. A stage is far more likely if the surrounding epochs share it — sleep moves through stages in slow, structured cycles, and a human scorer reads each epoch *in context*. A per-epoch classifier throws that context away. Sequence models keep it. A recurrent neural network (RNN) processes epochs in order, maintaining a hidden state that carries information forward; its refined descendants, the LSTM and GRU, add gates that let the network remember relevant context over long spans and forget the rest, curing the vanishing-gradient problem that crippled plain RNNs. The modern alternative is the transformer, built on attention: instead of passing information hand-to-hand along the sequence, attention lets every epoch look directly at every other and weight their relevance, learning which distant epochs inform the current one. Attention is what powers today's largest models, and it is the mechanism behind the foundation models of §14.6. In practice a strong sleep stager is often a hybrid: a CNN turns each epoch into a compact vector, and a sequence model reasons over the night's worth of vectors — features learned, then context applied.

14.6 Self-supervised learning and foundation models

The deepest problem in clinical deep learning is *labels*. Expert-scored sleep is scarce and expensive, and a network learning features from scratch is ravenous for it. Self-supervised

varkonyibarnabas@gmail.com

learning sidesteps the shortage by inventing a task that needs *no* human labels. Two recipes dominate. Masked reconstruction hides part of a signal and trains the network to fill it in, forcing it to learn the signal's structure. Contrastive learning teaches the network that two augmented views of the same segment should map to nearby representations while different segments map apart. Either way, a network can pretrain on *oceans* of unlabelled physiological data and emerge with a rich, general representation — before it has seen a single label.

A foundation model is exactly this: a large network pretrained by self-supervision on a broad corpus, producing general-purpose representations that transfer to many downstream tasks with little task-specific data. The workflow is transfer learning — take the pretrained encoder and fine-tune it (or just train a small head on top of its frozen features) on your small labelled set. Because the model has already learned a great deal about the structure of physiological signals during pretraining, it often needs far fewer labels to reach good performance on *your* problem. For low-data clinical tasks this is one of the more consequential recent shifts. The durable principle — the part worth learning because it will outlast any particular model — is the three-step recipe itself: pretrain a large encoder by self-supervision on abundant unlabelled data, transfer the learned representation to a new task, and fine-tune with the few labels you have. Two broad families of biosignal foundation model currently exist. General time-series foundation models cut any series into fixed-length patches (tokens) and pretrain by masked prediction, treating a biosignal as one more time series. Biosignal- and EEG-specific foundation models bake in physiological structure such as channels and montages, and pretrain by masked reconstruction of signal patches. The specific systems in each family are named in the dated box below rather than in the running text, because the field is young and moving fast and the honest posture is awareness of the *approach* rather than allegiance to any one model.

> State of the field (as of 2026). *This snapshot will age; treat it as a pointer, not a verdict, and check current work before relying on any named system.* At the time of writing, examples of general time-series foundation models include patch-based transformers such as the MOMENT family. Examples of biosignal- and EEG-specific foundation models include BIOT (a generic transformer across EEG, ECG, and wearable signals), CBraMod (an EEG model pretrained by masked reconstruction of signal patches), and LaBraM (an EEG model pretrained by masked prediction of discrete neural-tokenizer codes). These are illustrations of the two families, not endorsements or a ranking: which model leads on a given task changes quickly, benchmarks are not always comparable, and a model that tops a public leaderboard may not be the right choice for your data. What is durable is the *idea* — self-supervised pretraining, then transfer and fine-tuning — not the current roster of winners.

varkonyibarnabas@gmail.com

When a foundation model may transfer poorly. Pretraining does not confer immunity to distribution shift, and a model that excels on its pretraining corpus can degrade sharply on data that differs from it. Watch especially for: montage changes (a different electrode layout or reference than the model saw in pretraining); sensor changes (a different amplifier, wearable, or electrode type); sampling-rate changes (data recorded at a rate the model was not pretrained on); missing channels (fewer or different channels than the pretrained input assumes); disease-population shift (a clinical population — by age, comorbidity, or pathology — under-represented in pretraining); and label-definition shift (your labels follow a different scoring rule or ontology than the downstream examples the model was tuned on). Each of these can quietly erode transfer, so a fine-tuned foundation model should be validated on *your* population and hardware under the same subject-independent rules (§14.7, Chapter 12) as any other model — a strong pretraining score is not evidence that it will work on your data.

14.6.1 Pretraining data governance

The power of a foundation model comes from the ocean of data it pretrains on, and that ocean raises governance questions a small supervised project can often sidestep. Consent is the first: physiological recordings gathered for one purpose may not carry consent for reuse as pretraining fuel, and "it was public" is not the same as "it was consented for this." Population bias is the second: a pretraining corpus assembled from whatever data were easiest to collect tends to over-represent some populations and under-represent others, and the model inherits those imbalances — so a representation that looks general may be general only for the groups the corpus happened to cover. De-identification has limits: stripping names and dates reduces but does not eliminate re-identification risk, because physiological signals themselves can be distinctive, and aggregating many records raises the stakes of any leak. Finally, benchmark-overlap leakage is a subtle but serious hazard: if a public test set has found its way into the pretraining corpus — as public data readily can — then a headline "transfer" result may be partly memorisation, and the reported number overstates how well the model will generalise to genuinely unseen data. Guarding against it means knowing what went into pretraining and checking that your evaluation data were held out of it, which is not always possible with a model someone else pretrained. The honest posture is to treat a pretrained encoder as a component whose provenance matters, and to report both what it was pretrained on (where known) and how you ensured your test data were not part of it.

14.7 Practical reality: a fair fight with the forest

None of this repeals Chapter 12. Deep learning should be validated by the *same* subject-independent rules — leave-one-subject-out, everything learned fit inside the training folds, judged by kappa and the confusion matrix — because a high-capacity model tends to overfit and leak *more* eagerly, not less. And when the comparison is fair, the result is instructive. Figure 14.3 stages the head-to-head on a controlled synthetic example in which both models are really trained and scored on held-out subjects — no numbers are asserted. The lesson lives in the *shape* of panel (b): the compact feature model is data-thrifty but plateaus early and low, because its hand-picked summary caps what it can express; the high-capacity model on the raw epoch is data-hungry — mediocre when only a couple of subjects are labelled, then climbing steeply as labels accumulate. That contrast — a flat feature ceiling versus a rising learned

varkonyibarnabas@gmail.com

curve — is the practical lesson, and it is why the deep model "earns its keep" only once data is plentiful.

Two honesty caveats matter here. First, in *this* clean demo the raw model leads throughout rather than starting behind, because its band-power features are a strictly *lossy* view of a tidy synthetic signal, so the network — seeing strictly more — is never at a disadvantage. The textbook crossover, where the classical baseline actually *wins* at small data and is overtaken only later, is a real and well-documented pattern on clinical recordings; it depends on two things a clean toy cannot fake: expert features that encode strong domain priors a from-scratch net struggles to relearn from few examples, and raw signals messy and high-dimensional enough to make that net genuinely overfit. Second, the absolute kappas are properties of the simulation, not a benchmark on real data. Read the figure for its *trend*, not its coordinates.

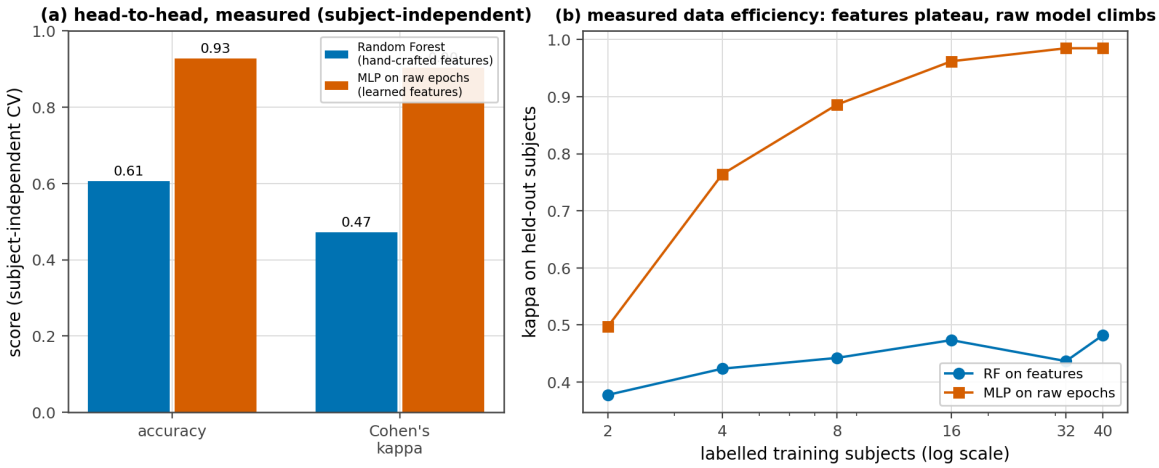
Aspect	Classical (RF on features)	Deep learning (CNN / sequence)
Features	hand-crafted, physiological	learned from data
Labelled-data hunger	modest	high (unless pretrained)
Small-clinical-data performance	strong, stable	often weaker; overfits
Interpretability	good (importances, rules)	poor (opaque)
Compute to train	seconds, CPU	minutes–hours, often GPU
Edge deployability (Ch.15)	excellent	needs compression
Best when	data scarce, interpretability matters	data abundant *or* a foundation model transfers

The forest is not obsolete; it is the reigning baseline you have to *beat*, and on the small datasets typical of a clinical study it often is not beaten by a from-scratch network at all. Deep learning earns its keep in two regimes: when labelled data is genuinely abundant, or when a foundation model lets you inherit representations learned elsewhere and fine-tune with the few labels you have — which is precisely why §14.6 matters so much for low-data health tasks.

14.8 Responsible AI in health

A deep model that scores sleep is a medical claim in waiting, and capacity brings hazards a forest largely lacks. Generalisation across populations is the first: a network trained on one demographic can quietly fail on another — different ages, skin tones (for optical sensors), electrode montages, or recording hardware — so subject-independent validation should extend to *population*-independent validation wherever the data allow. Bias can be learned from imbalanced or unrepresentative training data and then amplified at scale. Interpretability matters clinically: a forest can point to the features that drove a call, whereas a deep network's reasoning is largely opaque, which is why explainability methods (saliency, attention maps) are an active and necessary area rather than a nicety. In the language of Chapter 6, a deep network is the far black-box end of the modelling spectrum: its many weights fit the data superbly but admit no backward map to physiology, so — unlike a mechanistic model whose every parameter *means* something — one generally cannot say *why* it decided as it did, only that it did. And all of this sits inside a regulatory context — a model with a medical purpose is, in principle, a medical device, a theme Chapter 15 develops at awareness level. The responsible posture is humility: report honest subject- and population-independent numbers, disclose the training population, prefer the interpretable model when it is competitive, and avoid letting a benchmark score stand in for clinical evidence.

varkonyibarnabas@gmail.com



Synthetic data, REAL computation (every point is a model trained and scored on held-out subjects). The compact feature model is data-thrifty but PLATEAUS — its summary caps what it can learn; the high-dimensional raw model is DATA-HUNGRY — poor with 2 subjects, climbing steeply as labels accumulate. That data-hunger is the real lesson. (The classical-wins-at-small-data crossover needs expert features that encode strong priors and genuinely messy high-dim signals — a real-clinical pattern this clean toy does not reproduce; here the raw model, seeing strictly more, leads throughout.)

Figure 14.3. When deep learning is worth it — a fair comparison, every point really trained and scored on held-out subjects. (a) At a modest data budget, a random forest on compact hand-crafted features and a high-capacity model on the raw epoch are compared under the same subject-independent rules as Chapter 12; here the raw model leads because the band-power features are a strictly lossy view of this clean synthetic signal. (b) The measured data-efficiency curves carry the lesson: the feature model plateaus early and low (its summary caps what it can express), while the raw model is data-hungry — weak with two subjects, then climbing steeply as labels accumulate. On real clinical data, where expert features encode strong priors and raw signals are genuinely messy, the classical baseline typically *wins* at small data and is overtaken only past a crossover — a pattern this clean toy approximates in trend but does not reproduce in full. Read the curves for their shape, not their coordinates.

varkonyibarnabas@gmail.com

Three further duties deserve to be named rather than left implicit. Privacy is the first: physiological recordings, when linked or linkable to a person, are sensitive health data — they may constitute protected health information under HIPAA in the covered US healthcare contexts (which binds covered entities and their business associates), and personal, special-category data under the EU GDPR — subject to regimes that constrain how they may be stored, shared, and used for training, and that motivate de-identification and strict access control. One technical response is worth knowing: federated learning trains a shared model *without centralising the data* — each hospital computes model updates on its own recordings and sends only those updates (not the raw signals) to be averaged into a global model, so the data never leaves the institution. It is a natural fit for the multi-site theme of this book and for privacy-sensitive biosignals, though it brings its own difficulties (non-identically-distributed sites, communication cost, and the fact that model updates can themselves leak information unless protected). Fairness is the second duty: bias is not only a data problem to avoid but a property to *measure*, by reporting performance separately across the demographic and device subgroups that matter, so a model that works on average but fails a subpopulation is caught before deployment rather than after. And human oversight is the third: for a consequential clinical decision the responsible design keeps the model as *decision support* — a second reader whose suggestion a clinician can inspect and override — not an autonomous decider, and it treats informed consent and transparency about when and how an AI system is used as part of the clinical contract, not an afterthought. (A caution on the interpretability tools above: saliency and attention maps are themselves imperfect and can be misleading, so they inform human oversight rather than replace it.)

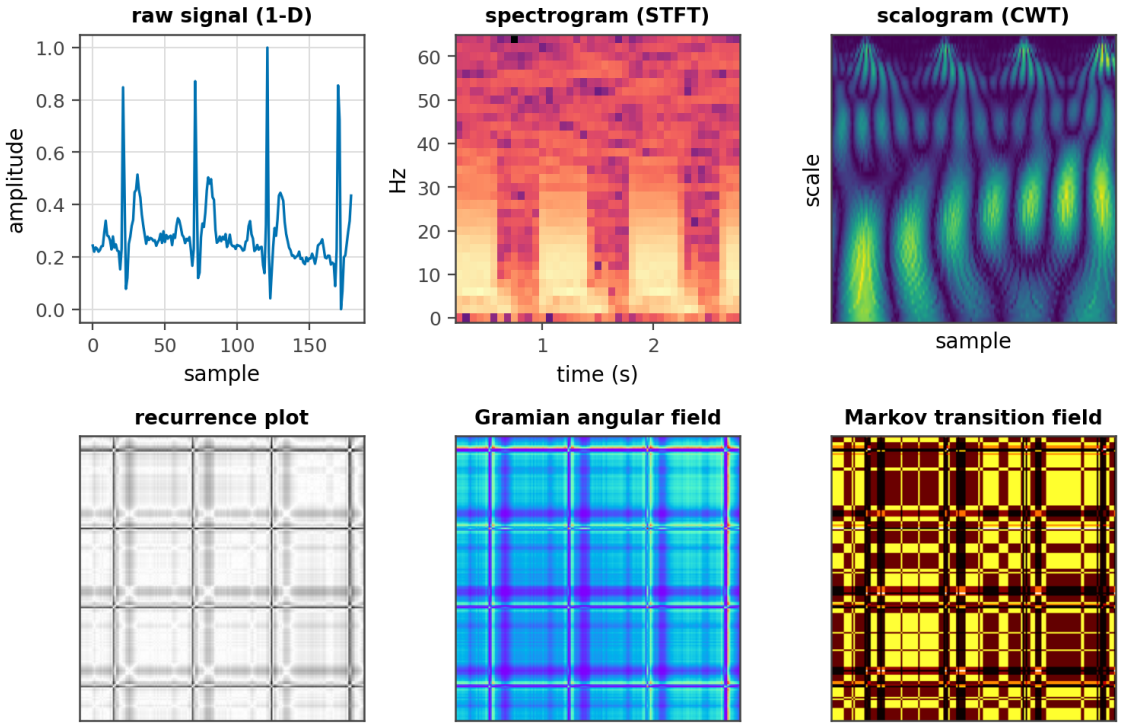
14.9 Signals as images: encodings and the 2-D analogue

Chapter 5 already turned a signal into a picture — the spectrogram and the scalogram are 2-D images of a 1-D signal, and §14.4's CNN can score a sleep epoch by *seeing* its time–frequency portrait. That observation generalises into a whole strategy for reusing the mature machinery of computer vision on biosignals: encode the 1-D signal as a 2-D image, then feed it to an image model. The motive is *transfer*. Vision backbones pretrained on ImageNet — millions of labelled photographs — have learned generic edge, texture, and shape detectors in their early layers, and those detectors *can* transfer well to the ridges and blobs of a signal-derived image — an empirical finding that varies with the task and the encoding, not a guarantee — so a biosignal task with only a few thousand labels can often borrow representational power it could never learn from scratch.

Several encodings are in common use, and Figure 14.4 shows five of them for a single ECG segment. The spectrogram (STFT) and scalogram (CWT) are the time–frequency images of Chapter 5. A recurrence plot marks, at pixel (i, j), how similar the signal is to itself at times i and j — periodic structure appears as a repeating block pattern, a change of regime as a block boundary. The Gramian angular field (GAF) re-expresses the signal in polar angles and encodes the pairwise relationship $\cos(\phi_i + \phi_j)$ as an image, preserving temporal correlations in a form a CNN reads well. The Markov transition field (MTF) quantises the signal into states and paints the transition probability between the states at times i and j, capturing its dynamics. None is uniquely correct; each foregrounds a different property, and which one wins is an empirical question settled by the honest validation of Chapter 12.

The image models these feed are the standard vision backbones. ResNet (whose residual con-

varkonyibarnabas@gmail.com



One 1-D signal, five 2-D encodings — spectrogram, scalogram, recurrence plot, Gramian angular field, Markov transition field — each of which an ImageNet-pretrained vision CNN can classify directly.

Figure 14.4. One 1-D signal, five 2-D encodings. The same three-beat ECG segment (top-left) rendered as a spectrogram and a scalogram (its time–frequency images from Chapter 5), a recurrence plot (self-similarity — the beats show up as a periodic block pattern), a Gramian angular field, and a Markov transition field. Each turns the waveform into an image that an ImageNet-pretrained vision network can classify, letting a biosignal task with few labels borrow the representational power of large-scale image pretraining.

varkonyibarnabas@gmail.com

nections make very deep networks trainable) and EfficientNet (which scales depth, width, and resolution together for accuracy-per-parameter) are the workhorse convolutional classifiers, applied to a spectrogram or a GAF exactly as they would be to a photograph. The Vision Transformer (ViT) splits the image into patches and attends across them, an increasingly strong alternative when data is plentiful. And U-Net, an encoder–decoder with skip connections built for image *segmentation*, is the natural choice when the task is to *localise* an event in time rather than label a whole epoch — segmenting the exact span of an apnea, a seizure, or a QRS complex on the 2-D encoding is the image analogue of event detection. A caution closes the loop with §14.8: a photograph-pretrained backbone was optimised for natural-image statistics, so the transfer, however useful, must still be *validated* on the biosignal task, not assumed.

> The 2-D analogue is where CNNs were born. Everything in §14.4 sharpens in two dimensions. The 2-D convolution that a CNN learns on a medical image *is* the 2-D convolution of the image-processing thread — an edge or texture detector — except its kernel is trained rather than designed. This is the payoff of the book's image thread: the same operation you applied by hand to sharpen or denoise a scan becomes, in a CNN, a *learned* bank of detectors, and stacking them yields the image classifiers that dominate medical imaging. A sleep epoch turned into a spectrogram is a small image, so a 2-D CNN can score sleep by *seeing* the time–frequency picture — spindles as bright ridges, delta as a low-frequency wash — closing the loop between the transforms of Chapter 5 and the learners here.

14.10 Companion notebook

The companion notebook really trains both contenders — a random forest on compact hand-crafted features and a small neural network on the raw epoch (a 1-D CNN in spirit) — on the *identical* subject-independent split, and prints both kappas so the fair fight of Figure 14.3(a) is measured rather than asserted. It then sweeps the training-set size to plot the measured data-efficiency curves of Figure 14.3(b), where the reader watches the feature model plateau while the raw model climbs; a short, honest note explains why this clean synthetic task does not manufacture the full small-data crossover (its features are strictly lossy) and what a real recording would add. It illustrates a representative 1-D convolution kernel to make the Chapter 9 equivalence tangible — in practice its taps would be learned by backpropagation — and a from-scratch NumPy 2-D CNN classifies one epoch rendered as a spectrogram (Figure 14.4 shows the other signal-to-image encodings as static visualisations), so the signal-to-image route of §14.9 is something the reader runs rather than reads. It is kept small enough to run on a modest GPU.

14.11 Case-study tie-in

In the sleep-staging case study, deep learning is an optional advanced track, framed as an extension rather than a requirement. The honest baseline remains the Chapter 13 random forest on hand-crafted features, cross-validated by subject; a strong team may try to push past its ceiling with a small CNN or a CNN-plus-sequence model, or by fine-tuning a pretrained encoder to squeeze more from limited labels. The rule that governs the attempt is Chapter 12's: the deep model is judged by the *same* LOSO kappa, on the *same* held-out subjects, with every learned step fit inside the training folds — so a bigger model earns its place only by beating

varkonyibarnabas@gmail.com

the forest under identical, honest conditions, not by looking impressive in a notebook.

14.12 Summary, questions, and further reading

Key formulas. Neuron $a = \sigma(\mathrm{w}^\top \mathrm{x} + b)$; ReLU $\max(0, z)$; the CNN layer is Chapter 9's convolution $y[n] = \sum_k h[k]\, x[n-k]$ with *learned* taps; training minimises cross-entropy by backpropagation and gradient descent.

Key ideas. Deep learning trades interpretability and data-thrift for capacity, learning features instead of hand-crafting them. A CNN convolution is the Chapter 9 convolution with a learned kernel — the clearest bridge from filtering to deep learning; the same operation runs on 1-D signals, spectrograms, and images. Sequence models (LSTM, transformer/attention) carry context across epochs. Self-supervised pretraining yields foundation models whose representations can transfer to small labelled tasks via fine-tuning — a promising route for low-data clinical problems, though an emerging and fast-moving one: learn the pretrain/fine-tune/transfer *principle* rather than any current model, watch for poor transfer under montage, sensor, sampling-rate, missing-channel, population, or label-definition shift, and mind the governance of the pretraining data (consent, population bias, de-identification limits, benchmark-overlap leakage). Deep learning is judged by the *same* subject-independent rules as Chapter 12; on small data the classical forest often wins, and deep learning tends to pay off mainly with abundant labels or a pretrained model that transfers. Responsible clinical use calls for population-independent validation, bias awareness, interpretability, and regulatory humility.

Conceptual questions. (1) At the level of the sliding weighted sum, what is the core difference between a CNN's first convolutional layer and a Chapter 9 FIR filter bank — and name two things a full convolutional layer adds beyond that sum? (2) Why might a small CNN *lose* to a random forest on a 20-subject dataset yet *win* on a 400-subject one? (3) What problem does self-supervised pretraining solve, and how? (4) Why does a higher-capacity model make honest, subject-independent validation *more* important, not less? (5) Name two responsible-AI risks that a deep model raises more sharply than a random forest. (6) Diagnose. A CNN sleep scorer trained at one centre (κ 0.80) falls to κ 0.45 at a second centre that uses a different EEG reference montage. List three candidate causes and, for each, one check that would confirm or rule it out.

Exploration tasks. Train a small 1-D CNN and the Chapter 13 forest on identical LOSO folds and compare kappa; then plot each model's performance against training-set size and explain why this clean synthetic task does *not* produce the small-data crossover (see §14.10). Visualise a trained CNN's first-layer kernels and check whether any resemble band-pass filters. Turn epochs into spectrograms and train a 2-D CNN, comparing it against the 1-D version. Fine-tune a pretrained time-series encoder on a shrinking fraction of the labels and measure how few labels it needs to match the forest.

Further reading. Goodfellow, Bengio & Courville, *Deep Learning* (networks, CNNs, training). LeCun, Bengio & Hinton's *Nature* review of deep learning. Vaswani et al., "Attention Is All You Need" (transformers). For biosignal and time-series foundation models at the time of writing: BIOT (a generic biosignal transformer), CBraMod (an EEG foundation model pretrained by masked reconstruction of signal patches) and LaBraM (an EEG foundation model pretrained by masked prediction of discrete neural-tokenizer codes), and MOMENT (a patch-based general

varkonyibarnabas@gmail.com

time-series foundation model); the field moves quickly, so treat the specific state of the art as a moving target and verify current models when you build. The PyTorch documentation for hands-on entry points.

varkonyibarnabas@gmail.com

Chapter 15 — From Notebook to System: Deployment Thinking for Biomedical Signal Models

A sleep scorer sitting in a notebook helps nobody — how might it become a feature in a wristband, a service in a hospital, or a cloud API, and what changes the moment it does?

This is a short chapter by design. It builds *design judgement* about where and how a biosignal model would run — not a course in embedded engineering, MLOps, or medical-device law, each of which is a field of its own. Think of it as the bridge between a model that *works in a notebook* and a model that *works in the world*, where latency, battery, connectivity, privacy, and regulation are no longer footnotes but the constraints that shape every choice. A recurring theme is that the classical signal-processing toolkit of Chapters 8–11 is not superseded by deployment — it becomes the *efficient front end* that makes edge deployment possible. And a recurring caution frames the whole chapter: a study-scale model is not, and should not be presented as, "regulator-ready"; the regulatory material here is *awareness with sourced pointers*, not a how-to.

15.1 Learning objectives

By the end of this chapter you should be able to:

- Place a model on the deployment spectrum — edge, embedded, server, cloud — and reason about the trade-offs of each (L4, L5).
- Decide what leaves the device — raw signal, features, or only the decision — and defend the choice on latency, privacy, and cost (L5).
- Explain edge constraints and the TinyML regime, and how quantization, pruning, and distillation make a model fit (L4).
- Describe streaming/windowed inference and the basics of server/cloud serving and MLOps, including model drift (L4).
- State, at *awareness* level and with sourced pointers, why clinical deployment carries privacy and regulatory weight beyond a study project (L5).

15.2 The deployment spectrum and the one pivotal choice

A trained model can live anywhere along a spectrum, and where you place it is the first and most consequential deployment decision. At one end is the edge: the model runs *on* the sensor device — a wristband, a microcontroller in a patch — with no round trip to any server. Next is embedded or gateway compute: a nearby phone or hub with more power than the sensor. Then an on-premises server inside the hospital, and finally the cloud: a scalable API somewhere on the internet. Moving *rightward* (toward the cloud) buys compute, memory,

varkonyibarnabas@gmail.com

and the ability to pool data across many users; moving *leftward* (toward the edge) buys lower latency, stronger privacy, and freedom from connectivity. Figure 15.1(a) lays out the spectrum and marks the choice that matters most.

That choice is what leaves the device. A wearable can transmit the *raw signal* to a big cloud model, or it can extract *features* on-device and send only those, or it can run the whole model on-device and emit *only the decision* ("N2"). These are radically different systems wearing the same description. Shipping raw signal off-device grants the most downstream capability but is the worst for privacy, bandwidth, and battery — a night of raw EEG is a large, sensitive stream. Shipping only the decision is the most private and frugal but pins all the computation to the device. The feature-based middle is often the sweet spot: extract compact features on-device (Chapter 11) and send those, keeping the raw physiology local. Features are *usually less revealing* than the raw signal — a handful of band powers discards most of the waveform — but they are not a privacy guarantee: engineered features can still be sensitive and are sometimes re-identifying, so "we only send features" reduces exposure rather than eliminating it. This single boundary — raw versus feature versus decision — strongly shapes the privacy posture, the bandwidth bill, the battery life, and much of the regulatory exposure of the whole product.

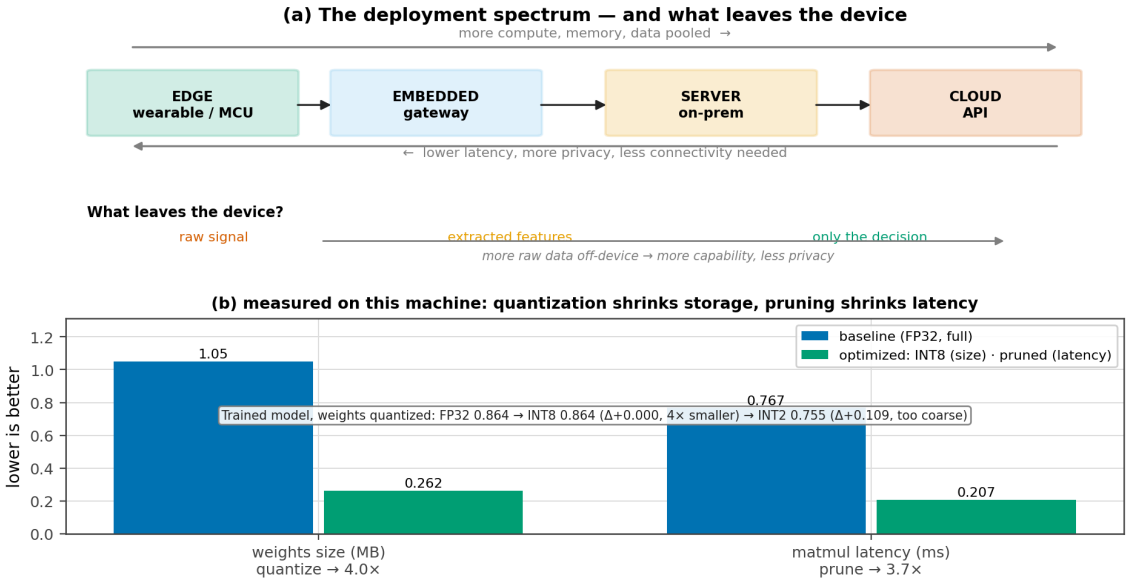


Figure 15.1. Two deployment decisions on one page. (a) The spectrum from on-device edge to cloud: rightward buys compute and pooled data; leftward buys latency, privacy, and offline operation. The pivotal design choice is *what leaves the device* — raw signal, extracted features, or only the decision — which sets the privacy, bandwidth, and battery profile of the whole system. (b) Making a model fit the edge, *measured on this machine*: quantizing FP32 weights to INT8 gives an exact 4× storage cut, while pruning whole channels cuts matmul latency directly by removing multiplies. The two levers act on different costs — and, tellingly, INT8 buys *no* CPU speed-up on this machine, because a naive integer path here hits no fast kernel — whether INT8 accelerates at all depends on the CPU's instruction set, runtime, and operators, and an edge NPU makes the gain reliable. Size shrinks universally; speed is hardware-dependent, which is the whole point.

The trade-offs behind the picture are worth naming because they pull in different directions. Latency: an arrhythmia alarm may need to fire in milliseconds, which effectively rules out a

varkonyibarnabas@gmail.com

cloud round trip; a sleep report can be computed overnight in the cloud. Connectivity: a home monitor must keep working when the Wi-Fi drops, which favours the edge. Privacy: keeping raw physiology on the device is usually the strongest privacy *mitigation* available — though no single choice guarantees privacy on its own. Power and battery: every byte transmitted and every multiply computed drains a coin cell, and radio transmission is often *more* expensive than local computation — a reason to compute at the edge and transmit little. Cost: cloud compute scales but bills per call; edge compute is paid once in silicon. The comparison table below turns these into a decision aid.

Criterion	Edge (on-device)	Cloud (server/API)
Latency	lowest (no round trip)	network-bound
Works offline?	yes	no
Privacy of raw signal	strongest (stays local)	weakest (leaves device)
Compute / model size	tightly limited	effectively unlimited
Battery impact	local compute; little radio	radio-heavy
Update / retrain	harder (push to fleet)	easy (central)
Best for	real-time, private, offline	heavy models, pooled data

15.3 Edge constraints, TinyML, and making the model fit

The edge is a hostile place for a model. A microcontroller may offer kilobytes of RAM, no floating-point unit, and a power budget measured in milliwatts — the regime called TinyML. A model that ran comfortably in a notebook will not fit, and three techniques bring it down to size. Quantization replaces 32-bit floating-point weights with 8-bit integers, shrinking the model by an exact fourfold, usually at negligible accuracy cost; the *speed-up*, however, does not follow automatically — it depends on the hardware, operators, memory bandwidth, and the quantization implementation. Figure 15.1(b) makes this concrete by measuring it on an ordinary CPU: the $4\times$ storage cut is real and immediate, but INT8 buys *no* latency gain on this machine, because a naive integer path here hits no fast kernel; whether it speeds up at all depends on the CPU's instruction set (e.g. AVX-512 VNNI or ARM dot-product), the runtime, and the operators — an edge NPU makes the gain reliable. Pruning removes weights or whole channels that contribute little, sparsifying the network; because it deletes multiplies outright, it cuts latency directly even on a CPU — the measured latency win in Figure 15.1(b) comes from pruning, not quantization. Knowledge distillation trains a small "student" model to mimic a large "teacher", capturing much of the teacher's skill in a fraction of the parameters. Alongside these, hardware acceleration (a DSP block or neural-processing unit on the chip) and portable formats — ONNX and TensorFlow Lite, and, in the PyTorch world, `torch.export` and the ExecuTorch embedded runtime in place of the now-deprecated TorchScript — let a model trained on a workstation run efficiently on target silicon. On the edge, inference is also streaming: rather than waiting for a whole night, the device processes the signal in a sliding window, emitting a stage every 30 seconds as the data arrive — the same windowing that produced features in Chapter 11, now driving real-time decisions.

varkonyibarnabas@gmail.com

Method: the classical DSP front end is your edge advantage. The whole toolkit of this book earns its keep at the edge. Doing the filtering (Chapters 9–10) and feature extraction (Chapter 11) *on-device* is usually cheaper, more robust, and more interpretable than streaming raw signals to a big remote model — and it shrinks *what leaves the device* from a raw waveform to a handful of features. A band-pass filter and a few band-power features cost almost nothing to compute and can feed a tiny classifier that runs on a coin cell. Deploying the classical pipeline is often the deployment: the efficient front end *is* the product.

15.4 Server, cloud, and MLOps

When the model is large or the data must be pooled, it lives on a server or in the cloud, and a new discipline takes over. Serving can be batch (score a night's recording after upload) or streaming (score a live feed), and the model is typically wrapped in a container behind an API, with autoscaling to meet demand and GPUs for heavy networks. But the model is the easy part; keeping it healthy over time is MLOps — the operational practice around a deployed model. Its pillars are versioning (of code, of data, and of the model, so any result can be reproduced), monitoring (of inputs and outputs in production), and the detection of drift. Drift is the quiet killer of deployed clinical models: the world changes — a new sensor, a new population, a new electrode montage — and the data the model meets in production slowly diverges from the data it was trained on, so its accuracy decays without any error ever being thrown. Catching drift requires monitoring the live data distribution and the model's confidence, and responding with a retraining loop. Underpinning all of it is a reproducible data lifecycle: pipelines, logging, and provenance so that every prediction can be traced back to the specific code, data, and model that produced it — the same engineering rigour the capstone of Chapter 16 develops.

Data governance and licensing. A deployed model is only as trustworthy as the data behind it, so the data itself needs governance — a discipline that becomes contractual, not just good practice, once a model ships. In brief: pin and cite the exact dataset version, confirm each license permits your intended use (research terms rarely cover commercial deployment, and mixed corpora inherit the union of their obligations), treat physiological data as sensitive personal data with real de-identification limits, and guard against benchmark contamination — the same subject, recording, or public benchmark leaking across training and evaluation. Appendix F.4 treats each of these in full and gives the per-dataset governance record that turns them into an auditable checklist.

15.5 Implications and responsibilities (awareness level)

The moment a model leaves the notebook and touches a real patient, it acquires responsibilities that a study project does not have, and this section flags them at *awareness* level, with cautious language and pointers to authoritative sources — not as a how-to. Privacy and data residency come first: physiological signals are sensitive personal data, and where they are stored and processed is governed by data-protection law; keeping raw signal on the device (§15.2) is often the strongest mitigation. Security matters because a networked medical device is an attack surface. Robustness and equity matter because a model that works for the population it was trained on may fail for another, and deployment at scale amplifies any such

varkonyibarnabas@gmail.com

bias. And a human should stay in the loop: an automated sleep score is a decision *support*, not a substitute for clinical judgement.

Then there is regulation, which is stated here for *awareness only* and is not legal or regulatory advice. The paragraph that follows is a date-stamped orientation, not a compliance guide; regulatory specifics change, jurisdictions differ, and only the official sources are authoritative. Software with a *medical purpose* is, in principle, a medical device — the concept of Software-as-a-Medical-Device (SaMD) — and making a clinical claim about it can trigger formal obligations. In the United States, the FDA generally regulates such software under its existing device framework, using the standard device classes and premarket pathways, with recent guidance addressing how AI-enabled devices manage change over their lifecycle (its Predetermined Change Control Plan guidance). In the European Union, an AI system with a medical purpose may fall under both the medical-device rules and the EU AI Act (Regulation 2024/1689), which entered into force in 2024 and applies in phases over the following years rather than all at once; under it, an AI system that is itself a medical device — or a safety component of one — and that already requires third-party conformity assessment is generally classed as high-risk (the condition set out in its Article 6), rather than every medical-purpose AI being automatically high-risk. Because the timeline is phased and the detailed obligations are still being clarified, treat any specific figure or deadline as time-sensitive and verify it against the primary text. The single most important thing to take from this is a matter of humility: a study-scale or course-scale sleep scorer, however good its kappa, is not a medical device, is not "regulator-ready," and carries no clinical claim. Turning it into one would require formal clinical evaluation, quality processes, and regulatory review far beyond the scope of any study or coursework. The correct posture is to *know that this weight exists*, to design as if privacy and generalisation mattered because they do, and to consult the authoritative sources — the FDA's SaMD guidance and the text of the EU AI Act — rather than any textbook paraphrase (including this one) when a real deployment is contemplated.

The main regulatory pathways, named. For orientation only — and, again, not regulatory advice — it helps to put names to the routes a medical-device claim actually travels, because they recur in any deployment conversation. In the United States, most software devices reach market through the 510(k) route by showing substantial equivalence to an already-cleared "predicate" device; a genuinely novel low-to-moderate-risk device with no predicate can instead use the De Novo route to create a new classification; and the highest-risk devices require full premarket approval (PMA) with clinical evidence. In the European Union, a device is CE-marked after a conformity assessment under the Medical Device Regulation (MDR, 2017/745), its rigour scaled to the device's risk class, and — as above — an AI-enabled device may additionally fall under the EU AI Act. Cutting across jurisdictions, ISO 13485 certifies the manufacturer's quality management system — the documented processes for design and traceability — ISO 14971 governs medical-device risk management specifically, and IEC 62304 governs the software life cycle; note that ISO 13485 certifies the *organisation's processes*, not the safety or clinical performance of any one device. The through-line to Chapter 12: these pathways evaluate the evidence appropriate to a device's intended use, risk class, and jurisdiction — they do not map one-to-one onto the pedagogical validation ladder, and a clearance is not a certificate that every rung was climbed. A 510(k) leans on substantial equivalence to a predicate plus analytical validation, while a novel high-risk AI diagnostic demands prospective clinical evidence.

varkonyibarnabas@gmail.com

Pathway (region)	When it applies	What it demonstrates
510(k) (US)	moderate-risk device with an existing predicate	substantial equivalence to a cleared device
De Novo (US)	novel low/moderate-risk device, no predicate	safety and effectiveness for a *new* classification
PMA (US)	high-risk device	full premarket approval backed by clinical evidence
CE marking / MDR (EU)	any device placed on the EU market	conformity assessment under Reg. 2017/745, scaled to risk class
ISO 13485 (international)	the manufacturer, not a single device	a certified quality-management system for medical devices

15.6 Companion notebook

The companion notebook makes Figure 15.1(b) concrete and *measurable* on your own machine. It builds a representative dense weight matrix — the bulk of a small model's parameters — and measures three effects directly: quantizing FP32 to INT8 shrinks storage by an exact four-fold; pruning whole channels cuts the matmul latency by removing multiplies; and, on a small *trained* classifier, INT8 quantization costs essentially no accuracy — but a too-coarse INT2 collapses it, a measured reminder that lower precision is a real design choice, not a free lunch past a point. The honest, slightly uncomfortable result on latency is that INT8 leaves CPU latency essentially *unchanged* — a general-purpose CPU has no fast integer-matmul path — so on this hardware the latency win comes from pruning, not quantization, and a real INT8 speed-up waits for an edge NPU. That is the whole point the figure separates: model size shrinks universally, but speed is hardware-dependent. Exporting a trained model to a portable format (ONNX/TFLite), estimating on-wearable memory, staging the on-device-vs-cloud architecture comparison of §15.2, and simulating windowed (streaming) inference over a night are left as the exploration tasks below, since each depends on target hardware and tooling the notebook cannot assume.

15.7 Case-study tie-in

Deployment is an *optional* lens on the sleep-staging case study, not a required build. It asks a single, clarifying question of the finished model: how might this ship? Two answers frame the trade-offs of the chapter. As a consumer home-sleep tracker, it would live at the edge — features and a small model on the wearable, only the nightly summary leaving the device, prized for privacy and battery life and making *no* clinical claim. As a clinical scoring service, it would live in the cloud — a heavier model, pooled data, central retraining — but would then confront exactly the privacy, robustness, and regulatory obligations of §15.5, and would need formal clinical evidence before any diagnostic claim. Naming which end of the spectrum a project targets, and what evidence that target would demand, is the deployment-thinking this short chapter exists to teach.

varkonyibarnabas@gmail.com

15.8 Summary, questions, and further reading

Key ideas. A model can run anywhere from edge to cloud; rightward buys compute and pooled data, leftward buys latency, privacy, and offline operation. The pivotal choice is what leaves the device — raw signal, features, or only the decision. The edge (TinyML) is tightly constrained, and quantization, pruning, and distillation shrink a model to fit — roughly $4\times$ smaller from FP32 to INT8, with speed and energy gains that depend on the hardware and implementation. Edge inference is streaming/windowed, and the classical DSP front end is the efficient edge advantage. Cloud serving brings MLOps: versioning, monitoring, and drift detection with retraining loops, plus data governance and licensing — dataset versioning, license compatibility, patient privacy/de-identification, and benchmark contamination. Clinical deployment carries privacy and regulatory weight — SaMD, the FDA framework, the EU AI Act (in force since 2024 with phased application) — but a study-scale model is *not* regulator-ready, and all of this is stated at awareness level with sourced pointers and a date stamp, not as legal guidance or a how-to.

Conceptual questions. (1) For a real-time cardiac-alarm wearable, argue for edge over cloud on two grounds. (2) Explain how sending only extracted features, rather than the raw signal, changes the privacy and battery profile. (3) What is model drift, why is it dangerous, and how would you detect it? (4) Why is a study-scale sleep scorer *not* a medical device, and what would turning it into one require? (5) How does the classical pipeline of Chapters 9–11 lower the cost of edge deployment? (6) Diagnose. A quantized on-wearable scorer passes bench tests but, in the field, its accuracy quietly degrades over months for a subset of users. Name three distinct causes — one each in the *model*, the *data*, and the *hardware* — and the one monitoring signal you would watch for each.

Exploration tasks. Quantize a trained classifier to INT8 and tabulate the change in model size, latency, and accuracy. Sketch two architectures for a wearable sleep scorer — "on-device features plus tiny model" versus "raw signal to cloud" — and score each on latency, privacy, and battery. Simulate windowed streaming inference over a night and compare it with batch scoring. Read the FDA's SaMD material and the EU AI Act summary, and write one paragraph on what evidence a clinical sleep-scoring claim would require.

Design brief. Design the deployment for a wearable arrhythmia monitor: it must alarm in real time, keep working when the phone loses connectivity, respect patient privacy, and last a multi-day battery. Spec: turn each requirement into a hard constraint (latency, offline operation, privacy, power). Design: place the model on the edge→cloud spectrum and decide what leaves the device — raw signal, features, or only the decision; pick the compression (quantization / pruning / distillation) if it runs on the edge. Evaluate: the diagnostic that shows the on-device model still meets the accuracy *and* latency budget. Motivate: justify the edge-vs-cloud split against the latency and connectivity constraints. Alternatives: name a defensible different placement and the spec change (e.g. a non-urgent overnight report) that would make cloud the better choice. Rule out: name a deployment choice that would be *wrong* here — e.g. streaming raw ECG to a cloud model for a real-time alarm, or assuming INT8 quantization guarantees the latency win — and the requirement it breaks.

Further reading. For edge and TinyML: Warden & Situnayake, *TinyML*, and the TensorFlow Lite / ONNX Runtime documentation for quantization and portable formats. For serving and MLOps: Huyen, *Designing Machine Learning Systems* (deployment, monitoring, drift). For regulation, consult the primary sources rather than paraphrases: the FDA's guidance on AI-

varkonyibarnabas@gmail.com

enabled device software functions and its *Marketing Submission Recommendations for a Predetermined Change Control Plan for Artificial Intelligence-Enabled Device Software Functions*, and the text of the EU AI Act (Regulation 2024/1689) together with the EU Medical Device Regulation (2017/745) and In Vitro Diagnostic Medical Device Regulation (2017/746). Treat all regulatory pointers as awareness-level and time-sensitive: verify the current guidance before any real deployment.

varkonyibarnabas@gmail.com

Chapter 16 — Capstone: Building, Evaluating & Delivering an Automatic Sleep-Scoring System

Every earlier chapter taught one skill in isolation. How do all those threads — a filter here, a feature there, a classifier, an honest metric — become one working, defensible system that a team builds together and stands behind?

This is the chapter where the book's promise comes due. We have acquired signals (Chapter 2), transformed them (Chapters 3–7), cleaned them (Chapters 8–10), turned them into features (Chapter 11), learned to validate honestly (Chapter 12), chosen classifiers (Chapters 13–14), and thought about where a model runs (Chapter 15). Now we assemble the whole thing into a single running system — an automatic sleep scorer that reads a night of physiology and returns a hypnogram a clinician could read — and we do it the way real work is done: as reusable modules, built iteratively, evaluated ruthlessly, engineered reproducibly, and delivered by a team that can explain and defend every choice. The sleep-staging problem is the worked case study throughout; the *method* generalises to any biosignal you have met in these pages.

16.1 Learning objectives

By the end of this chapter you should be able to:

- Assemble the end-to-end pipeline — data loading → preprocessing → feature extraction → feature selection → classification → inference → reporting — as reusable, testable modules (L5).
- Run an iterative build, from a simple baseline to a tuned ensemble, and explain why incremental development beats a single-shot attempt (L5, L6).
- Justify and document design decisions so choices are traceable, not accidental (L5).
- Evaluate a system in practice with subject-independent (LOSO) validation, Cohen's kappa, macro-F1, confusion matrices, hypnograms, and Bland–Altman agreement, finishing on a held-out set (L5).
- Make a project reproducible and engineered — config-driven runs, caching, unit tests, version control, pinned environments (L5).
- Work as an effective team: divide roles, collaborate in an agile loop, review each other's code, and disclose individual contributions honestly (L6).
- Communicate results in a technical report and a defence of your own work (L6).

16.2 The end-to-end pipeline as reusable modules

A capstone tempts you toward one enormous script that reads a file at the top and prints an accuracy at the bottom. Resist it. The professional shape is a pipeline of small, single-

varkonyibarnabas@gmail.com

purpose modules, each with a clear input and output, that you can build, test, cache, and swap independently. Figure 16.1 lays out the seven stages, and the discipline is that each stage is a function (or a class) you can call, test, and replace without touching the others.

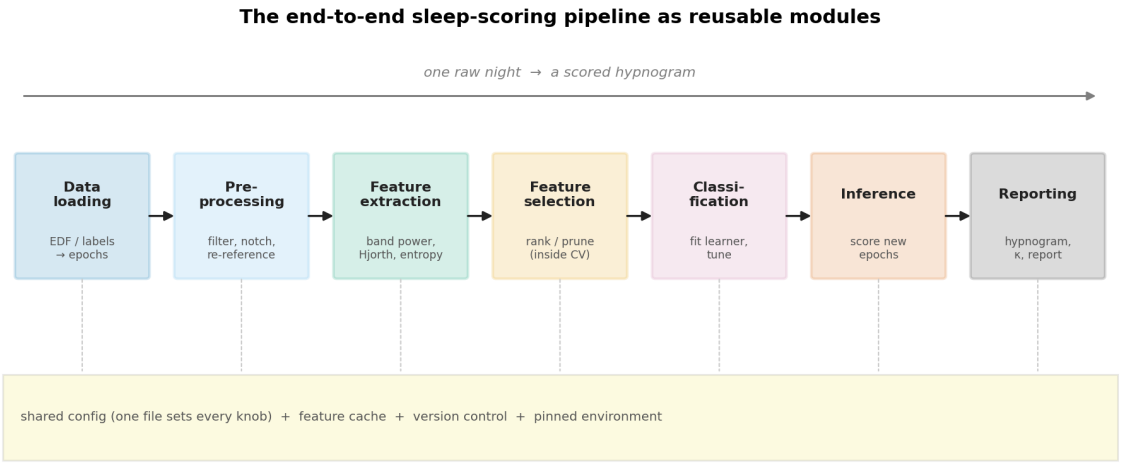


Figure 16.1. The sleep-scoring pipeline as seven reusable modules — data loading, preprocessing, feature extraction, feature selection, classification, inference, reporting — driven by one shared configuration, a feature cache, version control, and a pinned environment. The takeaway: build the system as swappable parts on a shared rail, not one monolithic script, so any single stage can be improved or tested in isolation.

Data loading turns raw recordings (an EDF file of channels plus an annotation file of expert labels) into a tidy table of 30-second epochs, each tagged with its stage. Preprocessing applies the Chapter 8–10 toolkit — band-pass filtering, a notch at the mains frequency, re-referencing, artifact handling — to each epoch. Feature extraction collapses each cleaned epoch into a fixed-length vector of the Chapter 11 descriptors: band powers, Hjorth parameters, spectral entropy, EOG movement features. Feature selection ranks and prunes those features — *inside* the cross-validation loop, per Chapter 12, never before. Classification fits a learner (Chapters 13–14). Inference applies the frozen pipeline to a new night, epoch by epoch. Reporting turns the predicted sequence into a hypnogram, the metrics, and the paragraph a human reads.

In Python the modular shape is deliberately plain — each stage is a function with an honest signature:

```
# pipeline.py — one function per stage, all driven by a config dict
def load_epochs(edf_path, label_path, cfg):
    """EDF + annotations -> (epochs[n, ch, samples], labels[n])."""
    ...
    return epochs, labels

def preprocess(epochs, cfg):
    """Band-pass, notch, re-reference each epoch. Returns cleaned epochs."""
    return np.stack([_clean(e, cfg) for e in epochs])

def extract_features(epochs, cfg):
```


varkonyibarnabas@gmail.com

```
    """Cleaned epochs -> feature matrix X[n, d] + feature names."""
    return X, names

def select_features(X, y, cfg):        # fit INSIDE the training fold only
    return selector.fit(X, y)

def fit_classifier(X, y, cfg):
    return model.fit(X, y)

def run(cfg):                          # the whole pipeline, config-driven
    ep, y = load_epochs(cfg["edf"], cfg["labels"], cfg)
    ep = preprocess(ep, cfg)
    X, names = extract_features(ep, cfg)
    return evaluate_loso(X, y, cfg)    # Chapter 12's honest harness
```

> MATLAB equivalent. The same modular shape is idiomatic in MATLAB: put each stage in its own function file (`load_epochs.m`, `preprocess.m`, `extract_features.m`, ...) and let a top-level `run_pipeline.m` thread them together from a `cfg` struct. Read EDFs with `edfread`, design filters with `designfilt/butter` and apply with `filtfilt`, compute spectra with `pwelch`, and train with `fitcknn`, `fitcsvm/fitcecoc`, or `TreeBagger`. Cross-validation lives in `cvpartition` (use `'Grouping'`/custom subject folds for LOSO), and `confusionmat/confusionchart` draw the confusion matrix. The language changes; the seven modules and the discipline do not.

Two properties make this shape pay off. First, each stage is independently testable — you can unit-test the feature extractor on a synthetic epoch whose band power you know analytically, without ever loading a real recording. Second, each stage is swappable: changing the classifier from k-NN to a random forest is a one-line edit to the config, because the interface between "features in" and "labels out" never moved. That swappability is exactly what the iterative build of the next section exploits.

16.3 An iterative build: baseline first, then earn the complexity

The single most important process lesson in this book is *do not try to build the final system in one shot*. Waterfall — design everything, build everything, then discover at the end that it does not generalise — is how capstones fail. Instead, build in iterations: each round produces a *complete*, *runnable* pipeline end to end, and each round is measured under the same honest harness so you can see whether a change actually helped. You start deliberately simple, get a number, and only then spend complexity where the evidence says it will pay.

A natural progression climbs the learner ladder — and, realistically, the feature set grows alongside it at almost every rung (see the table below): this is deliberately *not* the controlled, features-held-fixed comparison of Section 13.4, where swapping k-NN for a random forest on an *unchanged* feature set bought essentially zero kappa. Here model and features improve together, which is the more realistic way a capstone actually gets built, at the cost of not being able to cleanly attribute a given rung's gain to the model alone. Iteration 1 is the baseline: a small feature set and a k-NN classifier — the simplest capable learner (Chapter 13) — wired end to end and scored under Leave-One-Subject-Out. It will not be good, and that

varkonyibarnabas@gmail.com

is the point: it proves the pipeline *runs* and gives you a floor. Iteration 2 swaps in a scaled SVM for a smoother boundary and adds the Hjorth features. Iteration 3 moves to a random forest with class weights against the N2-heavy imbalance, on the full time+frequency feature set. Iteration 4 *tunes* that forest and expands the feature set further. Figure 16.2(b) shows the agreement climbing across these rounds, and the table below is the ladder written down.

The κ values in the table are illustrative indicative targets on a simulated example, not benchmark claims or expected real-world performance. They describe the *shape* of a healthy iteration — each rung a little better than the last — using round numbers from the synthetic companion pipeline. Read them as "roughly this much better than the previous rung," never as a score to expect on real recordings; genuine performance depends on the data, the labels, and the population, and honest numbers come only from your own subject-independent evaluation (Chapter 12).

Iteration	Learner	Key change	Feature set	Illustrative LOSO κ (simulated)
1	k-NN	end-to-end baseline; prove it runs	small (band powers)	≈ 0.36*
2	SVM (RBF, scaled)	smoother boundary; scale inside folds	small + Hjorth	≈ 0.46*
3	Random forest	bagging + class weights vs. N2 imbalance	full time+freq set	≈ 0.57*
4	Tuned RF (+ features)	tune trees/depth; add EOG & entropy features	expanded, selected	≈ 0.63*

*Illustrative simulated values — read the *trend*, not the absolute numbers (see the paragraph above).

Read the ladder as a story about *where the gains come from* — model and features rising together, not the model in isolation. The jump from k-NN to SVM is real but modest; the jump to a random forest is larger, and reflects both a model that tolerates mixed feature scales and imbalance gracefully *and* a bigger feature set; the final jump adds tuning on top of yet more features. Because the two levers move together at every rung, this ladder cannot by itself say how much of any jump is the model versus the features — Section 13.4's controlled, same-features comparison is where that question is actually isolated, and there the model alone bought almost nothing. Every rung is a full, defensible system on its own — if you run out of time at Iteration 2, you still have a working scorer with an honest number. That is the deep virtue of iterating: you are never one broken commit away from having nothing to show.

> Method: definition of "done" for an iteration. An iteration is finished only when it (1) runs end to end from raw data to a hypnogram, (2) reports Cohen's kappa under LOSO with its spread, (3) is committed to version control with a written note of *what changed and why*, and (4) is at least as good as the previous iteration or explains, in writing, why the change was kept despite not helping. Anything less is a work-in-progress, not an iteration.

varkonyibarnabas@gmail.com

16.4 Design decisions and their justification

A capstone is assessed as much on *why* as on *what*. Every consequential choice — which signals to use, which features to compute, which model to fit, which validation to trust — should be written down at the moment it is made, with its motivation, so the design is traceable rather than a pile of accidents you can no longer explain. This is not bureaucracy; it is the difference between a system you can defend and one you merely possess.

The choices have real structure. Signals: sleep staging leans hardest on EEG (the band-power signatures of each stage), with EOG to catch the eye movements of REM and wake, and EMG chin tone to separate REM's muscle atonia — a decision grounded in the physiology of Chapter 1, not convenience. Features: delta power discriminates N3, spindle-band activity flags N2, the alpha/theta balance separates wake from drowsiness — each feature earns its place by mapping to a known stage signature (Chapter 11), and you should be able to name that mapping for every feature you keep. Model: start with the interpretable, low-friction random forest that runs on a microcontroller and explains itself through feature importances; reach for a deep network (Chapter 14) only when the evidence says the hand-crafted features have been exhausted. Validation: subject-independent, because the clinical target here is a *new sleeper*, not a new epoch from a known one — so LOSO (or GroupKFold by subject) is the right split for a new-subject claim; a multi-site or multi-device claim would instead need site- or device-level splits, matching the unit of splitting to the deployment claim (Chapter 12). The habit to build is simple: for every knob you turn, write one sentence saying why, so that six weeks later — or in front of an examiner — the reasoning is still there.

16.5 Evaluation in practice

Chapter 12 laid down the rules of honest measurement; here we *apply* them to a whole system and read the results the way a clinician would. The headline is subject-independent (LOSO) performance: train on all sleepers but one, score the held-out sleeper, rotate, and report the mean kappa with its spread. The primary metric is Cohen's kappa, agreement corrected for chance,

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

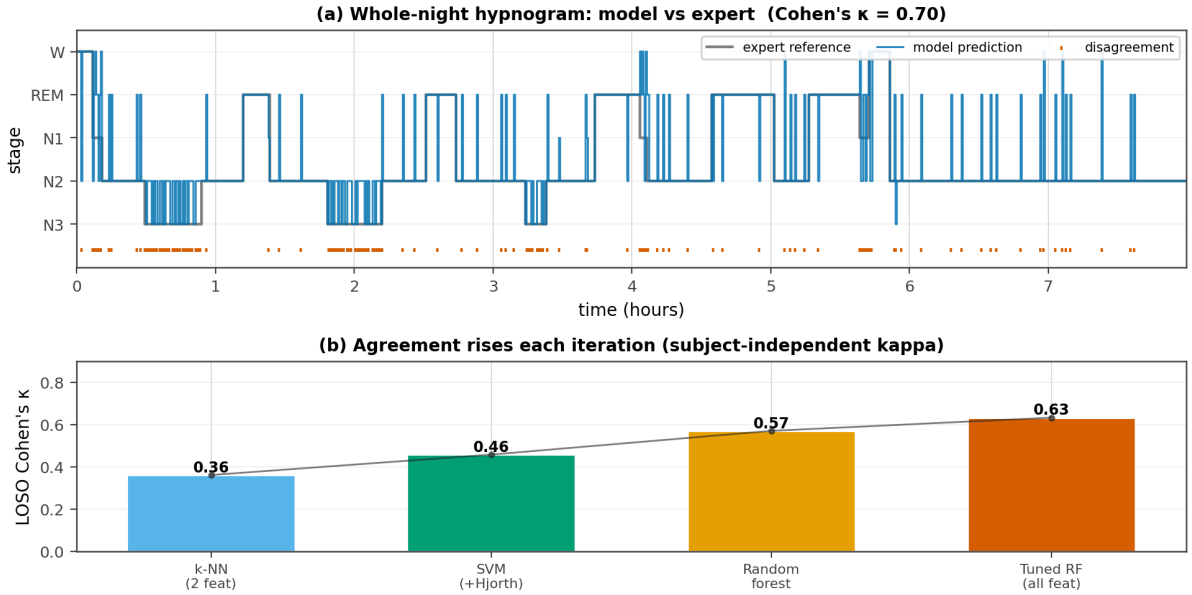
where p_o is observed agreement and p_e is chance agreement given the stage frequencies — a common sleep-scoring metric because, being chance-corrected, it is *less easily fooled* by the roughly 45%-N2 imbalance that would flatter raw accuracy. It is not, however, immune to prevalence and bias, so report it alongside per-class recall, macro-F1, balanced accuracy, the confusion matrix, and the clinical summaries rather than as a single headline. Macro-F1 (the unweighted mean of per-class F1) reports how well the *rare* stages fare, since it refuses to let the majority class dominate:

$$\text{macro-F1} = \frac{1}{K}\sum_{c=1}^{K}\frac{2\,\text{precision}_c \cdot \text{recall}_c}{\text{precision}_c + \text{recall}_c}.$$

The confusion matrix then shows *which* stages are confused for which — typically N1 bleeding

varkonyibarnabas@gmail.com

into N2 and wake, and REM occasionally mistaken for N1 — pointing directly at where the next feature should help. Figure 16.2(a) shows the corresponding whole-night view: the predicted hypnogram overlaid on the expert reference, with disagreements marked. Read this way, evaluation is not one number but a diagnosis: kappa says *how good overall*, the confusion matrix says *where it fails*, and the hypnogram says *whether the night's shape is right*.



Real leave-one-subject-out (LOSO) kappas on synthetic multi-subject data: errors cluster at stage transitions, and each build iteration genuinely lifts the measured kappa. Synthetic data, real computation.

Figure 16.2. Reading a sleep scorer two ways (illustrative, on a simulated night). (a) A whole-night hypnogram: the model's predicted stages (W/REM/N1/N2/N3) overlaid on the expert reference across an eight-hour night, with per-epoch disagreements marked — errors cluster at stage transitions and confuse neighbouring stages, at an illustrative Cohen's κ ≈ 0.70 on this synthetic example (one night, within the tuned model's 0.40–0.72 across-subject range). (b) The iterative build: subject-independent kappa rising from a k-NN baseline through SVM and random forest to a tuned RF. The takeaway: agreement is best judged as a rising curve across honest iterations, not a single lucky number; the specific κ values are illustrative, not benchmark results.

Two further checks separate a serious evaluation from a naive one. First, night-level agreement: a scorer can post a decent per-epoch kappa yet still mis-estimate the clinical summaries a physician actually reads — total sleep time (TST), sleep efficiency (TST as a fraction of time in bed), REM latency, wake after sleep onset (WASO), and overall hypnogram agreement — because per-epoch errors that cancel in a *count* may still bias a summary if they cluster. Report each of these summaries and compare them against the expert's with a Bland–Altman plot (model minus expert, against their mean), which reveals both a systematic bias and the limits of agreement ($\bar{d} \pm 1.96\, s_d$), telling you *how far off and in which direction* the clinically meaningful numbers run. Second, and finally, the held-out set: kappa reported during development is spent as you tune, so the honest final claim comes from scoring nights the pipeline has *not touched during any iteration* — a genuine test of generalisation, reported once, exactly as Chapter 12 sets out. On this simulated example a LOSO kappa in the mid-0.6s is a reasonable result for automatic staging — an illustrative figure, not a clinical claim (§16.3, Chapter 15).

varkonyibarnabas@gmail.com

There is one more ceiling to keep in view, and it is not the model's. Labels are measurements too (Chapter 12): expert sleep scores are not ground truth handed down from nature but human judgements, and independent scorers disagree on a meaningful fraction of epochs — inter-scorer agreement on staging is itself well short of perfect, especially for N1 and at stage transitions. That inter-scorer uncertainty is label-reliability context, not a universal ceiling: judged against a *single* noisy expert, a model cannot reliably appear to agree more closely than two experts agree with each other, so a kappa near the inter-rater agreement is doing about as well as *that* target allows — though a model scored against consensus or adjudicated labels can legitimately exceed pairwise human agreement. Chasing a higher number against one scorer risks overfitting that scorer's idiosyncrasies. Report your agreement, and read it against inter-scorer agreement, not against a hypothetical perfect one.

16.6 Reproducibility and engineering

A result nobody can reproduce is not a result. The engineering that surrounds the pipeline is what turns a lucky run into a defensible one, and it is assessed. Config-driven runs: every knob — sampling rate, filter cut-offs, feature list, model hyperparameters, random seed — lives in a single configuration file, so the whole system is defined by data, not by scattered magic numbers, and any run can be re-created by re-reading its config. Caching: feature extraction over a night is slow, so cache the computed feature matrix keyed by the config and the data version; the second run is instant, and — importantly — the cache is *invalidated* when the config changes, so you never accidentally evaluate stale features. Unit tests: each module gets a small test — the filter attenuates a tone at the notch frequency, the feature extractor returns the analytically known band power of a pure sinusoid, the LOSO splitter never lets a subject appear in both train and test. These tests are your seatbelt during the iterative build: a change that silently breaks a stage is caught by a failing test, not by a mysterious drop in kappa three iterations later. Version control: every iteration is a commit with a message saying what changed and why (§16.3), so the project's history *is* the design log. Pinned environments: record exact library versions (a lock file or environment file), because a scorer that gives $\kappa = 0.76$ today and $\kappa = 0.71$ after a silent library upgrade is not reproducible. Fix the random seed and pin the versions, and the number becomes a fact rather than an anecdote.

16.7 Teamwork

A capstone is usually built by a team, and collaboration is an assessed outcome, not a logistical detail. The pipeline's modularity (§16.2) is what makes teamwork tractable: divide by module. One member owns preprocessing, another features, another modelling, another integration and reporting — each working behind a stable interface so the parts fit when they meet. The workflow is agile: short iterations (§16.3) with a shared board of tasks, frequent integration so the parts stay compatible, and a brief regular check-in rather than a frantic merge at the end. Code review is where quality is enforced and knowledge spreads: no change reaches the shared pipeline without a teammate reading it, which catches leakage bugs (a scaler fit outside the fold), style drift, and untested edges before they compound. And honest disclosure of individual contributions is a matter of integrity: a team report should state, plainly and per-person, who did what — not to apportion blame but because being able to explain and defend *your own* contribution is the point of the exercise. A team that divides cleanly, integrates

varkonyibarnabas@gmail.com

often, reviews each other's code, and reports its contributions honestly will out-build a more talented team that does none of these.

16.8 Communicating results

The final deliverable is rarely the code — it is the report and the defence. A technical report has a predictable, teachable structure, and following it is most of the battle: an *introduction* that states the problem and why it matters; a *methods* section that describes the pipeline precisely enough to reproduce (the seven modules, the features, the validation); a *results* section led by the honest LOSO kappa, the confusion matrix, and a hypnogram, with the held-out number reported once; a *discussion* that reads the errors (where does it confuse N1?), names the limitations (study-scale, not clinical), and traces the design decisions of §16.4; and a *conclusion* with concrete next steps. Lead with the confusion matrix and kappa, never bare accuracy (Chapter 12). Presenting is the same story compressed: the problem, the pipeline diagram (Figure 16.1), the rising-kappa curve (Figure 16.2b), one hypnogram, and the honest final number — in that order, because it is the order that persuades. And defending individual work means being ready to answer *why* you chose this feature, this model, this validation — which you can, because you wrote the reasons down as you went (§16.4). A result you can explain is worth more than a slightly higher one you cannot.

16.9 A worked walk-through: one full iteration

To make the whole thing concrete, trace a single iteration end to end. Start from a night's EDF and its expert labels; `load_epochs` returns roughly 960 thirty-second epochs, each with its stage. `preprocess` band-passes the EEG to 0.5–40 Hz (the §9.6 case-study spec, wide enough to keep the 16–30 Hz beta band intact), notches the mains, and re-references. `extract_features` computes, per epoch, the delta/theta/alpha/sigma/beta band powers (sigma is the spindle band that flags N2, §16.4), their relative fractions, Hjorth mobility and complexity, spectral entropy, and an EOG movement index — a feature vector of a few dozen numbers, in line with Chapter 11's pipeline. The LOSO harness now holds out one sleeper, fits the scaler and feature selector on the *remaining* sleepers only, trains a random forest with class weights, and scores the held-out night. Rotating through every sleeper and averaging gives, on this simulated example, an illustrative $\kappa \approx 0.61$ with a spread across subjects (again a shape-of-a-result figure, per §16.3, not a benchmark). report renders the held-out night's predicted hypnogram against its reference (Figure 16.2a), prints the confusion matrix (N1 mostly lost to N2 and wake), and writes the results paragraph:

varkonyibarnabas@gmail.com

> *Under leave-one-subject-out validation the tuned random forest reached a mean Cohen's κ of 0.61 (range 0.34–0.73 across subjects) with a macro-F1 of 0.64. The confusion matrix shows the expected weakness on N1 — frequently scored as N2, and to a lesser extent REM or wake — while N3 and REM are recovered reliably. The whole-night hypnogram tracks the expert's architecture, and Bland–Altman analysis of the clinical summaries (total sleep time, sleep efficiency, REM latency, WASO) shows a small positive bias in total sleep time (a couple of minutes) with limits of agreement of roughly [−8, +12] minutes and no large systematic bias in the others. On a held-out subject not used to tune the model, κ was 0.61, indicating the estimate generalises. Read against inter-scorer agreement — experts themselves disagree on a meaningful share of epochs — this is near what a single-rater target allows. These are illustrative figures on a simulated example; this is a study-scale result and carries no clinical claim.*

That paragraph — honest, specific, and traceable — is the shape of a capstone result. It reports the right metric under the right validation, reads its own errors, quantifies its clinical agreement, confirms generalisation on truly unseen data, and states its own limits.

16.10 Companion notebook

The companion notebook is a minimal end-to-end run of exactly this pipeline: it loads a short synthetic night, preprocesses and extracts features, runs the LOSO harness, and produces the metrics, the confusion matrix, a predicted-versus-reference hypnogram, and a predictions file — mirroring the seven modules of Figure 16.1 in runnable form. It then walks the iteration ladder of §16.3, swapping k-NN for an SVM for a random forest with a single config change, and prints the kappa rising each round so the reader can reproduce Figure 16.2(b) on their own machine.

16.11 Summary, questions, and further reading

Key ideas. Build the sleep scorer as seven reusable modules — loading, preprocessing, features, selection, classification, inference, reporting — on a shared rail of config, cache, tests, and version control. Build iteratively: a simple k-NN baseline first, earning complexity only where evidence justifies it, so agreement rises each round and you always have a working system. Justify and record every design decision so it is traceable. Evaluate with subject-independent LOSO, leading with Cohen's kappa (chance-corrected but not immune to prevalence) alongside macro-F1 and the confusion matrix, checking the clinical summaries — TST, sleep efficiency, REM latency, WASO, hypnogram agreement — with Bland–Altman, and reporting a held-out number once; read all agreement against inter-scorer agreement (label-reliability context, not a universal ceiling), since labels are measurements too (Chapter 12). Make the whole thing reproducible — config-driven, cached, unit-tested, version-controlled, pinned. Collaborate by dividing along module boundaries, integrating often, reviewing code, and disclosing contributions honestly. Communicate in a structured report and a defence you can give because you wrote your reasons down. (As flagged throughout, the κ figures here are illustrative simulated values, not benchmarks or a clinical claim — see §16.3.)

Key formulas. Cohen's kappa $\kappa = (p_o - p_e)/(1 - p_e)$; macro-F1 = unweighted mean of per-class F1; Bland–Altman limits of agreement $= \bar{d} \pm 1.96\, s_d$; LOSO estimate = mean over subjects of held-out-subject performance.

varkonyibarnabas@gmail.com

Conceptual questions. (1) Why build the pipeline as separate modules rather than one script — name two concrete payoffs. (2) Why start an iterative build with a deliberately weak k-NN baseline instead of the best model you can imagine? (3) A teammate reports $\kappa = 0.85$ but split epochs randomly — what is wrong and what is the true number likely to be? (4) You post a good per-epoch kappa but the total-sleep-time estimate is 40 minutes off — how can both be true, and which plot would reveal it? (5) Why must the held-out set be scored only once, and what does version control add to reproducibility?

Exploration tasks. Assemble the seven-module pipeline and run one full iteration to a hypnogram and a kappa under LOSO. Walk the k-NN $\rightarrow$ SVM $\rightarrow$ RF $\rightarrow$ tuned-RF ladder with a single config change per rung and plot the rising kappa. Add a unit test that fails if a subject appears in both train and test. Compute total sleep time from the predicted and reference hypnograms and draw a Bland–Altman plot. Write the one-paragraph results statement for your run, leading with kappa and the confusion matrix, and finish by stating the study-scale limitation explicitly.

Design brief (capstone-scale). You are handed a spec: a sleep-staging system for a *pediatric* ward, single EEG + EOG channels, a report needed each morning, labels scored by one technician, and a hard rule that the report must not over-state confidence. Produce a one-page design across the whole pipeline — acquisition band, preprocessing, features, learner, validation scheme, and reported metrics — then, for each of the three most consequential choices, write two sentences of motivation and name the alternative you rejected together with the spec change that would reverse the decision. Defend it as you would at the oral defence: the grade is the quality of the trade-off reasoning, not a single "correct" pipeline. Rule out: for one stage of your pipeline, name a choice that would be outright *wrong* for this pediatric, confidence-bounded spec — not merely suboptimal — and the requirement it violates.

Further reading. For the sleep-scoring task and its metrics, the AASM scoring manual and the sleep-staging literature (including reports of automatic-staging kappa on public datasets). For honest evaluation, revisit Chapter 12 and Cohen's 1960 paper on kappa; for architecture-agreement analysis, Bland & Altman's method-comparison paper. For the engineering, Huyen's *Designing Machine Learning Systems* (pipelines, reproducibility, deployment) and general software-engineering-for-ML practice on testing, configuration, and version control. For the collaboration and reporting skills, any solid guide to technical writing and agile teamwork — the specifics matter less than the habits of dividing cleanly, integrating often, and defending what you built.

varkonyibarnabas@gmail.com

Appendix A — Getting Started with Python & MATLAB

The main chapters lead with intuition, but the intuition sticks only when you run the code and watch a figure move. This appendix gets you to that point: a working environment, a mental model of how a biosignal enters that environment, and a small bilingual cheat-sheet so you are never stranded whichever language your track uses.

A.1 An environment in five minutes

You do not need to install anything to begin. A hosted notebook service — the kind that runs Python in your browser with no local setup — is the fastest route in, and it is where the companion notebooks are designed to run. Open a notebook, run the first cell, and you have NumPy, SciPy, Matplotlib, and scikit-learn already available; the deep-learning chapters add PyTorch. If you prefer to work locally, install a scientific Python distribution and create an isolated environment so your book work cannot break your other projects:

```
python3 -m venv bsp-env
source bsp-env/bin/activate        # Windows: bsp-env\Scripts\activate
pip install numpy scipy matplotlib scikit-learn pywavelets mne
```

Pin your versions. A notebook that "worked last term" and mysteriously fails today has almost always been broken by a silent upgrade of a dependency. Every companion notebook therefore begins with a requirements cell that names exact versions, and you should adopt the same habit in your own work.

MATLAB users are equally well served: the Signal Processing Toolbox, the Wavelet Toolbox, and the Statistics and Machine Learning Toolbox cover most of what the core chapters need (a few methods reach into the Econometrics or DSP System Toolbox — Appendix H's table flags which), and that table points you to the matching function for every method in the book.

A.2 How a biosignal gets into memory

Biosignals arrive in a handful of standard container formats, and you rarely need to parse them by hand. The two you will meet most often are EDF (the European Data Format) and WFDB (the format PhysioNet uses for its waveform databases). Conceptually they store the same three things: the raw samples for one or more channels, the sampling rate for each channel, and a header of metadata (channel names, physical units, patient-anonymised labels, and often event annotations such as sleep-stage scores or beat labels).

You do not read them with generic file tools; you use a purpose-built reader that hands you back the samples already scaled into physical units. In Python, the `pyedflib` and `mne` libraries read EDF, and the `wfdb` package reads PhysioNet records; in MATLAB, `edfread` and the WFDB Toolbox do the same. The mental model to carry is simple: a reader turns a file on disk into an

varkonyibarnabas@gmail.com

array of numbers plus a sampling rate. Everything else in the book — every transform, filter, and feature — operates on that array. Once a signal $x[n]$ and its rate f_s are in memory, the file format has done its job and can be forgotten.

A word on annotations. Many biosignal files carry expert labels aligned to the samples — the sleep stage of each thirty-second epoch, the location of each R-peak. These labels are the *reference labels* (or *expert-consensus labels*) your later classifiers will be trained and judged against — not ground truth handed down by nature but human judgements that carry their own inter-scorer variability (Chapter 12), so a model can rarely be asked to agree with them more closely than two experts agree with each other. Read them with the same care you read the signal: check that their time base matches the signal's, and that an "epoch 0" in the annotations means the same instant as sample 0 in the array.

A.3 The shared notebook header

Every companion notebook opens with the same header cell, and it is worth understanding because it encodes three good habits at once:

```
# 1) reproducibility: one seed for the whole notebook
import numpy as np
rng = np.random.default_rng(2013)

# 2) a dataset loader with an open-data fallback
def load_signal():
    try:
        return load_from_open_dataset()   # e.g. a small PhysioNet sample
    except Exception:
        return synthesize_demo_signal(rng) # deterministic synthetic fallback

# 3) a consistent plotting style so every figure looks the same
import matplotlib.pyplot as plt
plt.rcParams.update({"figure.dpi": 150, "axes.grid": True})
```

The seed makes the run deterministic: the same code produces the same figure every time, which is what lets the book state an expected number and lets you self-check. The fallback makes the notebook robust: if a download fails or you are offline, a synthetic signal stands in so the *concept* still runs. And the shared style makes the figures consistent, so a spectrogram in Chapter 3 and a spectrogram in Chapter 12 are directly comparable.

A.4 A Python ↔ MATLAB mini cheat-sheet

The two languages are close cousins for signal work. The single biggest trap is indexing: Python arrays start at index 0, MATLAB arrays start at 1. Keep that in mind and the rest transfers almost mechanically.

Task	Python (NumPy/SciPy)	MATLAB
Make a time vector	`t = np.arange(N)/fs`	`t = (0:N-1)/fs;`

varkonyibarnabas@gmail.com

Task	Python (NumPy/SciPy)	MATLAB
First / last sample	`x[0]`, `x[-1]`	`x(1)`, `x(end)`
FFT of a signal	`X = np.fft.fft(x)`	`X = fft(x);`
Design a Butterworth filter	`b,a = signal.butter(4, wc)`	`[b,a] = butter(4, wc);`
Apply a filter (zero-phase)	`y = signal.filtfilt(b,a,x)`	`y = filtfilt(b,a,x);`
Convolution	`y = np.convolve(x,h)`	`y = conv(x,h);`
Mean / standard deviation	`x.mean()`, `x.std(ddof=1)`	`mean(x)`, `std(x)`
Plot a signal	`plt.plot(t,x)`	`plot(t,x)`

If a snippet in the main text is Python, this table is usually enough to translate it; for a specific method, Appendix H's table (§H.1) gives the matching MATLAB call and what to verify, and where a translation is genuinely subtle (windowing conventions, one-sided versus two-sided spectra) it says so there.

---

varkonyibarnabas@gmail.com

Appendix B — Complex Numbers & Euler's Formula

Fourier analysis is the backbone of this book, and it is built on complex numbers. If complex numbers feel like an arbitrary piece of algebra, Fourier will feel arbitrary too. This appendix aims to make them feel *inevitable* instead — the natural language for describing something that rotates and oscillates, which is exactly what a signal's frequency content is.

B.1 A number with a direction

A complex number $z = a + jb$ pairs a real part a with an imaginary part b, where $j = \sqrt{-1}$ (engineers write j; mathematicians write i). The move that unlocks everything is to stop thinking of z as "two numbers glued together" and start thinking of it as a point in a plane — the *complex plane* — with the real part on the horizontal axis and the imaginary part on the vertical. A complex number is then an arrow from the origin to that point, and an arrow has two natural descriptions: its length and the angle it makes with the horizontal.

The length is the magnitude (or modulus),

$$|z| = \sqrt{a^2 + b^2},$$

and the angle is the phase (or argument),

$$\angle z = \arg(z) = \operatorname{atan2}(b, a).$$

These two quantities — magnitude and phase — are the whole point. In signal processing, magnitude tells you *how much* of a given frequency is present, and phase tells you *when* it peaks (its timing, or alignment). Every spectrum in this book is, underneath, a collection of magnitudes and phases.

B.2 Euler's formula: the bridge to rotation

The single most useful fact about complex numbers, and the reason they and trigonometry are secretly the same subject, is Euler's formula:

$$e^{j\theta} = \cos\theta + j\sin\theta.$$

Read the right-hand side geometrically: $\cos\theta$ is the horizontal coordinate and $\sin\theta$ the vertical coordinate of a point on the unit circle at angle θ. So $e^{j\theta}$ is simply the point on the unit

varkonyibarnabas@gmail.com

circle at angle θ — a complex number of magnitude 1 and phase θ. If we now let the angle grow with time, $\theta = 2\pi f t$, the number

$$e^{j2\pi ft} = \cos(2\pi ft) + j\sin(2\pi ft)$$

is a point marching around the unit circle f times per second. That is a phasor: a rotating arrow whose rotation rate *is* the frequency. This is the deepest reason Fourier analysis prefers complex exponentials to sines and cosines. A pure sinusoid is what you get if you stand to one side and watch the phasor's shadow on one axis; the phasor itself carries both the cosine and the sine — both the magnitude and the phase — in a single compact object. Working with the rotating arrow is algebraically far cleaner than juggling two shadows.

A general phasor with amplitude A and starting phase ϕ is $Ae^{j\phi}e^{j2\pi ft}$, and its projection onto the real axis, $A\cos(2\pi ft+\phi)$, is the physical oscillation we actually measure. Multiplying two phasors adds their angles and multiplies their lengths — which is why a filter that "delays a frequency" is, in this language, just a multiplication by a phasor of magnitude 1 and some phase.

B.3 Why the Fourier transform needs them

With Euler's formula in hand, the Fourier transform stops being mysterious. The transform asks a single question of a signal: *how much of the rotating arrow $e^{j2\pi ft}$ is present at each frequency f?* It answers by multiplying the signal by $e^{-j2\pi ft}$ (a counter-rotation) and averaging; where the signal contains that frequency, the counter-rotation freezes it into a steady, non-zero average, and where it does not, the rotation averages to zero. The result at each f is a complex number $X(f)$ — and its magnitude $|X(f)|$ and phase $\angle X(f)$ are exactly the amount and timing of that frequency in the signal. Complex numbers are not a mathematical detour on the way to Fourier; they *are* the coordinate system in which frequency lives. Chapter 3 builds the transform in full on this foundation.

---

varkonyibarnabas@gmail.com

Appendix C — Convolution, Step by Step

Convolution is the operation behind every filter in this book, and it is also — perhaps surprisingly — behind the feature maps of the convolutional neural networks in Chapter 14. It deserves a picture, because the algebra hides how simple it really is. The picture has a name: flip and slide.

C.1 The flip-and-slide picture in one dimension

Suppose you have a signal $x[n]$ and a short kernel (also called an impulse response or filter) $h[n]$. Their convolution is a new signal $y[n]$ defined by

$$y[n] = (x * h)[n] = \sum_{k} x[k]\, h[n-k].$$

The sum looks abstract, but every symbol earns its place once you draw it. The term $h[n-k]$, read as a function of k, is the kernel flipped (because of the minus sign in front of k) and then shifted so that it sits at position n. To compute one output sample $y[n]$, you lay this flipped-and-shifted kernel over the signal, multiply each kernel tap by the signal sample beneath it, and add up the products. To compute the whole output, you slide the flipped kernel one step at a time across the signal and repeat.

That is the entire operation: flip the kernel once, then slide it, and at each position sum the overlapping products. Figure C.1 makes this concrete for a short three-tap averaging kernel sliding over a small signal.

Notice two things in the figure. First, because the kernel here is a moving average, each output sample is a local mean of the input, so sharp features are smoothed — this is exactly what a low-pass filter does, and it is why convolution and filtering are two names for the same act. Second, the output is *longer* than the input: a signal of length N convolved with a kernel of length M produces $N+M-1$ samples, because the kernel overhangs both ends as it slides in and out. In practice we often trim or pad back to the original length; Chapter 9 (filter design) explains the choices.

A useful sanity check: convolving any signal with a single unit spike (an impulse, $h = [1]$) returns the signal unchanged. That is why $h[n]$ is called the impulse response — it is literally what the system does to a single impulse, and by the flip-and-slide logic, that response is all you need to know how the system treats *any* input.

C.2 The same picture in two dimensions

Nothing about flip-and-slide is special to one dimension. For an image, the kernel becomes a small grid — a 3×3 or 5×5 patch of weights — and convolution slides that patch over every pixel, multiplying and summing at each location:

varkonyibarnabas@gmail.com

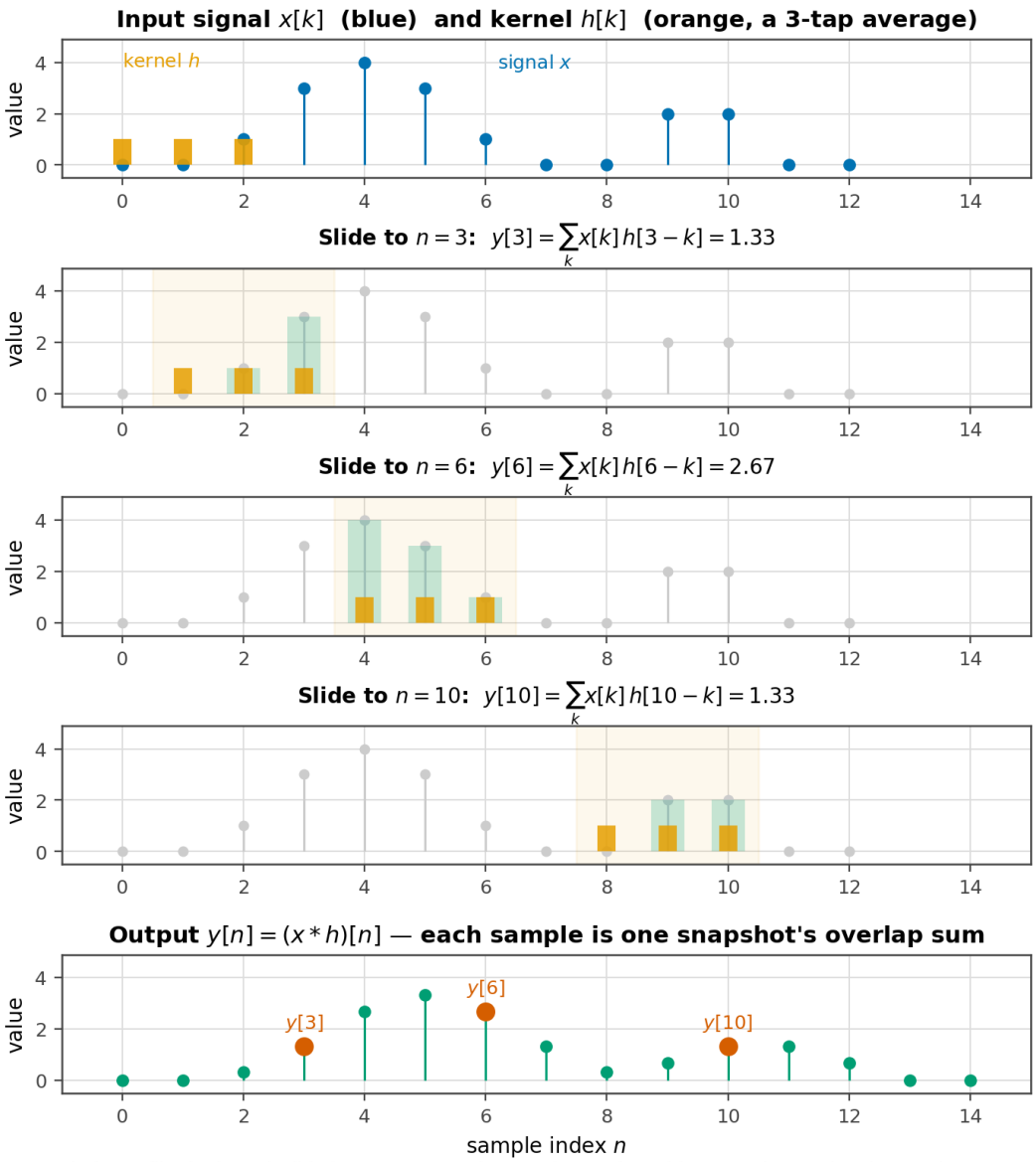


Figure C.1. Convolution as flip-and-slide. The kernel is reversed once, then shifted one sample at a time across the signal; at each position the overlapping products are summed to give one output sample. A smoothing kernel blurs the signal, and the same picture scales to 2-D image filters and CNN feature maps.

varkonyibarnabas@gmail.com

$$y[m, n] = \sum_i \sum_j x[i, j]\, h[m - i,\, n - j].$$

The mechanics are identical; there are simply two directions to slide in. A 3×3 averaging kernel blurs an image exactly as the three-tap kernel smoothed the signal above. Change the weights and you change the effect: a kernel with a positive centre and negative surround sharpens edges; an asymmetric kernel that differences neighbouring pixels detects gradients in a particular direction. This is the direct route from one-dimensional filtering to image filtering, and Chapter 9 develops it with worked examples.

C.3 The link to CNN kernels

Here is the payoff that ties the appendix to the modern-AI chapters. A convolutional neural network is, at heart, a stack of exactly these sliding kernels — except that the kernel weights are not designed by hand but learned from data. Where a classical engineer chooses a 3×3 edge detector, a CNN starts with random 3×3 kernels and adjusts their weights during training until they respond to whatever patterns help the network's task. The flip-and-slide operation is the same; the source of the weights is what differs. (A technical footnote: most deep-learning frameworks implement *cross-correlation* — the slide without the flip — and simply call it convolution. Because the weights are learned, the missing flip makes no practical difference; the network learns whatever orientation it needs.) Once you see a CNN as "convolution with learned kernels," the leap from hand-designed filters to learned features stops being a leap at all — it is the same picture, with the weights set free.

---

varkonyibarnabas@gmail.com

Appendix D — Correlation & Covariance

Where convolution *combines* two signals, correlation *compares* them. It is the tool for answering "how similar are these two signals?" and "is there a repeating structure hiding in this one?" — questions at the heart of spectral estimation and of timing tasks such as aligning a fetal heart-rate trace with uterine contractions.

D.1 Covariance: do two variables move together?

Start with two random variables X and Y. Their covariance measures whether they tend to rise and fall together:

$$\operatorname{cov}(X, Y) = \mathbb{E}\big[(X - \mu_X)(Y - \mu_Y)\big].$$

If large values of X coincide with large values of Y, the product of their deviations is usually positive and the covariance is positive; if one tends to be large when the other is small, the covariance is negative; if they are unrelated, it hovers around zero. Covariance has awkward units (the product of the two variables' units), so we usually normalise it into the correlation coefficient,

$$\rho_{XY} = \frac{\operatorname{cov}(X, Y)}{\sigma_X \, \sigma_Y}, \qquad -1 \le \rho_{XY} \le 1,$$

a pure number where $+1$ is perfect positive linear agreement, -1 perfect opposition, and 0 no linear relationship. Correlation measures *linear* association only — two variables can be strongly related through a curve and still have $\rho \approx 0$ — a caveat worth remembering before trusting a single number.

D.2 Cross-correlation: comparing signals across a lag

Signals live in time, which adds a crucial twist: two signals may be similar but *shifted*. The cross-correlation slides one signal past the other and reports the similarity at every relative lag ℓ:

$$r_{xy}[\ell] = \sum_{n} x[n] \, y[n + \ell].$$

The lag at which $r_{xy}[\ell]$ peaks is the time shift that best aligns the two signals — which is precisely how one measures a delay. This is the machinery behind estimating the time offset between a stimulus and a response, or between a uterine contraction and the fetal heart-rate deceleration that follows it: compute the cross-correlation, find the peak, read off the

varkonyibarnabas@gmail.com

lag. (The resemblance to convolution is not a coincidence — cross-correlation is convolution *without the flip*, which is why the two appendices are neighbours.) One convention note: `scipy.signal.correlate(x, y)` and MATLAB's `xcorr(x, y)` compute $\sum_n x[n+\ell]\,y[n]$ — the transpose of $r_{xy}[\ell]$ above, i.e. this appendix's $r_{yx}[\ell]$ — so if you read a lag straight off one of those functions per this section, the sign of the delay flips; check which signal your tool calls first.

> A note on scale. Unlike the normalised correlation coefficient ρ of §D.1, the raw cross-correlation sum $r_{xy}[\ell]$ (and the autocorrelation $r_{xx}[\ell]$ of §D.3) is not bounded to $[-1, 1]$: it scales with the signals' energy and with how much they overlap at each lag, so its magnitude alone means little. When you need a bounded, comparable measure — to say *how* similar two signals are, not just *where* they align — use the normalised cross-correlation, dividing $r_{xy}[\ell]$ by the *square root of the product of the two signals' energies* — by the Cauchy–Schwarz inequality this, and this specifically, is what bounds it to $[-1, 1]$ (for a strictly lag-wise bound, use the energies computed over the overlapping region at each lag).

D.3 Autocorrelation: a signal compared with itself

The most powerful special case sets $y = x$: the autocorrelation compares a signal with a delayed copy of itself,

$$r_{xx}[\ell] = \sum_n x[n]\,x[n+\ell].$$

At zero lag it is just the signal's energy. But at non-zero lags it reveals periodicity: if a signal repeats every P samples, then shifting it by P lines the copy back up with the original, and the autocorrelation shows a peak at lag $\ell = P$ (and at its multiples). This is how a pitch is estimated, how a heart rate can be found from an ECG without detecting individual beats, and — most importantly for this book — it is the direct bridge to the frequency domain. The Fourier transform of *this* (deterministic, energy) autocorrelation is the signal's energy spectral density $|X(f)|^2$. Chapters 6 and 7 use the random-process analogue: for a wide-sense-stationary process, the same Fourier-transform relationship holds between the *expectation*-based autocorrelation $R_X[k] = \mathbb{E}\{X[n]X[n+k]\}$ and its power spectral density — this is the Wiener–Khinchin relation proper — which is why Chapters 6 and 7 lean on autocorrelation so heavily: to characterise the frequency content of a random signal, you first characterise how it correlates with itself across time.

---

varkonyibarnabas@gmail.com

Appendix E — Probability & Statistics Refresher

Real biosignals are partly random, and the machine-learning models we build on them are statistical objects. This appendix collects the handful of probability ideas the book relies on, aiming for working intuition rather than measure-theoretic rigour.

E.1 Random variables and distributions

A random variable is a quantity whose value is uncertain, described not by a single number but by a distribution that says how probable each outcome is. Two distributions do most of the work in this book. The Gaussian (or normal) distribution, with probability density

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\!\left(-\frac{(x-\mu)^2}{2\sigma^2}\right),$$

is the familiar bell curve, parameterised by its mean μ (where it is centred) and its standard deviation σ (how wide it is). It is the default model for measurement noise, and for a deep reason: the central limit theorem says that a sum of many small independent effects tends toward a Gaussian regardless of the individual effects' shapes — and electronic and thermal noise is exactly such a sum. The uniform distribution, flat over an interval, models quantisation error, the small rounding that occurs when a continuous voltage is snapped to the nearest digital level.

E.2 Expectation and variance

Two summaries capture most of what we need from a distribution. The expectation (or mean) is the long-run average value,

$$\mathbb{E}[X] = \mu,$$

the balance point of the distribution. The variance measures spread — the average squared distance from the mean —

$$\mathrm{Var}(X) = \mathbb{E}\big[(X-\mu)^2\big] = \sigma^2,$$

and its square root σ, the standard deviation, is the spread expressed in the variable's own units, which makes it the natural measure of how "noisy" a signal is. A signal-to-noise ratio, encountered throughout the book, is at bottom a ratio of a signal's variance (its power) to the noise's variance.

varkonyibarnabas@gmail.com

Expectation is linear — $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$ always — which makes averaging easy to reason about. Variance is *not* linear in general, but it has one property that powers a central result of the book: for independent random variables, variances add. That single fact is the seed of the next section.

E.3 The $\sqrt{N}$ idea

Here is the most consequential piece of statistics in the whole book, and it follows directly from "variances of independent quantities add." Suppose you record the same event N times — N heartbeats, N evoked-response epochs — each a clean template s buried in independent noise of variance σ^2. Average the N records. The template, being identical every time, survives the average untouched. The noise, being independent from trial to trial, partly cancels: averaging N independent noise samples produces a result whose variance is σ^2/N, so its standard deviation shrinks by a factor of $\sqrt{N}$.

The signal is preserved while the noise amplitude falls as $1/\sqrt{N}$. The signal-to-noise ratio therefore improves as $\sqrt{N}$. Concretely, averaging $N = 100$ epochs buys a tenfold improvement in amplitude signal-to-noise ratio — about $20\,\mathrm{dB}$ — because $\sqrt{100} = 10$. This is why ensemble averaging pulls a clean cardiac template out of a noisy recording, and why a brain–computer interface repeats a stimulus dozens of times to lift a faint evoked response out of the ongoing EEG. It also sets the price of that gain: to *double* your signal-to-noise ratio you must *quadruple* the number of repetitions, a diminishing return that every experiment designer has to budget for. Whenever the book says "a law and a number," this is the archetype.

---

varkonyibarnabas@gmail.com

Appendix F — Datasets & Reproducibility

A method you cannot run on real data is a method you cannot trust, and a result you cannot reproduce is not yet a result. This appendix points to open biosignal data and lays out the practices that keep an analysis honest and repeatable.

F.1 Where to get open biosignal data

Two repositories cover almost everything the book touches. PhysioNet (physionet.org) is the largest open archive of physiological signals, hosting electrocardiograms (the classic MIT-BIH arrhythmia database and the large PTB-XL twelve-lead collection), electromyograms, cardiotocography (the CTU-UHB intrapartum database), and polysomnography with expert sleep-stage scores (the Sleep-EDF collection, the natural home of this book's case study). The UCI Machine Learning Repository hosts tabular and sensor datasets, including a Cardiotocography dataset and the Human Activity Recognition dataset of smartphone inertial-measurement-unit signals used for the accelerometry examples. Between them these two sources provide open ECG, EMG, CTG, IMU, and EEG data suitable for every demonstration in the book.

The guiding rule when choosing data for something a reader will run independently is: prefer openly licensed, small, and citable datasets. Openly licensed so anyone can obtain it; small so it runs in a hosted notebook in minutes rather than hours; citable so the work can be traced. Larger or access-controlled datasets have their place in research, but for a book companion the small open sample wins every time.

F.2 How to cite a dataset

A dataset is a scholarly artifact and is cited like one. Record enough that a reader could obtain exactly what you used: the dataset's name and version, its creators or contributing publication, the repository and access URL, and the license. PhysioNet additionally asks that you cite both the specific database *and* the PhysioNet platform paper, and many datasets specify their preferred citation on their landing page — use it verbatim. Getting this right is not bureaucracy; it is what lets someone else check, extend, or challenge your result.

F.3 Reproducibility practices

Reproducibility is a set of small disciplines, each cheap on its own and decisive together. Fix the random seed at the top of every analysis, so stochastic steps — a train/test split, a weight initialisation, a data augmentation — produce identical results on every run. Pin your dependencies to exact versions, because an unpinned library is a silent time-bomb. Separate data from code, loading through a documented function rather than hard-coding paths, so the analysis moves cleanly between machines. Provide a fallback, so that if a download fails a

varkonyibarnabas@gmail.com

synthetic or cached sample keeps the demonstration alive. And state expected outputs — a figure, a number, a metric — so a reader can confirm their run matches yours. One of the most common failures of reproducibility in this field, worth its own warning, is data leakage: letting information from the test set influence training, most often by splitting epochs from the same subject across the train and test sets. A subtler cousin is benchmark contamination — the same recording or public benchmark appearing in both a model's training data and its evaluation, which is easy to miss when a pretrained model (Chapter 14) has ingested public corpora. The remedy is subject-independent validation with test data provably held out of training, and Chapter 12 treats it in full.

The table below is the template for a dataset-governance record — one row per dataset, maintained alongside the code — that captures at a glance everything the practices above require.

Field	What it records	Example
Name / version	The dataset and its release	Sleep-EDF Expanded, v1.0.0
Used in	Which chapters or demos rely on it	Case study; Ch. 1, 3, 12
License / access	Terms and how to obtain it	Open (ODC-BY); direct download
License-use fit	Does the license cover *your* use?	Research/teaching OK; commercial: check
Privacy / de-identification	How subjects are protected	De-identified; no re-identification attempts
Approx. size	Storage and record count	~8 GB; 197 whole-night recordings
Notebook feasibility	Fits a hosted notebook?	Yes, via a small sampled subset
Held out of pretraining?	Guards against benchmark overlap	Test split excluded from any pretraining
Citation	The preferred, verbatim citation	Dataset paper + platform paper

Kept current, this single table answers "where did this come from, may I use it, is it clean of contamination, will it run, and how do I cite it?" for every signal in the book — and turns reproducibility from an aspiration into a checklist.

F.4 Data governance and licensing

Reproducibility answers *can I run it again?*; governance answers *am I allowed to, and is the result honest?* Four disciplines cover most of it.

Dataset versioning and citation. Datasets are revised — records added, labels corrected, formats changed — so pin the exact version you used and cite it with that version, as §F.2 describes. A result reported against "the Sleep-EDF database" without a version is hard to reproduce once the database moves on; a result reported against a named version, with its citation, can be checked years later.

License compatibility. Every open dataset carries a license, and openly *available* is not the same as openly *usable for any purpose*. A license may permit research and teaching but not commercial deployment; it may require attribution, forbid redistribution, or restrict re-identification attempts. Read each dataset's license and confirm it covers *your* intended use — and when you combine datasets, check that their terms are mutually compatible, because the obligations of a mixed corpus are the union of its parts. When a dataset feeds a

varkonyibarnabas@gmail.com

model that might one day ship (Chapter 15), the license question is a deployment question, not a footnote.

Patient privacy and de-identification. Physiological recordings are sensitive personal data even after names are removed. De-identification reduces re-identification risk but does not eliminate it: physiological signals can themselves be distinctive, and combining a "de-identified" record with outside information can sometimes re-identify a subject. Respect the consent under which data were collected, do not attempt re-identification, and treat even public biosignal data as carrying a duty of care — the same caution the deployment chapter applies to features leaving a device.

Benchmark contamination and train–test overlap. A performance number is only meaningful if the data it was measured on were genuinely unseen during training. The classic error is splitting a single subject's epochs across train and test (§F.3); the modern one is benchmark contamination, where a public test set has leaked into a large pretraining corpus (Chapter 14) so a model has effectively *memorised* the answers. Guard against both by keeping the unit of splitting matched to the deployment claim and by confirming — as far as a model's provenance allows — that your evaluation data were held out of every stage of training, pretraining included. When you cannot verify a pretrained model's corpus, say so, and treat its headline benchmark with corresponding caution.

Together with the reproducibility practices of §F.3, these turn a dataset from an anonymous file into an accountable, citable, license-clean, and contamination-checked component of the work.

---

varkonyibarnabas@gmail.com

Appendix G — A Task-to-Tool Atlas

This atlas is a quick-reference map from a *need* to a sensible *starting point*, a *next thing to try*, and — most importantly — *what to check before trusting the result*. It is deliberately compact: a starting point, not a decision procedure, and never a substitute for the reasoning each chapter teaches. Read it as "if this is the shape of my problem, begin here, and do not skip the last column." The task vocabulary (representation, denoising, detection, estimation, segmentation, classification, prediction, control) is defined in §1.8.

G.1 Need → tool → cautions

Need	Good starting point	Consider next	Check before trusting
Overall rhythmic content	FFT / Welch PSD (Ch 3, 7)	multitaper or AR spectrum (Ch 7)	stationarity, leakage, artifacts, record duration
Rhythm changing over time	STFT / spectrogram (Ch 3)	wavelets or Hilbert–Huang (Ch 5)	window/scale sensitivity; time–frequency resolution trade
Repeated event in noise	ensemble averaging (Ch 10)	matched template or supervised detector	alignment, trial independence, latency variability
Correlated reference artifact	regression / LMS (Ch 10)	RLS or Kalman	the reference must not contain the desired source
Multichannel mixture	PCA / ICA (Ch 10, 13)	spatial filtering or source models	component stability and physiological plausibility
Impulsive artifact	median / robust rejection (Ch 9)	segment repair or exclusion	narrow *real* events must not be deleted
Baseline drift	detrending / high-pass (Ch 8, 9)	model-based correction	low-frequency physiology and morphology (e.g. ST segment)
Event timing	peak / template detector (Ch 9)	learned detector (Ch 14)	tolerance window, refractory logic, false-alarm rate
Small labelled dataset	interpretable baseline (Ch 13)	regularised models, ensembles, SVM	subject-level split, calibration, class imbalance
Large unlabelled corpus	self-supervised learning (Ch 14)	foundation-model transfer	domain overlap, pretraining leakage, compute cost
Missing samples	preserve a mask; task-specific handling (Ch 12)	interpolation or an explicit missingness model	gap duration, missingness mechanism, added uncertainty
Multiple devices / channels	timestamp alignment + resampling (Ch 2)	drift correction or event-based alignment	clock drift, latency, unit consistency

varkonyibarnabas@gmail.com

G.2 Output type → how to evaluate it

The right metric is set by *what the method outputs*, not by habit. Evaluating one output type with another's metric is a common source of over-optimistic numbers (Chapter 12).

Output	Appropriate evaluation examples	Common mistake
Event times	sensitivity, PPV, timing error, false detections per hour	scoring every sample as independent
Continuous estimate	bias, MAE / RMSE, limits of agreement, calibration	correlation alone
Segmentation	overlap, boundary error, state-duration error	ignoring duration errors
Class probability	discrimination (AUC), calibration, decision utility	accuracy alone
Clinical summary	subject-level or episode-level error and uncertainty	pooling all epochs across subjects

G.3 How to use this atlas

1. Name the task (§1.8) and the claim (Chapter 12) first — they decide the columns that matter.
2. Start at the leftmost tool that fits; only move right when a checked limitation forces it.
3. Treat the last column as mandatory: a result you cannot defend against those checks is a candidate, not a conclusion.

varkonyibarnabas@gmail.com

Appendix H — Python Implementation Map

Choose and verify the method first; learn the function call second. This appendix maps each technique the book teaches to the Python call it uses (or, where the chapter only discusses the technique, recommends), the parameter that carries the concept, how to check the result, and the plausible-but-wrong use to rule out. It is a reference, not a tutorial: the worked code lives in the chapter notebooks (`figures_src/chNN_figures.py`), and the concept-level *selection* guide is the Task-to-Tool Atlas (Appendix G). Array conventions and environment setup are in Appendix A. Each row names the tool the book actually uses in its notebooks or, where a technique is only *discussed* (e.g. resampling, ICA), the recommended Python starting point.

The stack is exactly the pinned environment — NumPy, SciPy, scikit-learn, PyWavelets, Matplotlib (plus Pillow for images) — and no other library is required. Each row answers five questions: which tool, which parameter *is* the concept, which assumption must hold, how you know it worked, and which plausible mistake invalidates it. That last column is the Rule out move of §1.8 — "no single best method" is never "anything goes."

H.1 The master map

Read each row as: technique (chapter) → the Python call as the notebooks make it → MATLAB equivalent → the parameter that carries the concept → how to verify, and what to rule out.

Acquire, inspect, resample

Technique (ch)	Python (as used)	MATLAB	Concept in the parameters	Verify / rule out
Time vector (2)	`np.arange(n)/fs`	`(0:n-1)/fs`	`fs` is the sampling rate	verify duration = n/fs; avoid an implicit `fs`
Quality screen (8)	`np.isfinite`, `np.std`, `np.diff`	`isfinite`, `std`, `diff`	steps, clipping, dropouts	verify caught before filtering; avoid "just filter it"
Alias / quantization (2)	`bsp.alias_frequency`, `bsp.quantization_snr_db`	—	Nyquist; SNR = 6.02N + 1.76	verify measured SNR near the formula; avoid reconstructing at the cell edge (half-LSB bias)
Resample (2)	`signal.resample_poly` / `decimate`	`resample` / `decimate`	integer up/down ratio; anti-alias filter	verify no alias in the spectrum; avoid naive sample-dropping
Interpolate gaps (2)	`np.interp`	`interp1`	is piecewise-linear enough?	verify gaps are marked; avoid cubic overshoot near sparse points

Fourier and spectral estimation

varkonyibarnabas@gmail.com

Technique (ch)	Python (as used)	MATLAB	Concept in the parameters	Verify / rule out
FFT (3)	`np.fft.rfft`, `rfftfreq`	`fft`	resolution df = fs/N; one-sided scaling	verify the grid and units; avoid reading zero-padding as new information
Window (4)	`np.hanning`, `np.blackman`, `signal.windows.get_window`	`hann`, `window`	main lobe sets resolution, side lobe sets leakage	verify coherent gain and ENBW; avoid a rectangular window burying a weak neighbour
Periodogram (7)	`signal.periodogram`	`periodogram`	one-sided white mean = $2\sigma^2/fs$	verify variance does not fall with N; avoid reading variance as peaks
Welch PSD (7)	`signal.welch`	`pwelch`	`nperseg` is segment duration; `scaling`	verify band power via `bsp.bandpower`; avoid too-*short* `nperseg` (merges nearby peaks); too-long cuts the averages and can break stationarity
Multitaper (7)	`signal.windows.dpss` + averaging	`pmtm`	time-bandwidth NW, K tapers	verify Welch and multitaper agree; avoid calling a one-taper estimate multitaper — report NW and the taper count
AR spectrum (6, 7)	`bsp.ar_psd`	`pburg`, `pyulear`	PSD = sigma-sq times H-sq over fs	verify the peak sits at the rhythm; avoid the reciprocal (which inverts it)
Coherence (6)	`signal.coherence`, `csd`	`mscohere`, `cpsd`	`fs`, `nperseg`, `noverlap`	verify against a known pair; avoid volume conduction read as causation

Time-frequency and transforms

Technique (ch)	Python (as used)	MATLAB	Concept in the parameters	Verify / rule out
Spectrogram / STFT (3, 5)	`scipy.signal.spectrogram`	`stft`	window sets time-frequency resolution	verify resolution matches the window; avoid a too-long window that loses a transient
Analytic signal (5)	`scipy.signal.hilbert`	`hilbert`	must band-limit first	verify envelope and phase are sensible; avoid instantaneous frequency on a broadband signal
DWT + threshold (5, 10)	`pywt.wavedec`, `waverec`, `threshold`	`wavedec`	wavelet, level, extension mode	verify reconstruction error near zero; avoid leaving the boundary mode at its default

varkonyibarnabas@gmail.com

Technique (ch)	Python (as used)	MATLAB	Concept in the parameters	Verify / rule out
CWT (5)	`pywt.cwt`	`cwt`	scale-to-frequency mapping	verify the scale-frequency map; avoid reading the edge cone as signal
DCT (5)	`scipy.fft.dct`, `idct`	`dct`	energy compaction	verify reconstruction and coefficient accounting; avoid dropping coefficients blindly

Filters (design is not application)

Technique (ch)	Python (as used)	MATLAB	Concept in the parameters	Verify / rule out
IIR design (9)	`signal.butter`, `cheby1`, `ellip`	`butter`, `cheby1`, `ellip`	order and cutoff set the preserved band	verify by plotting `freqz` and marking the passband; avoid designing without plotting the response
Notch (9)	`signal.iirnotch`	`iirnotch` (DSP System Toolbox)	Q sets bandwidth, not depth	verify nearby content is preserved; avoid a Q mismatched to the mains
Zero-phase apply (9)	`signal.filtfilt`	`filtfilt`	doubles attenuation; offline only	verify edge transients are excluded; avoid `filtfilt` in a real-time path
Median (9)	`signal.medfilt`, `ndimage.median_filter`	`medfilt1`	window must be narrower than the feature	verify the QRS or edges survive; avoid a window wider than the feature

Correlation, AR, denoising

Technique (ch)	Python (as used)	MATLAB	Concept in the parameters	Verify / rule out
Cross-correlation (6)	`signal.correlate` + explicit normaliser	`xcorr`	lag convention; Cauchy-Schwarz bound	verify it is bounded to plus/minus one; avoid reading the raw sum as a coefficient
ACF / PACF (6)	`np.correlate` (Ch 6) or `statsmodels.acf/pacf`	`autocorr`, `parcorr` (Econometrics Toolbox)	tails-off versus cuts-off	verify against the confidence band; avoid reading the biased taper as decay
Ensemble average (10)	`np.mean` over aligned epochs	`mean`	the sqrt(N) SNR law	verify SNR rises as sqrt(N); avoid averaging a one-off artifact

varkonyibarnabas@gmail.com

Technique (ch)	Python (as used)	MATLAB	Concept in the parameters	Verify / rule out
PCA / ICA (10, 13)	`sklearn PCA`, `FastICA`	`pca`, `rica` (Statistics & ML Toolbox)	fit inside the fold; number of components or sources	verify retained variance or component stability; avoid fitting on the whole set (leakage)

Features, machine learning, validation, evaluation

Technique (ch)	Python (as used)	MATLAB	Concept in the parameters	Verify / rule out
Feature table (11)	`bsp.epoch_features` (one epoch -> a feature dict), `feature_table` (all nights -> `X, y, groups, names`)	table build	`feature_table` keeps the subject grouping as `groups`	verify rows carry provenance; avoid dropping the group id
Learners (13)	`KNeighborsClassifier`, `SVC`, `RandomForestClassifier` (plus logistic, NB, tree)	`fitcknn`, `fitcsvm`, `TreeBagger`	scale sensitivity	verify against a baseline; avoid plain accuracy under imbalance
Leakage-safe pipeline (12)	`make_pipeline(scaler, clf)`	pipeline workflow	preprocessing fitted inside the fold	verify no leakage; avoid scaling before the split
Grouped CV (12)	`GroupKFold`, `LeaveOneGroupOut` + `cross_val_predict`	grouped partition	the group is the subject, not the epoch	verify with `assert_no_subject_leak`; avoid a random split
Metrics (12)	`cohen_kappa_score`, `calibration_curve` (+ `sklearn.metrics`)	`perfcurve`	kappa vs accuracy; PR under imbalance	verify with a bootstrap CI; avoid an ROC hiding the base rate
Clustering (13)	`sklearn KMeans`, `scipy linkage`	`kmeans`, `linkage`	k, or the linkage rule	verify by overlay on the true stages; avoid reading clusters as classes

H.2 The tested helpers (`bsp`)

The companion ships a small, tested helper package so the operations that are easy to misuse have one correct, documented home; chapters and notebooks call these rather than re-deriving them, and the two highest-risk helpers (`ar_psd`, `bandpower`) plus the package API are covered by a build-time check (`tests/test_appendix_helpers.py`), so those cannot silently drift.

Helper	What it guarantees
`bsp.ar_psd(a, sigma2, fs)`	the AR PSD as sigma-sq times H-sq over fs — never the inverted reciprocal
`bsp.bandpower(x, fs, (lo, hi))`	a half-open band `[lo, hi)`, so adjacent bands do not double-count the shared edge
`bsp.snr_db`, `sqrtN_gain_db`, `alias_frequency`, `quantization_snr_db`	the book's DSP laws as one-liners

varkonyibarnabas@gmail.com

Helper	What it guarantees
`bsp.epoch_features`, `feature_table`	returns `X`, `y`, `groups`, and feature names, keeping the subject grouping intact
`bsp.default_classifier`, `loso_evaluate`	a fold-safe `Pipeline` scored under leave-one-subject-out
`bsp.assert_no_subject_leak`, `assert_fold_safe_scaling`	declarative guards: the first raises on train/test subject overlap; the second raises if you assert scaling was *not* fold-local
`bsp.expect_close`, `expected_output_table`	reproducible-number checks — assert the returned `ok` to fail a build on drift

H.3 The leakage-safe machine-learning pattern

The single most important pattern in the book — every learned transform fitted *inside* each fold, and the fold split by *subject*:

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import GroupKFold, cross_val_predict

model = make_pipeline(StandardScaler(), SVC())   # scale-sensitive SVM
pred  = cross_val_predict(model, X, y, cv=GroupKFold(5), groups=subject_id)
# fold-local scaling matters; score pred with cohen_kappa_score / f1_score
# (group = subject, never epoch); GroupKFold needs n_splits <= n_subjects.
```

Everything learned from data — scaling, imputation, PCA, feature selection, hyperparameter tuning, calibration, and the decision threshold — belongs inside the pipeline and the fold, never fitted on the whole set first (Chapter 12; TrustCV's `DataLeakageChecker` audits this automatically).

H.4 Reproducibility and verification checklist

Before reporting a result, confirm that:

- Python and package versions are recorded, and randomness comes from a seeded `np.random.default_rng`;
- the dataset name, version, licence, and a checksum are recorded;
- the split *unit* (epoch, subject, or site) is declared and matched to the claim;
- `fs`, units, and every filter and spectral parameter are explicit;
- each learned transform is fitted inside the training fold, never before the split;
- synthetic-versus-real mode is visible on the figure, and synthetic numbers are never reported as clinical;
- expected outputs and tolerances are stated (`bsp.expect_close`);
- every figure is traceable to the exact code, and conclusions are limited to the data and the validation design.

varkonyibarnabas@gmail.com

varkonyibarnabas@gmail.com

Appendix I — Abbreviations & Acronyms

Biomedical signal processing borrows freely from physiology, electronics, statistics, and machine learning, and each field arrives with its own initialisms. This table collects the abbreviations and acronyms used across the book, expanded once so a reader from any background can decode them at a glance. Terms (rather than acronyms) are defined in the Glossary that follows.

Acronym	Expansion
AASM	American Academy of Sleep Medicine (sleep-scoring standard)
ABP	Arterial Blood Pressure (waveform)
ADC	Analog-to-Digital Converter
AF	Atrial Fibrillation
AIC	Akaike Information Criterion (model-order selection)
AR	Autoregressive (model)
ARMA	Autoregressive Moving-Average (model)
AUC	Area Under the (ROC) Curve
BCG	Ballistocardiogram (whole-body cardiac recoil)
BCI	Brain–Computer Interface
BIA	Bioelectrical Impedance Analysis (body composition)
BIS	Bioimpedance Spectroscopy
CMRR	Common-Mode Rejection Ratio
CNN	Convolutional Neural Network
CSP	Common Spatial Patterns
CTG	Cardiotocography (fetal heart rate + uterine activity)
CVP	Central Venous Pressure (waveform)
CWT	Continuous Wavelet Transform
DCT	Discrete Cosine Transform
DFA	Detrended Fluctuation Analysis
DFT	Discrete Fourier Transform
DPSS	Discrete Prolate Spheroidal Sequences (Slepian tapers)
DSP	Digital Signal Processing
DWT	Discrete Wavelet Transform
EBI	Electrical Bioimpedance
ECG	Electrocardiogram (heart)
EDA	Electrodermal Activity (skin conductance; see GSR)
EDF	European Data Format (biosignal file format)
EEG	Electroencephalogram (brain)
EGG	Electrogastrogram (stomach)
EIT	Electrical Impedance Tomography
EMG	Electromyogram (muscle)
ENOB	Effective Number Of Bits
EOG	Electrooculogram (eye movement)
ERG	Electroretinogram (retina)
F1	F1 score (harmonic mean of precision and recall)
FFT	Fast Fourier Transform
FIGO	International Federation of Gynecology and Obstetrics (CTG guidelines)
FIR	Finite Impulse Response (filter)

varkonyibarnabas@gmail.com

Acronym	Expansion
FN / FP	False Negative / False Positive
fNIRS	functional Near-Infrared Spectroscopy
GAF	Gramian Angular Field (signal-to-image encoding)
GDPR	General Data Protection Regulation (EU)
GLCM	Grey-Level Co-occurrence Matrix (image texture)
GRU	Gated Recurrent Unit
GSR	Galvanic Skin Response (see EDA)
HIPAA	Health Insurance Portability and Accountability Act (US)
HRV	Heart-Rate Variability
ICA	Independent Component Analysis
ICG	Impedance Cardiography
ICP	Intracranial Pressure (waveform)
IEC	International Electrotechnical Commission (safety, e.g. IEC 60601)
IIR	Infinite Impulse Response (filter)
IMU	Inertial Measurement Unit (accelerometer + gyroscope)
IOP	Intraocular Pressure
IUPC	Intrauterine Pressure Catheter
k-NN	k-Nearest Neighbours (classifier)
LF / HF	Low-Frequency / High-Frequency (HRV bands)
LMS	Least Mean Squares (adaptive filter)
LOSO	Leave-One-Subject-Out (cross-validation)
LSTM	Long Short-Term Memory (recurrent network)
MAD	Median Absolute Deviation
MAR	Missing At Random
MCAR	Missing Completely At Random
MDL	Minimum Description Length (model-order selection)
MDR	Medical Device Regulation (EU 2017/745)
MEMS	Micro-Electro-Mechanical System (accelerometer/gyroscope)
MFCC	Mel-Frequency Cepstral Coefficients (acoustic features)
MI	Mutual Information
MLOps	Machine-Learning Operations (deployment, monitoring, retraining)
MLP	Multi-Layer Perceptron (fully-connected network)
MMG	Mechanomyogram (muscle vibration)
MNAR	Missing Not At Random
MTF	Markov Transition Field (signal-to-image encoding)
NPU	Neural Processing Unit (edge AI accelerator)
ODC-BY	Open Data Commons Attribution (open licence)
ONNX	Open Neural Network Exchange (portable model format)
PAP	Pulmonary Artery Pressure (waveform)
PCA	Principal Component Analysis
PCG	Phonocardiogram (heart sounds)
PMA	Premarket Approval (FDA device pathway)
PPG	Photoplethysmogram (optical blood-volume pulse)
PSD	Power Spectral Density
PTT	Pulse Transit Time
QRS	QRS complex (the ventricular-depolarisation deflections of the ECG beat; contains the R-peak)

varkonyibarnabas@gmail.com

Acronym	Expansion
ReLU	Rectified Linear Unit (activation function)
REM	Rapid Eye Movement (sleep stage)
RIP	Respiratory Inductance Plethysmography
RLS	Recursive Least Squares (adaptive filter)
RMS	Root Mean Square
RMSSD	Root Mean Square of Successive Differences (HRV)
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic (curve)
SaMD	Software-as-a-Medical-Device
SCG	Seismocardiogram (chest micro-motion)
SDNN	Standard Deviation of NN (normal-to-normal) intervals (HRV)
SNR	Signal-to-Noise Ratio
SoS / SOS	Second-Order Sections (biquad filter cascade)
SpO_2	Peripheral Oxygen Saturation
SSVEP	Steady-State Visually Evoked Potential
STFT	Short-Time Fourier Transform
STV / LTV	Short-Term / Long-Term Variability (fetal heart rate)
SVM	Support Vector Machine
TN / TP	True Negative / True Positive
TOCO	Tocodynamometer (external uterine-contraction sensor)
TST	Total Sleep Time
ViT	Vision Transformer
WASO	Wake After Sleep Onset
WSS	Wide-Sense Stationary

Sleep-report summaries used throughout the case study — total sleep time (TST), wake after sleep onset (WASO), REM latency (time from sleep onset to first REM), and sleep efficiency (TST as a fraction of time in bed) — are defined where they first appear in Chapters 12 and 16; transformer and other named neural architectures are introduced in Chapter 14.

varkonyibarnabas@gmail.com

Appendix J — Glossary

This glossary defines the core terms of the book in one line each; abbreviations and acronyms are expanded in Appendix I. It is a quick reference, not a substitute for the chapter that develops each idea. Terms are listed alphabetically.

Term	Definition
Aliasing	A high frequency masquerading as a lower one because the signal was sampled below the Nyquist rate; irreversible once it occurs.
Analytic signal	The complex signal built from a real signal and its Hilbert transform, from which instantaneous amplitude (envelope) and phase are read.
Anti-alias filter	An analog low-pass filter applied before the ADC to remove energy above the Nyquist frequency.
Autocorrelation	A signal's correlation with a time-shifted copy of itself; a measure of self-predictability and periodicity.
Band power	The signal power within a frequency band — a core spectral feature (e.g. EEG delta/alpha/sigma power).
Baseline wander	Slow low-frequency drift of a signal's baseline, typically from electrode motion or respiration.
Bioimpedance	The opposition of tissue to an applied alternating current; an induced measurement whose changes track respiration, blood flow, and body composition.
Cohen's kappa (κ)	An agreement metric that corrects observed accuracy for the agreement expected by chance.
Convolution	The operation combining an input with a system's impulse response to give its output — multiplication in the frequency domain.
Corneo-retinal potential	The eye's standing electrical dipole (cornea positive relative to retina) whose rotation the EOG records.
Cross-validation	Estimating performance by repeatedly training on part of the data and testing on the held-out remainder.
Data leakage	Any path by which test information influences training, inflating apparent performance (e.g. splitting one subject's epochs across train and test).
Domain shift	A change in data distribution between training and deployment (new device, subject, or site) that degrades a model.
Ensemble averaging	Averaging many stimulus-locked repetitions to raise SNR by $\sqrt{N}$, since the signal adds and independent noise partly cancels.
Epoch	A short fixed-length window of a signal (30 s in sleep scoring) treated as locally stationary for analysis.
Feature	A number computed from a signal (band power, RMS, …) that summarises it for a classifier — a level-2 quantity, not a raw signal.
FIR / IIR filter	Finite- and infinite-impulse-response digital filters; FIR has no feedback and can be designed for exactly linear phase, IIR is efficient but can distort phase.

varkonyibarnabas@gmail.com

Term	Definition
Foundation model	A large model pretrained on broad data and then adapted to specific tasks (Chapter 14).
Fourier transform	The representation of a signal as a sum of sinusoids, exposing its frequency content.
Heart-rate variability	Beat-to-beat variation in heart rate, a window on autonomic control (time- and frequency-domain indices).
Hypnogram	The time series of sleep stages across a night — a level-2 derived series, not a raw signal.
Impulse response	A linear time-invariant system's output to a unit impulse; it fully characterises the system.
Motion artifact	Corruption from movement disturbing the electrode interface and cables, appearing as low-frequency transients and baseline shifts.
Multitaper	A spectral estimator averaging several orthogonal (Slepian) tapered periodograms to cut variance without shortening the record.
Notch filter	A narrow band-stop filter removing a single frequency, typically the 50/60 Hz powerline frequency.
Nyquist frequency / rate	Half the sampling rate (highest representable frequency) / twice the signal's highest frequency (the minimum sampling rate).
Overfitting	A model fitting noise or quirks of the training data and so generalising poorly.
Pan–Tompkins	The classic algorithm for detecting QRS complexes (R-peaks) in the ECG.
Periodogram / Welch	The squared FFT magnitude as a raw spectral estimate / Welch's averaged, windowed version trading resolution for lower variance.
Power spectral density	How a signal's power is distributed across frequency.
Quantization	(1) Discretising a signal's amplitude into a finite number of ADC levels, introducing quantization noise (Ch. 2). (2) In deep-learning deployment (Ch. 15), reducing the numerical precision of a model's weights/activations, e.g. FP32→INT8.
Sampling	Reading a continuous signal at regular instants to produce a discrete sequence.
Spectral leakage	Smearing of spectral energy into neighbouring frequencies caused by analysing a finite (windowed) record.
Spectrogram / STFT	A time-frequency image from the short-time Fourier transform, showing how the spectrum evolves over time.
Stationarity	The property that a signal's statistics do not change over time; biosignals are usually only locally (within-window) stationary.
Tetrapolar measurement	A four-electrode bioimpedance method separating current injection from voltage sensing to cancel electrode-impedance error.
Wavelet transform	A multi-resolution transform using scaled, shifted wavelets: good time localisation at high frequency, good frequency localisation at low.

varkonyibarnabas@gmail.com

Zero-phase filtering	Filtering forward then backward (filtfilt) so the output has no phase distortion, preserving waveform landmarks.

varkonyibarnabas@gmail.com

References & Further Reading

The following works ground the book and are the natural next steps for a reader who wants to go deeper. They are grouped by purpose.

Accessible and biomedical foundations

- Samani, A. *An Introduction to Signal Processing for Non-Engineers.* CRC Press. The accessibility and tone benchmark for this book, covering the measurement pipeline, linear time-invariant systems, and spatial filters for a non-engineering reader.
- Rangayyan, R. M. *Biomedical Signal Analysis.* IEEE Press / Wiley. A comprehensive reference for biomedical signal processing across modalities.
- Semmlow, J. L., and Griffel, B. *Biosignal and Medical Image Processing.* CRC Press. A combined treatment of one-dimensional signals and medical images, mirroring this book's signals-and-images approach.

Classical signal processing (depth references)

- Oppenheim, A. V., and Schafer, R. W. *Discrete-Time Signal Processing.* Pearson. The standard rigorous reference for the mathematics of discrete-time signals and systems.
- Haykin, S. *Adaptive Filter Theory.* Pearson. The reference for LMS and RLS adaptive filtering and adaptive noise cancellation, underpinning Chapter 10 (noise reduction).

Machine learning and deep learning

- Bishop, C. M. *Pattern Recognition and Machine Learning.* Springer. Foundations of statistical machine learning.
- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow.* O'Reilly. A practical, code-first companion for the machine-learning and deep-learning pipeline.

Evaluation and agreement statistics

- Bland, J. M., and Altman, D. G. (1986). "Statistical methods for assessing agreement between two methods of clinical measurement." *The Lancet*, 327(8476), 307–310. The method-comparison analysis (difference-versus-mean plot with limits of agreement) used for night-level clinical-summary agreement in Chapters 12 and 16.
- Cohen, J. (1960). "A coefficient of agreement for nominal scales." *Educational and Psychological Measurement*, 20(1), 37–46. The origin of Cohen's kappa, the chance-corrected agreement coefficient that is the headline metric for sleep staging in Chapters 12 and 16.

varkonyibarnabas@gmail.com

Clinical grounding and data

- American Academy of Sleep Medicine. *The AASM Manual for the Scoring of Sleep and Associated Events.* The clinical scoring standard underpinning the sleep-staging case study.
- PhysioNet — physionet.org. Open physiological databases: ECG (MIT-BIH, PTB-XL), EMG, cardiotocography (CTU-UHB), and polysomnography (Sleep-EDF).
- UCI Machine Learning Repository — archive.ics.uci.edu. Open datasets including Cardiotocography and Human Activity Recognition (inertial-measurement-unit) data.

Intuition-first and visual references

- Downey, A. B. *Think DSP: Digital Signal Processing in Python.* O'Reilly / Green Tea Press. Learns digital signal processing by writing code first — a model for the companion notebooks.
- Schaedler, J. *Seeing Circles, Sines, and Signals.* An interactive, visual primer that builds Fourier intuition from rotation and phasors.
- 3Blue1Brown, Fourier-series and transform visualizations; and BetterExplained, "An Interactive Guide to the Fourier Transform." Intuition-first visual explanations of the ideas in Appendix B and Chapter 3.

Signal-processing education

- Wage, K. E., Buck, J. R., et al. "The Signals and Systems Concept Inventory." *IEEE Transactions on Education.* The standard instrument for measuring conceptual understanding and common misconceptions in signals-and-systems teaching, and a model for the book's conceptual questions.

varkonyibarnabas@gmail.com